Advanced Deep Learning

Paul Swoboda

HHU Düsseldorf

Summer Semester 2026

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Overall Goals

- Understand current deep learning state of the art (LLMs, vision, multimodal)
- Achieve mastery of deep neural networks
- Cover a wide range of topics, fast tempo, in-depth
- Coding whenever possible, conceptual and technical depth
- After the course you will be well-equipped for a Master's thesis

Side Effects

- Earning potential increased (!?)
- You will come to appreciate the bitter lesson [R. Sutton, *The Bitter Lesson*, 2019]
- Ability to follow and discern the scientific literature in deep learning
- You will become more productive and iterate faster
- You will learn to intuit: what is fundamentally important vs. what will fade away? Where do performance limitations come from?
- Better mental preparation for AGI?

Requirements

Necessary

- (Introduction to) Deep Learning
- Machine Learning

Desirable

- Reinforcement Learning (Milica Gasic)
- Implementing Transformers Practical (Carol van Niekerk)

Interaction

Please initiate discussion, pose questions etc.

Tempo

Due to the amount of material covered, I will pursue a higher speed.

Expectation

I expect a significant time investment from students. I also invest effort.

Literature

Deep Learning

- [Goodfellow et al., *Deep Learning*, 2016] – the classic
- [Prince, *Understanding Deep Learning*, 2023] – modern, visual
- [C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, 2024] – thorough, updated, my favorite
- [Fleuret, *The Little Book of Deep Learning*, 2023] – concise handbook

Machine Learning

- [Murphy, *Probabilistic Machine Learning: An Introduction*, 2022] – broad introduction
- [Murphy, *Probabilistic Machine Learning: Advanced Topics*, 2023] – advanced topics
- [C. M. Bishop, *Pattern Recognition and Machine Learning*, 2006] – a classic reference

Reinforcement Learning

- [R. S. Sutton and Barto, *Reinforcement Learning: An Introduction*, 2018]
- [Silver, *Lectures on Reinforcement Learning*, 2015]
- [Schneider and Wallkötter, *Reinforcement Learning Course Materials*, 2023]

Organization

People

- **Lecturer:** Paul Swoboda
  paul.swoboda@hhu.de
- **Assistant:** Paweł Batorski
  pawel.batorski@hhu.de
- **Tutor:** Abtin Pourhadi
  abtin.pourhadi@hhu.de

Administrative questions
(exam registration, study regulations, etc.)

- Claudia Forstinger
  claudia.forstinger@hhu.de

Communication

- Primary channel: Discord → or would you prefer rocketchat?

Materials

- Github
  https://github.com/pawelswoboda/ADL
- Please file corrections etc. through pull requests

Lecture Modality

- Assignments will be given each week/second week, depending on scope of exercise. Work on them in groups of three.
- Assignments will be corrected, but we will not count points (thank you OpenAI/Anthropic/...)
- Two or three mini-exams will determine the grade. One or two exams mid-semester, the final one at the end.
- Access to GPUs for completing exercises will be granted on request.

Topics

Transformers

- GPT-2 reimplementation (à la Karpathy)
- Recent architectural improvements
- Positional Encodings: RoPE, YaRN
- Efficient attention: {Flash|Ring|Tree} attention
- KV-Cache: Multi-head latent attention, grouped query attention
- Mixture of experts, load balancing
- Transformers for computer vision: ViT, DETR, TrackFormer, MaskFormer, SWin, ...
- Multimodal models: CLIP, SigLIP, PaliGemma

Recursive Models

- RNNs, LSTM, GRU
- Mamba{1|2}: Efficient computation with prefix sum & chunked matrix multiplication

Agentic

- Prompting: In-context learning, {chain|tree} of thought
- Memory
- Reasoning

Training

- Training: Pre-training, SFT, RLHF & post-training
- Efficiency: Mixed precision, kernel fusion
- Muon optimizer
- Parallelism: Distributed training, Data/model/pipeline parallelism, sharding
- Hyperparameters
- Fine-tuning: LoRA, Adapters
- Distillation
- Reinforcement learning: PPO, GRPO
- Scaling laws

Theme

I hope to cover all the essential technical content that allows deep technical understanding of the current iteration of intelligent machines.

Language Models: The Core Idea

A **language model** assigns probabilities to sequences of words (or tokens):

$$P(w_1, w_2, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1})$$

Claude Shannon (1951)

Modeled English as a stochastic process; estimated per-character entropy at $\sim$1 bit [Shannon, *"Prediction and Entropy of Printed English"*, 1951].

N-gram models

$P(w_t \mid w_{t-n+1}, \ldots, w_{t-1})$: count-based, with smoothing techniques like Kneser-Ney [Kneser and Ney, *Improved Backing-off for M-gram Language Modeling*, 1995]. Dominated NLP for decades.

Fundamental limitation

N-grams cannot generalize to unseen word combinations. Not scalable, no semantic understanding.

Approaches That Did Not Scale

Pattern: Encode human knowledge into the system → initial success → plateau → superseded by learning from data.

Symbolic / grammar-based
Hand-written grammars, rule-based parsers, knowledge bases. Linguists formalized syntax [Chomsky, *Syntactic Structures*, 1957], annotated treebanks [Marcus et al., *"Building a Large Annotated Corpus of English: The Penn Treebank"*, 1993], built rule-based MT systems. Brittle, expensive to maintain, poor on out-of-domain text.

"Every time I fire a linguist, the performance of the speech recognizer goes up."
– Frederick Jelinek, IBM (c. 1988) [Jelinek, *"Some of My Best Friends are Linguists"*, 2005]

Feature-engineered ML SVMs, CRFs (Conditional Random Fields), . . . with hand-crafted features (bag-of-words, TF-IDF, POS tags) [Joachims, *"Text Categorization with Support Vector Machines: Learning with Many Relevant Features"*, 1998]. Each NLP task required a separate pipeline with separately engineered features. No transfer, no shared representations, error propagation.

The transition Statistical methods beat symbolic ones. Neural networks [Collobert et al., *"Natural Language Processing (Almost) from Scratch"*, 2011] then replaced feature engineering entirely by learning representations end-to-end from raw text.

From N-grams to Neural Language Models

Core shift: Replace discrete, count-based representations with continuous, *learned* ones.

Learned word embeddings (2003–2014)

Neural networks learn to map words to dense vectors where proximity encodes semantic similarity [Bengio et al., *"A Neural Probabilistic Language Model"*, 2003; Mikolov et al., *"Distributed Representations of Words and Phrases and Their Compositionality"*, 2013; Pennington et al., *"GloVe: Global Vectors for Word Representation"*, 2014]. Representations emerge from the training objective, not from human annotation.

Sequence-to-sequence (2014)

Encoder-decoder architectures compress an entire input sequence into a vector, then decode it into an output sequence [Sutskever et al., *"Sequence to Sequence Learning with Neural Networks"*, 2014]. Enabled end-to-end learning for translation, summarization, etc.

Attention (2015)

Instead of compressing everything into a single vector, let the decoder attend to all encoder states [Bahdanau et al., *"Neural Machine Translation by Jointly Learning to Align and Translate"*, 2015]. Removes the information bottleneck. Precursor to the Transformer.

Recurring theme

Each advance removes a hand-designed component (feature engineering → embeddings, fixed context → attention) and replaces it with something learned.

The Transformer Revolution

Transformer (2017)
Vaswani et al. replaced recurrence entirely with **self-attention** [Vaswani et al., *"Attention Is All You Need"*, 2017]. Parallel training, long-range dependencies, superior scaling.

ELMo (2018)
Peters et al. introduced *contextualized* word representations from bidirectional LSTMs [Peters et al., *"Deep Contextualized Word Representations"*, 2018]. Same word gets different embeddings in different contexts.

GPT (2018)
Radford et al. showed that *unsupervised pre-training* of a Transformer decoder, followed by fine-tuning, achieves strong results across many tasks [Radford, Narasimhan, et al., *Improving Language Understanding by Generative Pre-Training*, 2018].

BERT (2019)
Devlin et al. pre-trained a bidirectional Transformer encoder with masked language modeling [Devlin et al., *"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding"*, 2019]. Dominated NLU benchmarks.

GPT-2 (2019)
Scaling up GPT to 1.5B parameters demonstrated impressive zero-shot text generation [Radford, J. Wu, et al., *Language Models are Unsupervised Multitask Learners*, 2019]. "Too dangerous to release."

Paradigm shift

Pre-train a large model on massive text corpora, then adapt to downstream tasks. The era of "foundation models" begins.

The Scaling Era

GPT-3 (2020)
175B parameters. In-context learning: the model performs tasks from a few examples in the prompt, no fine-tuning needed [Brown et al., *"Language Models are Few-Shot Learners"*, 2020].

Scaling laws (2020)
Kaplan et al. showed predictable power-law relationships between compute, data, model size, and loss [J. Kaplan et al., *"Scaling Laws for Neural Language Models"*, 2020].

PaLM (2022)
540B parameters. Breakthrough reasoning and code generation [Chowdhery et al., *"PaLM: Scaling Language Modeling with Pathways"*, 2023].

Chinchilla (2022)
Hoffmann et al. showed prior models were *undertrained*: optimal allocation balances model size and data volume [Hoffmann et al., *"Training Compute-Optimal Large Language Models"*, 2022].

GPT-4 (2023)
Multimodal. Human-level on many professional exams [OpenAI, *"GPT-4 Technical Report"*, 2023].

Open models
LLaMA [Touvron, Lavril, et al., *"LLaMA: Open and Efficient Foundation Language Models"*, 2023], Llama 2 [Touvron, Martin, et al., *"Llama 2: Open Foundation and Fine-Tuned Chat Models"*, 2023], Llama 3 [Grattafiori et al., *"The Llama 3 Herd of Models"*, 2024], Mistral [A. Q. Jiang et al., *"Mistral 7B"*, 2023], DeepSeek [DeepSeek-AI, *"DeepSeek-V3 Technical Report"*, 2024] democratized access.

The Reasoning Era (2024–)

Pre-training scaling is slowing down. The new frontier: **test-time compute** and **reinforcement learning on verifiable rewards (RLVR)**.

Chain-of-thought
: Models reason step-by-step before answering. Unlocks multi-step math, logic, and coding tasks [Wei et al., *"Chain-of-Thought Prompting Elicits Reasoning in Large Language Models"*, 2022].

o1 / o3 (OpenAI, 2024–25)
: Trained with large-scale RL to "think" internally before responding [OpenAI, *Learning to Reason with LLMs*, 2024; OpenAI, *Introducing OpenAI o3 and o4-mini*, 2025]. Spend more compute at inference time for harder problems.

DeepSeek-R1 (2025)
: Open-weight reasoning model. Emergent chain-of-thought arises purely from RL on verifiable math/code rewards [DeepSeek-AI, *"DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning"*, 2025].

RLVR
: RL with *verifiable* rewards (unit tests, proof checkers, math verification) instead of human preference labels. Scalable, objective, no reward hacking.

Shift in scaling axis

Pre-training: scale parameters and data (fixed compute at inference).
Reasoning: scale *inference-time compute* via RL-trained thinking (variable cost per query).

The Agentic Era (2025–)

LLMs move from passive text generators to **autonomous agents** that use tools, write code, and complete multi-step tasks.

Code agents
: LLMs integrated into development environments that read, write, and debug code autonomously. Claude Code (Anthropic), Codex (OpenAI), Gemini CLI (Google), Copilot (GitHub).

Tool use
: Models learn to call APIs, execute shell commands, browse the web, and manipulate files. Enables open-ended problem solving beyond text completion.

Agentic loops
: The model operates in a loop: observe $\rightarrow$ plan $\rightarrow$ act $\rightarrow$ observe. Requires grounding (tool results), memory (context management), and self-correction.

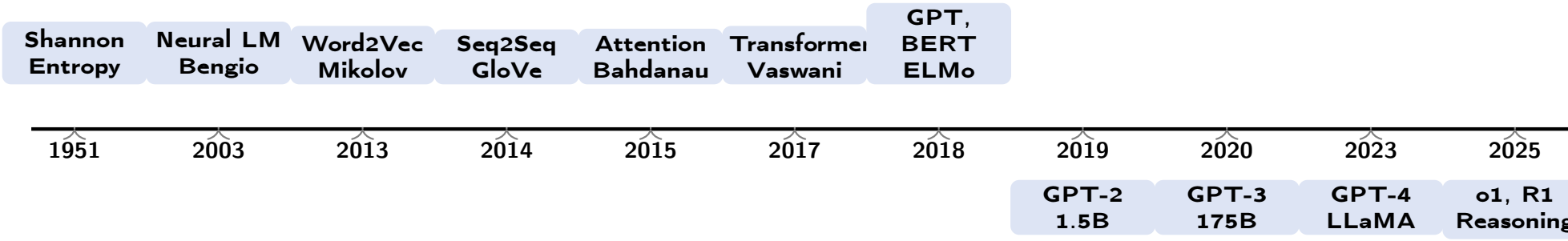
Computer use
: Agents that interact with GUIs: clicking, typing, navigating applications like a human user [Anthropic, *Introducing Computer Use, a New Claude 3.5 Sonnet, and Claude 3.5 Haiku*, 2024].

Multi-agent systems
: Multiple specialized agents collaborate on complex tasks. Orchestration, delegation, and communication between agents.

Shift

From "model as oracle" to "model as worker": the bottleneck moves from capability to reliability and trust.

Historical Timeline



Trend

Hand-crafted features → learned representations → pre-trained foundation models → scaling → reasoning via RL.

The Bitter Lesson [R. Sutton, *The Bitter Lesson*, 2019]

Core thesis: General methods leveraging computation beat methods leveraging human knowledge, and by a large margin.

Chess
Human-crafted evaluation functions dominated for decades, but massive brute-force search defeated Kasparov in 1997 [Campbell et al., *"Deep Blue"*, 2002].

Go Initial efforts exploited human knowledge of the game. AlphaGo Zero learned tabula rasa via self-play + search, surpassing all predecessors [Silver et al., *"Mastering the game of Go without human knowledge"*, 2017].

Speech
Hand-engineered phoneme/vocal-tract models lost to statistical HMMs in the 1970s DARPA competition. Deep learning pushed this further [Hinton, L. Deng, et al., *"Deep neural networks for acoustic modeling in speech recognition"*, 2012].

Vision
Edge detectors, SIFT features, all replaced by learned convolutions [Krizhevsky et al., *"ImageNet classification with deep convolutional neural networks"*, 2012].

The Lesson

1. Researchers build domain knowledge into agents. Helps short-term, satisfying personally.
2. In the long run it plateaus and *inhibits* further progress.
3. Breakthroughs come from scaling computation via **search** and **learning**.

"We want AI agents that can *discover* like we can, not which contain what we have discovered." – R. Sutton

Philosophical/Speculation

- AGI: I think it is a plausible hypothesis that LLMs (or incremental improvements of them) will lead to human-level intelligence in a vast array of settings.
- Timeline: I think timeslines are on the shorter side.
- Emergence of intelligence: It is interesting that relatively easy ideas result in emergence of intelligence.
- Ghosts: LLMs are ghost-like: They are an echo (statistical destillation) of humans organisms, but lack embodiment. Is this a deeper restriction?
- Criticism: Some people do not believe in LLMs:
  - World models: Yann LeCun thinks world models are necessary: Causal and physical understanding of the world is not present in LLMs.
  - Bitter lesson: Richard Sutton thinks that LLMs and transformers are not bitter-pilled enough. We only scale through pre- and post-training, in ah higly complex manner. A truly bitter-lesson pilled approach would require a single fundamental principle that learns through general interaction with the world, i.e. test-time training.
  - Jagged Intelligence: Right now LLMs have jagged intelligence: Superhuman at some complex tasks, struggling with some simple ones. I think that humans suffer from the same, though.

Table of Contents

1 Introduction

2 Language Generation
- Tokenization
- Training Objective
- Autoregressive Sampling

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Table of Contents

1 Introduction

2 Language Generation
- Tokenization
- Training Objective
- Autoregressive Sampling

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

From Text to Tokens

Raw text must be converted to a sequence of discrete **tokens** before a model can process it.

Character-level tokenization

`"Hello"` $\rightarrow$ `[H, e, l, l, o]`

- Tiny vocabulary ($\sim$100 symbols for ASCII/Unicode subsets)
- Long sequences; struggles to capture word-level structure

Word-level tokenization

`"Hello world"` $\rightarrow$ `[Hello, world]`

- Huge vocabulary; unknown-word problem for rare words

Subword tokenization (used in practice)

Split into frequent subword units: `"unhappiness"` $\rightarrow$ `[un, happiness]`

- Compact vocabulary ($\sim$50k tokens)
- Handles rare words gracefully via decomposition

Byte-Pair Encoding (BPE)

BPE [Sennrich et al., *"Neural Machine Translation of Rare Words with Subword Units"*, 2016] is the dominant subword algorithm (used in GPT-2 [Radford, J. Wu, et al., *Language Models are Unsupervised Multitask Learners*, 2019]/GPT-3/4, LLaMA, . . .).

Training algorithm:

1. Start with a vocabulary of individual bytes (256 entries)
2. Count all adjacent pair frequencies in the corpus
3. Merge the most frequent pair → add the merged token to the vocabulary
4. Repeat until vocabulary size V is reached

Example: training on corpus {"hug"×10, "pug"×5, "bug"×3}

Step 0:	h u g ×10	p u g ×5	b u g ×3	**(u,g): 18 → merge**
Step 1:	h ug ×10	p ug ×5	b ug ×3	**(h,ug): 10 → merge**
Step 2:	hug ×10	p ug ×5	b ug ×3	$\mathcal{V}$: h,u,g,p,b,ug,hug

At inference: apply learned merges greedily, left to right. Every byte sequence is encodable: **no unknown tokens.**

BPE Tokenization at Inference

Given a string, there are typically **many valid segmentations** into vocabulary tokens.

Example: `"tokenization"` could be split as

- `[token, ization]` (2 tokens)
- `[token, iz, ation]` (3 tokens)
- `[t, o, k, e, n, i, z, a, t, i, o, n]` (12 tokens – bytes)

Which segmentation does BPE choose?

1. Start from the byte-level decomposition of the input string
2. Find the pair of adjacent tokens whose merge rule has the **highest priority** (i.e. was learned earliest during training)
3. Merge that pair into a single token
4. Repeat until no more merge rules apply

Deterministic and unique

The merge priority table imposes a total order, so BPE always produces the **same unique tokenization** for a given string. The result tends to be close to the shortest possible segmentation (fewest tokens).

Vocabulary and Special Tokens

The tokenizer defines a **vocabulary** $\mathcal{V}$ mapping tokens $\leftrightarrow$ integer IDs.

Special tokens are added to control model behavior:

- `<|endoftext|>` / `<eos>`: marks end of a document
- `<|pad|>`: pads shorter sequences in a batch to equal length
- `<|bos|>`: beginning-of-sequence prompt for generation
- Instruction-tuned models add `<|user|>`, `<|assistant|>`, `<|system|>` etc.

Embedding lookup

Each token ID $t \in \{0, \ldots, V-1\}$ is mapped to a vector via an **embedding matrix** $E \in \mathbb{R}^{V \times d}$: the model's input is $x_i = E[t_i]$.

Note: the same matrix E is often used (transposed) to produce output logits (**weight tying**).

Try it yourself: https://tiktokenizer.vercel.app/

Table of Contents

1 Introduction

2 Language Generation
- Tokenization
- Training Objective
- Autoregressive Sampling

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Next-Token Prediction

An autoregressive language model defines a distribution over sequences by factoring it autoregressively:

$$p(x_1, x_2, \ldots, x_T) = \prod_{i=1}^{T} p(x_i \mid x_1, \ldots, x_{i-1})$$

The model is trained to predict the **next token** given all preceding tokens.

One block, T training examples:

Input context	Target
`[The]`	`cat`
`[The, cat]`	`sat`
`[The, cat, sat]`	`on`
`[The, cat, sat, on]`	`the`

Efficiency

Generation is autoregressive and slow: One token after another. Can we circumvent this limitation during training with a parallel pass through the data? We will affirmatively answer this for transformers and SSMs (Mamba).

Cross-Entropy Loss

The model outputs **logits** $z \in \mathbb{R}^V$ for the next token; convert to probabilities via softmax:

$$p_\theta(x = k \mid \text{context}) = \frac{\exp(z_k)}{\sum_{v=1}^{V} \exp(z_v)}$$

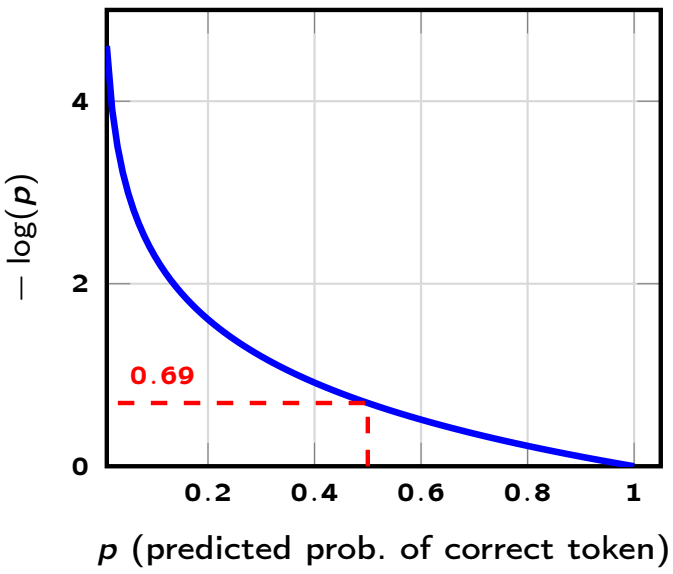
Training loss: average negative log-likelihood over all positions:

$$\mathcal{L} = -\frac{1}{T} \sum_{i=1}^{T} \log p_\theta(x_i \mid x_1, \ldots, x_{i-1})$$

Perplexity is the standard reporting metric:

$$\mathrm{PPL} = \exp(\mathcal{L})$$

A perplexity of k means the model is as uncertain as a uniform distribution over k tokens.



Teacher forcing

During training the *ground-truth* previous tokens are always fed as context, even if the model's own prediction was wrong. This decouples training from generation. We learn *offline*.

Why Next-Token Prediction Works

- **Self-supervised**: supervision signal is derived from raw text, no manual labels needed.
- **Scalable**: the internet provides virtually unlimited text data.
- **Versatile**: to predict the next token well, the model must implicitly learn grammar, facts, reasoning, style, (world model?) ...

Prediction is compression (Sutskever)

A model that predicts the next token well must build a **compressed representation** of the training data. Compression forces the model to discover the underlying structure: facts, reasoning patterns, world knowledge. The better the compression, the deeper the understanding.

Ilya Sutskever on this topic: https://www.youtube.com/watch?v=Yf1o0TQzry8 (excerpt: https://www.youtube.com/watch?v=YEUclZdj_Sc)

Emergent capabilities

As model size and data scale up, capabilities emerge that were not explicitly trained: few-shot learning, chain-of-thought reasoning, code generation, ... The compression view explains this: larger models can compress more structure, unlocking qualitatively new abilities.

Table of Contents

1 Introduction

2 Language Generation
- Tokenization
- Training Objective
- Autoregressive Sampling

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

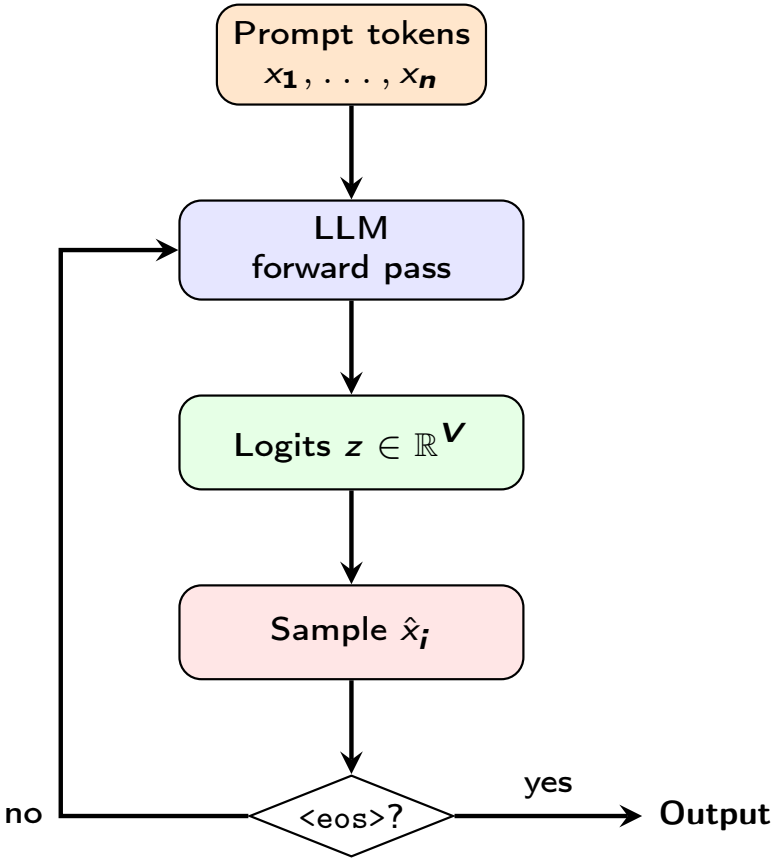
17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Generation Loop

At inference time we generate one token per step: $\hat{x}_i \sim p_\theta(\cdot \mid x_1, \ldots, x_{i-1}), \quad i = 1, 2, 3, \ldots$

1. Encode prompt into token IDs
2. Forward-pass $\rightarrow$ logits $z \in \mathbb{R}^V$
3. Apply **sampling strategy** to pick $\hat{x}_i$
4. Append $\hat{x}_i$ to context; go to 2
5. Stop at <eos> or length limit

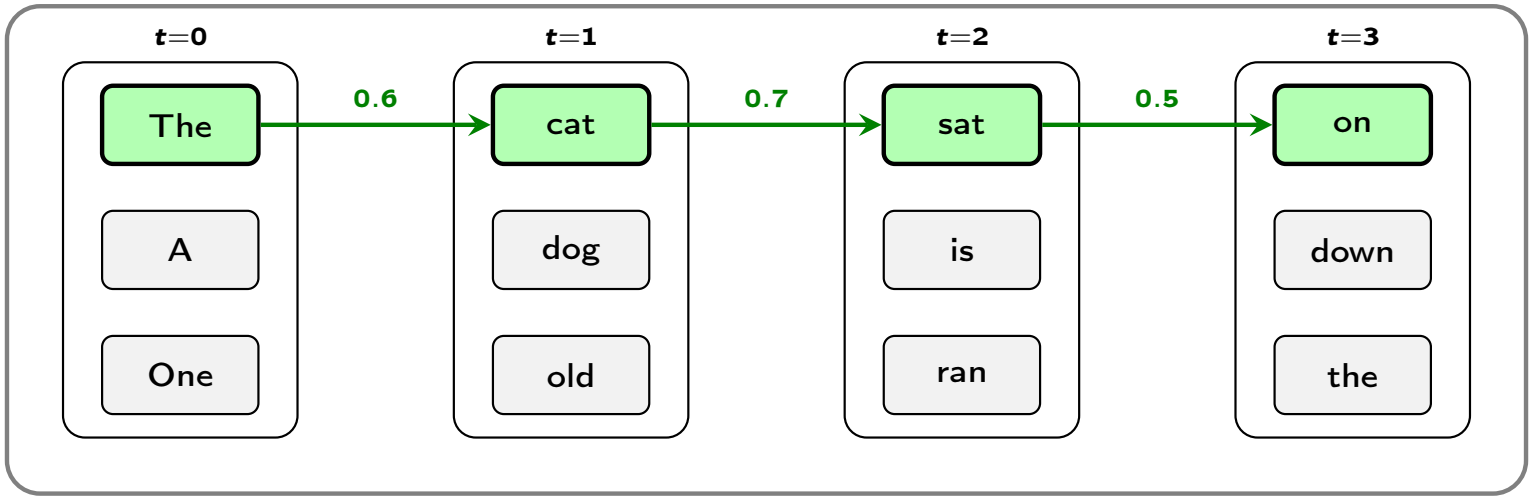


Distribution shift

Training uses ground-truth context; generation uses *model-predicted* context. Errors accumulate.

Greedy Decoding

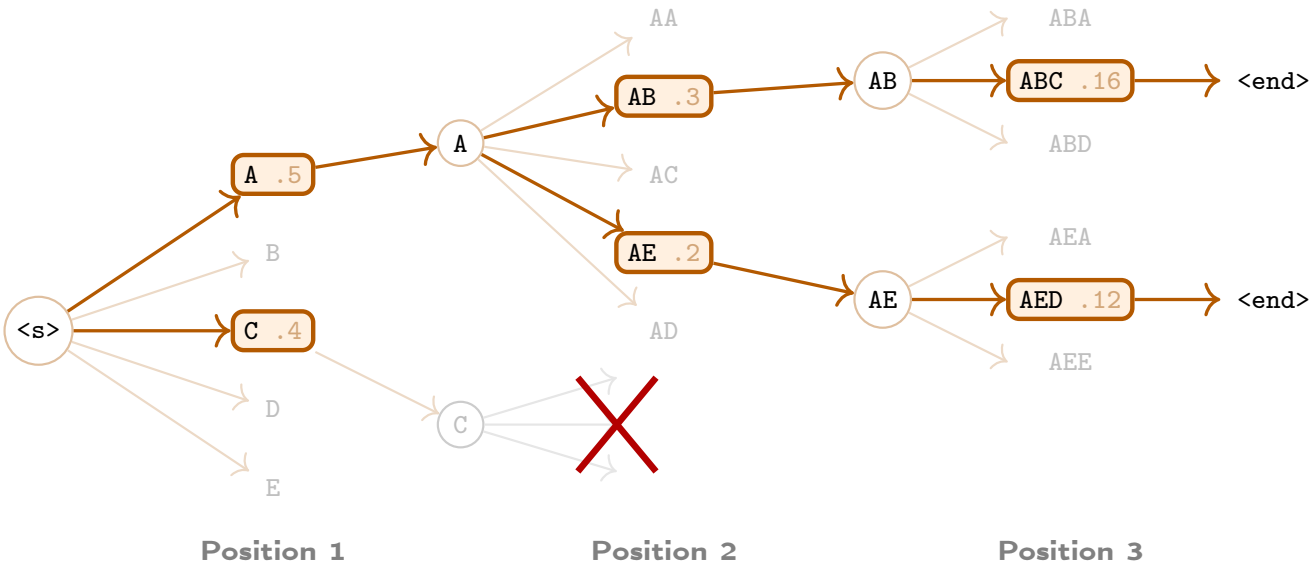
Greedy decoding: always pick the most probable token: $\hat{x}_i = \arg\max_k \; p_\theta(x = k \mid x_{<i})$



- Fast and deterministic, but often repetitive or suboptimal
- Can miss globally better sequences by being locally greedy

Beam Search

Maintain the B most probable *sequences* at each step ($B = 2$ below).



Algorithm 1: Beam Search (B = beam width)

Input: Model p_θ, prompt $x_{1:n}$, beam width B, max length T

$\mathcal{B}_0 \leftarrow \{(x_{1:n},\ 0)\}$ `// set of (sequence, log-prob) pairs`

for $t = 1, \ldots, T$ **do**

$$\mathcal{C}_t \leftarrow \bigcup_{(s,\ell) \in \mathcal{B}_{t-1}} \underset{k \in \mathcal{V}}{\text{top-}B} \left\{ \left(s \oplus k,\ \ \ell + \log p_\theta(k \mid s) \right) \right\} \qquad // \ \leq B^2 \text{ candidates}$$

$\mathcal{B}_t \leftarrow \underset{\mathcal{C}_t}{\text{top-}B}$ by score **if** *all beams end with* `<eos>` **then break**

return $\arg\max_{(s,\ell) \in \mathcal{B}_T} \ \ell \,/\, |s|^\alpha$ `// length norm.,` $\alpha \in [0.6, 1]$

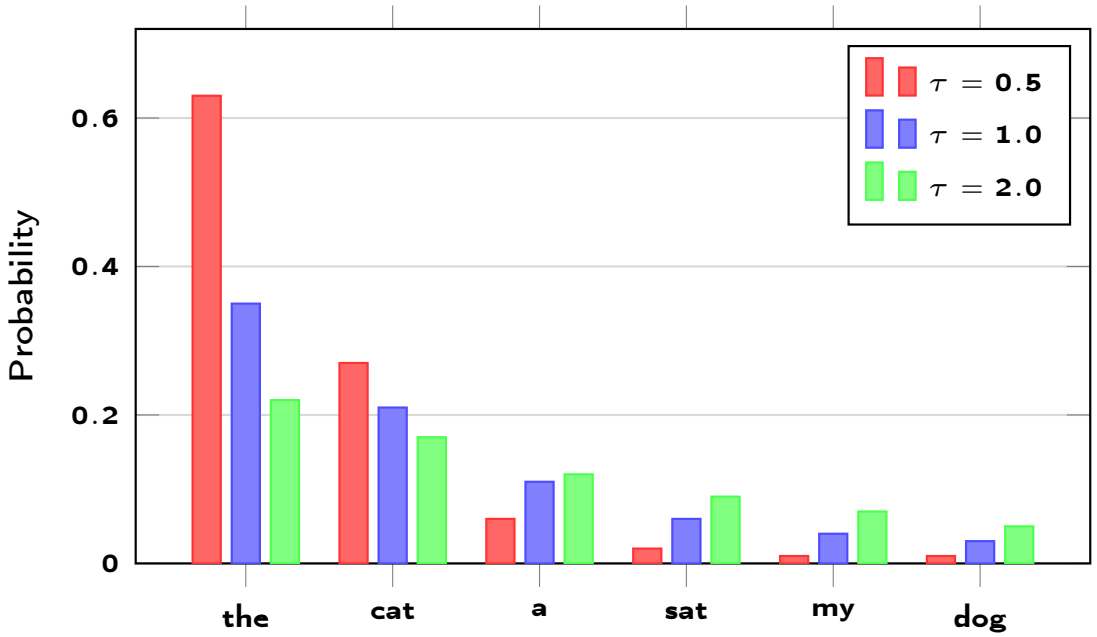
Temperature Sampling

Divide logits by a **temperature** $\tau > 0$ before softmax: $p_\tau(x = k) = \frac{\exp(z_k/\tau)}{\sum_v \exp(z_v/\tau)}$

- $\tau \to 0$: concentrates on mode $\to$ greedy
- $\tau = 1$: original distribution
- $\tau > 1$: flatter, more diverse

Intuition

Temperature controls the *sharpness*. In practice $\tau \in [0.7, 1.0]$.

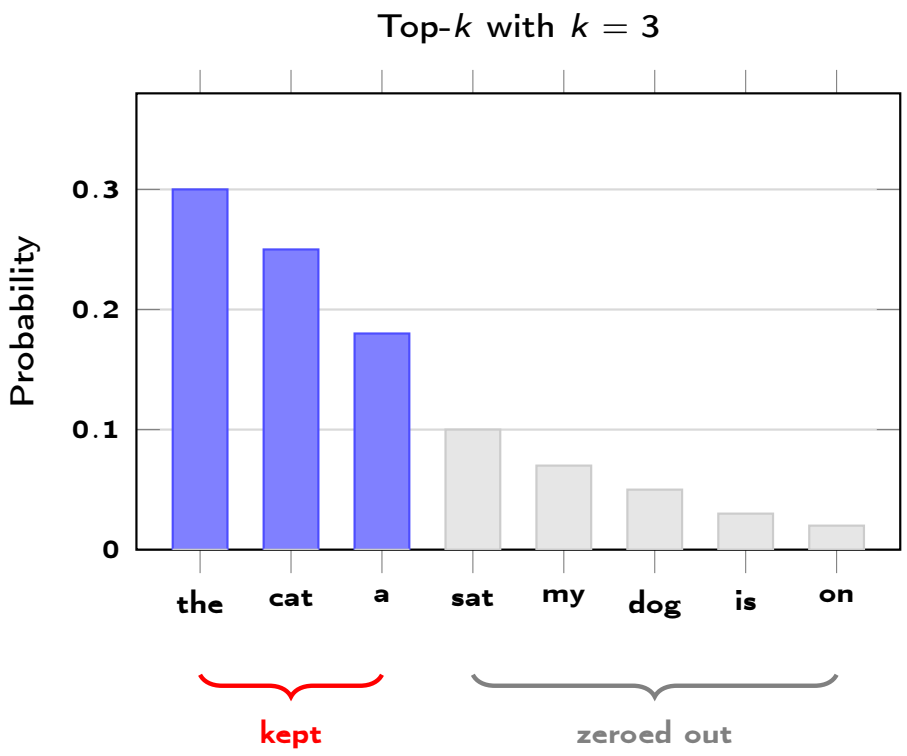


Top-k Sampling

Temperature alone may still assign probability mass to very unlikely tokens. **Truncation strategies** restrict sampling to a smaller candidate set.

Top-k sampling:

- Keep only the k tokens with highest probability; zero out the rest
- Redistribute mass among the kept tokens
- Problem: k is fixed – too restrictive when flat, too lenient when peaked

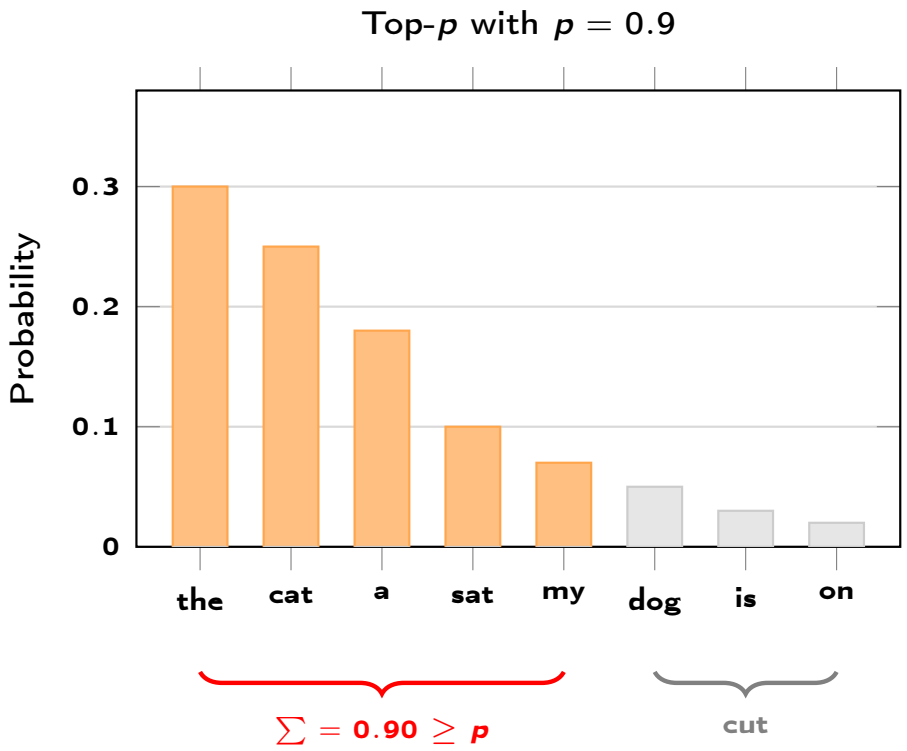


Nucleus (Top-p) Sampling

Nucleus sampling [Holtzman et al., *"The Curious Case of Neural Text Degeneration"*, 2020]:

- Sort tokens by descending probability
- Keep the smallest set whose cumulative probability $\geq p$
- Candidate set size **adapts** to the distribution shape
- Common default: $p = 0.9$

$$\mathcal{V}_p = \arg\min_{\mathcal{S}} \left\{ |\mathcal{S}| : \sum_{k \in \mathcal{S}} p_k \geq p \right\}$$



Top-k vs Top-p

Top-k always keeps exactly k tokens regardless of distribution shape. Top-p adapts: fewer tokens when confident, more when uncertain.

Sampling Strategies: Summary

Method	Deterministic?	Diversity
Greedy	Yes	None
Beam search	Yes	Low
Temperature	No	Tunable
Top-k	No	Medium
Top-p	No	Adaptive

Common combination

In practice: apply temperature τ, then top-p (or top-k), then sample.

Repetition penalty

Models sometimes loop. A simple fix: down-weight logits of previously generated tokens by a factor $\alpha > 1$ before sampling.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
   - Self-Attention
   - Multi-Head & Attention Variants
   - Transformer Architecture
   - Positional Encoding
   - KV-Cache
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
   - Self-Attention
   - Multi-Head & Attention Variants
   - Transformer Architecture
   - Positional Encoding
   - KV-Cache
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Motivation: Why Attention?

Consider the sentence:

*"The animal didn't cross the street because **it** was too tired."*

To understand what **it** refers to, a model must connect it to **animal**, five tokens away.

The challenge: When processing a sequence, each token must be able to selectively gather information from *any* other token, regardless of distance.

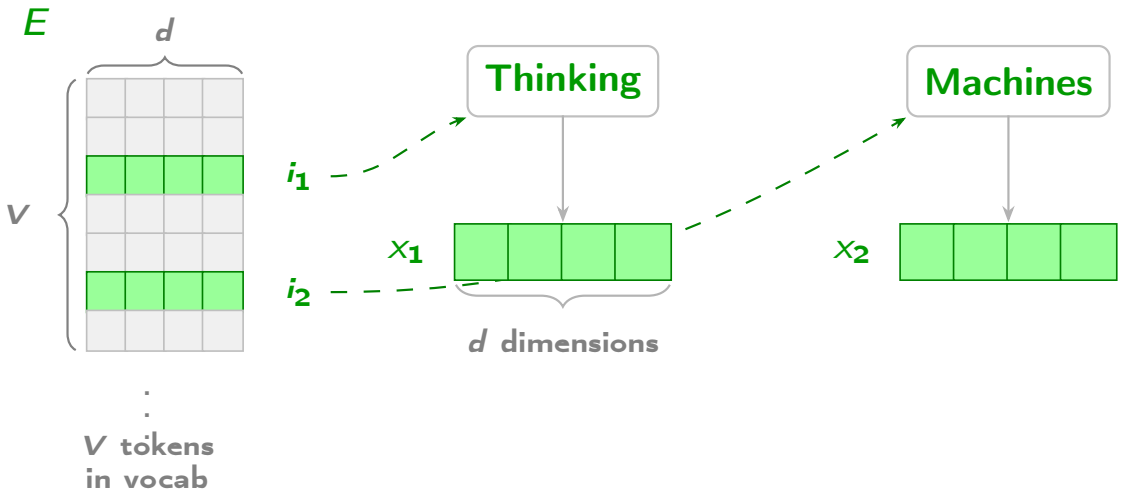
- RNNs pass information through a fixed-size hidden state, a bottleneck for long-range dependencies.
- **Attention** provides a direct, learnable connection between any two positions.
- Each token asks: *"Which other tokens are relevant to me, and how much?"*
- The answer is computed dynamically from the content of the tokens themselves.

References

- [Vaswani et al., *"Attention Is All You Need"*, 2017]
- [Alammar, *The Illustrated Transformer*, 2018]

Token Embeddings

- Each token is mapped to a learnable feature vector, its **token embedding**.
- The embedding matrix $E \in \mathbb{R}^{V \times d}$ is a lookup table: row i gives the embedding for token i.

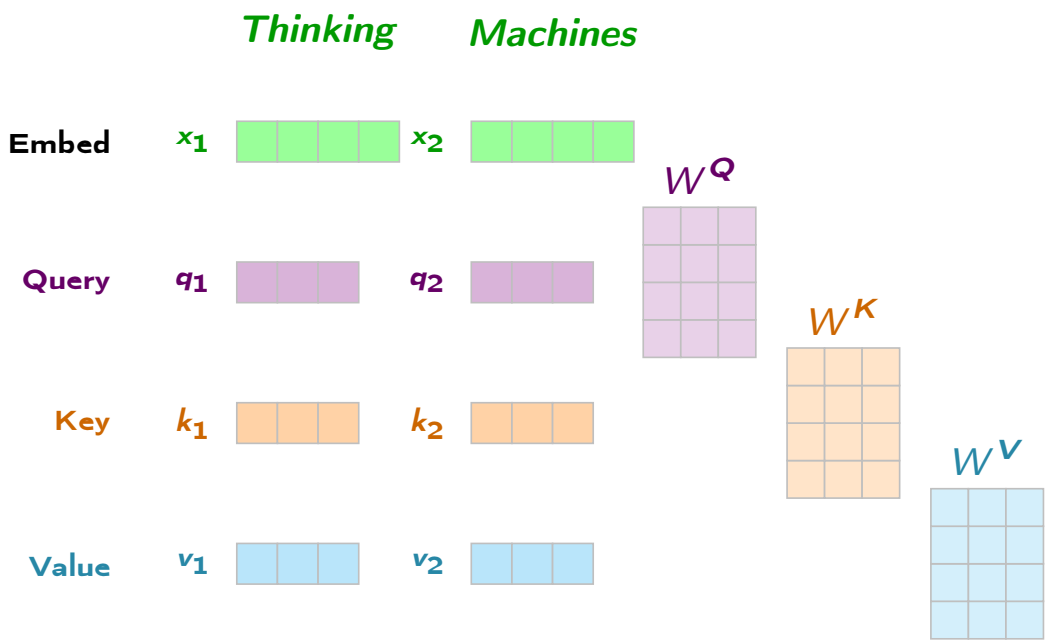


Self-Attention: Queries, Keys, and Values

Each token i projects its embedding $x_i \in \mathbb{R}^d$ into three vectors via learned matrices:

- **Query** $q_i = x_i W^Q$
  "What am I looking for?"
- **Key** $k_i = x_i W^K$
  "What do I contain?"
- **Value** $v_i = x_i W^V$
  "What will I contribute?"

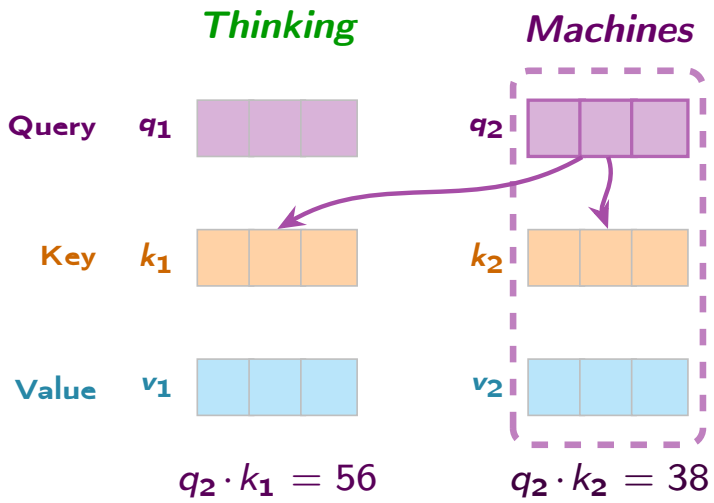
$W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$ are shared across all tokens.



Self-Attention: Computing Scores

The **attention score** from token i to token j is the dot product of their query and key:

$$s_{ij} = q_i^\top k_j$$



Numerical example ($d_k = 3$):

	Thinking				Machines		
q_1	·	·	·	q_2	4	6	2
k_1	5	4	6	k_2	2	4	3

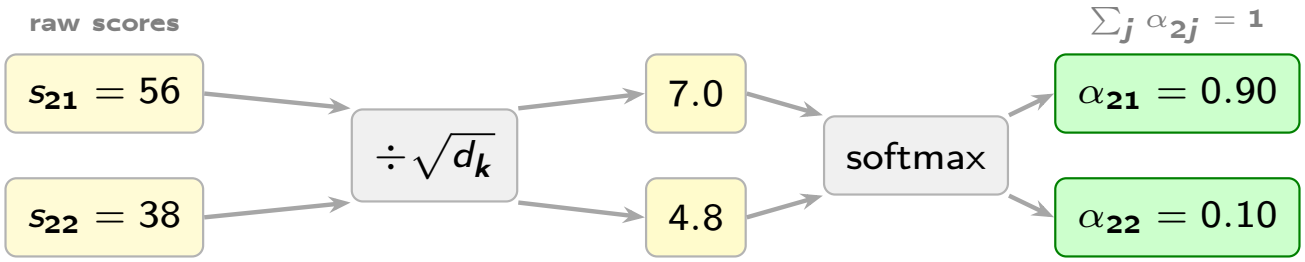
$$q_2 \cdot k_1 = 4 \cdot 5 + 6 \cdot 4 + 2 \cdot 6 = 56$$

$$q_2 \cdot k_2 = 4 \cdot 2 + 6 \cdot 4 + 2 \cdot 3 = 38$$

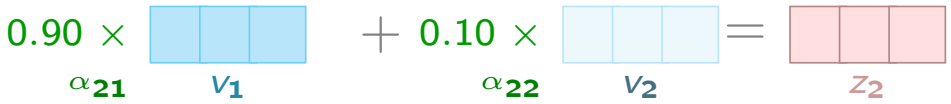
- Similar query and key vectors produce a high score.
- The score determines how much token i attends to token j when forming its output.

Self-Attention: Softmax & Weighted Output

Softmax normalization. Scale scores by $1/\sqrt{d_k}$, then apply softmax to get attention weights $\alpha_{ij} \geq 0$, $\sum_j \alpha_{ij} = 1$.



Weighted output. $z_i = \sum_j \alpha_{ij}\, v_j$: attention concentrates on the most relevant token.



Self-Attention: Matrix Form

- Stack all tokens row-wise into $X \in \mathbb{R}^{T \times d}$.
- All queries, keys, and values are computed in **one matrix multiply**.

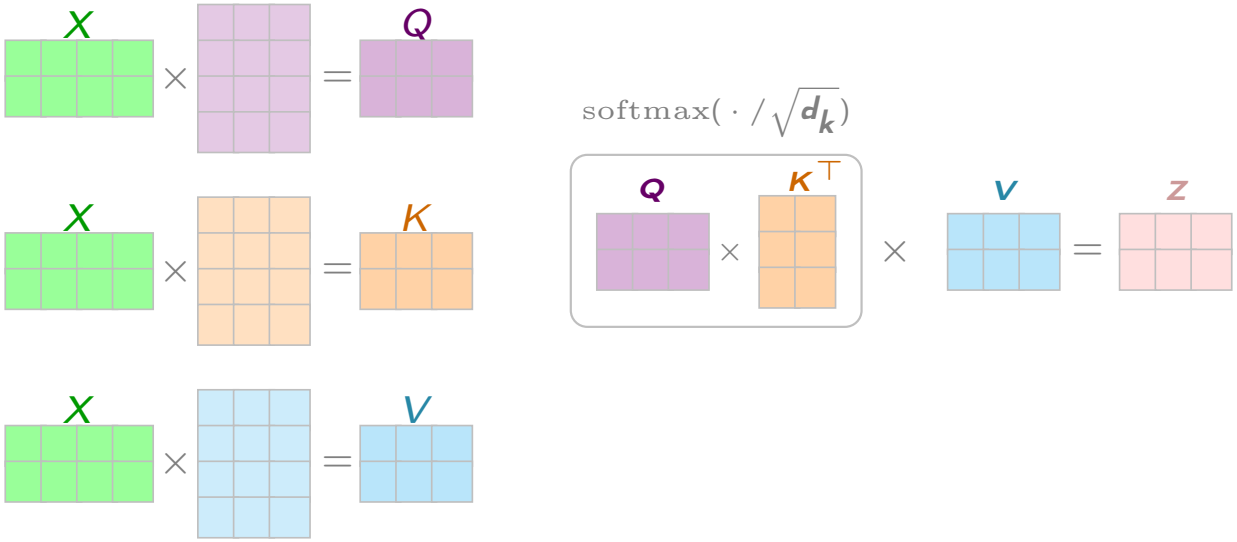


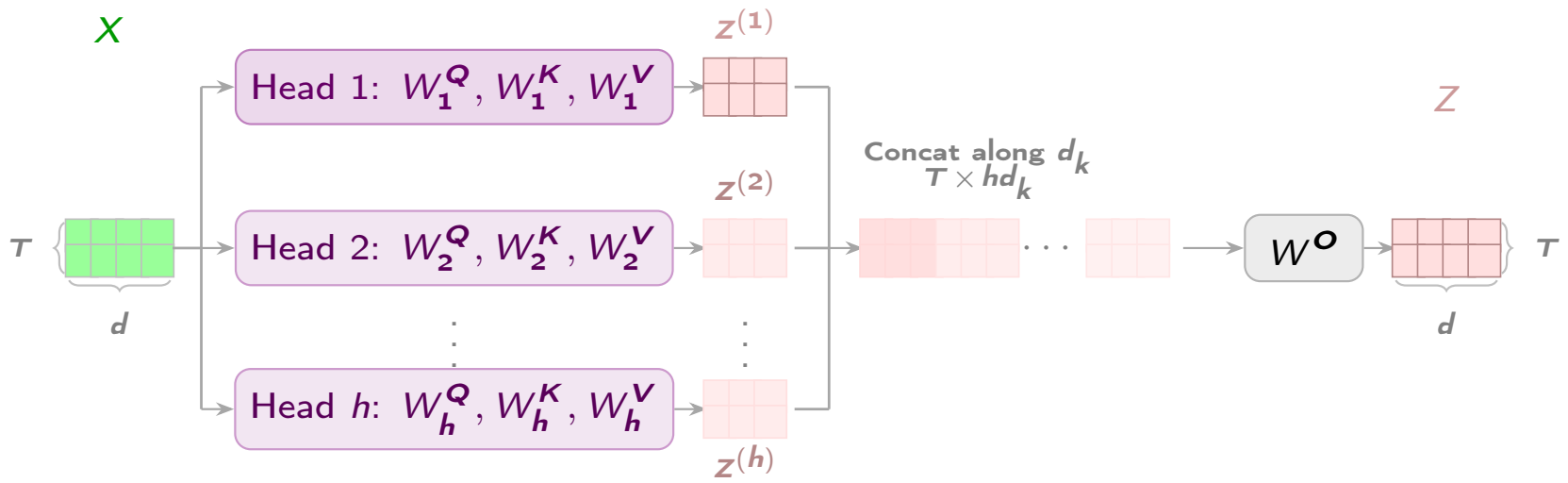
Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
   - Self-Attention
   - Multi-Head & Attention Variants
   - Transformer Architecture
   - Positional Encoding
   - KV-Cache
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Multi-Head Attention

- A single head may miss diverse relationship types.
- **Multi-head attention** runs h heads in parallel, each with its own W_i^Q, W_i^K, W_i^V.

$$\mathrm{MHA}(X) = \mathrm{Concat}(z^{(1)}, \ldots, z^{(h)})\, W^O$$



- Each head learns to attend to *different* aspects: syntax, coreference, proximity, ...
- $W^O \in \mathbb{R}^{hd_k \times d}$ projects the concatenated head outputs back to model dimension d.

Causal Self-Attention

During **autoregressive generation**, token i must not attend to future tokens $j > i$. We enforce this with an **upper-triangular mask** applied before softmax.

$$s_{ij} = \begin{cases} q_i^\top k_j / \sqrt{d_k} & j \leq i \\ -\infty & j > i \end{cases}$$

After softmax, $-\infty$ becomes 0, so future positions receive zero weight.

Key property

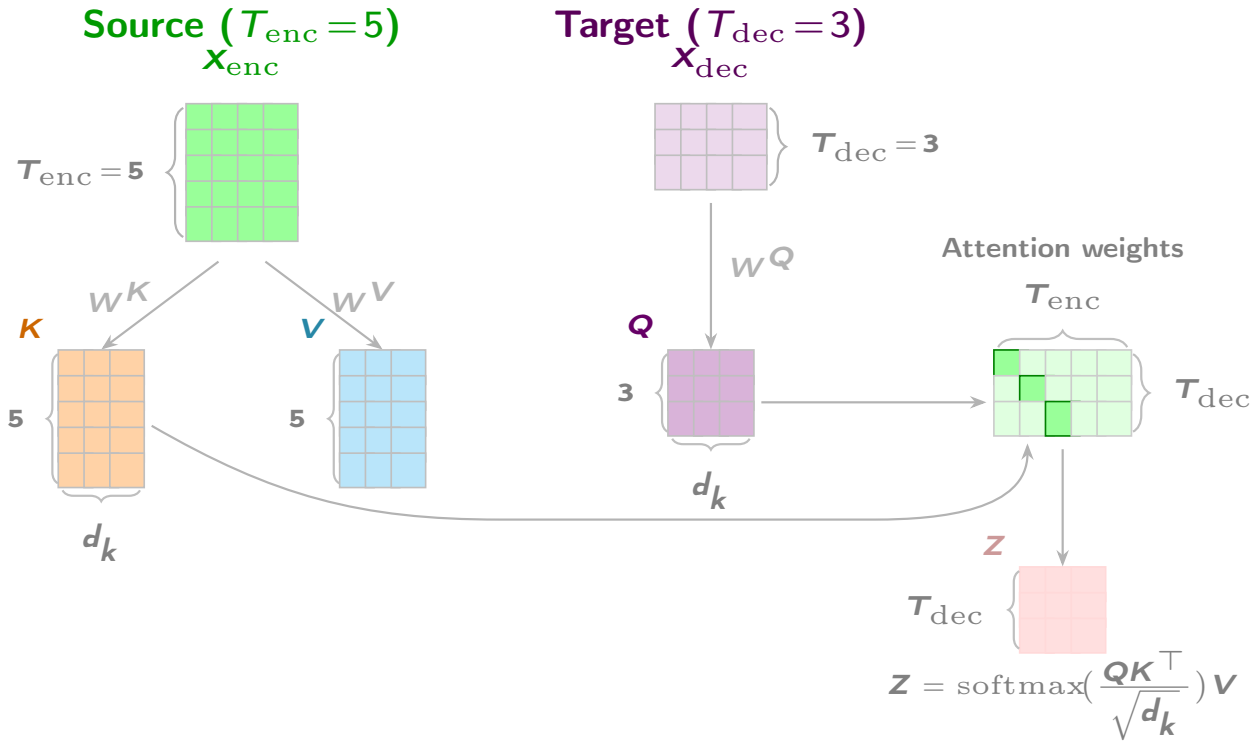
Training is fully **parallel**: the mask ensures valid supervision while all T positions are processed simultaneously.

score matrix (before softmax)

Cross-Attention

In **cross-attention**, queries come from one sequence and keys/values from another:

- Allows tokens of one sequence to attend to tokens of another, letting the model align and combine information across two different representations.
- **Decoder** tokens (T_{dec}) produce queries; **encoder** tokens (T_{enc}) provide keys and values.
- The attention weight matrix is $T_{\text{dec}} \times T_{\text{enc}}$ (not square!).



Cross-Attention: Machine Translation Example

- **Encoder** processes the source sentence (French) into key/value representations.
- **Decoder** generates the target sentence (English) one token at a time, using cross-attention to query the encoder output.
- Line thickness shows attention weight: each decoder token focuses on the most relevant source token.

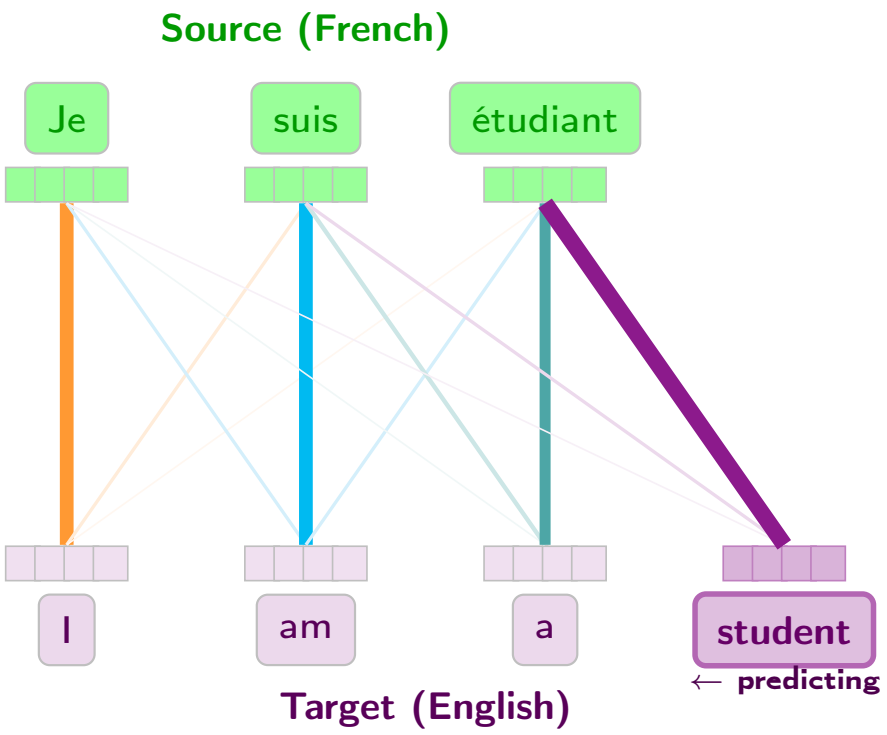
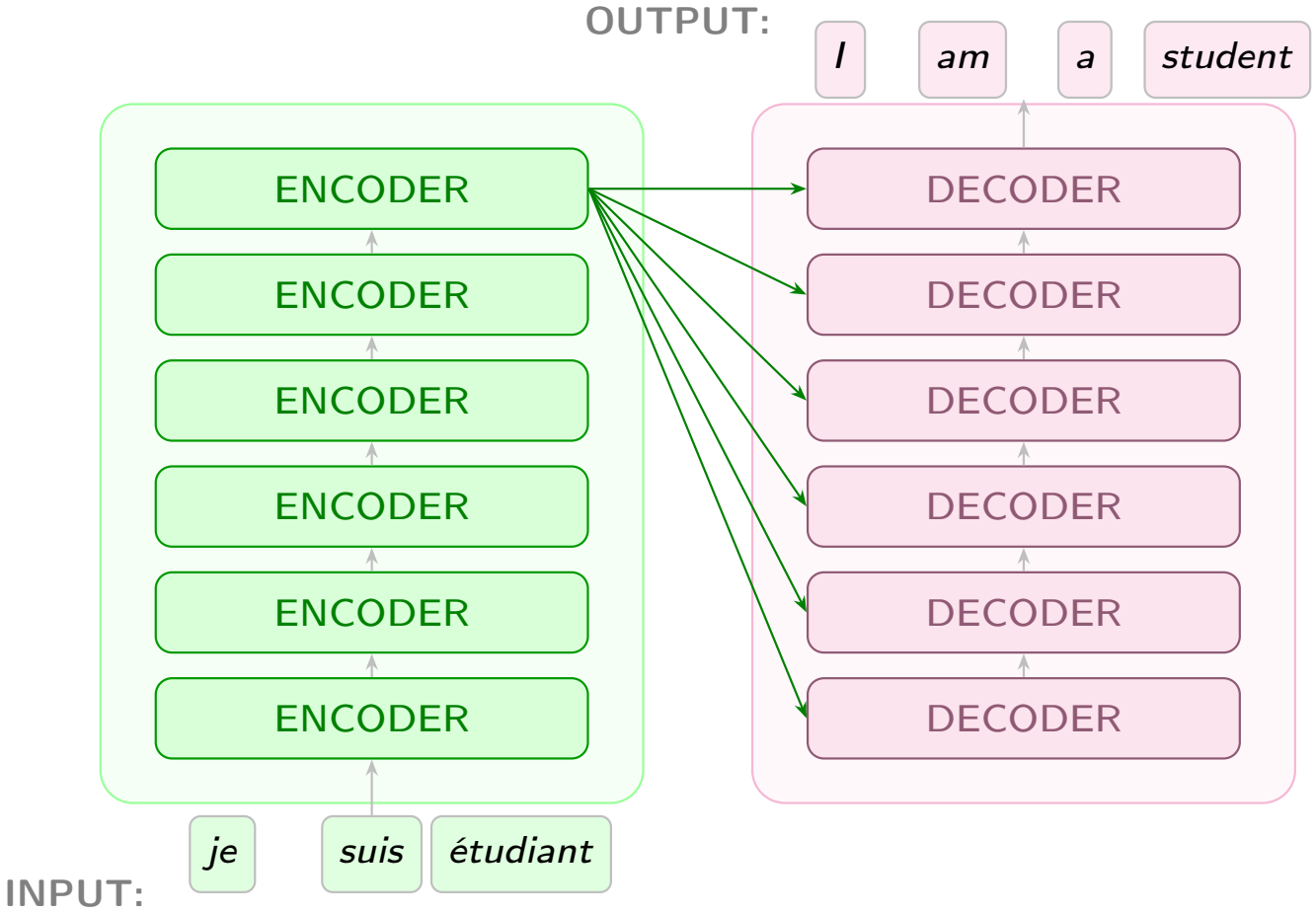


Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
   - Self-Attention
   - Multi-Head & Attention Variants
   - Transformer Architecture
   - Positional Encoding
   - KV-Cache
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Transformer Architecture: Encoder & Decoder

A transformer consists of an **encoder** and a **decoder**, each built from a stack of identical blocks.



- **Encoder**: processes the full input sequence into feature vectors
- **Decoder**: generates output tokens one at a time, conditioned on encoder features and past outputs

Encoder Block

Each encoder block has two sub-layers, each wrapped with a residual connection and layer normalization:

1. **Self-Attention**: exchanges information between tokens (mixing)
2. **Feed-Forward MLP**: per-token nonlinear transformation (no mixing)

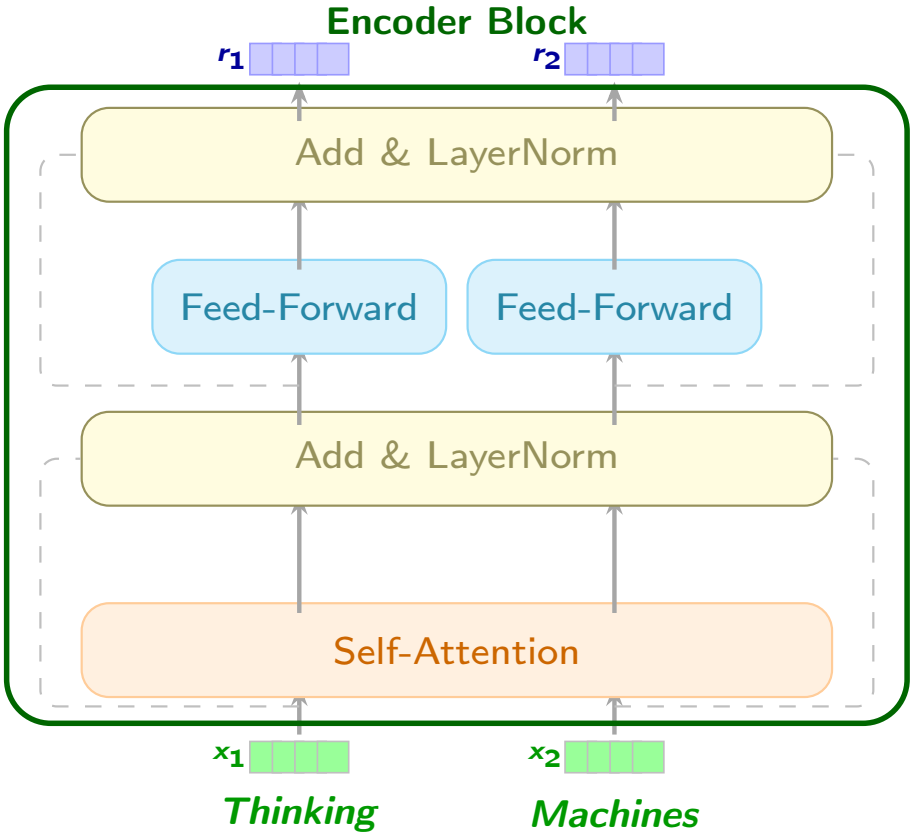
Sub-layer 1 (attention):

$$Z = \mathrm{MHA}(X)$$
$$X' = \mathrm{LayerNorm}(X + Z)$$

Sub-layer 2 (feed-forward):

$$F = \mathrm{FFN}(X')$$
$$= \underbrace{W_2}_{d \times d_{\mathrm{ff}}} \ \mathrm{GELU}(\underbrace{W_1}_{d_{\mathrm{ff}} \times d} \ X')$$
$$R = \mathrm{LayerNorm}(X' + F)$$

- $d_{\mathrm{ff}} = 4d$ (expand then contract).



- Dashed lines - - - = residual connections.

Decoder Block

Each decoder block has **three** sub-layers (encoder–decoder model) or two (decoder-only):

1. **Causal Self-Attention**: tokens attend only to past positions
2. **Cross-Attention**: queries from decoder, keys/values from encoder (omitted in decoder-only)
3. **Feed-Forward MLP**: per-token nonlinear transformation

Sub-layer 1 (causal attention):

$$Z = \mathrm{CausalMHA}(Y)$$
$$Y' = \mathrm{LN}(Y + Z)$$

Sub-layer 2 (cross-attention):

$$C = \mathrm{CrossMHA}(Y', R)$$
$$Y'' = \mathrm{LN}(Y' + C)$$

Sub-layer 3 (feed-forward):

$$F = \mathrm{FFN}(Y'')$$
$$O = \mathrm{LN}(Y'' + F)$$

- R = encoder output.

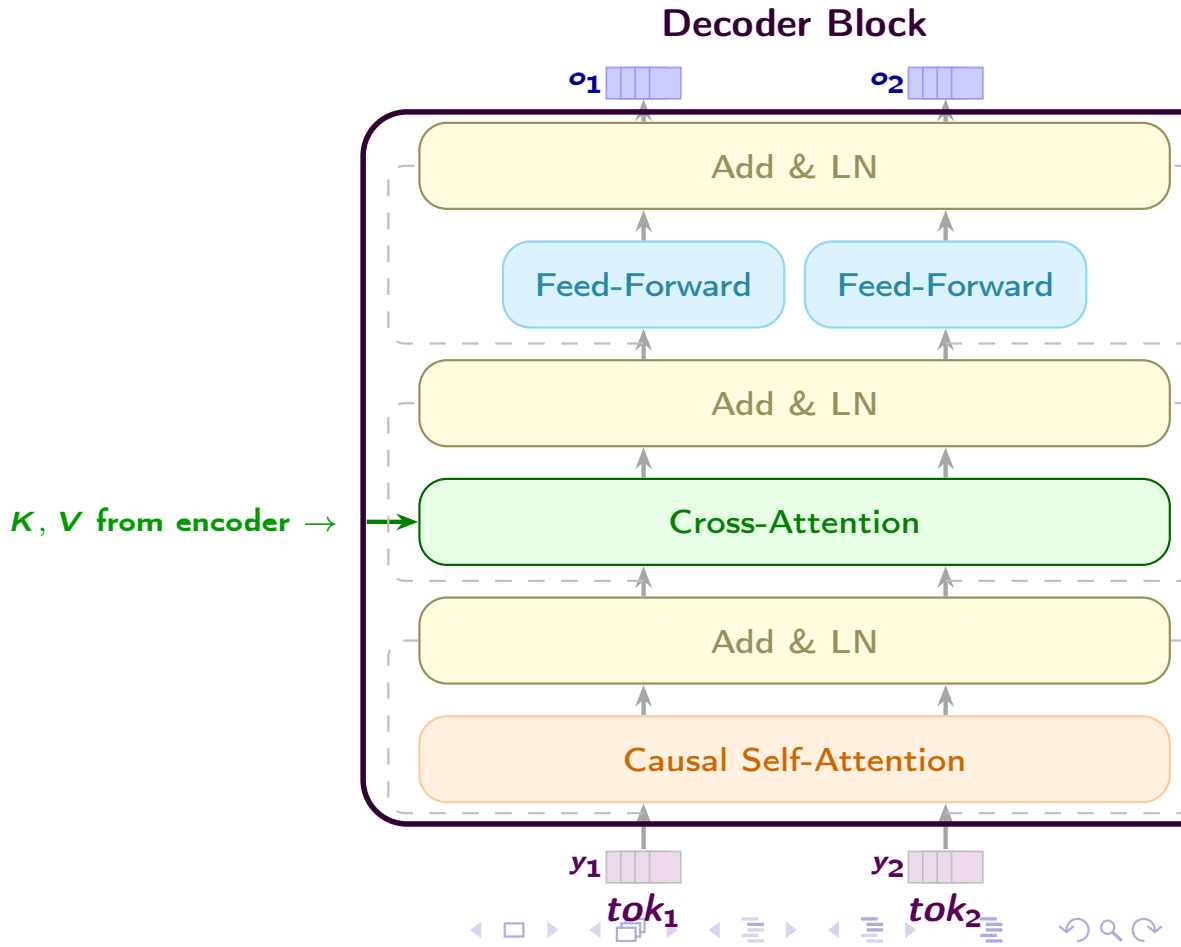
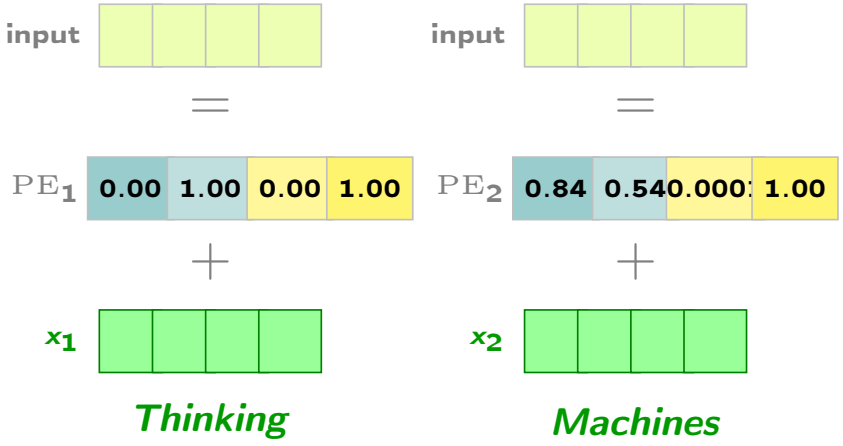


Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
   - Self-Attention
   - Multi-Head & Attention Variants
   - Transformer Architecture
   - Positional Encoding
   - KV-Cache
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Positional Encoding

Self-attention is **permutation equivariant**: it has no notion of token order. We add a **positional encoding** to each embedding to inject position information.



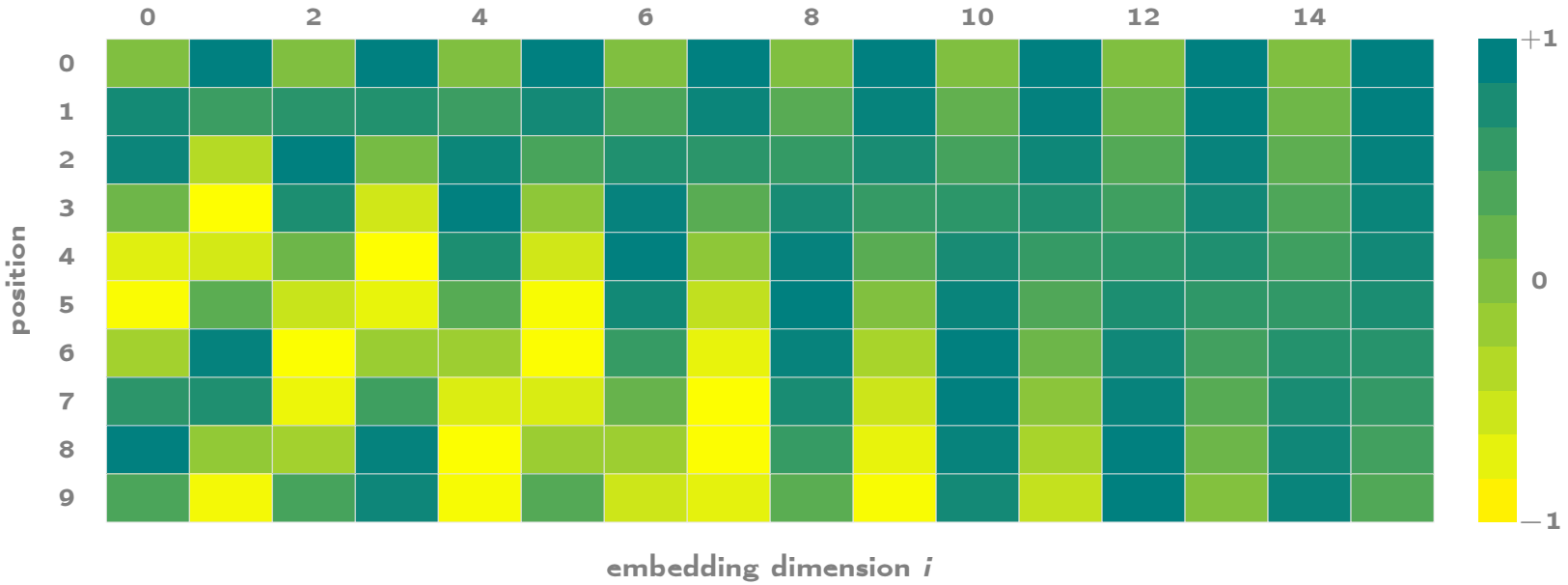
Classical sinusoidal encoding [Vaswani et al., *"Attention Is All You Need"*, 2017]:

$$\mathrm{PE}_{(pos,\,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad \mathrm{PE}_{(pos,\,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- pos = sequence position, $i = 0, \ldots, d/2-1$ = dimension pair index
- Each pair $(2i, 2i+1)$: different wavelength from 2π to $10000 \cdot 2\pi$
- Base 10000 chosen so the slowest sinusoid spans less than one cycle over typical sequence lengths

Positional Encoding: Visualization

Low dimensions oscillate fast (high frequency), high dimensions oscillate slowly.



Modern models use **learned** positional embeddings or **RoPE** instead.

Learned Positional Embeddings

Idea: Replace the fixed sinusoidal formula with a learnable embedding matrix.

- Embedding table $P \in \mathbb{R}^{L_{\max} \times d}$, initialized randomly
- Position m gets embedding $P[m, :]$, added to the token embedding
- Trained end-to-end with the rest of the model via backpropagation

Advantages:

- More expressive: each position has a free embedding
- Empirically matches or slightly outperforms sinusoidal PE
- Used in GPT-2, BERT, and many early Transformers

Limitations:

- Fixed maximum length $L_{\max}$: cannot generalize beyond it
- Absolute: attention depends on positions (m, n), not relative distance $m - n$
- Adds $L_{\max} \times d$ parameters (modest cost)

Limitations of Absolute Positional Encodings

Recall: sinusoidal or learned PE are **added** to token embeddings before attention.

Problems:

- Attention scores depend on *absolute* positions, not relative distance
- Sinusoidal PE: dot product $\mathrm{PE}(m)^\top \mathrm{PE}(n)$ depends on both m and n, not just $m - n$
- Learned PE: cannot generalize beyond the maximum training length
- Adding PE to content embeddings mixes positional and semantic information

Desiderata

- Attention score $\mathrm{q}_m^\top \mathrm{k}_n$ should depend on relative distance $m - n$
- Positional information should be *injected into* Q and K, not added to content
- No learned parameters

RoPE: Core Idea [Su et al., *"RoFormer: Enhanced Transformer with Rotary Position Embedding"*, 2024]

Key insight: Encode position by *rotating* query and key vectors.

- 2D subspace, position m, rotate by $m\theta$:

$$R(m\theta) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix}$$

- Query and key at positions m, n:

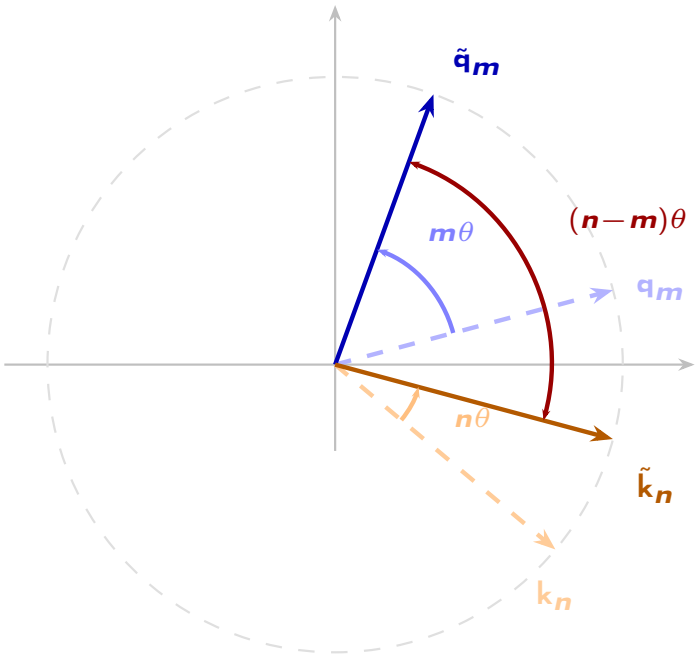
$$\tilde{\mathrm{q}}_m = R(m\theta)\,\mathrm{q}_m, \quad \tilde{\mathrm{k}}_n = R(n\theta)\,\mathrm{k}_n$$

- Dot product:

$$\begin{aligned} \tilde{\mathrm{q}}_m^\top \tilde{\mathrm{k}}_n &= \mathrm{q}_m^\top R(m\theta)^\top R(n\theta)\,\mathrm{k}_n \\ &= \mathrm{q}_m^\top R\big((n-m)\theta\big)\,\mathrm{k}_n \end{aligned}$$

- Since $R(\alpha)^\top = R(-\alpha)$ (orthogonal):

$$R(\alpha)^\top R(\beta) = R(-\alpha)\,R(\beta) = R(\beta - \alpha)$$



Result

The attention score depends only on relative position $n - m$ and the content vectors.

RoPE: Complex Number View

Equivalently, represent each dimension pair (x_{2i}, x_{2i+1}) as a complex number:

$$z_i = x_{2i} + j\, x_{2i+1} \in \mathbb{C}$$

RoPE at position m: multiply by a unit complex number

$$\tilde{z}_i = z_i \cdot e^{j\, m\theta_i}$$

The dot product between rotated queries and keys becomes:

$$\mathrm{Re}\left(\tilde{z}_i^{(q)} \cdot \overline{\tilde{z}_i^{(k)}}\right) = \mathrm{Re}\left(z_i^{(q)} \cdot \overline{z_i^{(k)}} \cdot e^{j\,(m-n)\theta_i}\right)$$

Implementation in practice

- Reshape Q, K into pairs: $(d/2)$ complex numbers
- Precompute $e^{j\, m\theta_i}$ for all positions m and frequencies i
- Element-wise complex multiply (or equivalent real-valued formula)

RoPE: Full Rotation Matrix

For head dimension d, apply different rotation frequencies θ_i to each pair:

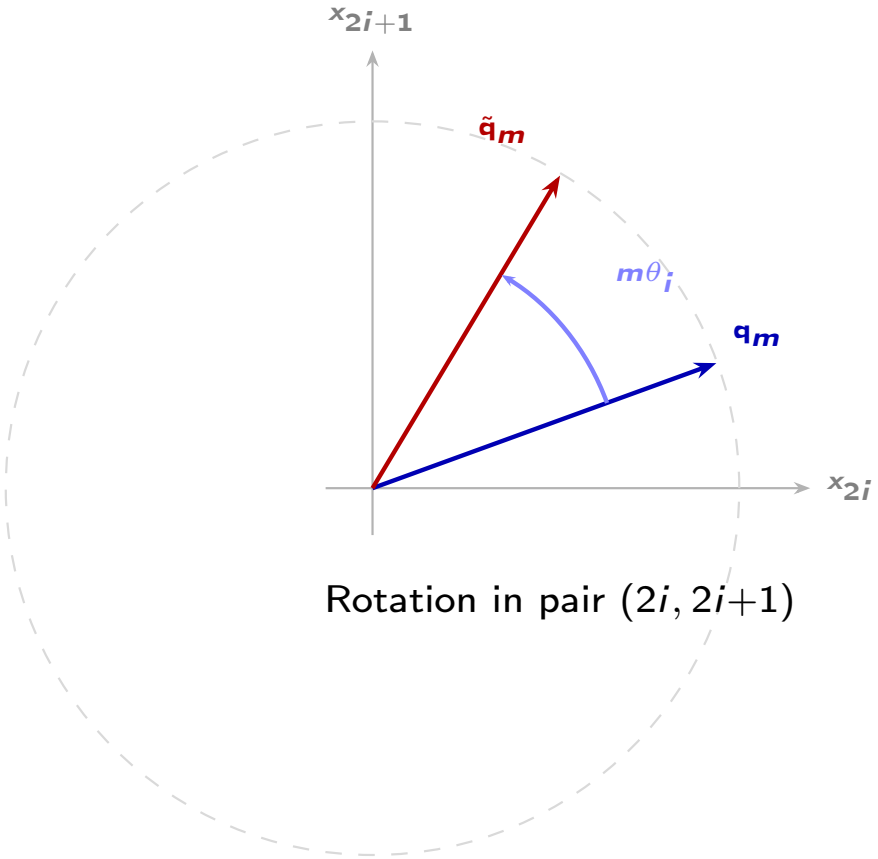
$$\theta_i = \frac{1}{b^{2i/d}}, \quad i = 0, 1, \ldots, d/2 - 1$$

where $b = 10{,}000$ is the base (same convention as sinusoidal PE).
The full rotation matrix at position m is block-diagonal:

$$R_m = \begin{pmatrix} R(m\theta_0) & & & \\ & R(m\theta_1) & & \\ & & \ddots & \\ & & & R(m\theta_{d/2-1}) \end{pmatrix}$$

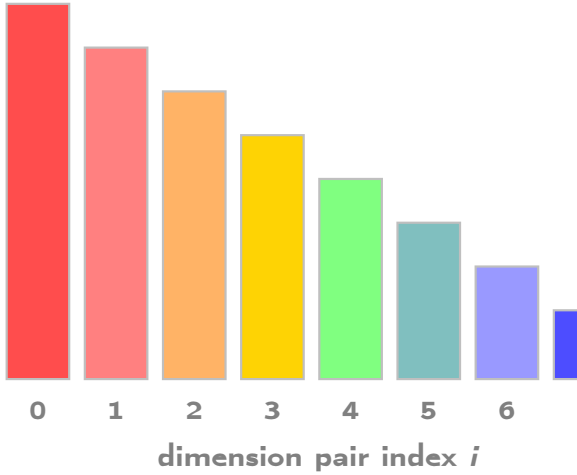
Low dimensions: large $\theta_i \Rightarrow$ fast rotation $\Rightarrow$ captures fine-grained local position
High dimensions: small $\theta_i \Rightarrow$ slow rotation $\Rightarrow$ captures long-range position
Analogous to the frequency spectrum in sinusoidal PE.

RoPE: Visualization



Rotation in pair $(2i, 2i+1)$

Rotation speed per dimension p



- Each dimension pair is rotated independently at its own frequency
- Low-index pairs rotate fast (local patterns), high-index pairs rotate slowly (global patterns)
- Applied to Q and K **after** projection, **before** dot product

RoPE: Properties

Relative PE: $\tilde{q}_m^\top \tilde{k}_n$ depends on q_m, k_n, and $m-n$ only

No learned params: Fully determined by base b and head dim d

Norm-preserving: $\|R_m q\| = \|q\|$ (orthogonal)

Long-range decay: As $|m-n|$ grows, rotating phases cancel $\Rightarrow$ inner product decays

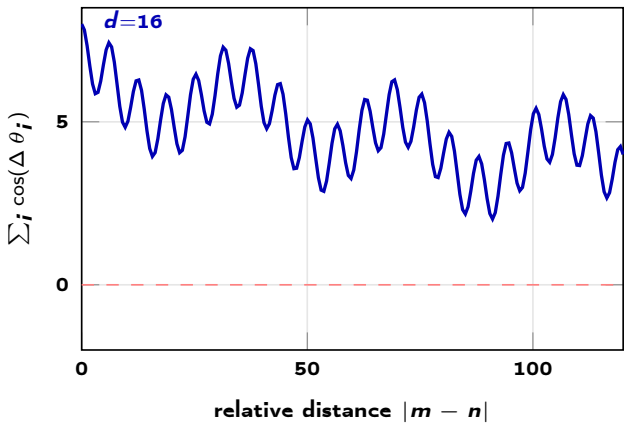
Flexible: Per-head, works with MHA, GQA, MQA

Adoption

Used by LLaMA, Mistral, Qwen, Gemma, DeepSeek, and most modern open-weight LLMs.

Long-range decay for unit vectors $q = k$:

$$\tilde{q}_m^\top \tilde{k}_n = \sum_{i=0}^{d/2-1} \cos\big((m-n)\,\theta_i\big)$$



High-freq pairs oscillate and cancel; low-freq pairs decay slowly. Peak at $\Delta=0$ (all cosines $=1$).

The Extrapolation Problem

RoPE is trained on sequences up to length L_{train} (e.g. 4096 tokens).

At inference, we want to handle $L_{\text{test}} \gg L_{\text{train}}$.

What goes wrong:

- Positions $m > L_{\text{train}}$ produce rotation angles $m\theta_i$ never seen during training
- Attention logits become unpredictable for out-of-distribution positions
- Performance degrades sharply beyond training length

Approaches:

Position interpolation: Scale positions: $m \to m \cdot \frac{L_{\text{train}}}{L_{\text{test}}}$ [S. Chen et al., *"Extending Context Window of Large Language Models via Positional Interpolation"*, 2023].
All angles stay in $[0, L_{\text{train}} \cdot \theta_i]$. Requires fine-tuning.

NTK-aware interpolation: Scale the base b instead of positions [bloc97, *NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and target length training*, 2023]. Changes frequencies non-uniformly. Can work without fine-tuning.

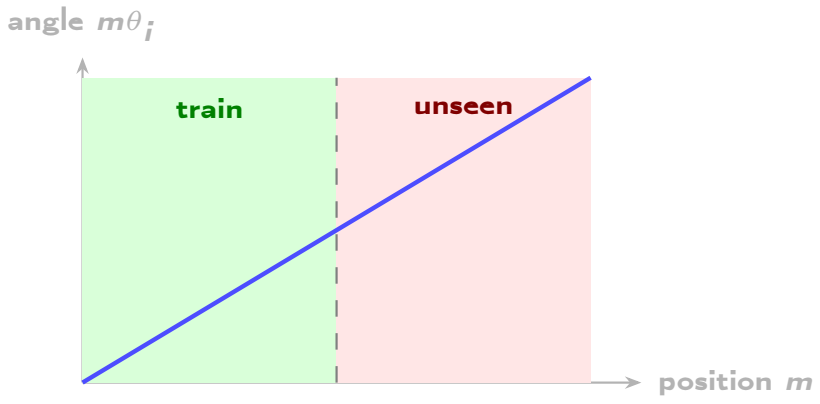
YaRN: Combines NTK-aware interpolation with attention scaling [B. Peng et al., *"YaRN: Efficient Context Window Extension of Large Language Models"*, 2023].

Position Interpolation vs. Extrapolation

Let $s = L_{\text{test}}/L_{\text{train}}$ be the scaling factor.

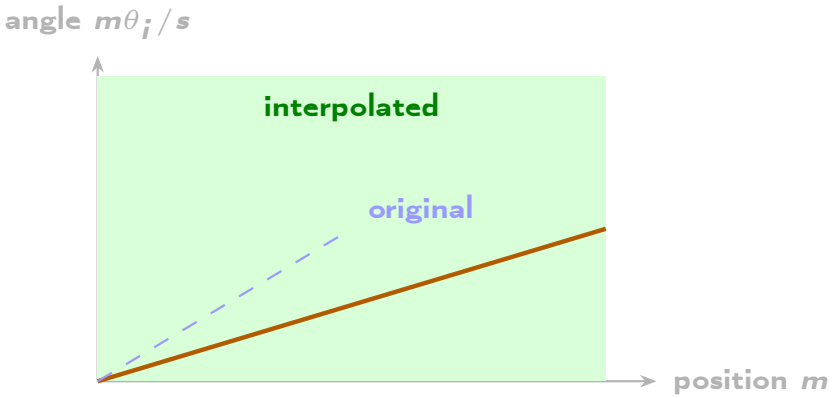
Extrapolation (naive):

- Use original θ_i at all positions
- Positions $> L_{\text{train}}$: angles unseen during training
- High-frequency pairs: multiple full extra rotations



Position Interpolation:

- Scale: $m \to m/s$
- All angles map into training range
- Tokens are "squeezed" closer together
- Fine-tuning needed: model must adapt to denser positions



NTK-aware Interpolation

Problem with uniform interpolation: position scaling $m \to m/s$ affects all frequencies equally, but high-frequency dimensions already resolve fine positions well.

Idea [bloc97, *NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and target length training*, 2023]**:** Scale the RoPE base instead:

$$b' = b \cdot s^{d/(d-2)}$$

This changes the frequencies $\theta_i = 1/b'^{2i/d}$ non-uniformly:

High-freq (small i): barely changed, preserves local resolution

Low-freq (large i): significantly stretched, accommodates longer sequences

Intuition

High-frequency components already cycle many times within L_{train}. Stretching them would lose local discriminability.
Low-frequency components have room to stretch without ambiguity.

Works reasonably well even **without fine-tuning** for moderate extensions (2–4×).

YaRN: Yet another RoPE extensioN [B. Peng et al., *"YaRN: Efficient Context Window Extension of Large Language Models"*, 2023]

YaRN combines three ideas for robust context extension:

1. Frequency-dependent interpolation

Partition dimension pairs into three groups based on wavelength $\lambda_i = 2\pi b^{2i/d}$:

No scaling: $\lambda_i < L_{\text{train}}$ (high-freq). These already complete full cycles. Leave unchanged.

Interpolation: $\lambda_i > L_{\text{test}}$ (low-freq). Apply full linear interpolation: $\theta_i \to \theta_i/s$.

NTK blend: In between. Smoothly interpolate between no scaling and full interpolation using a ramp function $\gamma(\lambda_i) \in [0, 1]$.

Effective frequency for each pair:

$$\theta_i' = (1 - \gamma(\lambda_i))\,\frac{\theta_i}{s} + \gamma(\lambda_i)\,\theta_i$$

YaRN: Attention Scaling and Fine-Tuning

2. Attention scaling

Interpolation compresses the angle differences, reducing attention logit magnitudes.

YaRN compensates with a temperature factor:

$$a_{m,n} = \frac{\tilde{\mathrm{q}}_m^\top \tilde{\mathrm{k}}_n}{\sqrt{d} \cdot t}$$

where $t = \sqrt{0.1 \ln s + 1}$ (empirically determined).
This restores the entropy of the attention distribution to match training behavior.

3. Fine-tuning

- Only 400 steps of fine-tuning on long sequences needed
- $\sim$0.1% of original training compute
- Achieves strong performance at 64$\times$–128$\times$ extension

Example: LLaMA 2

LLaMA 2 trained on 4K context. With YaRN: extends to 128K context while maintaining performance on short-context benchmarks.

Context Extension: Summary

Method	Fine-tuning	Max extension	Local resolution
Extrapolation (naive)	None	$\sim 1\times$	Preserved
Position Interpolation	Required	$\sim 8\times$	Reduced
NTK-aware scaling	Optional	$\sim 4\times$	Preserved
YaRN	Minimal	$\sim 128\times$	Preserved

Takeaway

RoPE provides a clean, parameter-free relative positional encoding. Its rotation structure enables principled context length extension through frequency-aware interpolation (YaRN), making it the default choice in modern LLMs.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers
- Self-Attention
- Multi-Head & Attention Variants
- Transformer Architecture
- Positional Encoding
- KV-Cache

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

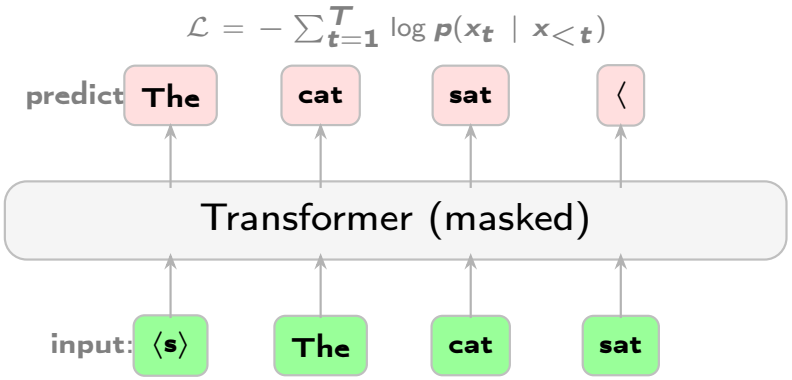
17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Training vs. Inference

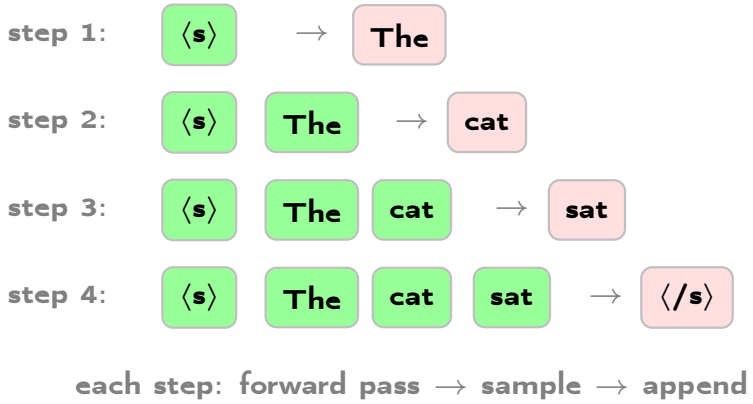
Training (parallel):

- Feed the full sequence at once
- Causal mask ensures each position only sees past tokens
- All T next-token losses computed simultaneously



Inference (autoregressive):

- Generate one token at a time
- Append each new token to the context
- KV-cache avoids recomputing past keys/values

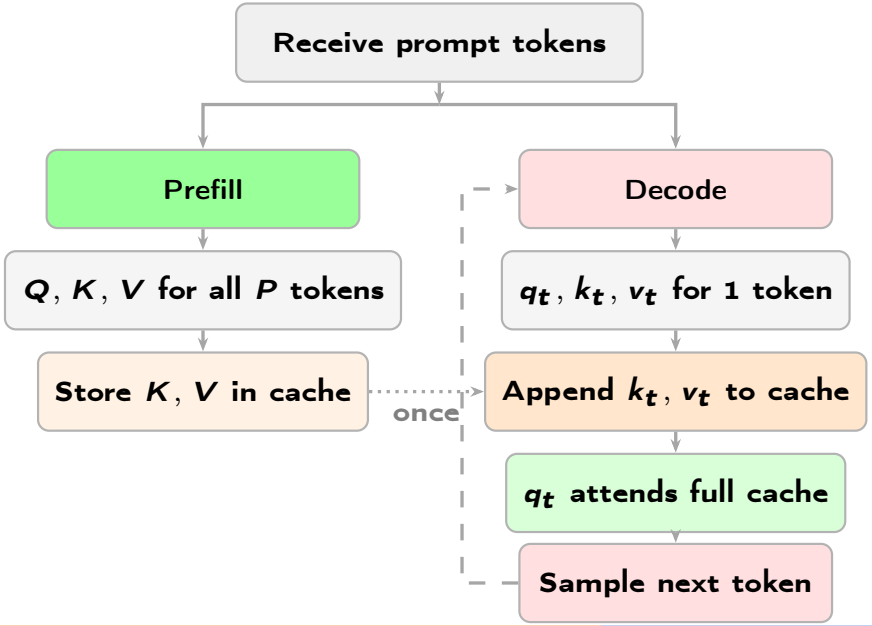


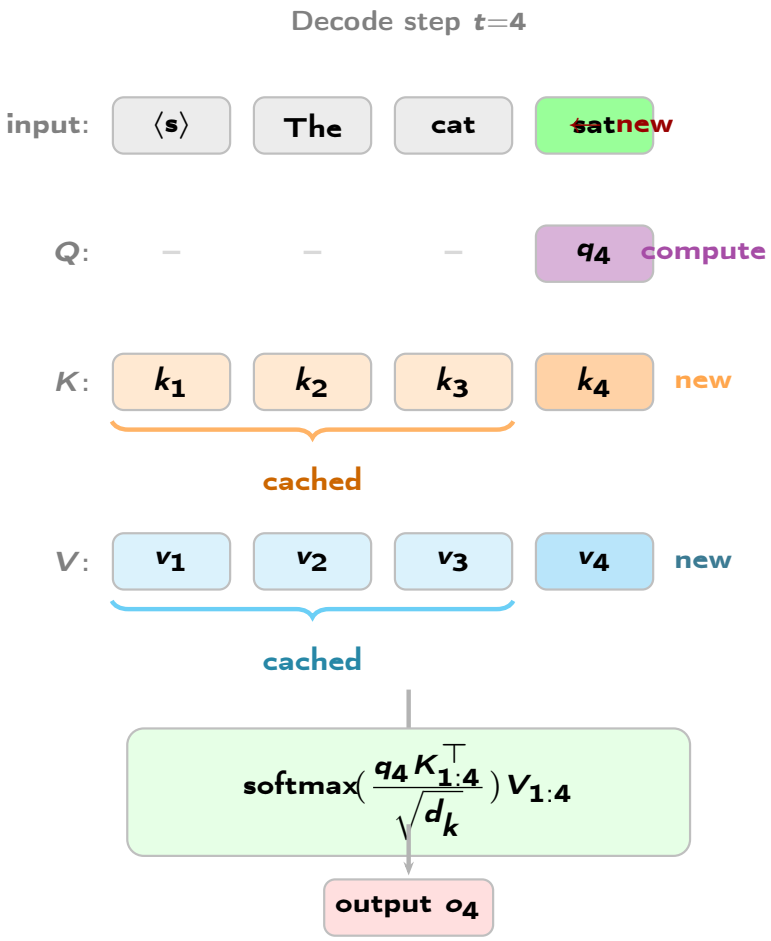
KV-Cache: Avoiding Redundant Computation

Key insight: In causal attention, K and V for past positions never change. Naïve generation recomputes them at every step – wasteful!

Complexity:

- **Prefill** $O(T^2)$: all T tokens attend to each other (full attention matrix).
- **Decode** $O(T)$ per token: only one new q_t attends over cached $K_{1:t}, V_{1:t}$. No recomputation of past keys/values needed.





KV-Cache: Prefill vs. Decode

Prefill (compute-bound):

- Large matrix multiplies, high arithmetic intensity
- Naïve cost: $O(T \cdot d)$ projections + $O(T^2 \cdot d)$ attention

Decode (memory-bound):

- Reading KV-cache dominates; only 1 query vector per step
- $O(d)$ projection + $O(t \cdot d)$ attention per step

Memory: cache grows as $O(L \cdot h \cdot T \cdot d_k)$
(L layers, h heads, T seq. length, d_k head dim).

Optimizations

KV-cache memory can become very large. Techniques for reduction: see next slides.

KV-Cache: The Memory Problem

Recall: KV-cache stores K and V for all past tokens, across all layers and heads.

Total KV-cache size:

$$\text{KV-cache} = 2 \cdot L \cdot h \cdot T \cdot d_k \cdot b \quad \text{bytes}$$

L = layers, h = heads, T = sequence length, d_k = head dim, b = bytes per element.

Hypothetical example (LLaMA-2 70B dimensions with MHA, BF16, $T = 4096$):

- $L = 80$, $h = 64$, $d_k = 128$, $b = 2$
- KV-cache $= 2 \times 80 \times 64 \times 4096 \times 128 \times 2 \approx 10.7\,\text{GB}$ **per sequence**
- At $T = 128\text{k}$: $\approx 335\,\text{GB}$ – larger than the model itself!
- Note: LLaMA-2 70B actually uses GQA (g=8), reducing this by 8×.

Approaches to Reduce KV-Cache

Fewer KV heads	MQA, GQA – share K,V across query heads
Compress KV	MLA – low-rank latent compression
Fewer tokens	Sliding window – bound the context
Share across layers	Cross-layer attention

Multi-Query Attention (MQA)

Idea [Shazeer, *"Fast Transformer Decoding: One Write-Head is All You Need"*, 2019]: Keep h separate query heads, but use a *single shared* key head and value head.

Standard MHA (per head i):

$$Q_i = XW_i^Q,\ K_i = XW_i^K,\ V_i = XW_i^V$$

Each head has its own W_i^K, W_i^V.

MQA:

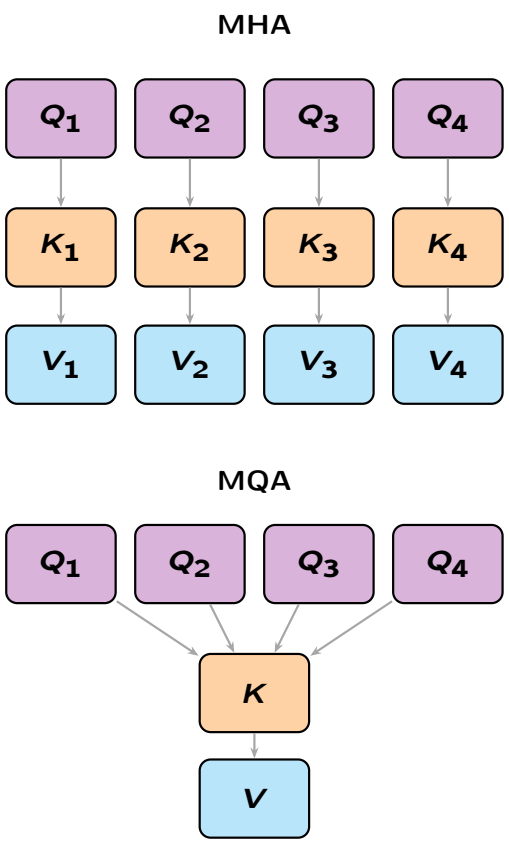
$$Q_i = XW_i^Q,\ K = XW^K,\ V = XW^V$$

All heads share a single W^K, W^V (no subscript i).

KV-cache reduction: $h\times$

- MHA: $2 \cdot h \cdot T \cdot d_k$ per layer
- MQA: $2 \cdot 1 \cdot T \cdot d_k$ per layer

Trade-off: Significant memory savings, but some quality degradation since K,V have reduced capacity.



Grouped Query Attention (GQA)

Idea [Ainslie et al., *"GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints"*, 2023]: Interpolate between MHA and MQA. Divide h query heads into g groups; each group shares one K,V head.

For group j and query head $i \in$ group j:

$$Q_i = XW_i^Q, \quad K_j = XW_j^K, \quad V_j = XW_j^V$$

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_j^\top}{\sqrt{d_k}}\right) V_j$$

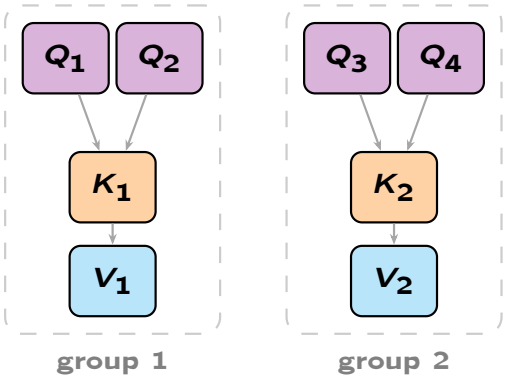
$g = h$ $\Rightarrow$ MHA (each head has own K,V)

$g = 1$ $\Rightarrow$ MQA (all heads share K,V)

$1 < g < h$ $\Rightarrow$ GQA (intermediate)

KV-cache per layer: $2 \cdot g \cdot T \cdot d_k$
Reduction vs. MHA: h/g

GQA (h=4, g=2)



Model	h	g
LLaMA-2 70B	64	8
LLaMA-3 8B	32	8
Mistral 7B	32	8

GQA: Uptraining from MHA Checkpoints

A pretrained MHA model can be converted to GQA without training from scratch [Ainslie et al., *"GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints"*, 2023].

Recipe:

1. **Assign** the h KV heads to g groups (each group has h/g heads).
2. **Mean-pool** the KV projection weights within each group:

$$W_j^K = \frac{1}{h/g} \sum_{i \in \text{group } j} W_i^K \qquad W_j^V = \frac{1}{h/g} \sum_{i \in \text{group } j} W_i^V$$

3. **Uptrain** with original data for ∼5% of original pretraining compute.

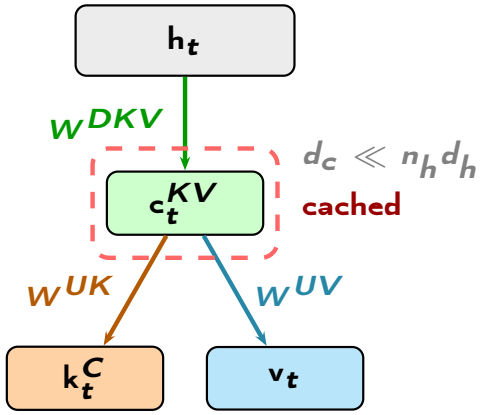
Key Results

- Mean-pool initialization significantly outperforms random initialization.
- GQA with $g = 8$ achieves quality close to MHA while approaching MQA inference speed.
- LLaMA-2 70B was released with GQA ($g = 8$), reducing KV-cache by 8× vs. MHA.

Multi-head Latent Attention (MLA)

Idea [DeepSeek-AI, *"DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model"*, 2024]: Compress the joint KV representation into a *low-rank latent vector* $c_t^{KV} \in \mathbb{R}^{d_c}$ with $d_c \ll n_h \cdot d_h$. Only c_t^{KV} is cached during inference.

- **Down-project** (KV compression): $c_t^{KV} = W^{DKV} h_t \in \mathbb{R}^{d_c}$
- **Up-project** to reconstruct K,V: $k_t^C = W^{UK} c_t^{KV}, \quad v_t = W^{UV} c_t^{KV}$
- **Queries** (also compressed): $c_t^Q = W^{DQ} h_t$, $q_t^C = W^{UQ} c_t^Q$
- **Key insight:** KV information across all heads lies in a low-dimensional subspace. Cache the compressed representation, reconstruct on the fly.



MLA: Decoupled RoPE

- **Problem:** RoPE applies position-dependent rotations R_t to queries and keys: $\text{score}_{t,s} = (R_t \mathrm{q}_t^C)^\top (R_s \mathrm{k}_s^C)$
- Without RoPE, we could absorb the up-projections: $(\mathrm{q}_t^C)^\top \mathrm{k}_s^C = (\mathrm{c}_t^Q)^\top \underbrace{(W^{UQ})^\top W^{UK}}_{\text{precomputable}} \mathrm{c}_s^{KV}$ and **never materialize full keys**. But R_s between W^{UK} and c_s^{KV} breaks this.
- **Solution: Decoupled RoPE keys.** Add a small separate key carrying positional information:

$$\mathrm{k}_t^R = R_t\, W^{KR}\, \mathrm{h}_t \in \mathbb{R}^{d_R}, \qquad \mathrm{q}_t^R = R_t\, W^{QR}\, \mathrm{c}_t^Q \in \mathbb{R}^{d_R}$$

- Attention score splits into content + position parts:

$$a_{t,s} = \frac{(\mathrm{q}_t^C)^\top \mathrm{k}_s^C + (\mathrm{q}_t^R)^\top \mathrm{k}_s^R}{\sqrt{d_h + d_R}}$$

- **Cache per token per layer:** $\mathrm{c}_t^{KV} \in \mathbb{R}^{d_c}$ and $\mathrm{k}_t^R \in \mathbb{R}^{d_R} \Rightarrow$ total $d_c + d_R$ elements.

DeepSeek-V2 Numbers

n_h=128, d_h=128, d_c=512, d_R=64. MHA cache: $2 \times 128 \times 128 = 32{,}768$. MLA cache: $512 + 64 = 576$. **Compression:** $\sim 57\times$.

Sliding Window Attention

Idea [A. Q. Jiang et al., *"Mistral 7B"*, 2023]: Each token attends only to the w most recent tokens instead of the full history.

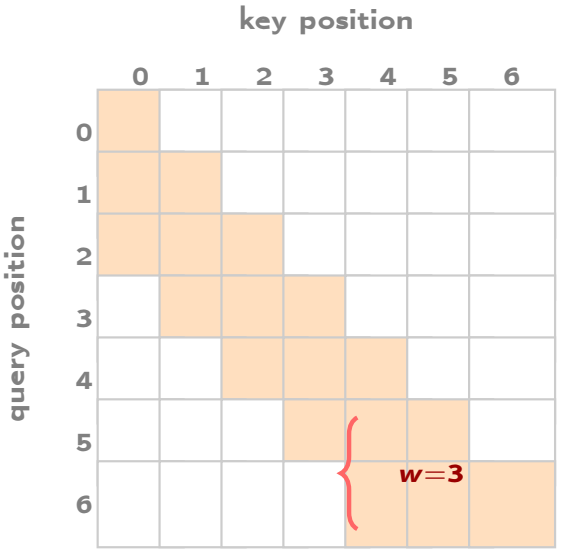
Token at position t attends to $\{t-w+1, \ldots, t\}$:

$$a_{t,s} = \begin{cases} \frac{\exp(q_t^\top k_s / \sqrt{d_k})}{\sum_{s'} \exp(\cdot)} & \text{if } s \geq t-w+1 \\ 0 & \text{otherwise} \end{cases}$$

Rolling buffer: cache positions indexed as $t \bmod w$. Old entries are overwritten $\Rightarrow$ **constant memory**.

KV-cache bound per layer: $2 \cdot h \cdot w \cdot d_k$
(independent of total sequence length T)

Effective receptive field: With L layers, information propagates up to $L \times w$ tokens back.



Attention mask for sliding window (w=3). Orange = attended positions.

Hybrid: Some models (Gemma 2, Jamba) alternate sliding window layers with full attention layers for bounded cache with some global context.

Cross-Layer Attention (CLA)

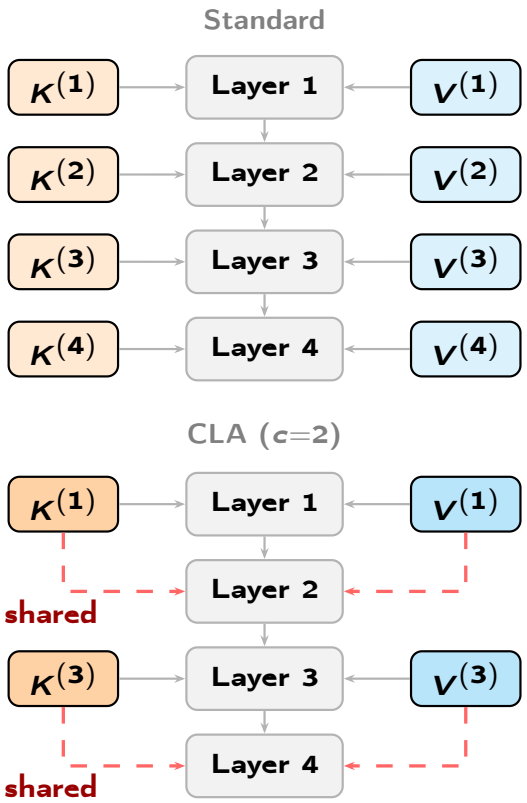
Idea [Brandon, M. Mishra, et al., *"Reducing Transformer Key-Value Cache Size with Cross-Layer Attention"*, 2024]: Adjacent layers *share* the same KV-cache. Only every c-th layer computes K,V; others reuse.

For layer ℓ, anchor layer $a(\ell) = c \cdot \lfloor \ell / c \rfloor$:

$$K^{(\ell)} = K^{(a(\ell))}, \quad V^{(\ell)} = V^{(a(\ell))}$$

Each layer keeps its own W^Q.

- **Reduction:** $c\times$ less KV-cache (across layers).
- CLA is *orthogonal* to GQA/MQA – combine for multiplicative savings.



KV-Cache Reduction: Summary

Method	Core idea	Cache per layer per token	vs. MHA
MHA	Separate K,V per head	$2\,h\,d_k$	$1\times$
MQA	Single shared K,V	$2\,d_k$	$1/h$
GQA-g	g groups share K,V	$2\,g\,d_k$	g/h
MLA	Low-rank KV compression	$d_c + d_R$	$\sim 1/57\times$
Sliding window	Attend to last w tokens	$2\,h\,d_k$ (capped at w)	w/T
CLA (factor c)	Share KV across c layers	$2\,h\,d_k/c$ (amortized)	$1/c$

Key points:

- These techniques are largely **orthogonal and composable**.
- GQA is the current standard (LLaMA-2/3, Mistral, Gemma).
- MLA (DeepSeek-V2/V3) achieves the strongest compression while retaining multi-head expressiveness.
- In practice, models combine multiple techniques: e.g. Mistral = GQA + sliding window; DeepSeek-V3 = MLA + other efficiencies.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model
- Architecture
- Optimization
- Mixed Precision Training
- Compilation and Kernel Fusion
- Distributed Training and Data Loading
- Hyperparameters

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model
- Architecture
- Optimization
- Mixed Precision Training
- Compilation and Kernel Fusion
- Distributed Training and Data Loading
- Hyperparameters

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Introduction

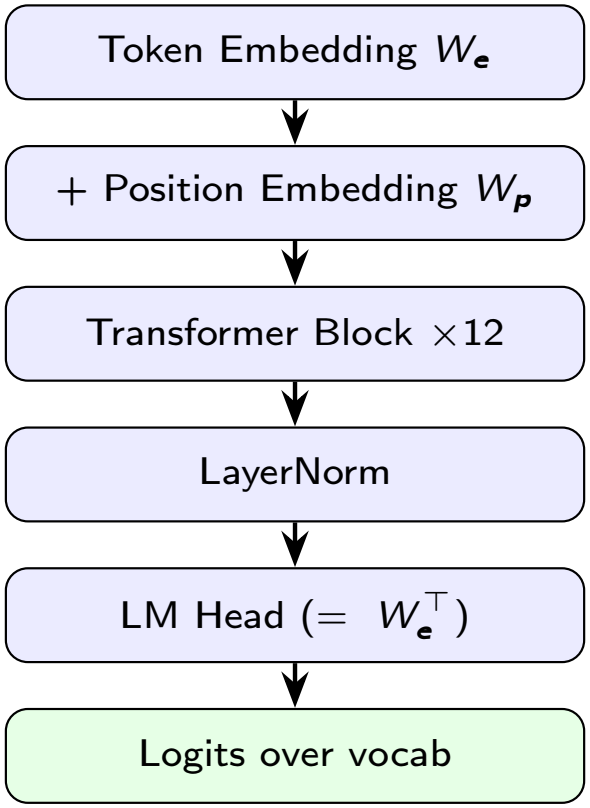
A *decoder-only* Transformer [Radford, J. Wu, et al., *Language Models are Unsupervised Multitask Learners*, 2019] for autoregressive language modeling.

Why GPT-2?

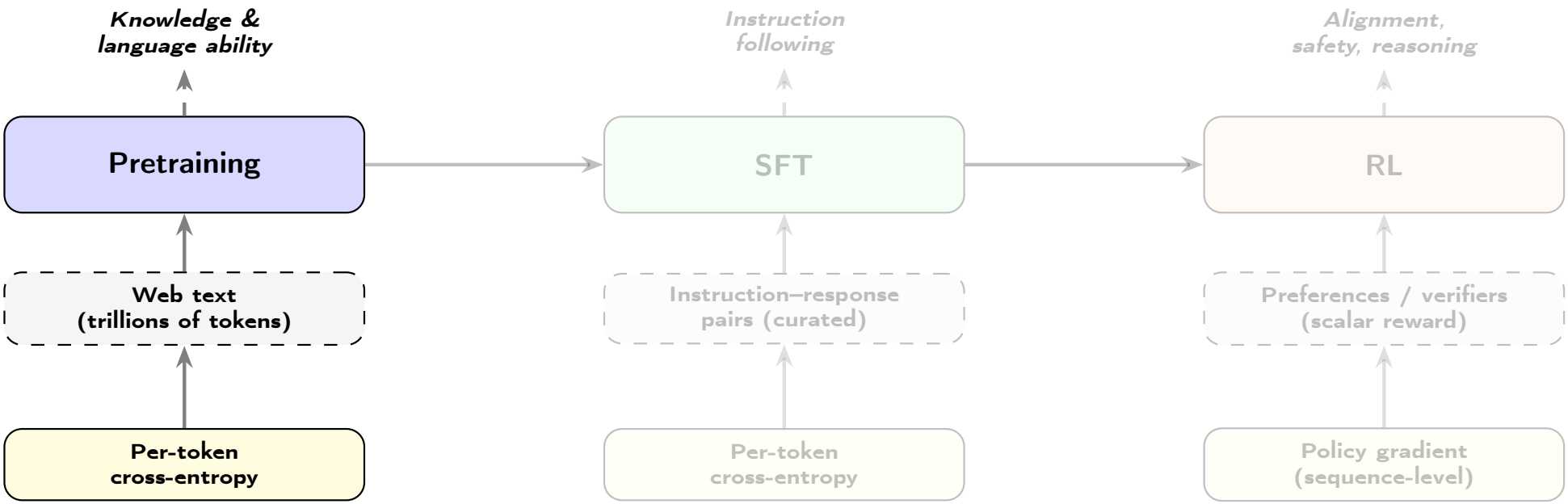
- Demonstrated that *scaling up* a simple language model yields surprisingly capable text generation
- Showed that unsupervised pretraining on diverse web text transfers to many tasks *without fine-tuning* (zero-shot)
- Architecture became the blueprint for GPT-3, ChatGPT, and most modern LLMs
- Small enough (124M) to train from scratch on a single GPU, ideal for studying training techniques

Base configuration (124M params):

- 12 layers, 12 heads, 768-dim embeddings
- Context length 1024 tokens
- Vocabulary: 50304 tokens (BPE)



Where Does Pretraining Fit?



In this chapter we focus on **pretraining**: training a language model from scratch on web-scale text data using next-token prediction.

Architecture: Transformer Block (Pre-Norm)

Each of the 12 blocks applies:

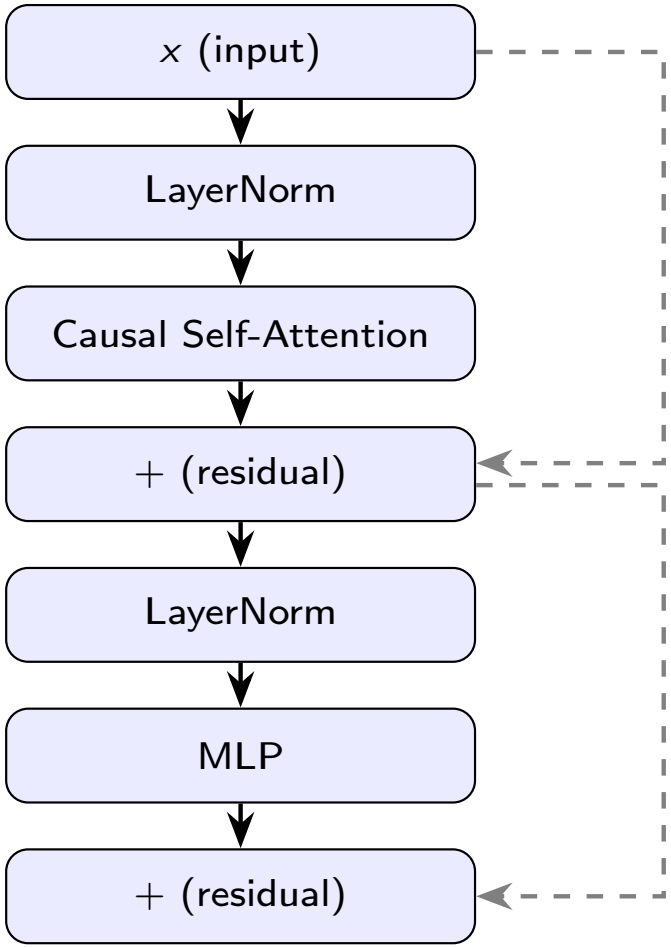
$$x \leftarrow x + \mathrm{Attn}(\mathrm{LN}_1(x))$$

$$x \leftarrow x + \mathrm{MLP}(\mathrm{LN}_2(x))$$

Pre-norm (LayerNorm *before* sublayer) differs from the original Transformer [Vaswani et al., *"Attention Is All You Need"*, 2017] which used post-norm. Pre-norm:

- Stabilizes training at large scale
- Gradients flow more cleanly to early layers

LayerNorm with optional bias: `bias=False` is slightly better and faster for pretraining.



Architecture: Causal Self-Attention

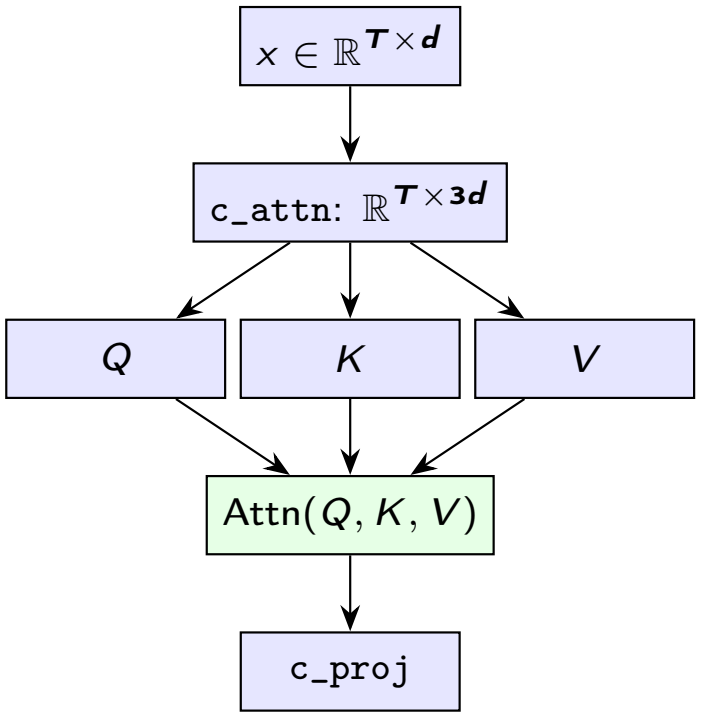
Standard multi-head self-attention with a **causal mask** (no peeking at future tokens):

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

where $M_{ij} = -\infty$ for $j > i$ (upper triangle).

Projections: Q, K, V are computed jointly via a single `c_attn` linear layer ($d \to 3d$), split afterwards.

Flash Attention (PyTorch $\geq$ 2.0): fused CUDA kernel for attention that avoids materializing the $T \times T$ matrix, reducing memory from $O(T^2)$ to $O(T)$.



Effect

Flash Attention gives the same result as standard attention but is 2–4× faster and uses far less memory.

Architecture: MLP Block

Each block's MLP is a two-layer network with a 4× expansion:

$$\mathrm{MLP}(x) = W_2 \cdot \mathrm{GELU}(W_1 x + b_1) + b_2$$

$$W_1 \in \mathbb{R}^{4d \times d}, \quad W_2 \in \mathbb{R}^{d \times 4d}$$

GELU activation (Gaussian Error Linear Unit):

$$\mathrm{GELU}(x) = x \cdot \Phi(x) \approx x \cdot \sigma(1.702x)$$

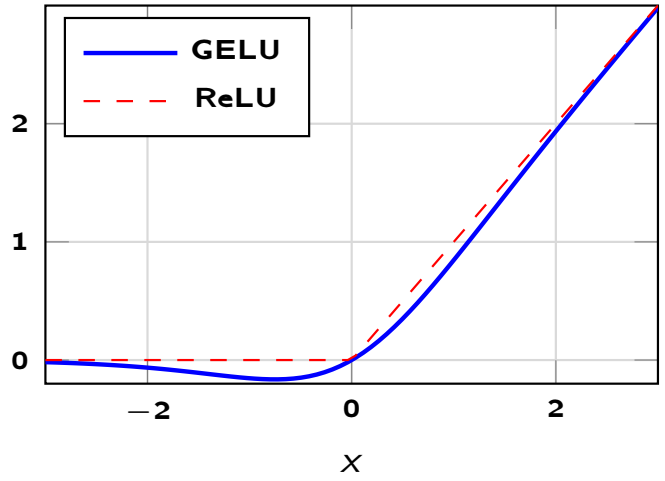
where Φ is the standard normal CDF.

Preferred over ReLU in transformers: smooth, non-zero for negative inputs, empirically better.

Why the approximate version? The original GPT-2 used the tanh approximation:

$$x \cdot \tfrac{1}{2}\left[1 + \tanh\left(\sqrt{2/\pi}(x + 0.044715x^3)\right)\right]$$

PyTorch's default `nn.GELU` computes the exact form. To reproduce GPT-2 weights, use `nn.GELU(approximate='tanh')`. See pytorch/pytorch#39853.



Weight Tying & Initialization

Weight tying: the token embedding matrix W_e and the LM head are *shared*:

```
lm_head.weight = transformer.wte.weight
```

- Saves $V \times d = 50304 \times 768 \approx 38\text{M}$ parameters
- Input and output representations are coupled; works well in practice

Initialization:

- All weights: $\mathcal{N}(0, 0.02)$
- Biases: zero
- **Residual projections** scaled by $\frac{1}{\sqrt{2L}}$ where L = number of layers:

$$\sigma = \frac{0.02}{\sqrt{2 \times 12}}$$

Prevents residual stream variance from growing with depth.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model
- Architecture
- Optimization
- Mixed Precision Training
- Compilation and Kernel Fusion
- Distributed Training and Data Loading
- Hyperparameters

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Optimizer: AdamW with Selective Weight Decay

AdamW = Adam + decoupled weight decay. Weight decay acts as L2 regularization on the weights directly (not via the gradient).

Key insight: not all parameters should be decayed.

Decay (2D+ tensors):

- Weight matrices in attention (Q, K, V, proj)
- Weight matrices in MLP
- Embedding matrices

No decay ($<$ 2D tensors):

- Biases
- LayerNorm scales (1D)

Why not decay everything?

Biases and LayerNorm parameters are scale/shift terms. Penalizing their magnitude does not help as regularization and can harm training.

Fused AdamW: PyTorch offers a CUDA-fused implementation that combines the optimizer update into a single kernel, giving a meaningful speedup.

Learning Rate Schedule: Cosine with Warmup

Three-phase schedule:

1. **Linear warmup** (0 to `warmup_iters`):

$$\eta_t = \eta_{\max} \cdot \frac{t+1}{\texttt{warmup_iters} + 1}$$

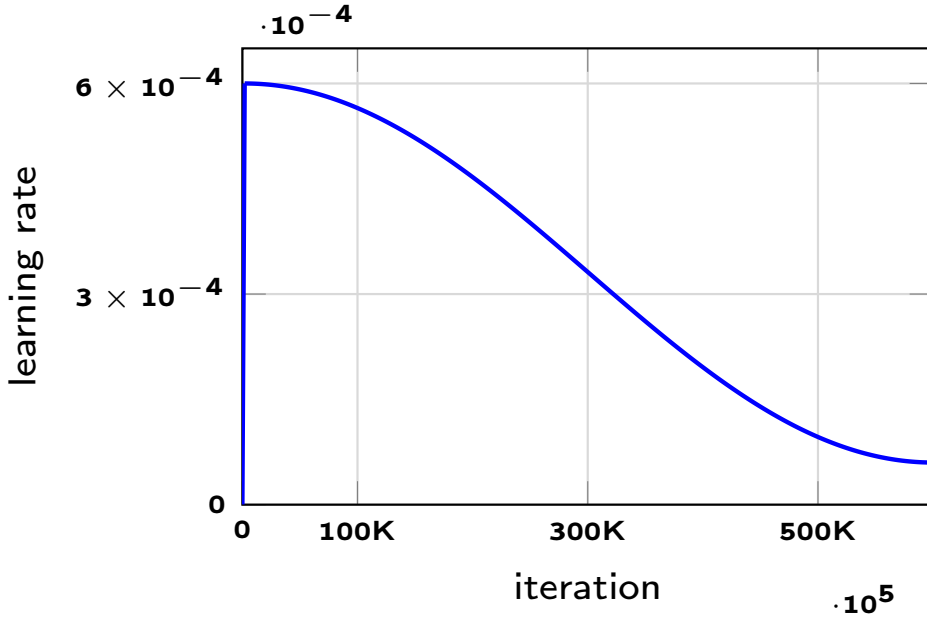
2. **Cosine decay** (`warmup_iters` to `lr_decay_iters`):

$$\eta_t = \eta_{\min} + \frac{1+\cos(\pi r)}{2}(\eta_{\max} - \eta_{\min})$$

   where $r = \frac{t - \texttt{warmup}}{\texttt{decay} - \texttt{warmup}}$

3. **Constant** $\eta_{\min}$ afterwards

Follows Chinchilla scaling law recommendations [Hoffmann et al., *"Training Compute-Optimal Large Language Models"*, 2022]: $\eta_{\min} \approx \eta_{\max}/10$.



Gradient Accumulation

Problem: large effective batch sizes don't fit in GPU memory.

Solution: accumulate gradients over multiple micro-batches before doing one optimizer step.

With `gradient_accumulation_steps` $= G$:

1. For $g = 1, \ldots, G$: forward + backward on micro-batch, *accumulate* gradients (no optimizer step)
2. After G steps: `optimizer.step()`, then `zero_grad()`

Mathematically equivalent to training on one big batch of size $G \times B$, since gradients add linearly.

Important: divide the loss by G before each backward pass to keep the scale consistent:

```
loss = loss / gradient_accumulation_steps
```

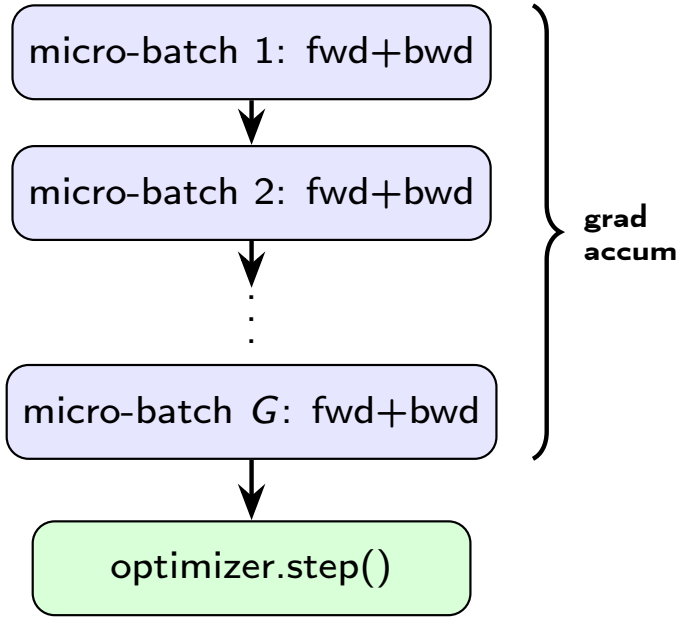


Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model
- Architecture
- Optimization
- Mixed Precision Training
- Compilation and Kernel Fusion
- Distributed Training and Data Loading
- Hyperparameters

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

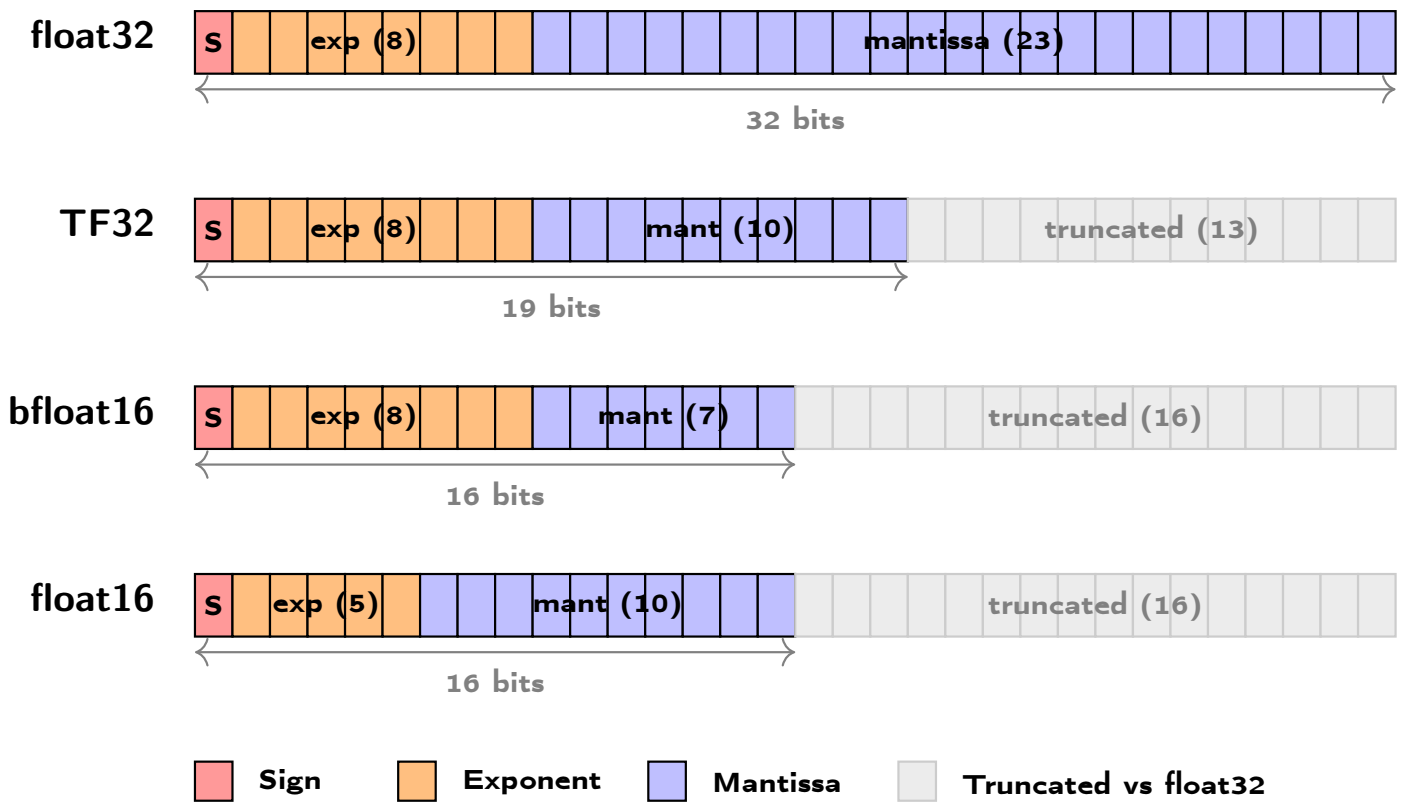
16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Floating Point Formats for Deep Learning

IEEE 754 representation: $(-1)^S \times 2^{E-\text{bias}} \times (1 + \sum_{i=1}^{m} b_i\, 2^{-i})$, bias $= 2^{e-1} - 1$.



4 bytes/param. Highest precision slowest. Used for master weights & optimizer states.

NVIDIA internal format (Ampere+). Tensor cores use TF32 for float32 matmuls transparently: 8× faster, no code changes.

2 bytes/param. Full float32 range, low precision. Preferred for training: no overflow issues.

2 bytes/param. Limited range ($\sim 6 \times 10^{-5}$ to 6.5×10^{4}) but higher precision than bfloat16. Prone to overflow.

Key trade-off: more exponent bits ⇒ larger dynamic range; more mantissa bits ⇒ higher precision. bfloat16 and TF32 keep float32's 8-bit exponent (same range), while float16 trades range for precision.

Mixed-Precision Matrix Multiplication

Given $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{n \times p}$, compute $C = A \cdot B$ where $C_{ij} = \sum_{k=1}^{n} A_{ik} \cdot B_{kj}$.

Mixed-precision approach: store and multiply in low precision, accumulate in high precision:

$$C_{ij} = \underbrace{\sum_{k=1}^{n} \overbrace{\mathrm{bf16}(A_{ik})}^{\text{cast to bf16}} \cdot \overbrace{\mathrm{bf16}(B_{kj})}^{\text{cast to bf16}}}_{\text{accumulated in float32}} \quad \in \text{float32}$$

Step by step:

1. Cast A, B from float32 to bfloat16
2. Each product $A_{ik} \cdot B_{kj}$: bf16 $\times$ bf16
3. Running sum $C_{ij} \mathrel{+}= \ldots$ kept in float32
4. Result C is in float32

Why not accumulate in bf16?
With $n = 768$, summing 768 small products in bf16 (7-bit mantissa $\approx$ 2 decimal digits) leads to severe rounding error. Float32 (23-bit mantissa) preserves accuracy.

Speedup sources:

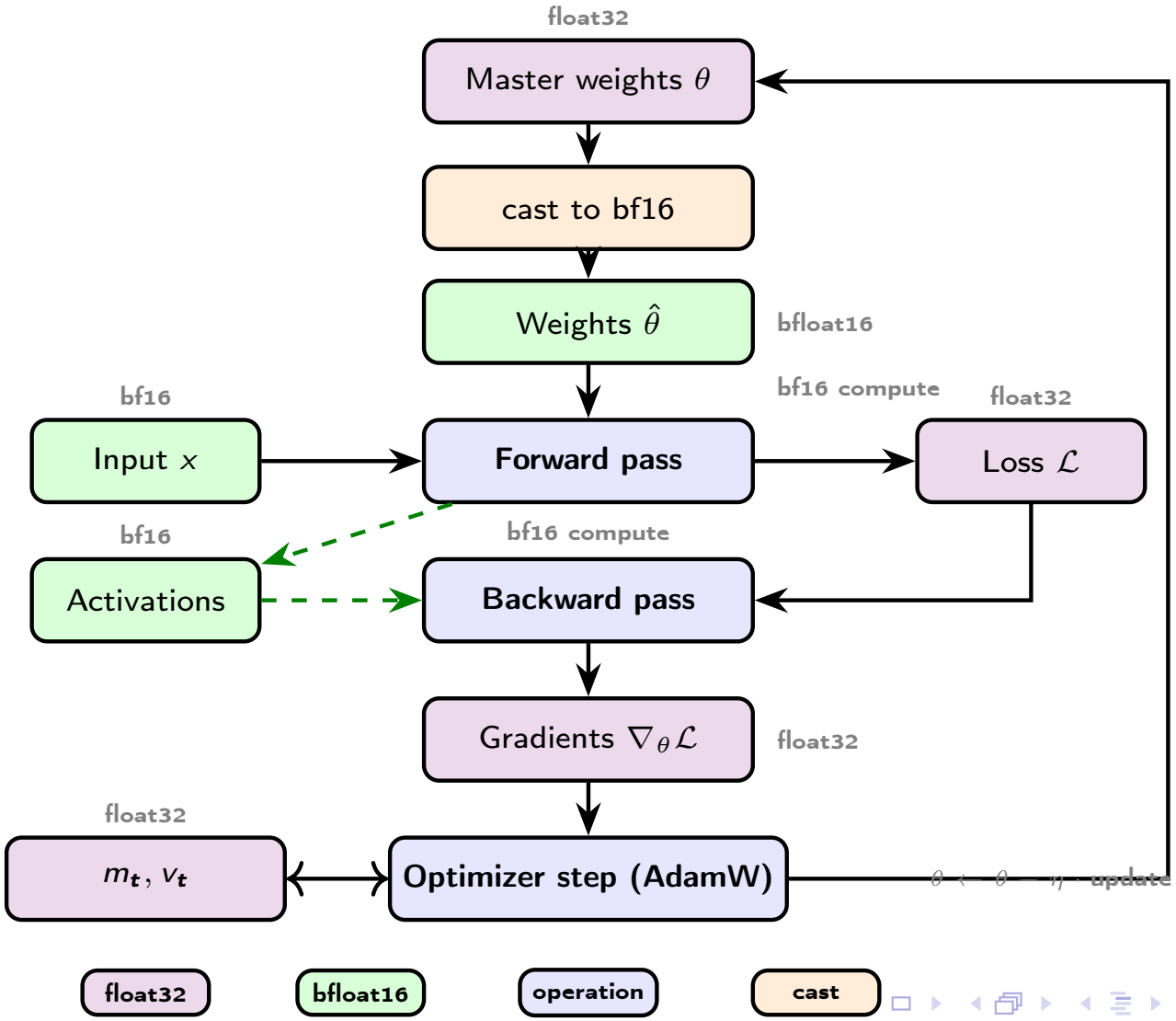
- bf16 values are $2\times$ smaller $\Rightarrow$ $2\times$ less memory bandwidth
- Tensor cores process bf16 at 2–8$\times$ the float32 throughput
- Float32 accumulation is "free", built into the tensor core hardware

In PyTorch

`torch.amp.autocast` automatically casts matmul inputs to bf16 while the accumulator stays in float32. No manual casting needed.

Mixed Precision Training

Goal: use reduced-precision floats to save memory and speed up computation, while keeping full precision where it matters.



GradScaler: Handling float16 Underflow

Problem: float16 has a small dynamic range ($\approx 6 \times 10^{-5}$ to 6.5×10^4). Gradients during training are often tiny and **underflow to zero**.

Solution: loss scaling.

1. Multiply loss by a large scale factor S before `backward()`
$$\tilde{\mathcal{L}} = S \cdot \mathcal{L}$$
2. Gradients are also scaled by S (by chain rule)
3. Before optimizer step: divide gradients by S
4. Check for `inf/nan`:
   - If found: **skip** the optimizer step, reduce S
   - Otherwise: proceed, slowly increase S

Not needed for `bfloat16` (same range as float32).

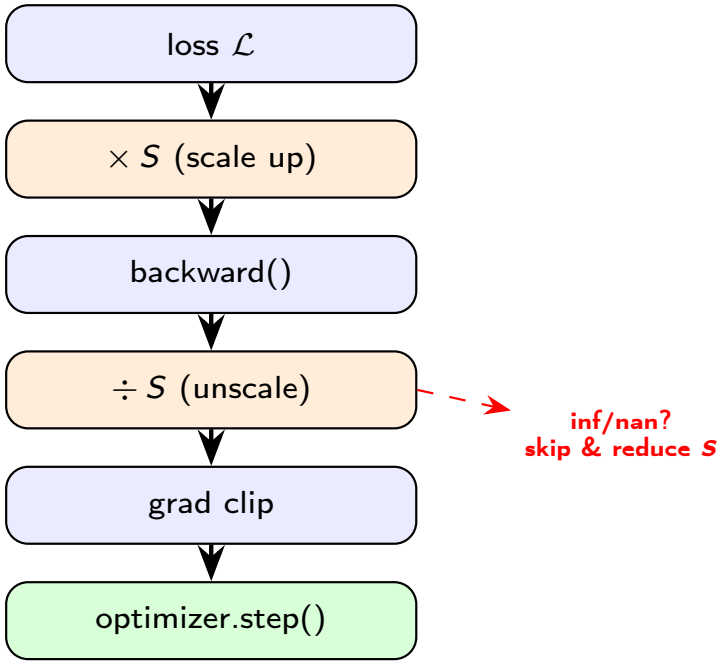


Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model
- Architecture
- Optimization
- Mixed Precision Training
- Compilation and Kernel Fusion
- Distributed Training and Data Loading
- Hyperparameters

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

torch.compile

PyTorch 2.0 introduced `torch.compile(model)`, which JIT-compiles the model graph.

How it works:

1. **TorchDynamo**: captures the computation graph by tracing Python bytecode
2. **TorchInductor**: backend compiler that generates optimized Triton (GPU) or C++ (CPU) code

What it optimizes:

- **Kernel fusion**: merges adjacent operations into one kernel (e.g., LayerNorm + dropout), reducing memory bandwidth
- **Memory layout**: reorders ops to maximize cache reuse
- Eliminates Python overhead in the hot path

Cost

First forward pass triggers compilation (takes ∼1 minute). Subsequent passes use the cached compiled graph.

Typical speedup

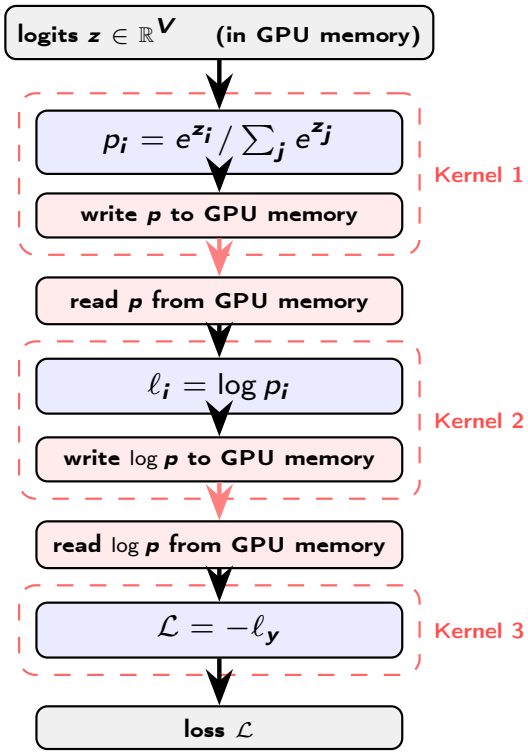
15–30% throughput improvement on A100 for GPT-2 scale models. Larger gains at bigger model sizes due to more ops to fuse.

Kernel Fusion: Softmax + Cross-Entropy (Forward)

Given logits $z \in \mathbb{R}^V$ and target class y:

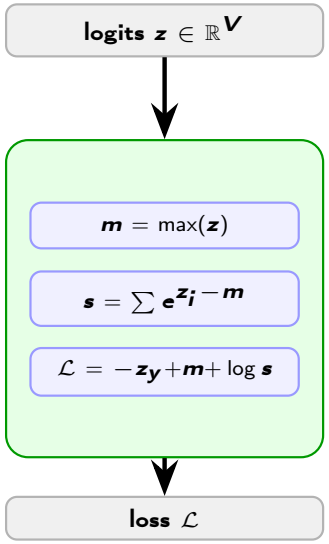
$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad \text{(softmax)}, \qquad \mathcal{L} = -\log p_y \;=\; -z_y + \log \sum_j e^{z_j} \quad \text{(cross-entropy)}$$

Unfused: 3 kernel launches



Each kernel: read from memory, compute, write back

Fused: 1 kernel launch



No GPU memory writes of e^z, p, $\log p$: all computed in registers.

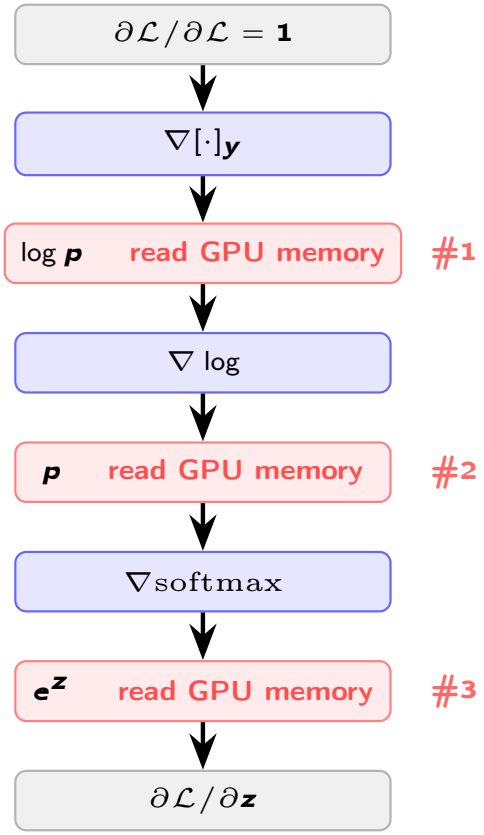
Cross-entropy computed directly from logits, no need to materialise p.

Kernel Fusion: Softmax + Cross-Entropy (Backward)

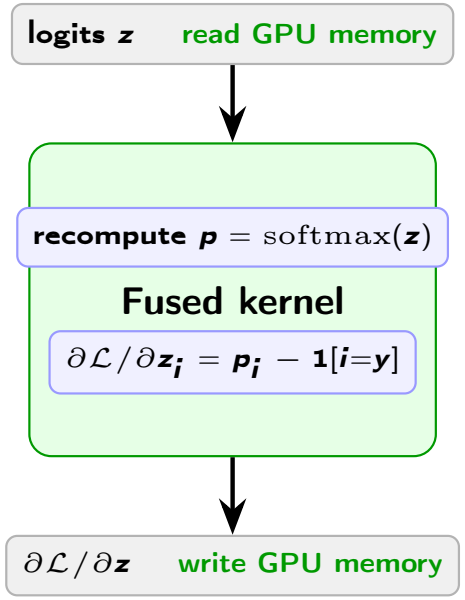
Gradient of cross-entropy w.r.t. logits:

$$\frac{\partial \mathcal{L}}{\partial z_i} = p_i - 1[i = y], \qquad p = \mathrm{softmax}(z)$$

Unfused backward



Fused backward



Only 1 read + 1 write. p recomputed in registers, no stored intermediates needed.

Gradient Clipping

Problem: in deep networks, gradients can occasionally explode, destabilizing training.

Solution: clip the global gradient norm before the optimizer step.

$$\text{if } \|\nabla\|_2 > \texttt{grad_clip}: \quad \nabla \leftarrow \frac{\texttt{grad_clip}}{\|\nabla\|_2} \cdot \nabla$$

- The global norm is computed over *all* parameters jointly
- Only rescales when the norm exceeds the threshold (here: 1.0)
- Does not change gradient direction, only magnitude
- Applied *after* unscaling (when using GradScaler with float16)

In code

```
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(
  model.parameters(), 1.0)
scaler.step(optimizer)
```

A few large gradient events per training run is normal; frequent clipping suggests a learning rate that is too high.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
   - Architecture
   - Optimization
   - Mixed Precision Training
   - Compilation and Kernel Fusion
   - Distributed Training and Data Loading
   - Hyperparameters
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Distributed Data Parallel (DDP)

Idea: each GPU holds a full model replica and processes a different mini-batch. Gradients are **all-reduced** across GPUs after each backward pass.

Gradient sync trick: only sync at the *last* micro-step during gradient accumulation:

```
model.require_backward_grad_sync
= (step == grad_accum_steps - 1)
```

Effective batch: scales linearly with number of GPUs. Throughput scales near-linearly.

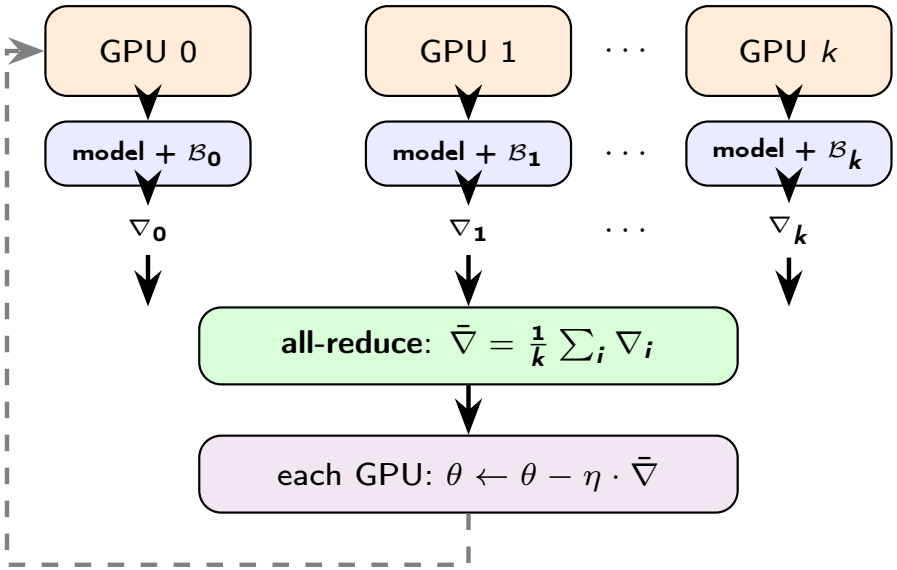


Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
   - Architecture
   - Optimization
   - Mixed Precision Training
   - Compilation and Kernel Fusion
   - Distributed Training and Data Loading
   - Hyperparameters
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Hyperparameters

Category	Parameter	Value
Model	`n_layer`	12
	`n_head`	12
	`n_embd`	768
	`block_size`	1024 tokens
	`vocab_size`	50304 (50257 rounded to $\times 64$)
	`bias`	False
Optimizer	`learning_rate`	6×10^{-4}
	`weight_decay`	0.1
	(β_1, β_2)	$(0.9, 0.95)$
	`grad_clip`	1.0
Schedule	`warmup_iters`	2000
	`lr_decay_iters`	600000
	`min_lr`	6×10^{-5} $(= \frac{\text{lr}}{10})$
Batch	`batch_size`	$\approx$ 491K tokens $(= 480 \times 1024)$

Vocabulary size rounded up to a multiple of 64 for GPU efficiency (better utilization of tensor cores).

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
   - Motivation & GPU Memory Hierarchy
   - The Flash Attention Algorithm
   - GPU Programming: CUDA and Triton
   - Triton Implementation
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

The Problem with Standard Attention

Recall: $\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$

The bottleneck is *not* arithmetic, it is **memory**:

- The $T \times T$ matrix $S = QK^\top$ must be materialized in HBM
- Writing S to HBM and reading it back dominates wall-clock time

Memory-bound, not compute-bound

Standard attention spends most time on memory transfers, not on FLOPs.

[Dao, Fu, et al., *"FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness"*, 2022] [Dao, *"FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning"*, 2023] [Jamil, *Flash Attention Explained*, 2024]

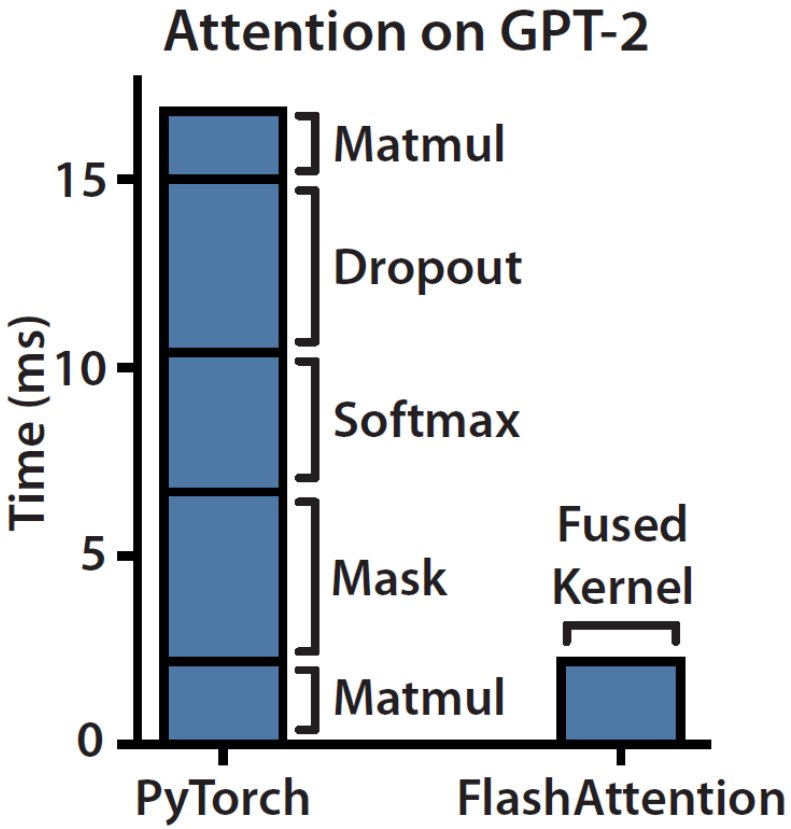


Table of Contents

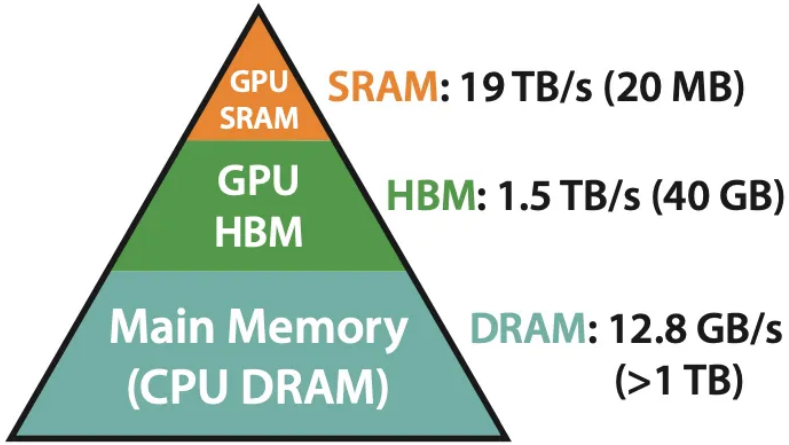
1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
   - Motivation & GPU Memory Hierarchy
   - The Flash Attention Algorithm
   - GPU Programming: CUDA and Triton
   - Triton Implementation
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

HBM vs. SRAM

GPUs have a two-level memory hierarchy:

HBM (off-chip): Stores model parameters, activations, optimizer states. Every kernel reads inputs from and writes outputs to HBM.

SRAM (on-chip): Each streaming multiprocessor has its own SRAM (shared memory). Data must be explicitly loaded into SRAM before compute.



Implication for attention

Standard attention writes $S, P \in \mathbb{R}^{T \times T}$ to HBM between steps. Flash attention computes S and P block-wise in SRAM on-the-fly, avoiding the HBM round-trip.

Standard Attention: Memory Access Pattern

Algorithm 2: Standard Attention ■ Load ■ Compute ■ Store

Input: $Q, K, V \in \mathbb{R}^{T \times d}$ in HBM
Output: $O \in \mathbb{R}^{T \times d}$

Load Q, K from HBM	// $2Td$
Compute $S = QK^\top \in \mathbb{R}^{T \times T}$	
Write S to HBM	// T^2
Load S from HBM	// T^2
Compute $P = \text{softmax}(S) \in \mathbb{R}^{T \times T}$	
Write P to HBM	// T^2
Load P, V from HBM	// $T^2 + Td$
Compute $O = PV \in \mathbb{R}^{T \times d}$	
Write O to HBM	// Td
return O	

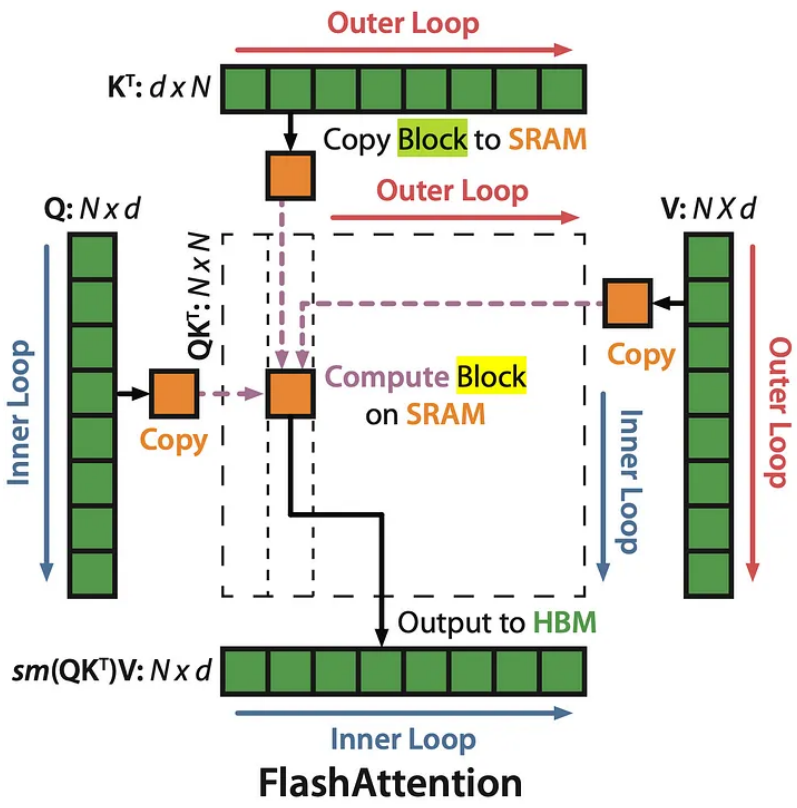
Total HBM access: $\mathcal{O}(T^2 + Td)$. Each step is a separate kernel (no fusion). S, P are $T \times T \rightarrow$ quadratic memory. Must store P for backward pass.

Flash Attention: Key Ideas

Do not materialize the $T \times T$ attention matrix.

Instead:

1. **Tiling**: Process Q, K, V in blocks that fit in SRAM
2. **Online softmax**: Compute softmax incrementally across blocks
3. **Kernel fusion**: Compute S, P, O in a single kernel
4. **Recomputation**: In backward pass, recompute P from Q, K instead of storing it



Memory-bound → compute-bound

Flash Attention trades extra computation for fewer memory accesses. This is faster because modern GPUs have far more compute than memory bandwidth.

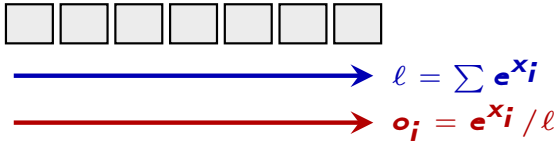
Softmax: Three Versions

Naive: 2 passes. Numerically unstable for large x_i.

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Algorithm 3: Naive Softmax

Input: $x = (x_1, \ldots, x_n)$
$\ell \leftarrow 0$
for $i = 1$ **to** n **do** $\ell \leftarrow \ell + e^{x_i}$
for $i = 1$ **to** n **do** $o_i \leftarrow e^{x_i}/\ell$
return $(o_1, \ldots, o_n)$

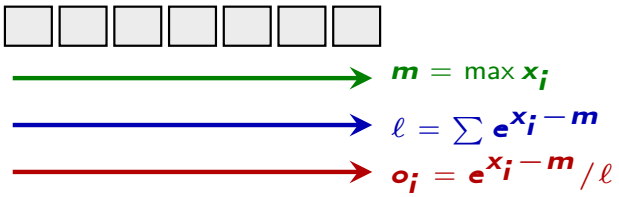


Stable: 3 passes. Subtract max for numerical stability.

$$\text{softmax}(x)_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j$$

Algorithm 4: Stable Softmax

Input: $x = (x_1, \ldots, x_n)$
$m \leftarrow -\infty$
for $i = 1$ **to** n **do** $m \leftarrow \max(m, x_i)$
$\ell \leftarrow 0$
for $i = 1$ **to** n **do** $\ell \leftarrow \ell + e^{x_i - m}$
for $i = 1$ **to** n **do** $o_i \leftarrow e^{x_i - m}/\ell$
return $(o_1, \ldots, o_n)$



Streaming (Online) Softmax

Key insight: Fuse max and sum into a *single pass*.

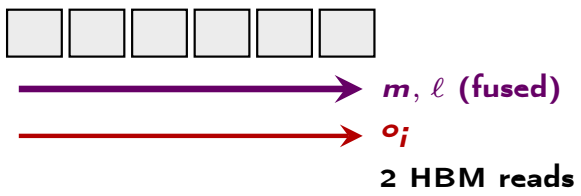
Maintain running max m and rescaled sum ℓ. When seeing x_i:

$$m' = \max(m, x_i)$$
$$\ell' = \ell \cdot e^{m-m'} + e^{x_i - m'}$$

The correction $e^{m-m'}$ rescales the previous sum to the new max.

Algorithm 5: Online Softmax

Input: $x = (x_1, \ldots, x_n)$
$m \leftarrow -\infty,\ \ell \leftarrow 0$
for $i = 1$ **to** n **do**
  $m' \leftarrow \max(m, x_i)$
  $\ell \leftarrow \ell \cdot e^{m-m'} + e^{x_i - m'}$
  $m \leftarrow m'$
for $i = 1$ **to** n **do** $o_i \leftarrow e^{x_i - m}/\ell$
return $(o_1, \ldots, o_n)$



Why this matters

Online softmax lets us process K blocks one at a time without knowing the global max, exactly what tiled attention needs.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
   - Motivation & GPU Memory Hierarchy
   - The Flash Attention Algorithm
   - GPU Programming: CUDA and Triton
   - Triton Implementation
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Attention with Two Blocks

Consider a single query block Q attending to keys and values split into two blocks. The score matrix and values are partitioned as:

$$S = \begin{bmatrix} S^{(1)} & S^{(2)} \end{bmatrix}, \qquad V = \begin{bmatrix} V^{(1)} \\ V^{(2)} \end{bmatrix}$$

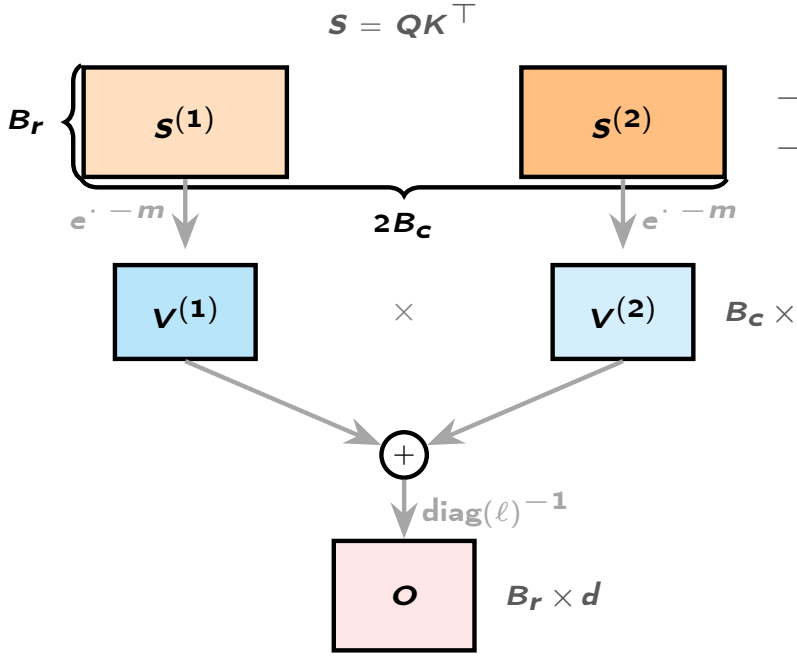
To compute softmax over the full row we need the **global** max and sum:

$$m = \max\big(\mathrm{rowmax}(S^{(1)}),\ \mathrm{rowmax}(S^{(2)})\big)$$

$$\ell = \mathrm{rowsum}(e^{S^{(1)}-m}) + \mathrm{rowsum}(e^{S^{(2)}-m})$$

The softmax-weighted output becomes:

$$O = \mathrm{diag}(\ell)^{-1}\Big(e^{S^{(1)}-m}V^{(1)} + e^{S^{(2)}-m}V^{(2)}\Big)$$



Challenge

Softmax needs m and ℓ across *all* blocks, but we want to process one block at a time.

Online Softmax for Blocked Attention

Solution: Apply streaming softmax to process one block at a time.

After block 1: local statistics and partial output

$$m^{(1)} = \text{rowmax}(S^{(1)})$$

$$\ell^{(1)} = \text{rowsum}(e^{S^{(1)} - m^{(1)}})$$

$$O^{(1)} = \text{diag}(\ell^{(1)})^{-1}\, e^{S^{(1)} - m^{(1)}}\, V^{(1)}$$

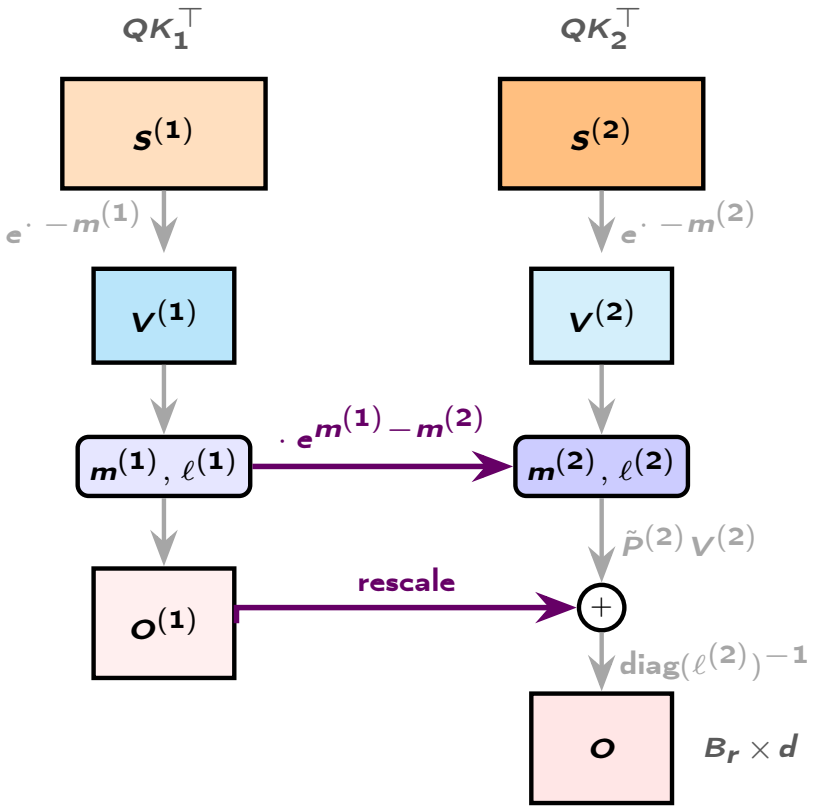
After block 2: update with correction $e^{m^{(1)} - m^{(2)}}$

$$m^{(2)} = \max(m^{(1)},\ \text{rowmax}(S^{(2)}))$$

$$\ell^{(2)} = e^{m^{(1)} - m^{(2)}}\, \ell^{(1)} + \text{rowsum}(e^{S^{(2)} - m^{(2)}})$$

$$O^{(2)} = \frac{\ell^{(1)}}{\ell^{(2)}} e^{m^{(1)} - m^{(2)}}\, O^{(1)} + \frac{1}{\ell^{(2)}}\, e^{S^{(2)} - m^{(2)}}\, V^{(2)}$$

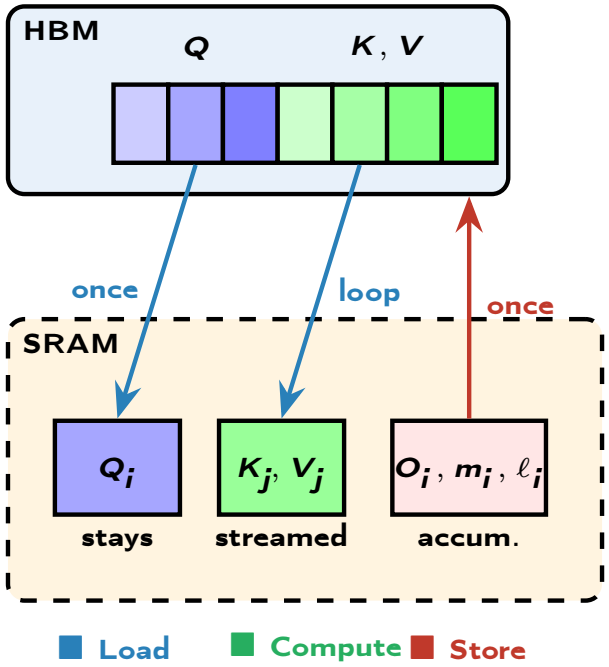
After the last block: $m^{(2)} = m$, $\ell^{(2)} = \ell$, $O^{(2)} = O$: exact result.



Flash Attention 2: Forward Pass

Algorithm 6: FlashAttention-2 Forward

Input: $Q, K, V \in \mathbb{R}^{T \times d}$ in HBM; block sizes B_r, B_c
Output: $O \in \mathbb{R}^{T \times d}$, $L \in \mathbb{R}^{T}$
Divide Q into T_r blocks, K, V into T_c blocks
for $i = 1, \ldots, T_r$ **do**
  Load Q_i from HBM to SRAM
  $O_i \leftarrow 0$, $\ell_i \leftarrow 0$, $m_i \leftarrow -\infty$
  for $j = 1, \ldots, T_c$ **do**
    Load K_j, V_j from HBM to SRAM
    $S_{ij} \leftarrow Q_i K_j^\top$
    $\tilde{m}_i \leftarrow \max(m_i, \mathrm{rowmax}(S_{ij}))$
    $\tilde{P}_{ij} \leftarrow \exp(S_{ij} - \tilde{m}_i)$
    $\ell_i \leftarrow e^{m_i - \tilde{m}_i} \ell_i + \mathrm{rowsum}(\tilde{P}_{ij})$
    $O_i \leftarrow \mathrm{diag}(e^{m_i - \tilde{m}_i}) O_i + \tilde{P}_{ij} V_j$
    $m_i \leftarrow \tilde{m}_i$
  $O_i \leftarrow \mathrm{diag}(\ell_i)^{-1} O_i$ // Trick 1
  $L_i \leftarrow m_i + \log(\ell_i)$ // Trick 2
  Write O_i, L_i to HBM
return O, L



Key: Q_i loaded once, K_j, V_j streamed. No $T \times T$ matrix materialized. $O(T)$ extra memory.

Trick 1: Accumulate unnormalized O_i, divide by ℓ_i once at the end.

Trick 2: Store $L_i = m_i + \log(\ell_i)$ instead of m_i and ℓ_i separately (for backward pass).

Attention Backward Pass: Gradient Derivation

Forward: $S = QK^\top, \quad P = \text{softmax}(S), \quad O = PV.$ Given dO, derive dQ, dK, dV.

Step 1: Gradients through $O = PV$

$$dV = P^\top dO \qquad\qquad dP = dO\, V^\top$$

Step 2: Gradient through softmax $P_i = \text{softmax}(S_i)$ (row-wise).

The Jacobian of softmax is $\frac{\partial P_i}{\partial S_i} = \text{diag}(P_i) - P_i P_i^\top$ (symmetric). By the chain rule, the row gradient dS_i is obtained by multiplying dP_i with this Jacobian:

$$dS_i = \frac{\partial P_i}{\partial S_i}\, dP_i = \left(\text{diag}(P_i) - P_i P_i^\top\right) dP_i = P_i \odot dP_i - \underbrace{(P_i^\top dP_i)}_{=:\, D_i} P_i$$

Componentwise: $dS_{ij} = P_{ij}(dP_{ij} - D_i)$, where the scalar $D_i = \sum_j P_{ij}\, dP_{ij}$.

Substituting $dP_{ij} = (dO\, V^\top)_{ij} = \sum_k dO_{ik} V_{jk}$:

$$D_i = \sum_j P_{ij} \sum_k dO_{ik} V_{jk} = \sum_k dO_{ik} \underbrace{\sum_j P_{ij} V_{jk}}_{=O_{ik}} = \sum_k dO_{ik}\, O_{ik}$$

Key insight: $D = \text{rowsum}(dO \odot O)$ can be precomputed from dO and O alone. No need to materialize P or dP.

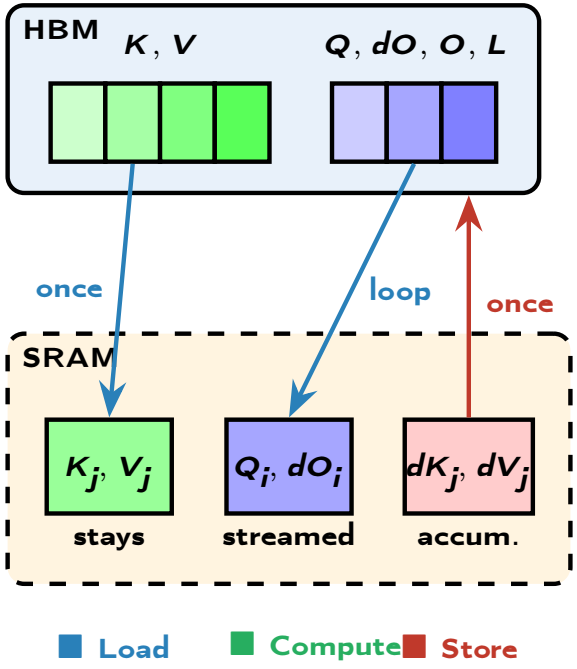
Step 3: Gradients through $S = QK^\top$

$$dQ = dS \cdot K \qquad\qquad dK = dS^\top \cdot Q$$

Flash Attention 2: Backward Pass

Algorithm 7: FlashAttention-2 Backward

Input: $Q, K, V, O, dO \in \mathbb{R}^{T \times d}$, $L \in \mathbb{R}^{T}$ in HBM;
block sizes B_r, B_c
Output: $dQ, dK, dV \in \mathbb{R}^{T \times d}$
Divide K, V into T_c blocks; Q, O, dO, dQ into T_r blocks
$D \leftarrow \text{rowsum}(dO \odot O)$ // precompute
for $j = 1, \ldots, T_c$ **do**
  Load K_j, V_j from HBM
  $dK_j \leftarrow 0$, $dV_j \leftarrow 0$
  for $i = 1, \ldots, T_r$ **do**
    Load Q_i, O_i, dO_i, L_i, D_i
    $S_{ij} \leftarrow Q_i K_j^\top$
    $P_{ij} \leftarrow \exp(S_{ij} - L_i)$ // recompute
    $dV_j \leftarrow dV_j + P_{ij}^\top \, dO_i$
    $dP_{ij} \leftarrow dO_i \, V_j^\top$
    $dS_{ij} \leftarrow P_{ij} \odot (dP_{ij} - D_i)$
    $dQ_i \leftarrow dQ_i + dS_{ij} \, K_j$
    $dK_j \leftarrow dK_j + dS_{ij}^\top \, Q_i$
  Write dK_j, dV_j to HBM
return dQ, dK, dV



Key differences from forward:

- Outer loop over K, V blocks (not Q)
- P_{ij} recomputed from Q_i, K_j, L_i
- D_i avoids storing P

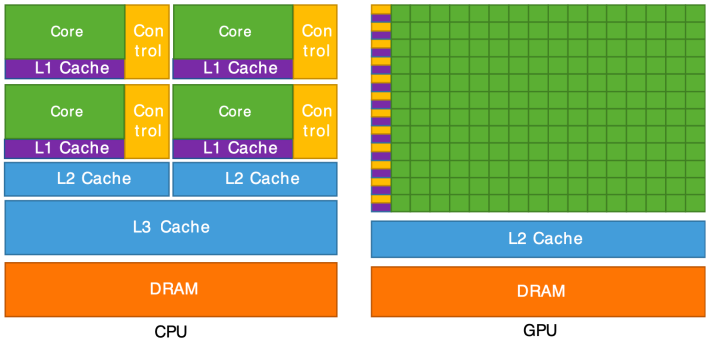
Table of Contents

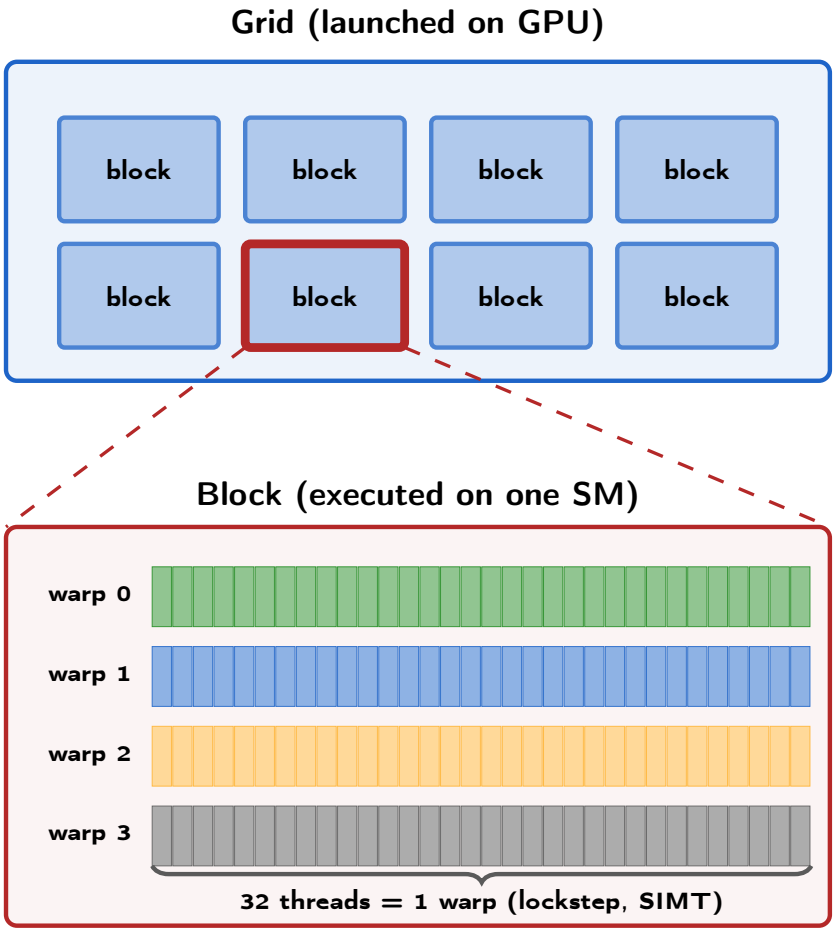
1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
   - Motivation & GPU Memory Hierarchy
   - The Flash Attention Algorithm
   - GPU Programming: CUDA and Triton
   - Triton Implementation
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

CUDA Programming Model

CUDA is NVIDIA's programming model for GPUs.

- **Thread**: runs the kernel
- **Warp**: 32 threads executing in lockstep (SIMT); implicit, not exposed in CUDA code
- **Block**: up to 1024 threads, runs on one SM
- **Grid**: collection of blocks, launched on the GPU





CUDA Example: Vector Addition

```
__global__ void cuda_vector_add(float *OUT, float *A, float *B, int N)
{
    int i = threadIdx.x;
    if (i < N) { OUT[i] = A[i] + B[i]; }
}

// Allocate device memory and copy host vectors to GPU
float *d_A, *d_B, *d_OUT;
cudaMalloc(&d_A,   N * sizeof(float));
cudaMalloc(&d_B,   N * sizeof(float));
cudaMalloc(&d_OUT, N * sizeof(float));
cudaMemcpy(d_A, h_A, N * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, N * sizeof(float), cudaMemcpyHostToDevice);

// Launch: one block, N threads
cuda_vector_add<<<1, N>>>(d_OUT, d_A, d_B, N);
```

`__global__` **Marks a function as a *kernel*: callable from the host, runs on the device.**

`cudaMalloc` / `cudaMemcpy` **Allocate GPU memory and transfer data from host to device.**

`threadIdx.x` **Built-in: index of the current thread within its block (here the only block).**

`<<<1, N>>>` **Launch configuration: 1 block of *N* threads. Each thread computes one element** `OUT[i] = A[i] + B[i]`.

CUDA Example: Matrix Addition

```
__global__ void cuda_matrix_add(float *OUT, float *A, float *B,
                                int NUM_ROWS, int NUM_COLS)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < NUM_ROWS && col < NUM_COLS) {
        size_t idx = row * NUM_COLS + col;
        OUT[idx] = A[idx] + B[idx];
    }
}

// 2D grid and block layout
dim3 grid(ceil(NUM_COLS/16), ceil(NUM_ROWS/16), 1);
dim3 block(16, 16, 1);
cuda_matrix_add<<<grid, block>>>(d_OUT, d_A, d_B, NUM_ROWS, NUM_COLS);
```

Uses 2D indexing: `blockIdx` selects the tile, `threadIdx` selects the element within the tile.

Triton: A Higher-Level GPU Language

Triton [Tillet et al., *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations*, 2019] abstracts away low-level CUDA details:

Concept	CUDA	Triton
Parallelism unit	Thread	Program (block-level)
Memory access	Per-element load/store	Block load/store with masks
Shared memory	Manual management	Automatic
Synchronization	`__syncthreads()`	Implicit

- Programs ("tasks") replace blocks; threads are not directly exposed
- Block-level operations: `tl.load`, `tl.store`, `tl.dot`
- `tl.constexpr` annotations enable compile-time optimizations
- `pid = tl.program_id(axis)` identifies which block we are in

Triton Example: Vector Addition

```
@triton.jit
def add_kernel(x_ptr, y_ptr, output_ptr, n_elements,
               BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(0)
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    result = x + y
    tl.store(output_ptr + offsets, result, mask=mask)

# Allocate vectors directly on the GPU (PyTorch tensors)
x      = torch.rand(n, device='cuda')
y      = torch.rand(n, device='cuda')
output = torch.empty(n, device='cuda')

# Launch
grid = lambda meta: (triton.cdiv(n, meta['BLOCK_SIZE']),)
add_kernel[grid](x, y, output, n, BLOCK_SIZE=256)
```

`@triton.jit` **JIT-compiles the Python function into a GPU kernel.**

`tl.constexpr` **Marks a kernel parameter as a compile-time constant (specialised per value).**

`tl.program_id(0)` **ID of the current program instance along axis 0 (like CUDA** `blockIdx.x`**).**

`tl.arange(0, BLOCK_SIZE)` **Vector of indices [0, BLOCK_SIZE); the kernel operates on whole tiles, not single threads.**

`mask` **Boolean vector that disables lanes past** `n_elements` **for the tail block.**

`tl.load` / `tl.store` **Masked, vectorised reads/writes from/to global memory.**

`grid = lambda meta: ...` **Number of program instances to launch, computed from the chosen** `BLOCK_SIZE`**.**

Triton Example: Matrix Addition

```
@triton.jit
def mat_add_kernel(A, B, C, M, N,
                   BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
    A_block = tl.load(A + offs_m[:, None] * N + offs_n[None, :], mask=mask)
    B_block = tl.load(B + offs_m[:, None] * N + offs_n[None, :], mask=mask)
    tl.store(C + offs_m[:, None] * N + offs_n[None, :],
             A_block + B_block, mask=mask)

# 2D grid launch
grid = lambda meta: (triton.cdiv(M, meta['BLOCK_M']),
                     triton.cdiv(N, meta['BLOCK_N']))
mat_add_kernel[grid](A, B, C, M, N, BLOCK_M=64, BLOCK_N=64)
```

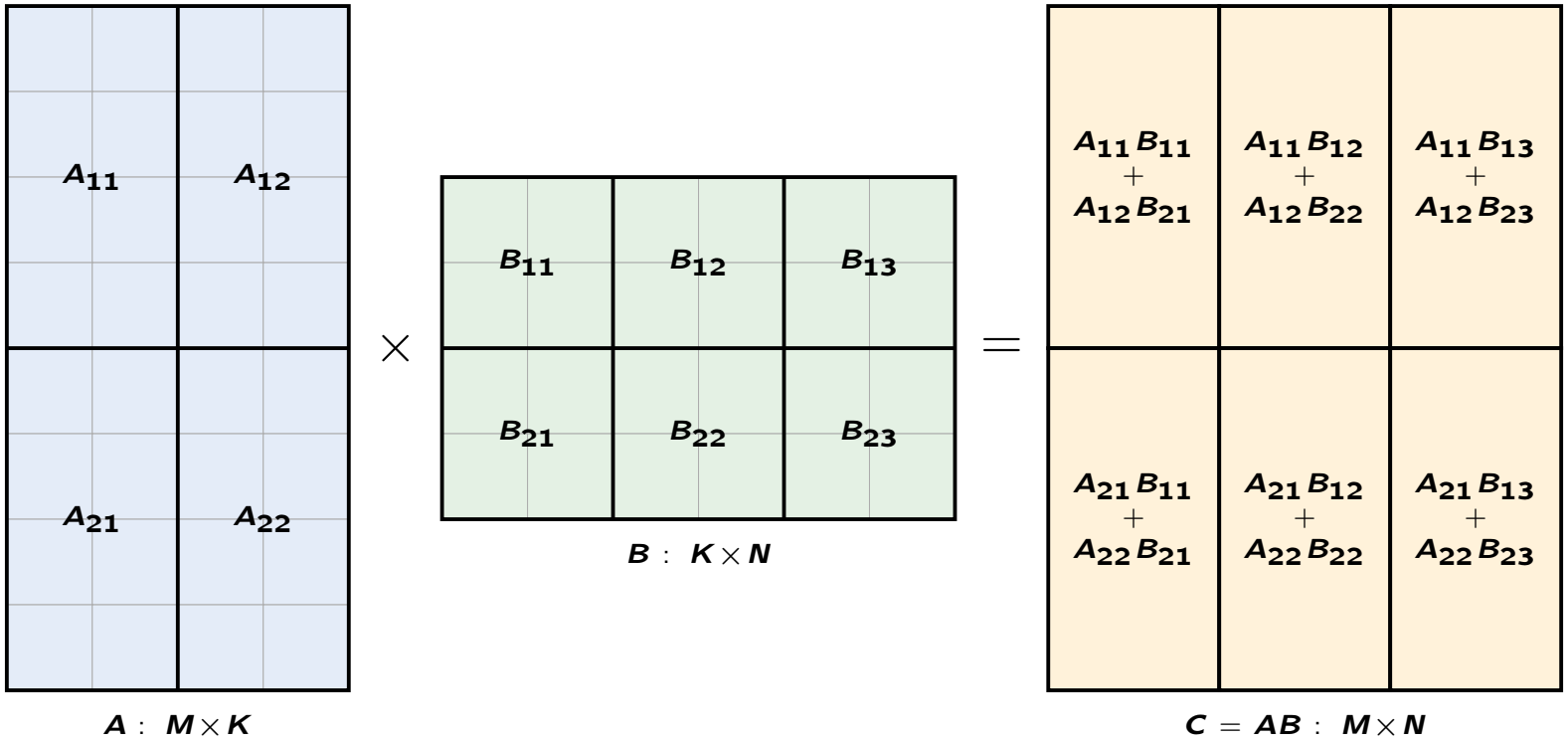
2D program grid: `program_id(0)` for rows, `program_id(1)` for columns.

Block Matrix Multiplication in Triton

For computing $S = QK^\top$ in tiles, we need blockwise matrix multiplication. Same formula as for scalars, but each entry is a sub-matrix:

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \end{bmatrix}, \quad C_{ij} = \sum_k A_{ik} B_{kj}$$

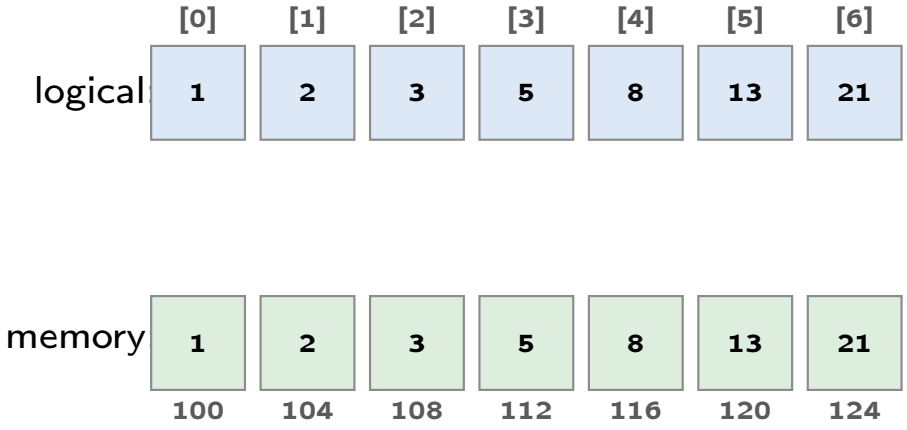
- Each Triton program computes one output block C_{ij}, iterating over the K dimension in chunks of `BLOCK_SIZE_K`



Tensor Memory Layout

Tensors are flat arrays in memory, described by **shape** and **strides**:

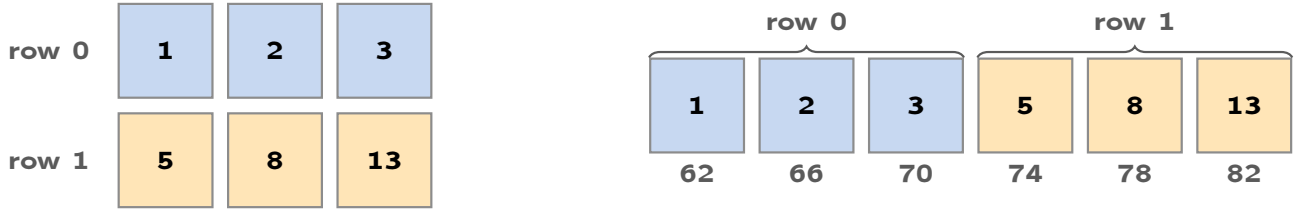
- **shape**: number of elements per dimension
- **stride**: number of elements to skip in memory to advance one step in that dimension



1D vector: shape $= [7]$

stride $= [1]$

logical (2×3) memory (row-major)



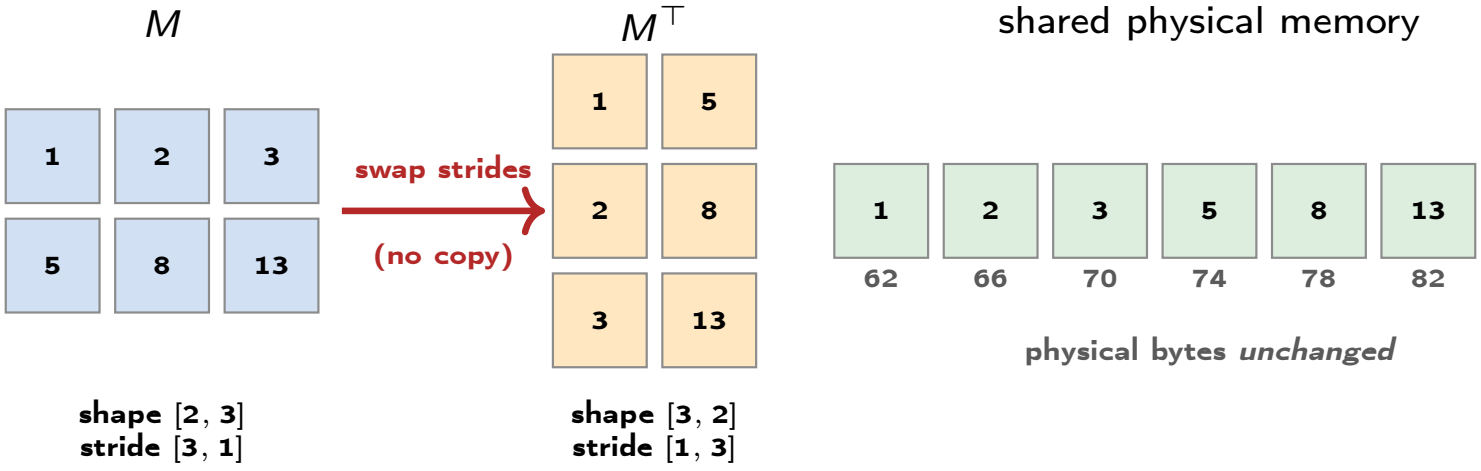
2D matrix: shape $= [2, 3]$

stride $= [3, 1]$

next row $\rightarrow +3$, next col $\rightarrow +1$

Transpose via Strides

Key trick: Transposing a tensor only swaps its *shape* and *strides*. The physical memory is untouched, so the operation costs nothing.



- Resulting view is non-contiguous: stride along the new fastest axis is no longer 1
- Flash Attention uses this to load $K^\top$ for free by swapping the last two strides of K

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
   - Motivation & GPU Memory Hierarchy
   - The Flash Attention Algorithm
   - GPU Programming: CUDA and Triton
   - Triton Implementation
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Forward Pass: Inner Loop

```
@triton.jit
def _attn_fwd_inner(O_block, l_i, m_i, Q_block, K_block_ptr, V_block_ptr,
                    block_index_q, softmax_scale, BLOCK_SIZE_Q: tl.constexpr,
                    BLOCK_SIZE_KV: tl.constexpr, STAGE: tl.constexpr,
                    offs_q: tl.constexpr, offs_kv: tl.constexpr,
                    SEQ_LEN: tl.constexpr):
    if STAGE == 1:        # Left of diagonal (causal)
        lo, hi = 0, block_index_q * BLOCK_SIZE_Q
    elif STAGE == 2:      # Diagonal block (causal)
        lo, hi = block_index_q * BLOCK_SIZE_Q, (block_index_q + 1) * BLOCK_SIZE_Q
    else:                 # Non-causal: full range
        lo, hi = 0, SEQ_LEN
    K_block_ptr = tl.advance(K_block_ptr, (0, lo))
    V_block_ptr = tl.advance(V_block_ptr, (lo, 0))
    for start_kv in range(lo, hi, BLOCK_SIZE_KV):
        K_block = tl.load(K_block_ptr)
        QK_block = tl.dot(Q_block, K_block)
        # ... apply mask for STAGE==2, scale, compute new max ...
        m_ij = tl.maximum(m_i, tl.max(QK_block, 1) * softmax_scale)
        QK_block = QK_block * softmax_scale - m_ij[:, None]
        P_block = tl.math.exp(QK_block)
        l_ij = tl.sum(P_block, 1)
        alpha = tl.math.exp(m_i - m_ij)   # correction factor
        l_i = l_i * alpha + l_ij
        V_block = tl.load(V_block_ptr)
        O_block = O_block * alpha[:, None] # rescale previous output
        O_block = tl.dot(P_block.to(tl.float16), V_block, O_block)
        m_i = m_ij
        V_block_ptr = tl.advance(V_block_ptr, (BLOCK_SIZE_KV, 0))
        K_block_ptr = tl.advance(K_block_ptr, (0, BLOCK_SIZE_KV))
    return O_block, l_i, m_i
```

Forward Pass: Main Kernel

```
@triton.jit
def _attn_fwd(Q, K, V, softmax_scale, M, O,
              stride_Q_batch, stride_Q_head, stride_Q_seq, stride_Q_dim,
              stride_K_batch, stride_K_head, stride_K_seq, stride_K_dim,
              stride_V_batch, stride_V_head, stride_V_seq, stride_V_dim,
              stride_O_batch, stride_O_head, stride_O_seq, stride_O_dim,
              BATCH_SIZE, NUM_HEADS: tl.constexpr,
              SEQ_LEN: tl.constexpr, HEAD_DIM: tl.constexpr,
              BLOCK_SIZE_Q: tl.constexpr, BLOCK_SIZE_KV: tl.constexpr,
              STAGE: tl.constexpr):
    block_index_q = tl.program_id(0)      # which Q block
    index_batch_head = tl.program_id(1)   # which batch x head
    # Set up block pointers for Q, K, V, O using tl.make_block_ptr
    # K is loaded transposed by swapping strides!
    Q_block = tl.load(Q_block_ptr)  # stays in SRAM
    m_i = tl.zeros([BLOCK_SIZE_Q], dtype=tl.float32) - float("inf")
    l_i = tl.zeros([BLOCK_SIZE_Q], dtype=tl.float32) + 1.0
    O_block = tl.zeros([BLOCK_SIZE_Q, HEAD_DIM], dtype=tl.float32)
    # Call _attn_fwd_inner for each stage
    # Epilogue: store logsumexp and normalized output
    m_i += tl.math.log(l_i)    # L = m + log(l) for backward
    O_block = O_block / l_i[:, None]
    tl.store(m_ptrs, m_i)
    tl.store(O_block_ptr, O_block.to(O.type.element_ty))
```

Grid layout: `(ceil(SEQ_LEN / BLOCK_Q), BATCH × HEADS, 1)`
Each program processes one Q-block for one head of one batch element.

Backward Pass: Inner Loop

```
@triton.jit
def _attn_bwd_inner(dQ_block, Q_block, K_block_ptr, V_block_ptr,
                    softmax_scale, L_i, D_i, dO_block,
                    BLOCK_SIZE_Q: tl.constexpr,
                    BLOCK_SIZE_KV: tl.constexpr,
                    SEQ_LEN: tl.constexpr):
    lo, hi = 0, SEQ_LEN
    K_block_ptr = tl.advance(K_block_ptr, (0, lo))
    V_block_ptr = tl.advance(V_block_ptr, (lo, 0))
    for start_kv in range(lo, hi, BLOCK_SIZE_KV):
        K_block = tl.load(K_block_ptr)
        V_block = tl.load(V_block_ptr)
        # Recompute P from Q, K and saved logsumexp L
        QK_block = tl.dot(Q_block, K_block)
        P_block = tl.math.exp(QK_block * softmax_scale - L_i[:, None])
        # dS = P * (dO @ V^T - D)
        dP_block = tl.dot(dO_block, tl.trans(V_block))
        dS_block = P_block * (dP_block - D_i[:, None])
        # Accumulate dQ
        dQ_block += tl.dot(dS_block.to(tl.float16), tl.trans(K_block))
        K_block_ptr = tl.advance(K_block_ptr, (0, BLOCK_SIZE_KV))
        V_block_ptr = tl.advance(V_block_ptr, (BLOCK_SIZE_KV, 0))
    return dQ_block
```

Backward Pass: Main Kernel

```
@triton.jit
def _attn_bwd(Q, K, V, softmax_scale, dO, dQ, dK, dV, M,
              SEQ_LEN: tl.constexpr, HEAD_DIM: tl.constexpr,
              BLOCK_SIZE_Q: tl.constexpr, BLOCK_SIZE_KV: tl.constexpr,
              ... ):  # stride arguments omitted
    # Phase 1: compute dQ (outer loop over Q blocks)
    block_index_q = tl.program_id(0)
    Q_block = tl.load(Q_block_ptr)
    dO_block = tl.load(dO_block_ptr)
    L_i = tl.load(M + offs_q)                  # saved logsumexp
    D_i = tl.sum(dO_block * tl.load(O_ptr), 1) # D = rowsum(dO * O)
    dQ_block = tl.zeros([BLOCK_SIZE_Q, HEAD_DIM], dtype=tl.float32)
    dQ_block = _attn_bwd_inner(dQ_block, Q_block, K_ptr, V_ptr,
                               softmax_scale, L_i, D_i, dO_block, ...)
    tl.store(dQ_block_ptr, dQ_block.to(tl.float16))
    # Phase 2: compute dK, dV (outer loop over KV blocks)
    block_index_kv = tl.program_id(0)
    K_block = tl.load(K_block_ptr)
    V_block = tl.load(V_block_ptr)
    dK_block = tl.zeros([BLOCK_SIZE_KV, HEAD_DIM], dtype=tl.float32)
    dV_block = tl.zeros([BLOCK_SIZE_KV, HEAD_DIM], dtype=tl.float32)
    for start_q in range(0, SEQ_LEN, BLOCK_SIZE_Q):
        Q_block = tl.load(Q_block_ptr)  # stream Q blocks
        dO_block = tl.load(dO_block_ptr)
        L_i = tl.load(M + offs_q)
        D_i = ...  # precomputed or recomputed
        QK_block = tl.dot(tl.trans(K_block), Q_block) # K^T @ Q
        P_block = tl.math.exp(QK_block * softmax_scale - L_i[None, :])
        dV_block += tl.dot(P_block, dO_block)         # P^T @ dO
        dP_block = tl.dot(V_block, tl.trans(dO_block)) # V @ dO^T
        dS_block = P_block * (dP_block - D_i[None, :])
        dK_block += tl.dot(dS_block, Q_block)          # dS^T @ Q
    tl.store(dK_block_ptr, dK_block.to(tl.float16))
    tl.store(dV_block_ptr, dV_block.to(tl.float16))
```

PyTorch Integration

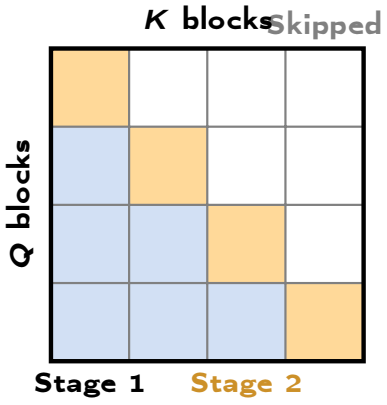
```
class TritonAttention(torch.autograd.Function):
    @staticmethod
    def forward(ctx, Q, K, V, causal, softmax_scale):
        BATCH_SIZE, NUM_HEADS, SEQ_LEN, HEAD_DIM = Q.shape
        O = torch.empty_like(Q)
        M = torch.empty((BATCH_SIZE, NUM_HEADS, SEQ_LEN),
                        device=Q.device, dtype=torch.float32)
        stage = 3 if causal else 1
        grid = lambda args: (triton.cdiv(SEQ_LEN, args["BLOCK_SIZE_Q"]),
                             BATCH_SIZE * NUM_HEADS, 1)
        _attn_fwd[grid](Q=Q, K=K, V=V, softmax_scale=softmax_scale,
            M=M, O=O, ..., STAGE=stage)  # strides omitted
        ctx.save_for_backward(Q, K, V, O, M)
        ctx.softmax_scale = softmax_scale
        ctx.causal = causal
        return O

    @staticmethod
    def backward(ctx, dO):
        Q, K, V, O, M = ctx.saved_tensors
        dQ = torch.empty_like(Q)
        dK = torch.empty_like(K)
        dV = torch.empty_like(V)
        grid = lambda args: (triton.cdiv(SEQ_LEN, args["BLOCK_SIZE_Q"]),
                             BATCH_SIZE * NUM_HEADS, 1)
        _attn_bwd[grid](Q=Q, K=K, V=V, softmax_scale=ctx.softmax_scale,
            dO=dO, dQ=dQ, dK=dK, dV=dV, M=M, ...)
        return dQ, dK, dV, None, None
```

Causal vs. Non-Causal Attention

In **causal** (autoregressive) attention, token i can only attend to tokens $j \leq i$.

Flash Attention handles this efficiently:

1. **Stage 1**: Blocks fully left of the diagonal $\rightarrow$ no masking needed
2. **Stage 2**: The diagonal block $\rightarrow$ apply causal mask



Blocks fully *right* of the diagonal are skipped entirely, saving computation.

For non-causal attention, all blocks are processed in a single stage.

Flash Attention: Summary

1. **Problem**: Standard attention is *memory-bound* due to materializing $T \times T$ matrices in HBM
2. **Solution**: Tile the computation, keep blocks in SRAM, use online softmax to avoid materializing the full attention matrix
3. **Streaming softmax**: Enables incremental softmax computation across blocks with a running max and sum
4. **Kernel fusion**: Single fused kernel for the entire forward pass. No intermediate HBM writes
5. **Backward pass**: Recompute attention weights from Q, K instead of storing them; use saved logsumexp L
6. **Result**: Exact attention (not an approximation!) with $O(T)$ memory and significantly faster wall-clock time

Impact

Flash Attention is now the default attention implementation in most deep learning frameworks and has enabled training with much longer context lengths. We will later discuss ring attention and tree attention for scaling to even larger contexts.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
   - Overview
   - RNN
   - LSTM
   - GRU
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
   - Overview
   - RNN
   - LSTM
   - GRU
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Sequence Models: Overview

Many tasks require processing **ordered sequences** of variable length:

- Natural language (words, subwords, characters)
- Time series (sensor data, finance, audio)
- Biological sequences (DNA, protein)

High-level idea: Accumulate information in a **fixed-size state vector** $h_t \in \mathbb{R}^n$ that summarizes everything seen so far. At each time step, update the state by combining it with the new input:

$$\boxed{h_t = f_\theta(h_{t-1}, x_t)}$$

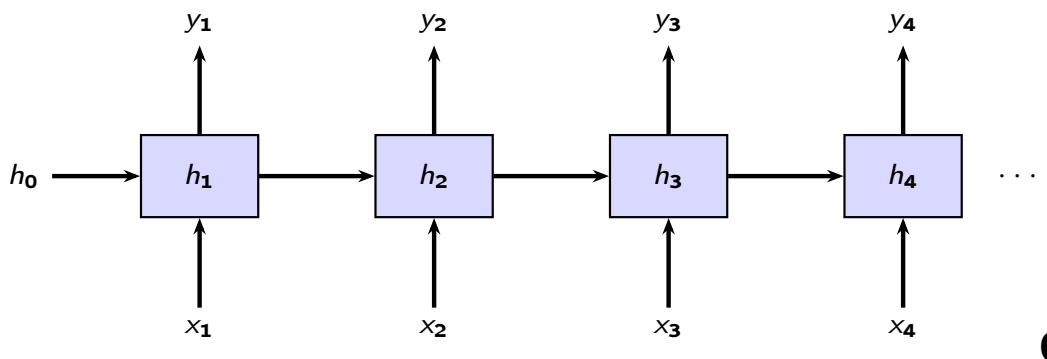
Key properties of recurrent state models

Constant memory: State size is fixed regardless of sequence length T

Causal by construction: Output at time t depends only on $x_1, \ldots, x_t$

Online / streaming: Can process inputs one token at a time

The Recurrent State Paradigm



The state h_t acts as a **lossy compression** of the input history $x_1, \ldots, x_t$ into a fixed-dimensional vector.

Central trade-off:

- Fixed state $\Rightarrow$ **constant cost** per step ($\mathcal{O}(1)$ memory and compute)
- But also an **information bottleneck**: must decide what to remember and what to forget

Challenges:

- How to learn *what* to store in h_t?
- How to avoid vanishing gradients during training?
- Can we parallelize the sequential computation?

Recurrent Model Families

Different architectures address these challenges in different ways:

	RNN	LSTM / GRU	SSM (Mamba)
Year	1990	1997 / 2014	2023
What to store	Learned implicitly	**Gating** mechanism	**Selective** mechanism
Gradient flow	Vanishing / exploding	Gated information highway	Structured recurrence
Training	Sequential	Sequential	**Parallelizable**
Inference / step	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$

Progression

Each generation addresses a key limitation of its predecessor:

- **LSTM/GRU:** solve vanishing gradients via gates that control information flow
- **SSMs (Mamba):** enable parallel training while keeping $\mathcal{O}(1)$ recurrent inference

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models
- Overview
- RNN
- LSTM
- GRU

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

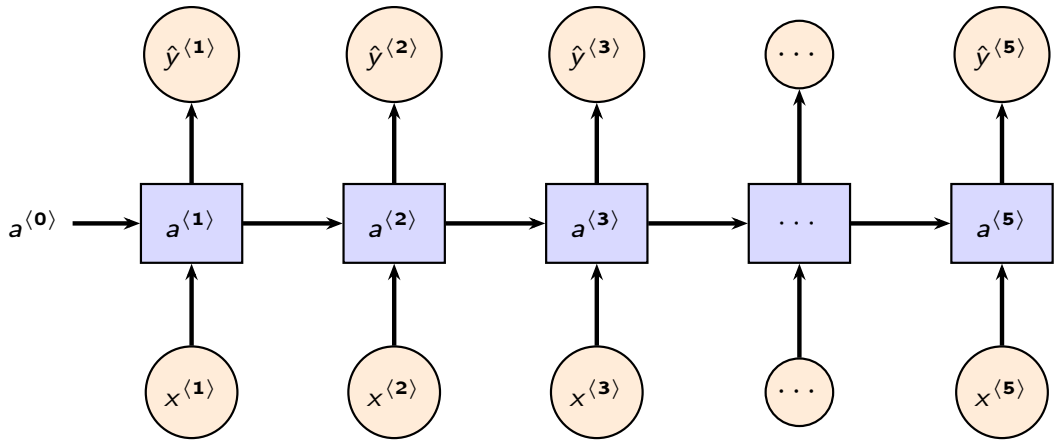
16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Recurrent Neural Networks (RNNs)

The simplest sequence model: process tokens one at a time, maintaining a hidden state:



Update equations:

$$\boxed{a^{\langle t \rangle} = g_1\left(W_{aa}\, a^{\langle t-1 \rangle} + W_{ax}\, x^{\langle t \rangle} + b_a\right)}$$

$$\boxed{y^{\langle t \rangle} = g_2\left(W_{ya}\, a^{\langle t \rangle} + b_y\right)}$$

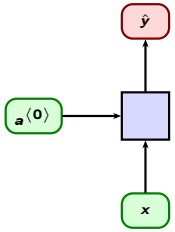
Parameters W_{aa}, W_{ax}, W_{ya}, b_a, b_y are **shared across time steps**.

g_1 is typically tanh; g_2 depends on the task (softmax for classification, linear for regression).

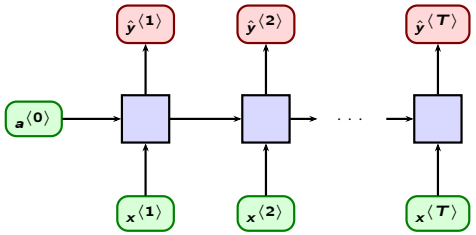
RNN Applications

RNNs support flexible input/output configurations:

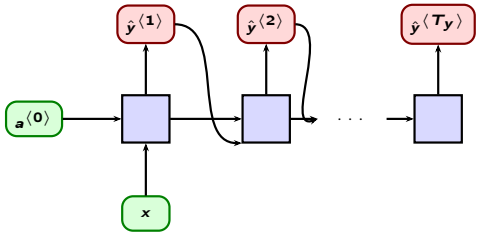
One-to-One – Classification, $T_x = T_y = 1$



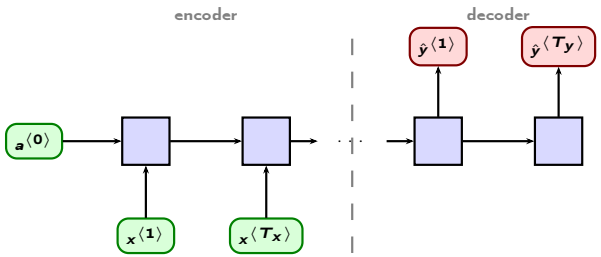
Many-to-Many $(T_x = T_y)$ – Seq. tagging



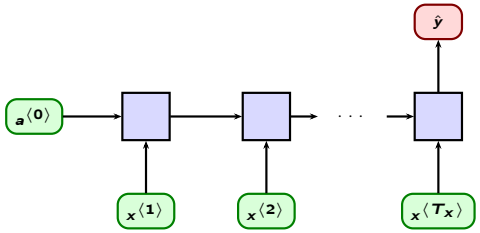
One-to-Many – Image captioning, $T_x = 1 < T_y$



Many-to-Many $(T_x \neq T_y)$ – Translation



Many-to-One – Sentiment, $T_x > T_y = 1$

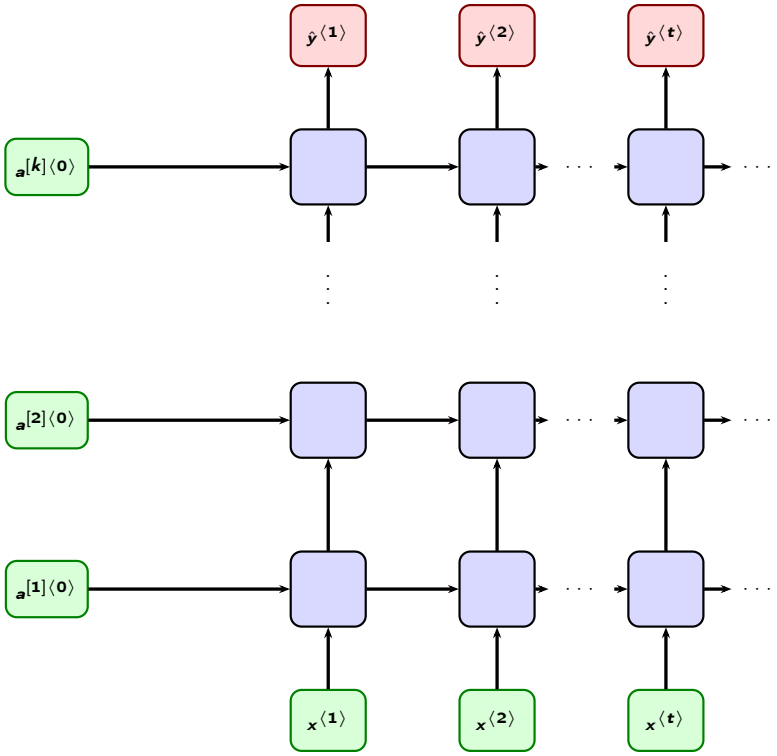


Deep RNNs

A single-layer RNN has limited capacity.

Solution: Stack multiple RNN layers, analogous to stacking Transformer blocks.

- Layer l receives $h^{[l-1]\langle t \rangle}$ as input
- More expressive representations
- Output from topmost layer



RNN: Advantages and Drawbacks

Advantages

- Can process input of *any length*
- Model size does not grow with input length
- Hidden state captures historical context
- Weights shared across time
- $\mathcal{O}(T)$ memory (vs. $\mathcal{O}(T^2)$ for attention)

Drawbacks

- **Slow training:** sequential computation, no parallelism across time
- **Long-range dependencies:** hidden state must compress all history, so information decays
- **Gradient problems:** vanishing/exploding gradients through many time steps
- Cannot natively look ahead (but bidirectional RNNs exist)

The Vanishing/Exploding Gradient Problem

During backpropagation through time (BPTT), gradients involve products of Jacobians:

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial W}, \qquad \frac{\partial a^{\langle t \rangle}}{\partial a^{\langle k \rangle}} = \prod_{i=k+1}^{t} \frac{\partial a^{\langle i \rangle}}{\partial a^{\langle i-1 \rangle}}$$

Each Jacobian factor is $\frac{\partial a^{\langle i \rangle}}{\partial a^{\langle i-1 \rangle}} = W_{aa}^{\top} \cdot \mathrm{diag}\big(g'(W_{aa}\, a^{\langle i-1 \rangle} + W_{ax}\, x^{\langle i \rangle} + b)\big)$, so

$$\left\| \frac{\partial a^{\langle t \rangle}}{\partial a^{\langle k \rangle}} \right\| = \left\| \prod_{i=k+1}^{t} W_{aa}^{\top} \mathrm{diag}(g'(\cdot)) \right\| \leq \big(\|W_{aa}\| \cdot \gamma\big)^{t-k} \quad \text{where } \gamma = \sup |g'|$$

- $\|W_{aa}\| \cdot \gamma > 1$: gradients **explode** exponentially in $t-k$
- $\|W_{aa}\| \cdot \gamma < 1$: gradients **vanish** exponentially in $t-k$

Remedies

Exploding gradients: Gradient clipping: rescale gradient if $\|\nabla\| > \theta$ (typically $\theta = 1$)

Vanishing gradients: Architectural solutions: **LSTM** and **GRU** with gating mechanisms

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models
- Overview
- RNN
- LSTM
- GRU

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

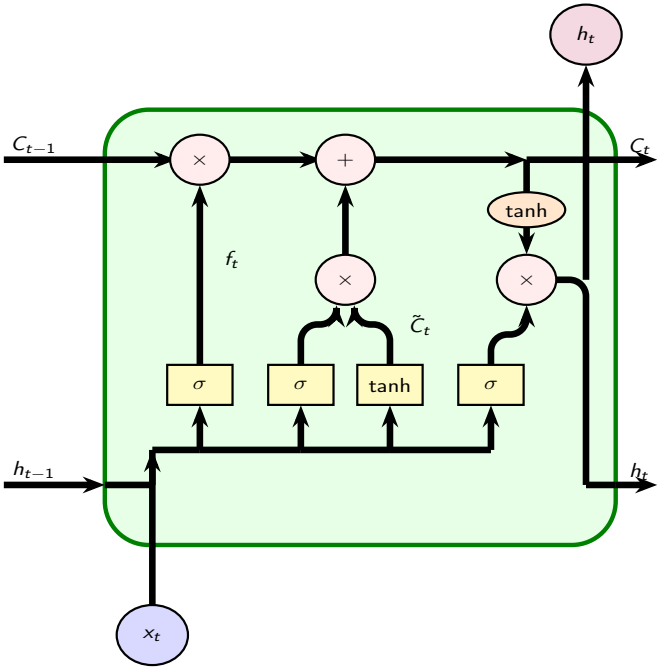
LSTM: Long Short-Term Memory

[Hochreiter and Schmidhuber, *"Long Short-Term Memory"*, 1997] introduced two key ideas to address vanishing gradients:

- **Gates** for selectively adding/removing information to/from cell state
- An **information highway** C_t (similar to residual connections) for unimpeded gradient flow

Two state vectors:

- C_t: Cell state (long-term memory, upper line)
- h_t: Hidden state (short-term output, lower line)



Each gate has the form $\sigma(W[h_{t-1}, x_t] + b) \in [0, 1]$, with its own learned W, b.

LSTM: Forget Gate

Forget gate f_t:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate i_t, candidate $\tilde{C}_t$:

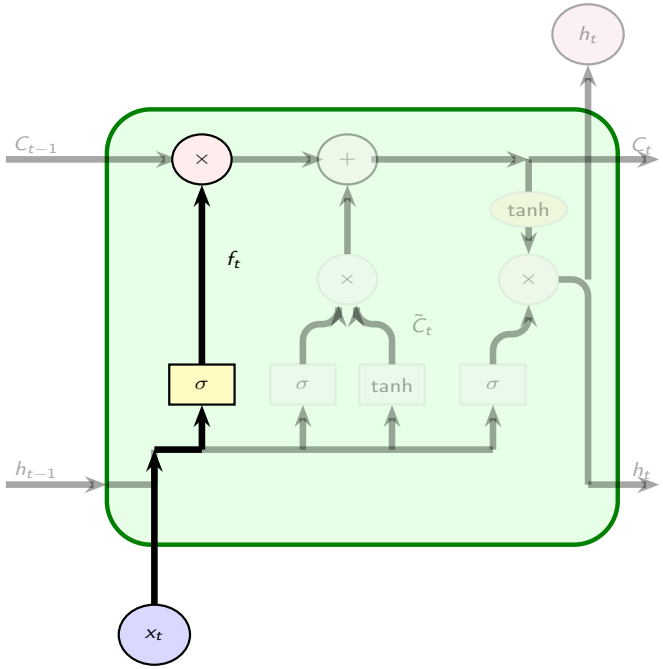
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell state update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(C_t)$$



LSTM: Input Gate

Forget gate f_t:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate i_t:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Candidate $\tilde{C}_t$:

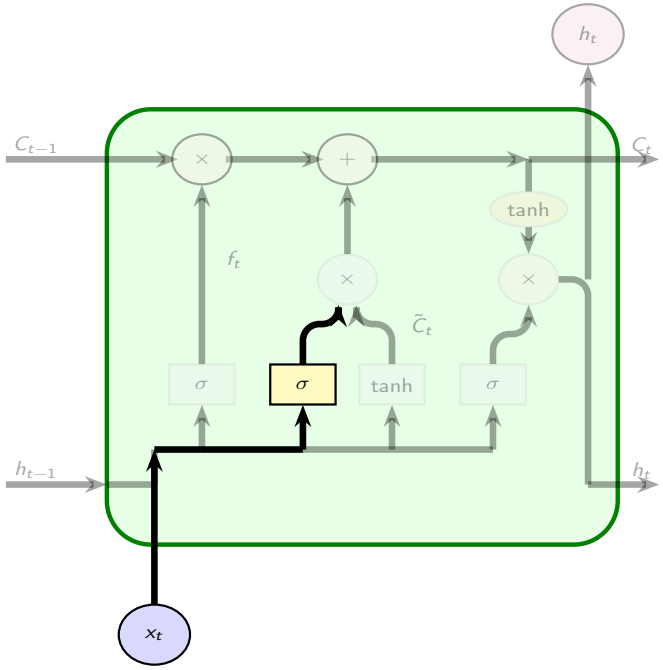
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell state update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(C_t)$$



LSTM: Candidate

Forget gate f_t:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate i_t:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

Candidate $\tilde{C}_t$:

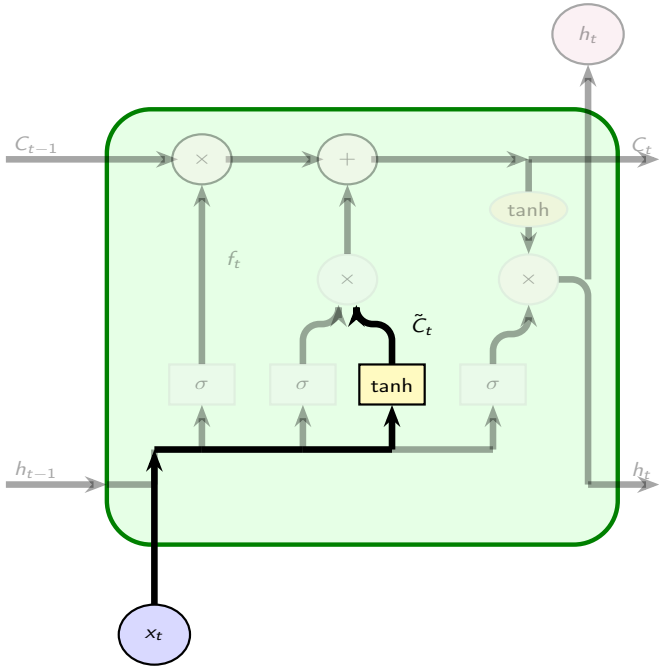
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell state update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(C_t)$$



LSTM: Cell State Update

Forget gate f_t:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate i_t, candidate $\tilde{C}_t$:

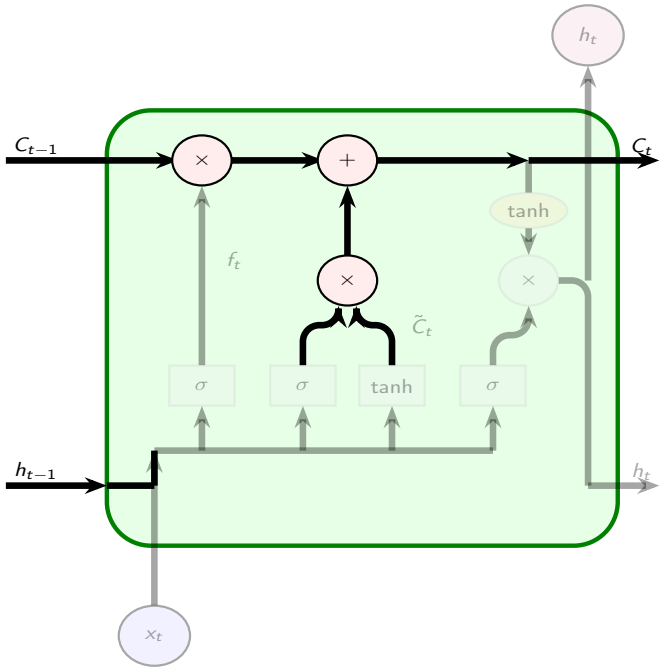
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell state update:

$$\boxed{C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t}$$

Output:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(C_t)$$



LSTM: Output Gate

Forget gate f_t:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Input gate i_t, candidate $\tilde{C}_t$:

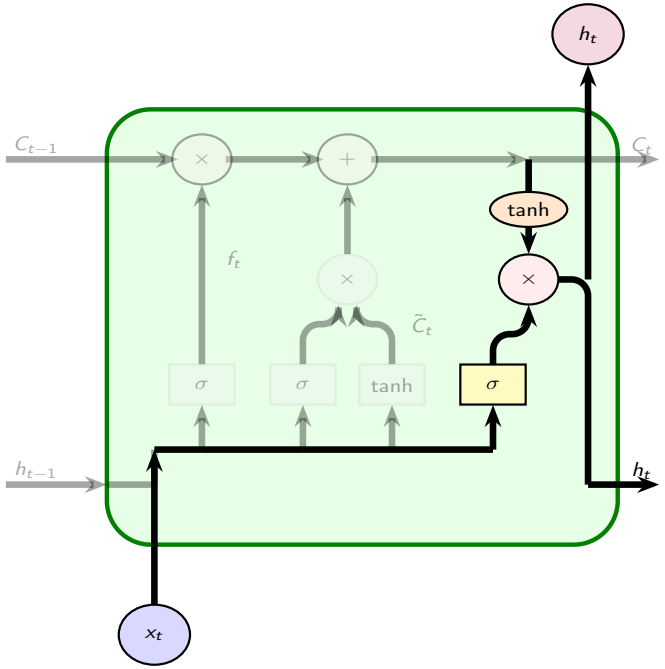
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Cell state update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output gate o_t:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(C_t)$$



LSTM: Summary of Gates

Gate	Equation	Purpose
Forget f_t	$\sigma(W_f[h_{t-1}, x_t] + b_f)$	Erase cell state features
Input i_t	$\sigma(W_i[h_{t-1}, x_t] + b_i)$	Select features to update
Candidate $\tilde{C}_t$	$\tanh(W_C[h_{t-1}, x_t] + b_C)$	Proposed new values
Output o_t	$\sigma(W_o[h_{t-1}, x_t] + b_o)$	Filter cell state for output
Cell update	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$	
Hidden state	$h_t = o_t \odot \tanh(C_t)$	

Why LSTM helps with vanishing gradients

$\frac{\partial C_t}{\partial C_{t-1}} = f_t$: when $f_t \approx 1$, gradients flow through unchanged (information highway), similar to residual connections.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models
- Overview
- RNN
- LSTM
- GRU

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

GRU: Gated Recurrent Unit

The GRU [Cho et al., *"Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation"*, 2014] simplifies the LSTM by merging cell and hidden state into a single state h_t, using only two gates:

Update gate z_t (combines forget + input):

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Reset gate r_t:

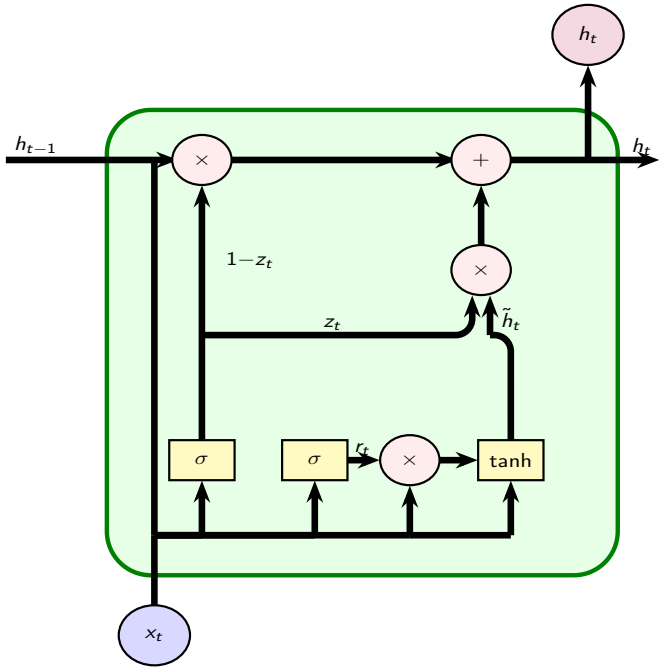
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

Hidden state update:

$$\boxed{h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t}$$



GRU: Update Gate

Update gate z_t (combines forget + input):

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Reset gate r_t:

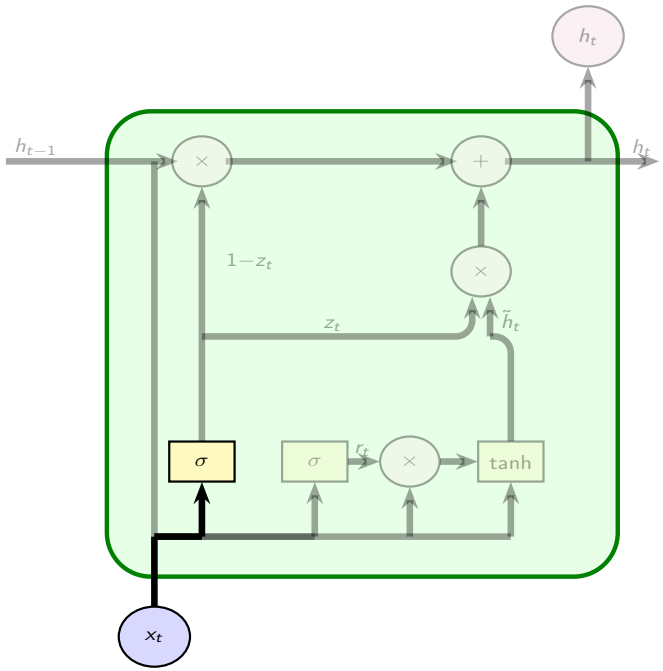
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

Hidden state update:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$



GRU: Reset Gate

Update gate z_t (combines forget + input):

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Reset gate r_t:

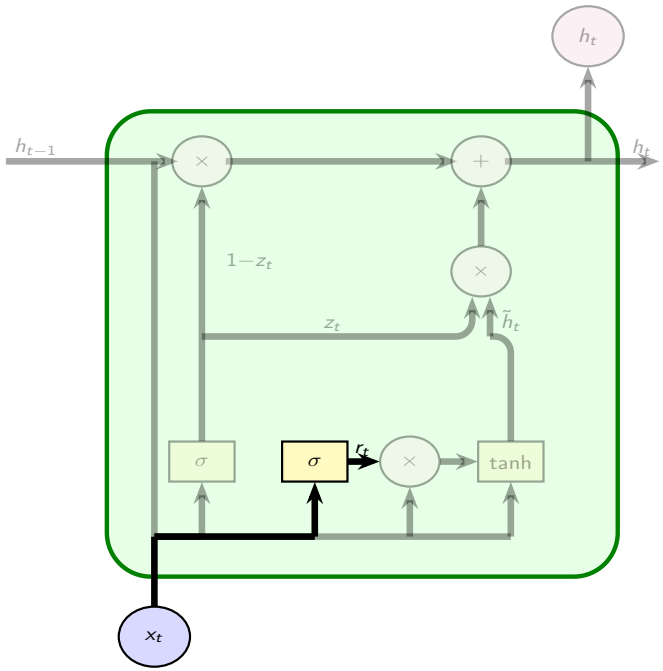
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

Hidden state update:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$



GRU: Candidate Hidden State

Update gate z_t (combines forget + input):

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Reset gate r_t:

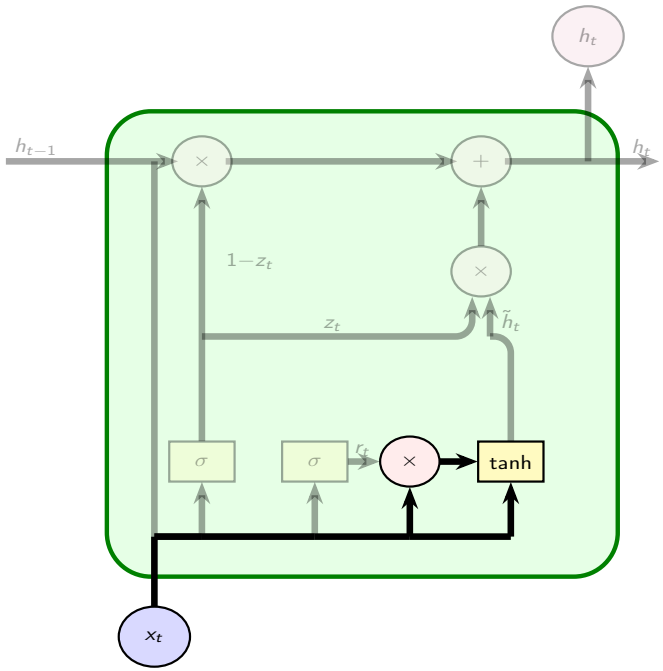
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

Hidden state update:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$



GRU: Hidden State Update

Update gate z_t (combines forget + input):

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Reset gate r_t:

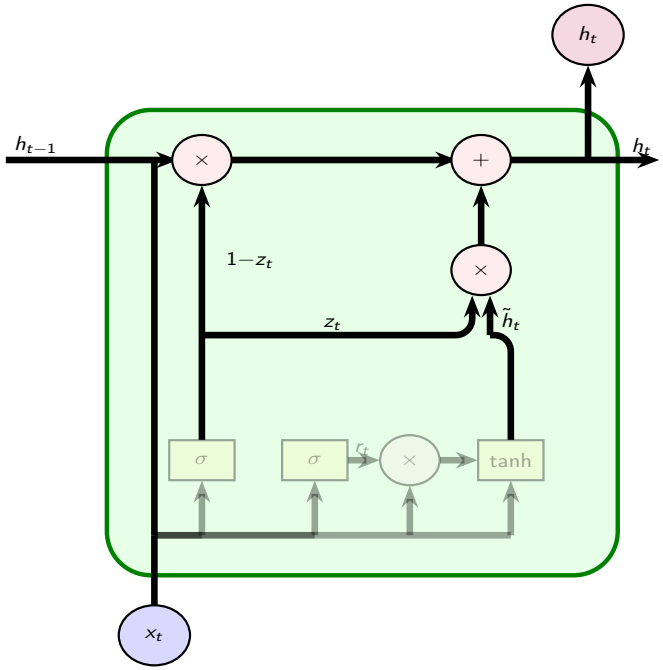
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

Hidden state update:

$$\boxed{h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t}$$



GRU vs. LSTM

	LSTM	GRU
States	2 (C_t, h_t)	1 (h_t)
Gates	3 (forget, input, output)	2 (update, reset)
Parameters	$4 \cdot (n_h^2 + n_h \cdot n_x + n_h)$	$3 \cdot (n_h^2 + n_h \cdot n_x + n_h)$

When to use which?

- GRU: fewer parameters $\Rightarrow$ faster training, good for smaller datasets
- LSTM: more expressive, often better on longer sequences and larger datasets
- In practice: try both; performance differences are often task-dependent

Historical note

LSTMs [Hochreiter and Schmidhuber, *"Long Short-Term Memory"*, 1997] (1997) and GRUs [Cho et al., *"Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation"*, 2014] (2014) were the dominant sequence models before Transformers [Vaswani et al., *"Attention Is All You Need"*, 2017] (2017). They remain relevant for streaming/real-time applications due to $\mathcal{O}(1)$ per-step computation.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
   - State Space Models
   - Mamba: Selective State Spaces
   - Mamba-2: State Space Duality
   - The Efficient SSD Algorithm
   - The Mamba-2 Architecture
   - Comparison and Results
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
   - State Space Models
   - Mamba: Selective State Spaces
   - Mamba-2: State Space Duality
   - The Efficient SSD Algorithm
   - The Mamba-2 Architecture
   - Comparison and Results
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Motivation: Recurrence vs. Attention

RNNs / LSTMs

- $O(1)$ memory per step, streaming
- Sequential $\Rightarrow$ not parallelizable during training

Transformers

- Fully parallel training
- $O(T^2)$ attention cost
- KV cache grows linearly at inference

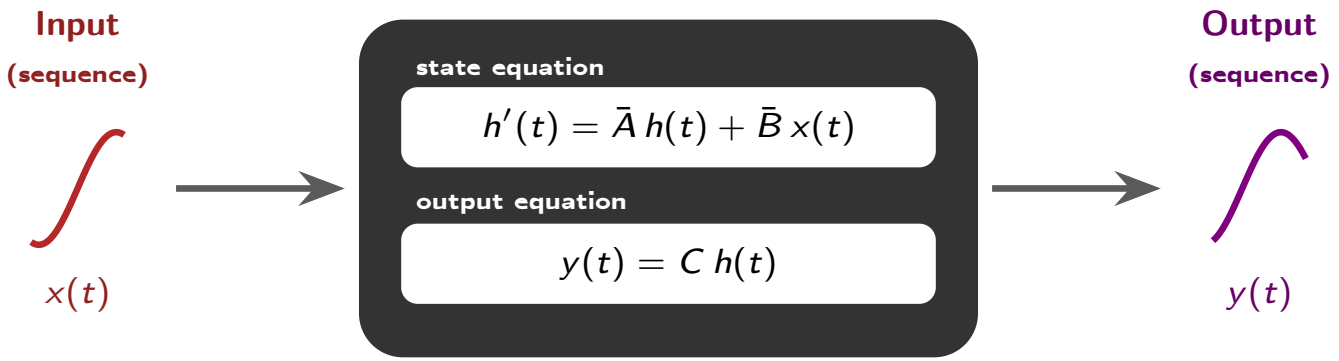
Can we get the best of both?

State Space Models aim to combine:

- Constant-size hidden state $\Rightarrow$ $O(1)$ inference per step (like RNNs)
- Parallel training via convolutional or attention-like view (like Transformers)

Continuous-Time State Space Models

SSMs originate from control theory. A continuous-time, linear system maps input $x(t) \in \mathbb{R}$ to output $y(t) \in \mathbb{R}$ via a latent state $h(t) \in \mathbb{R}^N$:



- $\bar{A} \in \mathbb{R}^{N \times N}$: state transition matrix
- $\bar{B} \in \mathbb{R}^{N \times 1}$: input projection
- $C \in \mathbb{R}^{1 \times N}$: output projection

Linear Time-Invariance (LTI)

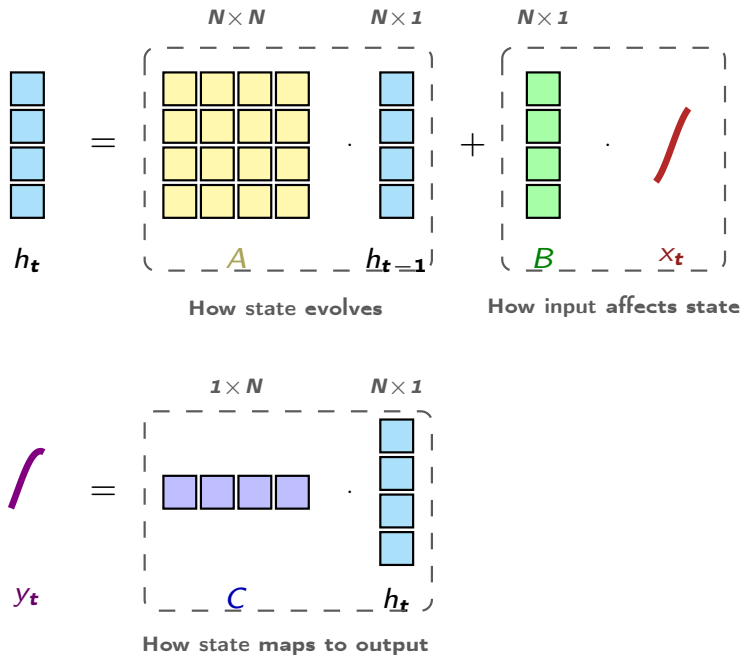
$(\bar{A}, \bar{B}, C)$ are *constant*: they do not depend on the input.

Discretization

To process a discrete sequence $x_0, x_1, \ldots, x_T$ with step size Δ, we discretize via the **zero-order hold** (ZOH):

$$A = \exp(\Delta \bar{A}), \qquad B = (\Delta \bar{A})^{-1}(\exp(\Delta \bar{A}) - I) \cdot \Delta \bar{B}$$

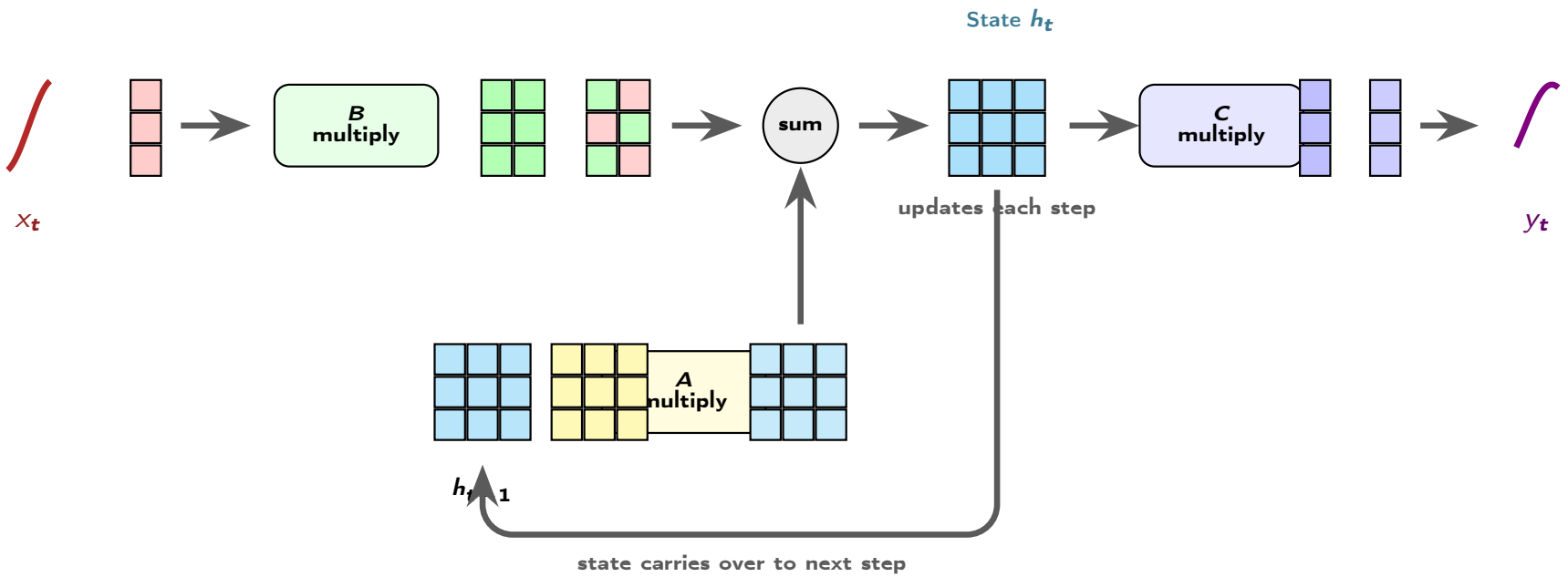
This gives the **discrete SSM recurrence**:



- Looks like a linear RNN, but with structured matrices A, B, C
- The step size Δ controls the resolution: how much continuous time each discrete step represents
- A large Δ focuses on coarse/slow features; a small Δ on fine/fast features

SSM: Computational Graph

One step of the recurrence $h_t = A\,h_{t-1} + B\,x_t, \quad y_t = C\,h_t$:



The Dual View: Recurrence vs. Convolution

For LTI systems (A, B, C constant), unrolling the recurrence yields:

$$y_t = CA^t B\, x_0 + CA^{t-1} B\, x_1 + \cdots + CB\, x_t = \sum_{k=0}^{t} \underbrace{CA^k B}_{\bar{K}_k}\, x_{t-k}$$

This is a **causal convolution** $y = \bar{K} * x$ with kernel $\bar{K} \in \mathbb{R}^T$.

Recurrent mode

- $O(N)$ per step
- Sequential: good for inference

Convolutional mode

- Compute $\bar{K}$, then use FFT
- $O(T \log T)$: good for training

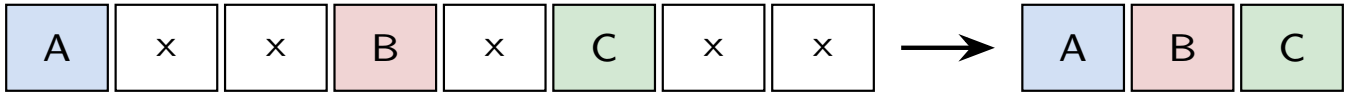
S4 [Gu, Goel, et al., *"Efficiently Modeling Long Sequences with Structured State Spaces"*, 2022]

Structured State Spaces for Sequences (S4) uses HiPPO initialization and diagonal structure for $\bar{A}$ to make this practical. First SSM to match Transformers on long-range benchmarks.

The Limitation of LTI Models

LTI SSMs apply the *same* convolution kernel to every input, regardless of content.

Example: Consider the task of selectively copying tokens from context.



- An LTI model treats every token identically. It cannot *selectively* remember some tokens and ignore others based on content
- A fixed kernel is fundamentally content-agnostic
- Transformers solve this via content-dependent attention weights

Insight

To make SSMs truly powerful, the parameters must be **input-dependent**. This breaks LTI and the convolutional mode, but enables *selectivity*.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
   - State Space Models
   - Mamba: Selective State Spaces
   - Mamba-2: State Space Duality
   - The Efficient SSD Algorithm
   - The Mamba-2 Architecture
   - Comparison and Results
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Selectivity: The Core Idea of Mamba [Gu and Dao, *"Mamba: Linear-Time Sequence Modeling with Selective State Spaces"*, 2024]

Mamba makes the SSM parameters *functions of the input*:

$$B_t = \text{Linear}_B(x_t), \quad C_t = \text{Linear}_C(x_t), \quad \Delta_t = \text{softplus}(\text{Linear}_\Delta(x_t))$$

The discrete recurrence becomes **time-varying**:

$$\boxed{h_t = A_t\, h_{t-1} + B_t\, x_t, \qquad y_t = C_t^\top h_t}$$

where $A_t = \exp(\Delta_t \cdot \bar{A})$ and B_t, C_t depend on x_t.

Intuition for Δ_t (step size):

- **Large Δ_t**: Focus on current input, reset state $\approx$ "remember this token"
- **Small Δ_t**: Ignore current input, preserve state $\approx$ "skip this token"

The model learns *when* to update its state. This is the selection mechanism.

The Hardware Challenge

Making parameters input-dependent breaks the convolutional mode:

- Time-varying $(A_t, B_t, C_t) \Rightarrow$ no fixed convolution kernel
- The recurrence must be computed sequentially $\Rightarrow$ back to the RNN problem?

The problem is not FLOPs; it's memory access.
Modern GPUs have:

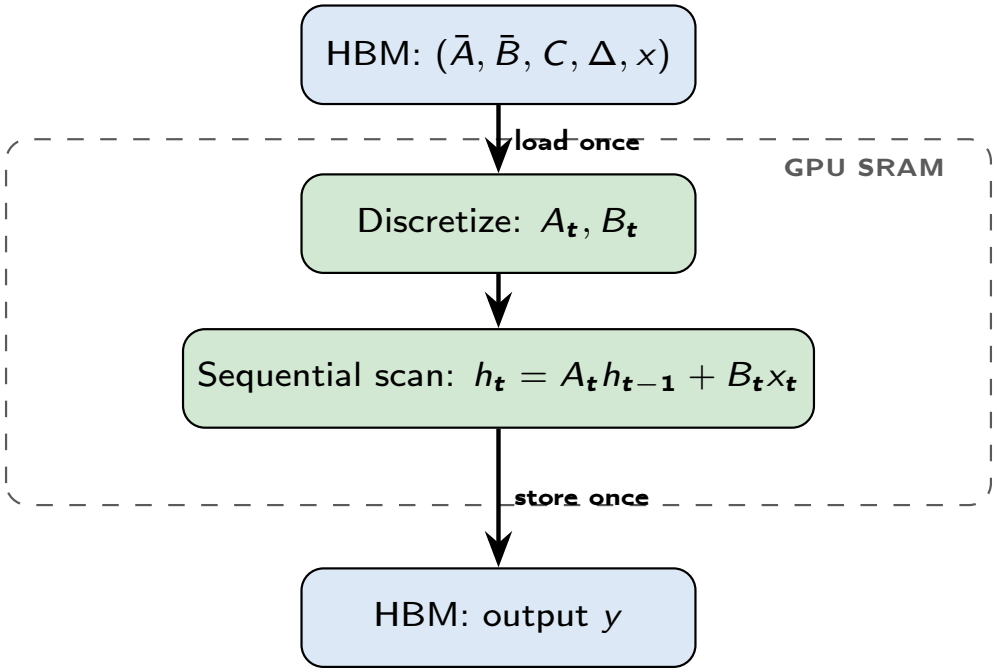
	SRAM (on-chip)	**HBM (off-chip)**
Size	~20 MB	~80 GB
Bandwidth	~19 TB/s	~3 TB/s

A naive scan reads/writes the full state from HBM at every step. Mamba's selective scan is *memory-bound*, not compute-bound.

Solution: Hardware-aware selective scan

Fuse the entire recurrence into a single GPU kernel that keeps the state in **SRAM**, avoiding repeated HBM round-trips. Same idea as FlashAttention [Dao, Fu, et al., *"FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness"*, 2022], applied to scans.

Hardware-Aware Selective Scan



- Discretization + scan + output projection all happen **in SRAM**
- Intermediate states h_t never written to HBM. Only the final output y
- Recomputation in backward pass instead of storing activations

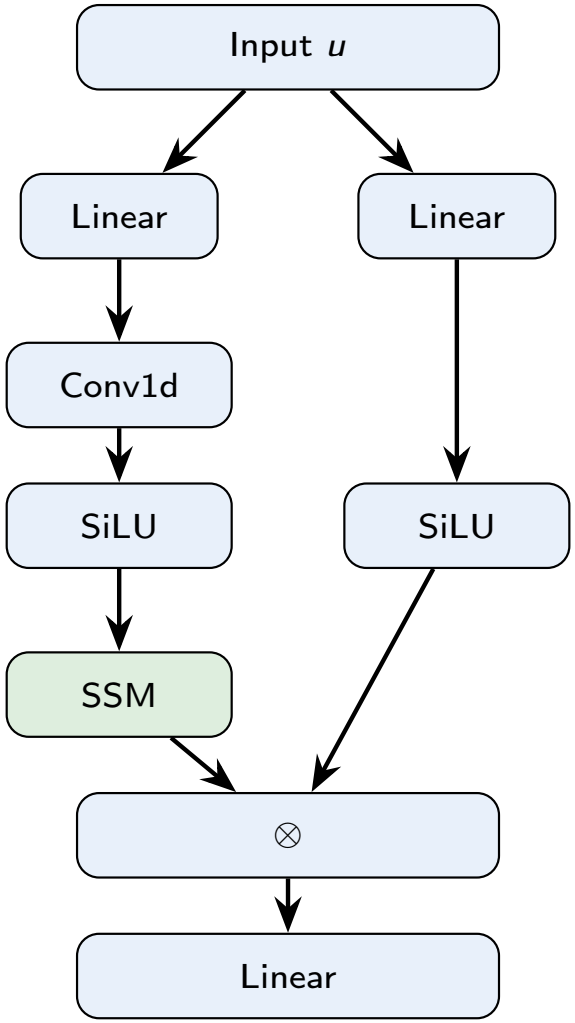
Mamba-1 Architecture

Each Mamba layer processes tokens via:

1. **Two parallel branches** from input projection:
   - Branch A: Linear $\rightarrow$ Conv1d $\rightarrow$ SiLU $\rightarrow$ SSM
   - Branch B: Linear $\rightarrow$ SiLU (gate)
2. SSM branch produces B_t, C_t, Δ_t from the activated input
3. Selective scan computes output
4. **Gated output**: multiply SSM output with Branch B, then project

Key design choices:

- A is diagonal with N elements per channel
- $P = 1$: each channel has its own SSM (no shared heads)
- Typical: $N = 16$, d_{model} up to 2048+



Mamba-1: Results

Language modeling (trained on the Pile, Chinchilla scaling [Hoffmann et al., *"Training Compute-Optimal Large Language Models"*, 2022]):

- Matches Transformer quality at all scales (125M to 1.4B parameters)
- **5× higher throughput** than Transformer at inference (long sequences)
- Linear scaling in sequence length vs. quadratic for attention

Long-range benchmarks:

- State-of-the-art on Long Range Arena, speech, genomics
- Scales to sequences of 1M+ tokens at inference

Remaining issues

- Training uses a **sequential scan**, no matrix multiplications
- GPUs/TPUs are optimized for matmuls, not sequential operations
- Despite kernel fusion, training throughput is lower than Transformers
- Each channel has an independent SSM ($P = 1$), limited parameter sharing

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
   - State Space Models
   - Mamba: Selective State Spaces
   - Mamba-2: State Space Duality
   - The Efficient SSD Algorithm
   - The Mamba-2 Architecture
   - Comparison and Results
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Two Problems with Mamba-1

Problem 1: Understanding

- State space models and attention mechanisms look very different
- Are they fundamentally different, or related?
- Can we unify them in a single framework?

Problem 2: Efficiency

- Despite optimization efforts, Mamba-1 training remains less hardware-efficient than attention
- SSMs lack native support for matrix multiplications
- GPUs/TPUs are built to do matmuls fast

The SSD Framework [Dao and Gu, *"Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality"*, 2024]

Both problems are solved together: by revealing that a scalar-structured SSM *is* a form of attention, we unlock matmul-based hardware-efficient training.

Restricting A: Scalar-Times-Identity

The one structural change in Mamba-2:

Structure	Shape of A	Description
General	(T, N, N)	Unrestricted matrix
Diagonal	(T, N)	Mamba-1
Scalar $\times$ Identity	(T)	**Mamba-2 / SSD**

By forcing all diagonal elements of A_t to be the *same* scalar a_t:

$$A_t = a_t \cdot I_N, \qquad a_t \in \mathbb{R}$$

- Fewer parameters per step, but enables a fundamentally new algorithm
- Opens the door to the **quadratic (attention) mode**

Multihead SSMs

For real language models, inputs have $\mathtt{P}$ channels: $X \in \mathbb{R}^{(T,P)}$

$$Y^{(T,P)} = \mathrm{SSM}\left(A^{(T)},\, B^{(T,N)},\, C^{(T,N)}\right)\left(X^{(T,P)}\right)$$

Multihead structure (like Transformers):

- Heads of dimension $\mathtt{P} \in \{64, 128\}$ (vs. $\mathtt{P} = 1$ in Mamba-1)
- Within a head, the *same* (a_t, B_t, C_t) are applied to all $\mathtt{P}$ channels
- Across heads, (a, B, C) are independent

Key advantage over Mamba-1

Larger state dimensions become tractable: $\mathtt{N} = 16$ (Mamba-1) $\rightarrow \mathtt{N} \in [64, 256]$ (Mamba-2), while *simultaneously* training faster.

Unrolling the SSM: The Attention Mode

With scalar $A_t = a_t$, unroll the recurrence over the full sequence. Define:

$$a^{\times}_{i:j} \;=\; a_i \cdot a_{i-1} \cdots a_{j+1} \qquad \text{(cumulative product of decays)}$$

The output at time i becomes:

$$y_i = \sum_{j \le i} C_i^{\top} \left(a^{\times}_{i:j} \right) B_j \, x_j$$

Stacking all y_i gives a matrix equation $Y = MX$ where:

$$M = L \circ CB^{\top} \in \mathbb{R}^{(T,T)}$$

L is the lower-triangular **decay matrix**:

$$L_{ij} = \begin{cases} a^{\times}_{i:j} & i \ge j \\ 0 & i < j \end{cases}$$

The Decay Matrix L

$$L = \begin{bmatrix} 1 & & & & \\ a_1 & 1 & & & \\ a_2 a_1 & a_2 & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ a_{T-1} \cdots a_1 & a_{T-1} \cdots a_2 & \cdots & a_{T-1} & 1 \end{bmatrix}$$

Special case: $a_t = 1$ for all t:

$$L = \text{lower-triangular causal mask}$$

and the model becomes *causal linear attention*:

$$Y = (L \circ QK^\top)\, V$$

where $(C, B, X) \leftrightarrow (Q, K, V)$.

Interpretation of a_t

The scalar decays $a_t \in (0, 1)$ act as **input-dependent relative positional encodings**: they control how much past information is retained at each step.

State Space Duality: Same Model, Two Views

$$(A^{(T)},\ B^{(T,N)},\ C^{(T,N)},\ X^{(T,P)}) \mapsto Y^{(T,P)}$$

SSM (Linear) Mode

$$h_t = a_t h_{t-1} + B_t x_t, \quad y_t = C_t^\top h_t$$

- Constant-size state $\mathtt{N}$
- $O(T)$ computation
- No matmuls $\rightarrow$ slow on hardware
- Efficient at *inference*

Attention (Quadratic) Mode

$$Y = (L \circ CB^\top)\, X$$

- Materializes $T \times T$ matrix
- $O(T^2 N)$ computation
- Pure matmuls $\rightarrow$ fast on hardware
- Efficient at *training* (small T)

The duality

These are **mathematically identical**. The same model can be computed either way.

SSD vs. Standard Attention: Two Differences

	Standard Attention	**SSD**
Sequence mixing matrix	$(L_{\text{causal}} \circ \text{softmax}(QK^\top))\, V$	$(L \circ CB^\top)\, X$
Softmax	✓	×
Multiplicative mask L	causal only ($a_t = 1$)	✓(input-dependent a_t)
State size at inference	$O(T)$ (KV cache)	$O(N)$ **(constant!)**
Training complexity	$O(T^2 N)$	$O(TN^2)$

- Removing softmax: state stays *constant* size, inference is $O(1)$ per step
- The decay mask L is what distinguishes SSD from linear attention and encodes *selectivity*

Efficiency Comparison

	Attention	**SSM**	**SSD**
State size	T	N	N
Training FLOPs	T^2N	TN^2	TN^2
Inference FLOPs	TN	N^2	N^2
Naive memory	T^2	TN^2	TN
Matrix multiplications	✓	×	✓

The best of both worlds

SSD achieves **constant inference state** (like SSMs) and **matrix multiply-based training** (like attention) simultaneously.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
   - State Space Models
   - Mamba: Selective State Spaces
   - Mamba-2: State Space Duality
   - The Efficient SSD Algorithm
   - The Mamba-2 Architecture
   - Comparison and Results
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

The Chunked SSD Algorithm

Neither pure mode is ideal for training on long sequences. The solution: **chunk the sequence**.

Split T tokens into $Q = T/C$ chunks of size C:

1. **Intra-chunk** (quadratic/attention mode):
   - Within each chunk, compute $Y_{\text{intra}} = (L \circ CB^\top)X$ via matmul
   - Cost: $O(C^2 \cdot P)$ per chunk, parallelizable across batch and heads
2. **Inter-chunk** (SSM/linear mode):
   - Compute a *summary state* h at the end of each chunk
   - Pass states across chunk boundaries sequentially (Q steps, not T)
   - For each chunk, add the contribution of all past states: Y_{inter}
3. **Combine:** $Y = Y_{\text{intra}} + Y_{\text{inter}} + D \odot X$

Overall cost: $O\left(\frac{T}{C} \cdot C^2 N\right) = O(TCN)$, optimal for $C = \sqrt{T}$.

Chunked Algorithm: Diagram

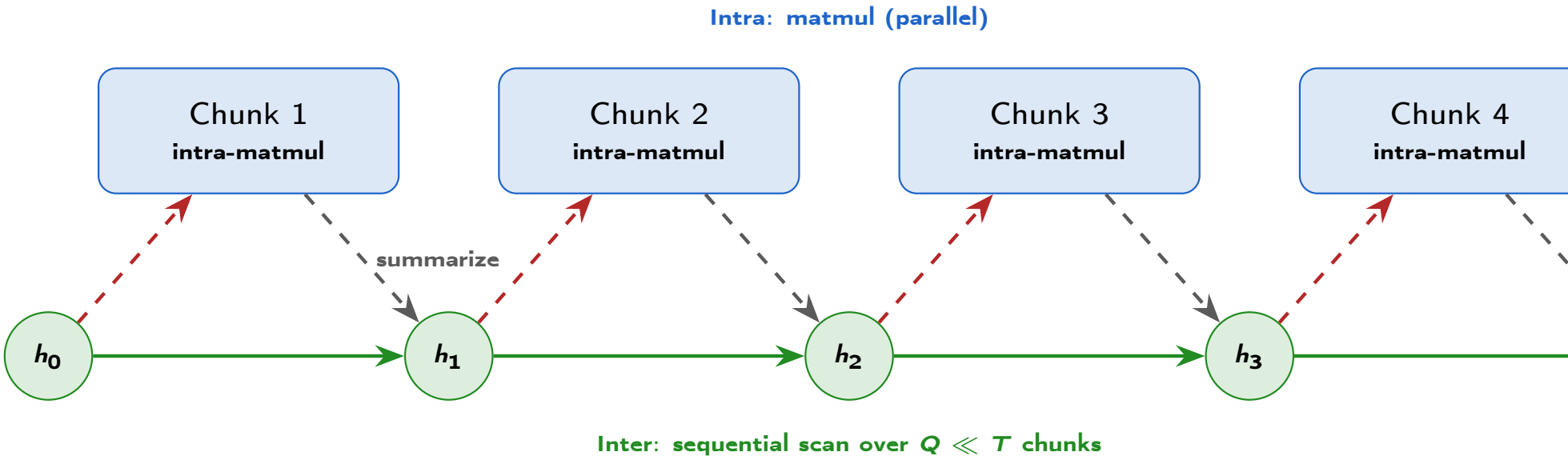


Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
   - State Space Models
   - Mamba: Selective State Spaces
   - Mamba-2: State Space Duality
   - The Efficient SSD Algorithm
   - The Mamba-2 Architecture
   - Comparison and Results
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Mamba-2 Architecture Overview

Key architectural change from Mamba-1:

(A, B, C) are produced *in parallel* with input X, not sequentially.

Per-layer computation:

1. Linear projection: $[Z, X, B, C, \Delta] = \text{Linear}(u)$
2. Conv1d (causal, depthwise) on X
3. $A \leftarrow -\exp(\texttt{A_log})$, $\Delta \leftarrow \text{softplus}(\Delta + \texttt{dt_bias})$
4. SSD block: $Y = \text{SSD}(A \cdot \Delta,\ B,\ C,\ X \cdot \Delta)$
5. Output: $\text{out_proj}(\text{RMSNorm}(Y) \cdot \text{SiLU}(Z))$

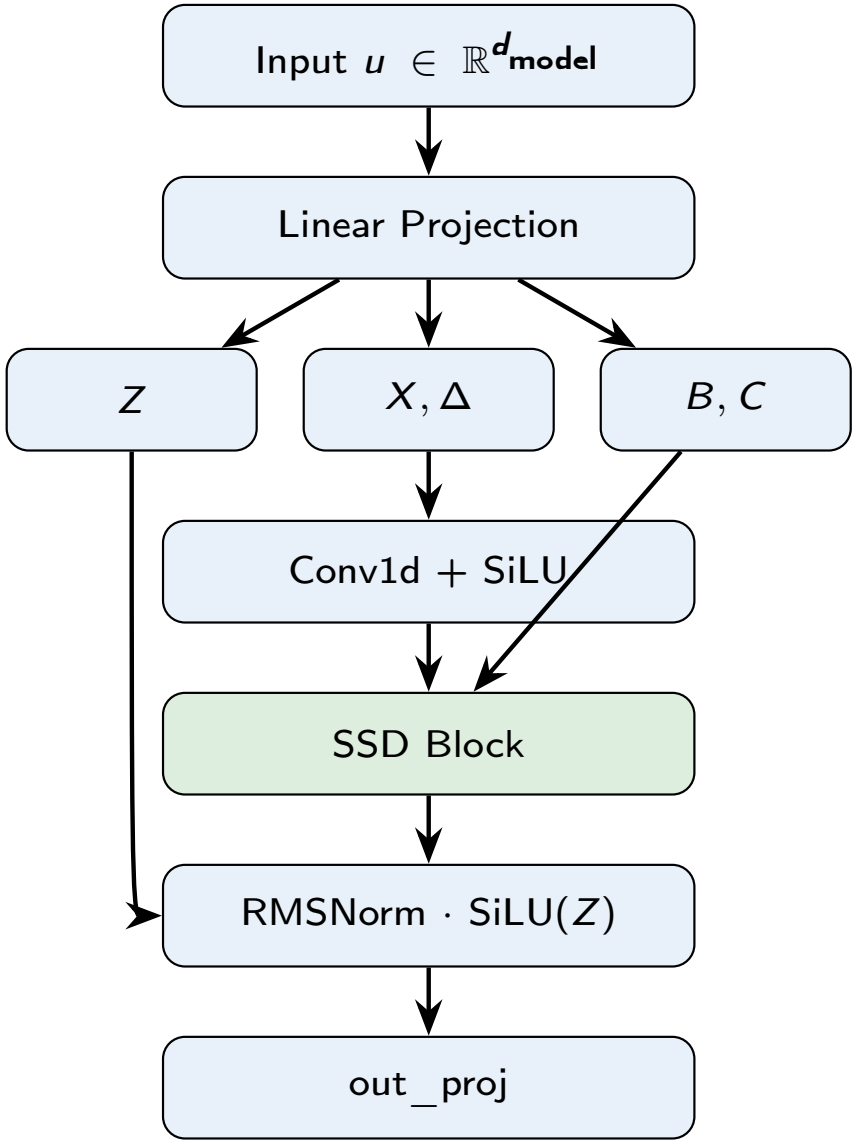


Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
   - State Space Models
   - Mamba: Selective State Spaces
   - Mamba-2: State Space Duality
   - The Efficient SSD Algorithm
   - The Mamba-2 Architecture
   - Comparison and Results
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Mamba-2 vs. Mamba-1: Key Differences

	Mamba-1	**Mamba-2**
Structure on A	Diagonal	Scalar-times-identity
Head dimension P	1 (per channel)	64–128
State dimension N	16	64–256+
(A, B, C) generation	Sequential	Parallel with X
Training algorithm	Selective scan	Chunked SSD (matmuls)
Tensor parallelism	Harder	Native support
Training speed	Slower	**2–8× faster**
Inference	Faster (smaller N)	Slightly slower

Note on inference

Mamba-1 may still be preferable for pure inference workloads due to its smaller state dimension. The larger N in Mamba-2 costs more per recurrence step.

Language Modeling: Scaling Results

Pile dataset (Chinchilla scaling laws [Hoffmann et al., *"Training Compute-Optimal Large Language Models"*, 2022]):

- Mamba-2 matches or slightly exceeds Mamba-1 at all scales
- Competitive with Transformer++ baseline
- Trained at 2–8× higher throughput

Multi-Query Associative Recall (MQAR):

- Synthetic task requiring long-range memory
- Mamba-2 substantially outperforms Mamba-1
- Advantage holds even at *equal* state size N
- Larger N (up to 16×) amplifies the gap

Why MQAR matters

MQAR tests whether a model can store and retrieve key-value pairs from context. Larger state N directly increases capacity. Mamba-2's jump from $N = 16 \rightarrow N = 256$ is crucial here.

Summary: State Space Models and Mamba

1. **State Space Models**: Linear systems discretized from continuous time; dual recurrent/convolutional view enables parallel training for LTI models
2. **Mamba-1**: Makes SSM parameters *input-dependent* (selective); hardware-aware scan keeps computation in SRAM; matches Transformer quality with linear-time inference
3. **State Space Duality**: Scalar-structured SSMs and causal linear attention are the *same* model viewed differently
4. **Mamba-2**: Chunked SSD algorithm: $O(TCN)$ compute, constant inference state, full matmul utilization; 2–8$\times$ faster training than Mamba-1

References

[Gu, Goel, et al., *"Efficiently Modeling Long Sequences with Structured State Spaces"*, 2022]
[Gu and Dao, *"Mamba: Linear-Time Sequence Modeling with Selective State Spaces"*, 2024]
[Dao and Gu, *"Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality"*, 2024]

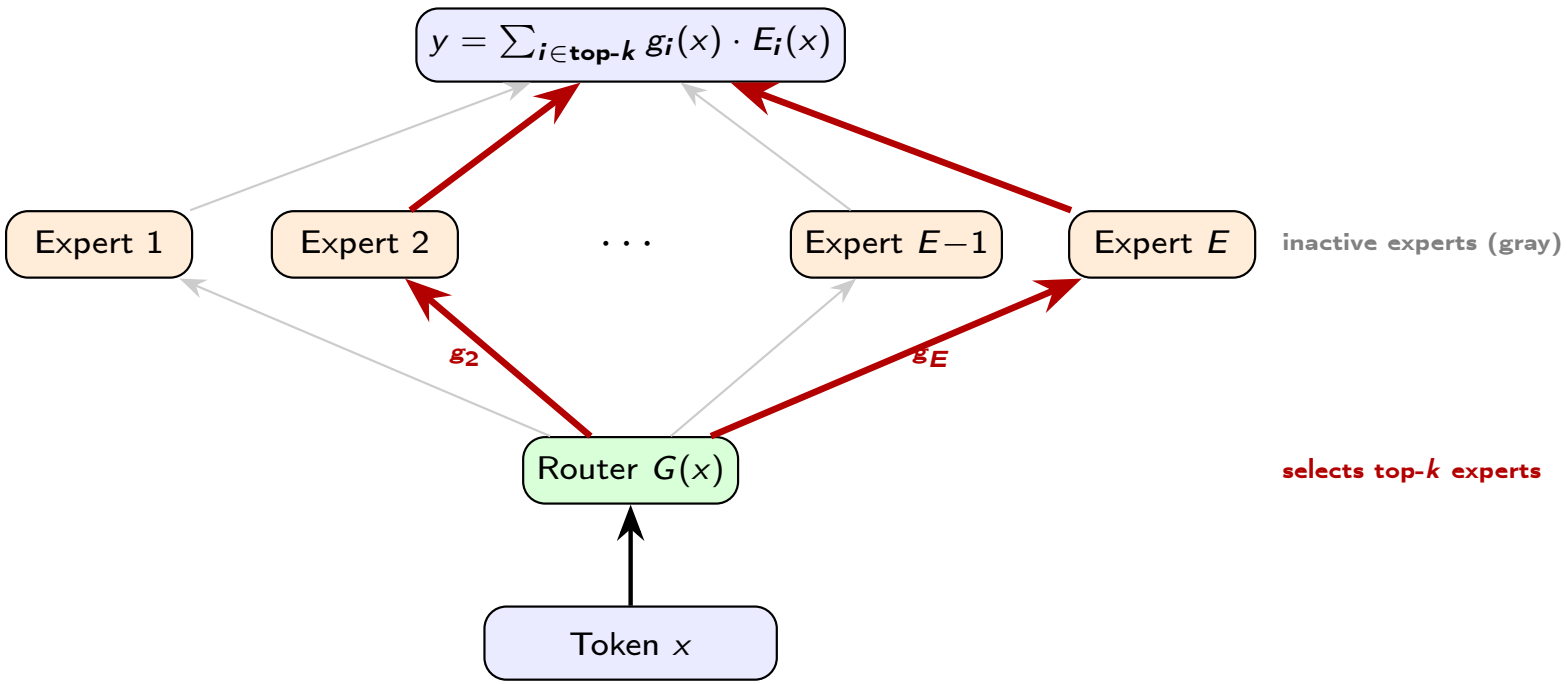
Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transf
   - Summary
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - Summary
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

MoE: High-Level Idea



- Replace the dense FFN in each Transformer block with a **sparse MoE layer**
- Maintain E expert sub-networks, each a standard FFN
- A **router** (gating network) selects the top-k experts per token
- Output: weighted combination $y = \sum_{i \in \text{top-}k} g_i(x) \cdot E_i(x)$

Why Mixture of Experts?

Dense models: every parameter is used for every input token.

- Scaling parameters $\Rightarrow$ scaling FLOPs proportionally
- Training cost grows with model size
- Inference cost grows with model size

Key insight: Not all tokens need the same computation. Can we *decouple* parameter count from compute cost?

Analogy: Like consulting a team of specialists: you don't ask every expert for every question, only the relevant ones.

Mixture of Experts (MoE)

- Maintain *many* expert sub-networks
- For each token, activate only a *few* experts
- $\Rightarrow$ Large model capacity with sparse computation

Result: Models with N total parameters but only $\sim k/E \cdot N$ active parameters per token (k active out of E experts).

In practice:

- Experts tend to *specialize* on different token types, languages, or domains
- MoE models match dense model quality at 2–4$\times$ less training compute
- Inference latency similar to a small dense model (only k experts run)
- Most successful modern LLMs use MoE: Mixtral, DeepSeek-V3, Grok

Challenges of MoE

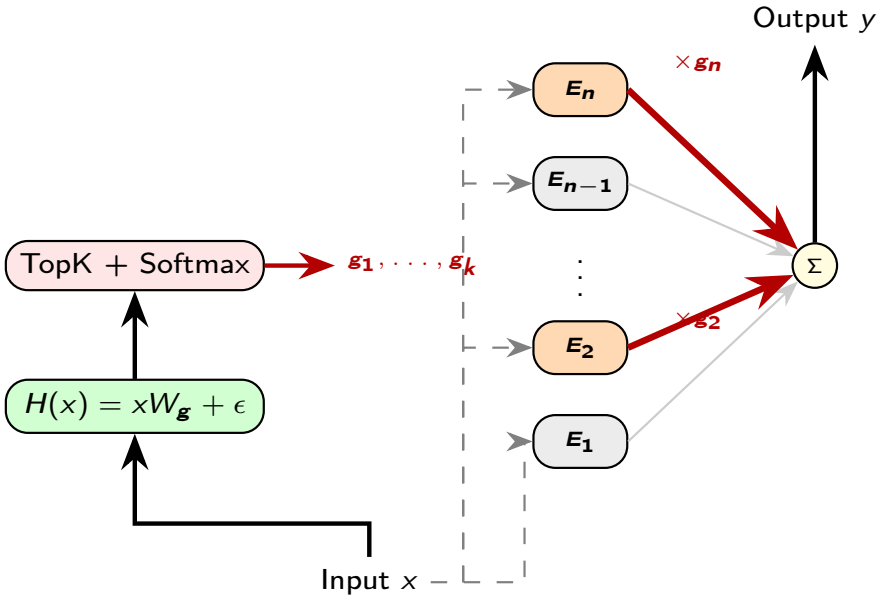
1. **Top-k is not differentiable**
   - The discrete selection of which k experts to activate is a hard, non-differentiable choice
   - Gradients cannot flow through $\mathrm{argtopk}$
   - **Workaround**: Multiply each expert's output by its continuous gating weight $g_i(x)$. Gradients flow through g_i via softmax over the selected experts
   - Router weights W_g are trained by the gating weights, not the selection decision
2. **Load balancing**
   - Router tends to collapse: sends most tokens to a few "favorite" experts
   - Favored experts improve faster $\Rightarrow$ attract even more tokens (self-reinforcing)
   - Requires auxiliary losses or other mechanisms
3. **Communication overhead**
   - Experts distributed across GPUs
   - Tokens must be dispatched to the right device via **all-to-all** communication
   - Poor load balance $\Rightarrow$ stragglers and wasted compute
4. **Memory vs. compute trade-off**
   - All expert weights must reside in memory, even though most are idle per token
   - Total parameter count $\gg$ active parameters
   - Inference requires careful memory management (e.g. expert offloading)
5. **Training instability**
   - Router learning is noisy early in training
   - Requires careful initialization and precision (e.g. router in `float32`)

Table of Contents

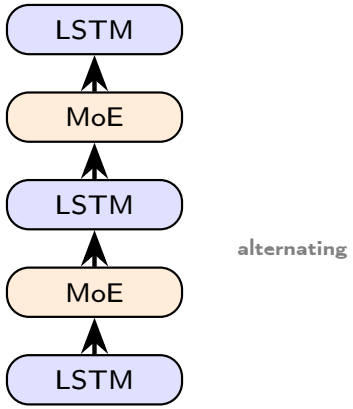
1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - Summary
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

Sparsely-Gated MoE: Architecture

[Shazeer et al., *"Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer"*, 2017]: up to **131,072 experts**, top-k gating (k=2).



Placement in network:



- MoE layers inserted *between* stacked LSTM layers
- Each MoE receives the preceding LSTM's output
- LSTMs: sequential modeling
- MoE: adds capacity without proportional compute
- Up to 5 LSTM + MoE pairs

The Sparsely-Gated Mixture-of-Experts Layer

Gating network for input x:

$$G(x) = \mathrm{softmax}\Big(\mathrm{TopK}\big(H(x),\ k\big)\Big)$$

$$\mathrm{TopK}(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \\ -\infty & \text{otherwise} \end{cases}$$

Noisy gating (training only, disabled at inference):

$$H(x)_i = (x \cdot W_g)_i + \underbrace{\mathcal{N}(0,1) \cdot \mathrm{Softplus}\big((x \cdot W_{\text{noise}})_i\big)}_{\text{learned, input-dependent noise}}$$

- W_{noise} is a second learned weight matrix controlling per-expert noise magnitude
- Noise encourages *exploration*: prevents the router from always picking the same experts
- At inference, noise is turned off $\Rightarrow$ deterministic routing

MoE output: $y = \sum_{i=1}^{E} G(x)_i \cdot E_i(x)$ (only k terms non-zero)

Load Balancing

As discussed earlier, the router tends to collapse without intervention. Noisy gating helps but is not sufficient alone.

Auxiliary load-balancing loss (added to main loss):

For a batch of T tokens across E experts:

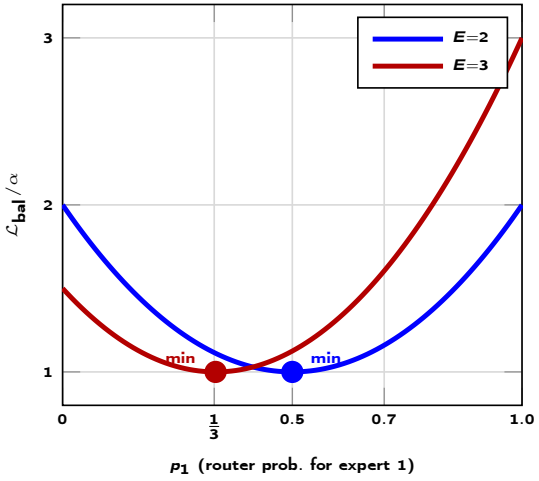
- $f_i = \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}[\text{token } t \to \text{expert } i]$ (hard, non-differentiable)
- $p_i = \frac{1}{T} \sum_{t=1}^{T} \mathrm{softmax}(H(x_t))_i$ (soft, differentiable)

$$\mathcal{L}_{\text{bal}} = \alpha \cdot E \cdot \sum_{i=1}^{E} f_i \cdot p_i$$

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \mathcal{L}_{\text{bal}}$$

- Minimized when $f_i = p_i = 1/E$ (uniform)
- Gradients flow through p_i, pushing router probabilities toward uniformity
- α small (e.g. 10^{-2}) to not dominate

Illustration (E=3, assuming $f_i \approx p_i$):



Loss as a function of how much probability is concentrated on one expert (others split equally). Minimum at uniform $p_i = 1/E$.

Trade-off: Too small $\alpha \Rightarrow$ imbalance; too large $\Rightarrow$ hurts quality by forcing uniformity.

Results: Scaling with Sparse MoE

[Shazeer et al., *"Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer"*, 2017] evaluated on language modeling and machine translation:

- $>$**1000**$\times$ **improvement in model capacity** with only minor losses in computational efficiency
- **Language modeling** (100B word Google News corpus): scaled up to **65,536 experts** with k=4, reaching **68B total parameters**. Only a tiny fraction active per token
- **Machine translation** (WMT'14): up to 2,048 experts per MoE layer, achieved state-of-the-art BLEU scores with 1/6 of the training time of Google's production system
- Performance improved up to 65,536 experts but degraded at 131,072, likely due to insufficient batch size per expert

Key Takeaway

Sparse MoE enables *conditional computation*: the model can store vastly more knowledge in its parameters without proportionally increasing the compute per example.

Challenges identified:

- Load balancing is critical and non-trivial
- Communication overhead when experts are distributed across devices
- Batch size must be large enough so each expert sees sufficient tokens

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - Summary
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

Switch Transformer: Simplifying MoE Routing

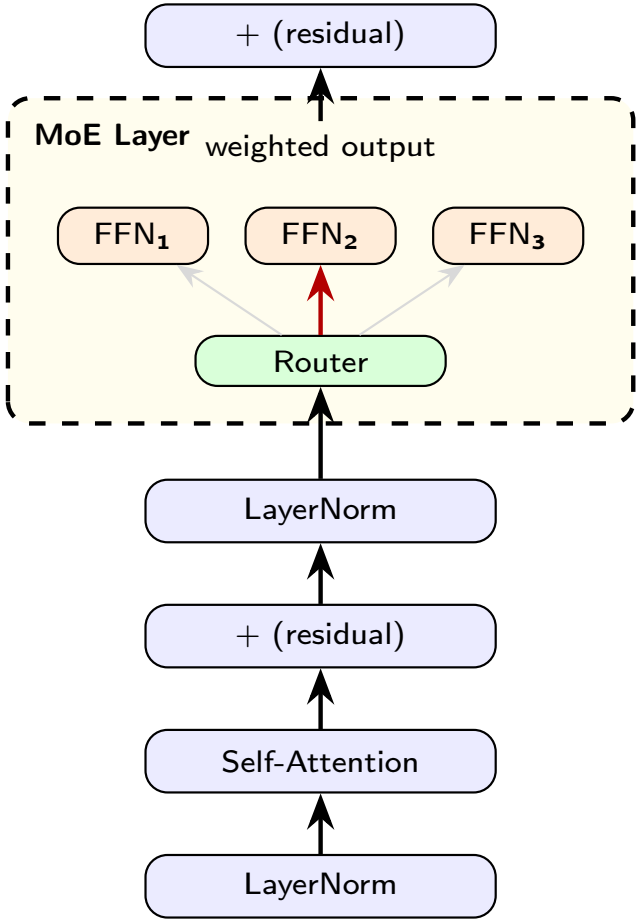
[Fedus et al., *"Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity"*, 2022] simplified the MoE approach with two key changes:

1. **Top-1 routing** ("Switch" routing):
   - Each token goes to *exactly one* expert
   - Simpler than top-k: no need to combine outputs
   - Reduces communication and computation

$$i^* = \arg\max_i (x \cdot W_r)_i$$

$$y = G(x)_{i^*} \cdot E_{i^*}(x)$$

2. **Expert capacity factor**:
   - Each expert processes at most C tokens per batch
   - $C = \frac{k \cdot T}{E} \cdot c$, with capacity factor $c > 1$ (e.g. $c = 1.25$)
   - Overflow tokens are passed through residual connection ("dropped")



Switch Transformer: Model Configurations

All Switch models are **FLOP-matched** to their dense T5 counterparts. Extra parameters go into expert FFN layers, but only 1 expert is active per token.

Model	Total Params	Experts	Layers	d_{model}	FLOPs/seq
T5-Base	0.2B	—	12	768	124B
Switch-Base	7.4B	128	12	768	124B
T5-Large	0.7B	—	24	1024	425B
Switch-Large	26B	128	24	1024	425B
T5-XXL	11B	—	24	4096	6.3T
Switch-XXL	395B	64	24	4096	6.3T
Switch-C	1,571B	2048	15	2080	890B

- Switch-Base has **37**× more parameters than T5-Base at *identical* FLOPs
- Switch-C: expert parallelism only (no model parallelism) ⇒ smaller d_{model}, fewer layers, but 2048 experts
- Training: 550k steps on C4 [Raffel et al., *"Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer"*, 2020], batch size 2^{20} tokens, $\alpha_{\text{bal}} = 10^{-2}$

Switch Transformer: Results

Pre-training speedup (FLOP-matched, same hardware):

- Switch-Base reaches T5-Base quality in **7**× **fewer training steps** (60k vs. 450k steps, both at 124B FLOPs/seq)
- Multilingual: **5**× **mean step speedup** over mT5-Base across 101 languages
- Switch-C reaches T5-XXL quality in ∼**4**× **fewer steps** in wall-clock time, but *not* FLOP-matched (890B vs. 6.3T FLOPs/seq)

Downstream tasks (fine-tuning):

- SuperGLUE: 75.1 (T5-Base) → **79.5** (Switch-Base)
- Winogrande: 66.6 → **73.3**
- XSum: 18.7 → **20.3**

Training stability mitigations:

- **Selective precision**: router computation in `float32`, rest in `bfloat16`. Pure `bfloat16` diverges; selective precision matches `float32` quality at `bfloat16` speed
- **Reduced init scale**: $s = 0.1$ (10× smaller than default). With $s = 1.0$: high variance, unstable
- **Expert dropout**: 0.4 at expert layers during fine-tuning (vs. 0.1 elsewhere)

Design Principles

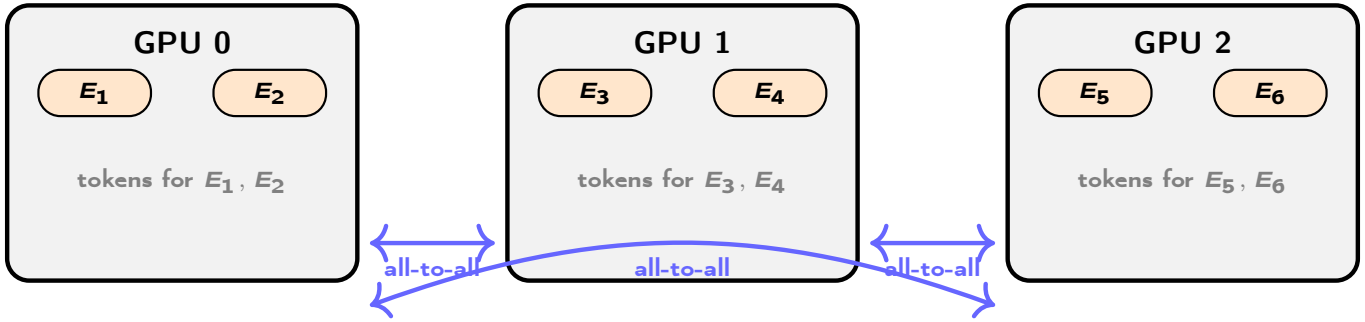
1. Top-1 routing is sufficient
2. More experts > larger experts
3. Auxiliary loss for load balancing
4. Numerical precision matters

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - Summary
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

Expert Parallelism

When experts are distributed across devices, MoE introduces a new parallelism dimension:



- **Dispatch**: **All-to-all** communication sends each token to the GPU hosting its assigned expert
- **Combine**: After expert computation, **all-to-all** sends results back
- **Cost**: Communication volume depends on load balance. Poor balance $\Rightarrow$ congestion

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - Summary
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

DeepSeek-MoE: Fine-Grained Expert Specialization

[Dai et al., *"DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models"*, 2024] proposed two innovations to improve expert specialization:

1. **Fine-grained expert segmentation**:
   - Instead of E large experts, split into mE smaller experts (each $1/m$ the size)
   - Activate mk experts (same total compute)
   - Allows more flexible, fine-grained combinations of knowledge

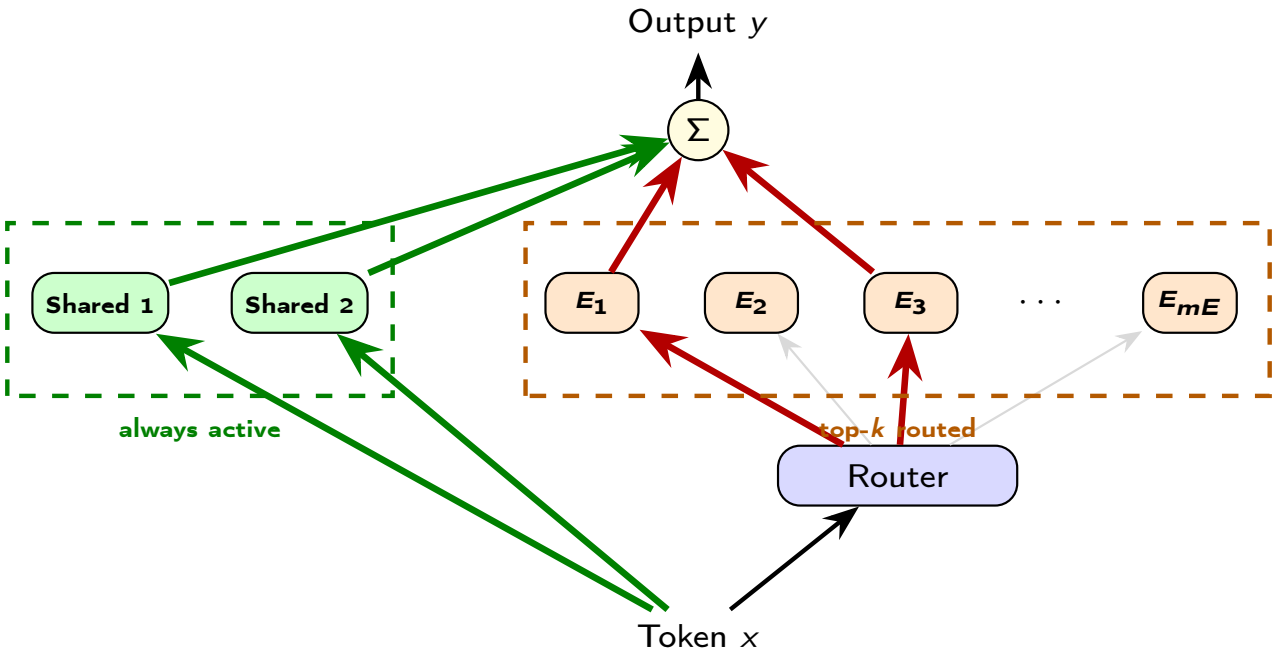
Example: Instead of 16 experts with top-2, use 64 experts with top-8 (same compute, more combinatorial flexibility).

2. **Shared experts**:
   - Dedicate K_s experts as **shared**, always active for every token
   - Remaining experts are **routed** as usual
   - Shared experts capture common knowledge; routed experts specialize

Output

$$y = \sum_{i=1}^{K_s} E_i^{(s)}(x) + \sum_{i \in \text{top-}k} g_i \cdot E_i^{(r)}(x)$$

DeepSeek-MoE: Architecture



- Shared experts reduce *redundancy* among routed experts
- Fine-grained segmentation provides $\binom{mE}{mk}$ possible expert combinations vs. $\binom{E}{k}$ with standard MoE

DeepSeek-MoE: Model Configurations

Fine-grained segmentation factor $m = 4$ throughout. Trained on 2T tokens.

Model	Total	Active	Routed	Shared	Top-k	Layers	d_{model}	d_{expert}
DeepSeek-MoE 16B	16.4B	2.8B	64	2	6	28	2048	1408
DeepSeek-MoE 145B	144.6B	22.2B	128	4	12	62	4096	2048
Dense baselines for comparison:								
DeepSeek 7B	6.9B	6.9B	—	—	—	30	4096	—
LLaMA-2 7B	6.7B	6.7B	—	—	—	32	4096	—
DeepSeek 67B	67B	67B	—	—	—	95	8192	—

- Active experts per token: k routed + K_s shared (e.g. $6 + 2 = 8$ for 16B)
- Each fine-grained expert has $1/m$ the intermediate dim of a standard FFN
- Context length 4096, AdamW optimizer, cosine LR schedule

DeepSeek-MoE: Results

DeepSeek-MoE 16B (2.8B active):

- Comparable to LLaMA-2 7B and DeepSeek 7B dense
- ≈**40%** of the compute of an equivalent dense model
- Active params: 2.8B vs. 6.9B dense

DeepSeek-MoE 145B (22.2B active):

- Approaches DeepSeek 67B dense performance
- Only **1/3** the active params (22.2B vs. 67B)
- Confirms fine-grained + shared experts scale

Ablation results ($m = 4$ best):

- Fine-grained segmentation alone: significant gain over standard top-k MoE
- Shared experts alone: moderate gain
- Combined: largest improvement
- $\binom{64}{6} \approx 74$M combinations vs. $\binom{16}{2} = 120$ for standard MoE

Design Recipe

Fine-grained segmentation (m=4) + shared experts (K_s=2) consistently outperforms standard MoE and dense models at matched compute.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
   - Motivation and Basic Idea
   - Sparsely-Gated MoE (Shazeer et al., 2017)
   - Switch Transformer
   - MoE in Practice: Integration into Transformers
   - Summary
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

DeepSeek-V3: Auxiliary-Loss-Free Load Balancing

[DeepSeek-AI, *"DeepSeek-V3 Technical Report"*, 2024] identified a key problem with auxiliary balancing losses:

Problem

The auxiliary loss $\mathcal{L}_{\text{bal}}$ conflicts with the language modeling objective. Tuning α is difficult: too small $\Rightarrow$ imbalance, too large $\Rightarrow$ hurts model quality.

Solution: Bias-based load balancing (no auxiliary loss):

- Add a learnable **bias** term b_i to each expert's gating score for routing:

$$i^* = \arg\max_i \left(s_i(x) + b_i\right)$$

- The bias is **not** used in the gating weight (only for routing decisions):

$$g_i(x) = \frac{s_i(x)}{\sum_{j \in \text{top-}k} s_j(x)}$$

- b_i is adjusted dynamically: if expert i is overloaded, decrease b_i; if underloaded, increase b_i

$\Rightarrow$ Load balancing is *decoupled* from the training loss.

DeepSeek-V3: Architecture Overview

DeepSeek-V3: 671B total parameters, 37B active per token.

MoE configuration:

- 61 Transformer layers
- First 3 layers: dense FFN
- Layers 4–61: MoE layers
- 256 routed experts + 1 shared expert per MoE layer
- Top-8 routing (out of 256)
- Fine-grained segmentation following DeepSeek-MoE

Additional innovations:

- Multi-head Latent Attention (MLA) for KV cache compression
- Multi-token prediction as auxiliary training objective
- FP8 mixed-precision training

Training efficiency:

- Trained on 14.8T tokens
- Training cost: **2.788M H800 GPU hours**
- ≈$5.5M (remarkably economical)
- Comparable to GPT-4 / LLaMA-3 405B quality

Why so efficient?

- MoE: only 37B of 671B params active
- FP8 training: ∼2× throughput vs. BF16
- Efficient expert parallelism with minimal communication

Comparison: Dense vs. MoE Approaches

Model	Total Params	Active Params	MoE Design
LLaMA-3 405B	405B	405B	Dense
Mixtral 8×7B	47B	13B	Top-2 of 8
Mixtral 8×22B	141B	39B	Top-2 of 8
DeepSeek-V2	236B	21B	Top-6 of 160 + shared
DeepSeek-V3	671B	37B	Top-8 of 256 + shared

Trends in MoE design:

1. **More, smaller experts**: from 8 large → 256 fine-grained
2. **Higher top-k**: from top-1/2 → top-6/8 with smaller experts
3. **Shared experts**: reduce redundancy, capture common knowledge
4. **Better load balancing**: from auxiliary losses → bias-based methods

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts
- Motivation and Basic Idea
- Sparsely-Gated MoE (Shazeer et al., 2017)
- Switch Transformer
- MoE in Practice: Integration into Transformers

- Summary

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

MoE: Summary and Open Challenges

Key ideas:

- MoE *decouples* model capacity (total params) from compute cost (active params)
- Router selects a sparse subset of experts per token
- Load balancing is essential for training stability and efficiency

Evolution:

- **Shazeer (2017)**: Sparsely-gated MoE layer, noisy top-k gating, auxiliary loss
- **Switch (2022)**: Simplified top-1 routing, scaled to 1.6T params
- **DeepSeek-MoE (2024)**: Fine-grained experts + shared experts
- **DeepSeek-V3 (2024)**: Auxiliary-loss-free balancing, 671B params at low cost

Open challenges:

- **Inference**: All expert weights must be in memory, even though most are idle
- **Expert collapse**: Some experts may remain underutilized
- **Communication**: Expert parallelism requires all-to-all communication
- **Distillation**: Can we compress MoE back into smaller dense models?

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models
- Vision Transformer (ViT)
- DeiT: Data-efficient Image Transformers
- DETR
- Deformable DETR
- TrackFormer
- MaskFormer
- Mask2Former

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

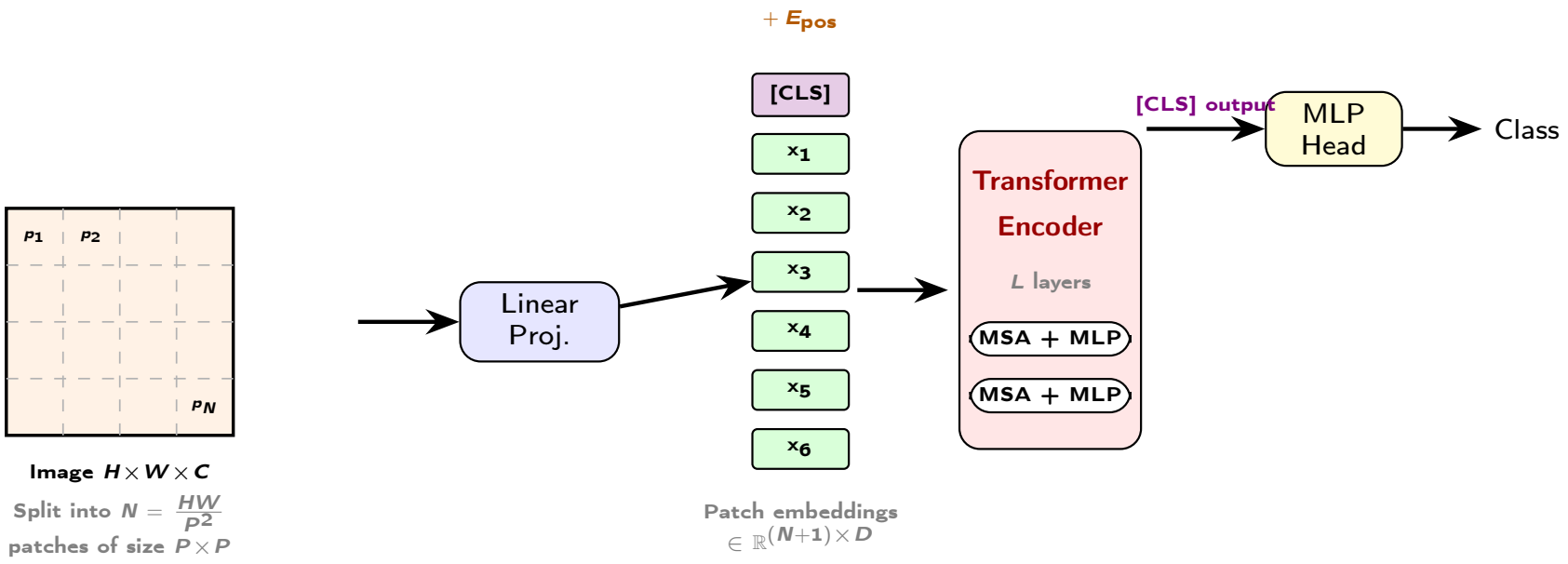
17 Using LLMs: Prompting, Tools, RAG, and

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
   - Vision Transformer (ViT)
   - DeiT: Data-efficient Image Transformers
   - DETR
   - Deformable DETR
   - TrackFormer
   - MaskFormer
   - Mask2Former
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

ViT: An Image is Worth 16×16 Words

[Dosovitskiy et al., *"An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale"*, 2021]: Apply a **standard Transformer encoder** directly to sequences of image patches, no convolutions needed.



- Split image into fixed-size patches (P=16), flatten and linearly project to D dimensions
- Prepend a learnable **[CLS]** token; add learnable positional embeddings
- Standard Transformer encoder; classify from [CLS] output
- Requires large-scale pre-training (JFT-300M) to outperform CNNs

ViT: Patch Embedding & Transformer Block

Patch embedding:

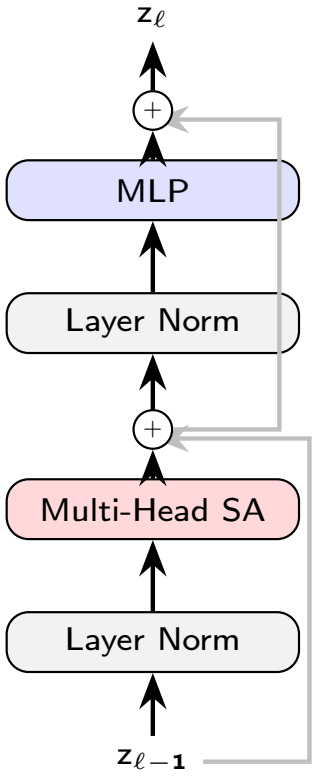
$$z_0 = [x_{\text{class}};\ x_p^1 E;\ \ldots;\ x_p^N E] + E_{\text{pos}}$$

- $E \in \mathbb{R}^{(P^2 C) \times D}$: projection matrix
- $E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$: positional embeddings
- Implemented as `Conv2d` with kernel and stride $= P$

Transformer encoder block (pre-norm):

$$z'_\ell = \text{MSA}(\text{LN}(z_{\ell-1})) + z_{\ell-1}$$
$$z_\ell = \text{MLP}(\text{LN}(z'_\ell)) + z'_\ell$$

Classification: $y = \text{LN}(z_L^0)$



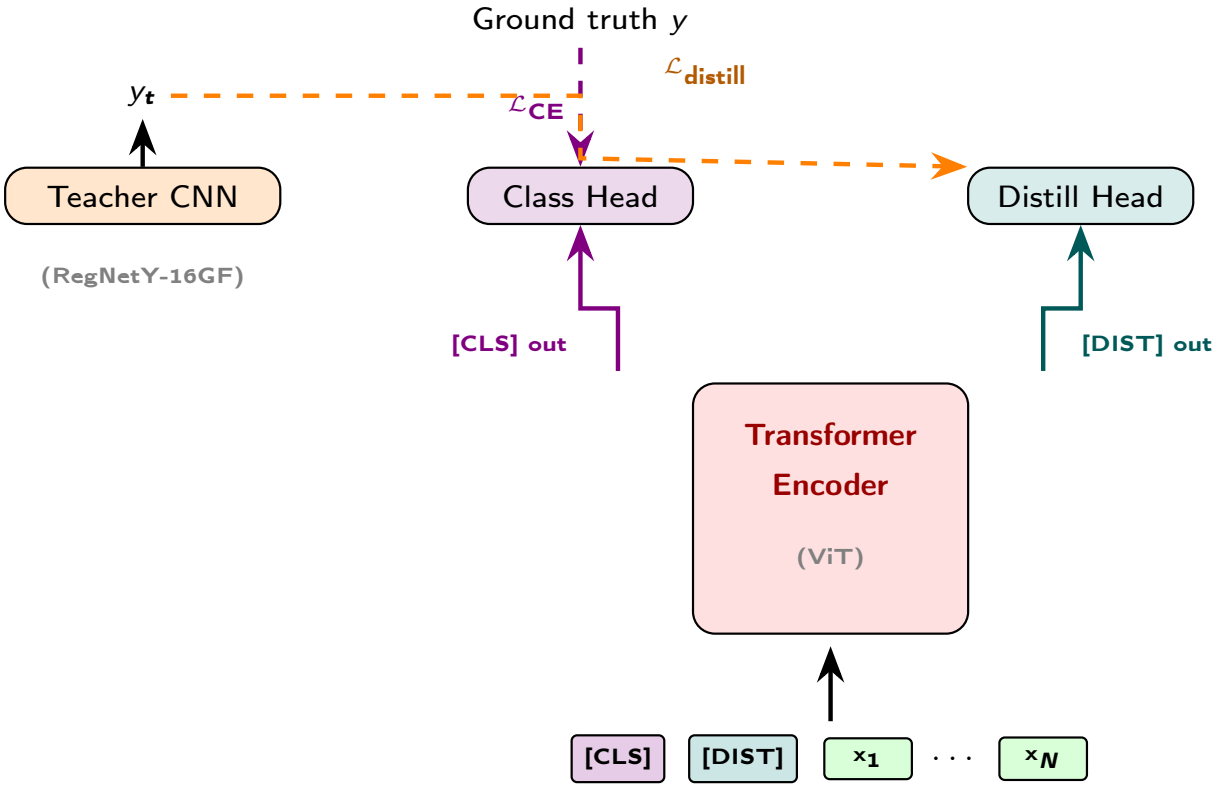
Variant	Layers	Hidden D	MLP dim	Heads	Params
ViT-B/16	12	768	3072	12	86M
ViT-L/16	24	1024	4096	16	307M
ViT-H/14	32	1280	5120	16	632M

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
   - Vision Transformer (ViT)
   - DeiT: Data-efficient Image Transformers
   - DETR
   - Deformable DETR
   - TrackFormer
   - MaskFormer
   - Mask2Former
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

DeiT: Knowledge Distillation for ViT

[Touvron, Cord, et al., *"Training data-efficient image transformers & distillation through attention"*, 2021]: Train ViT **on ImageNet-1K only** (no JFT-300M) via a **distillation token** and strong data augmentation.



- Input: [CLS]; [DIST]; x_1; ...; x_N (N+2 tokens)
- **[CLS]** head: cross-entropy vs. ground truth labels
- **[DIST]** head: cross-entropy vs. teacher's hard labels (**hard distillation** works best)
- At inference: average both heads' predictions

DeiT: Hard vs Soft Distillation

Soft distillation (KL divergence):

$$\mathcal{L}_{\text{soft}} = (1-\lambda)\,\mathcal{L}_{\text{CE}}(\psi(Z_s), y) + \lambda\,\tau^2\,\text{KL}(\psi(Z_s/\tau) \| \psi(Z_t/\tau))$$

- Z_s, Z_t: student/teacher logits
- τ: temperature, ψ: softmax

Hard distillation (best results):

$$\mathcal{L}_{\text{hard}} = \frac{1}{2}\,\mathcal{L}_{\text{CE}}(\psi(Z_s), y) + \frac{1}{2}\,\mathcal{L}_{\text{CE}}(\psi(Z_s), y_t)$$

where $y_t = \arg\max Z_t$ (teacher's hard prediction)

Training strategy (data-efficient):

- ImageNet-1K only (1.2M images)
- Strong augmentation: RandAugment, Mixup, CutMix, Random Erasing
- Regularization: Stochastic Depth, repeated augmentation
- AdamW + cosine schedule
- Trains in < 3 days on 8 GPUs

Variant	Dim	Params
DeiT-Ti	192	5M
DeiT-S	384	22M
DeiT-B	768	86M

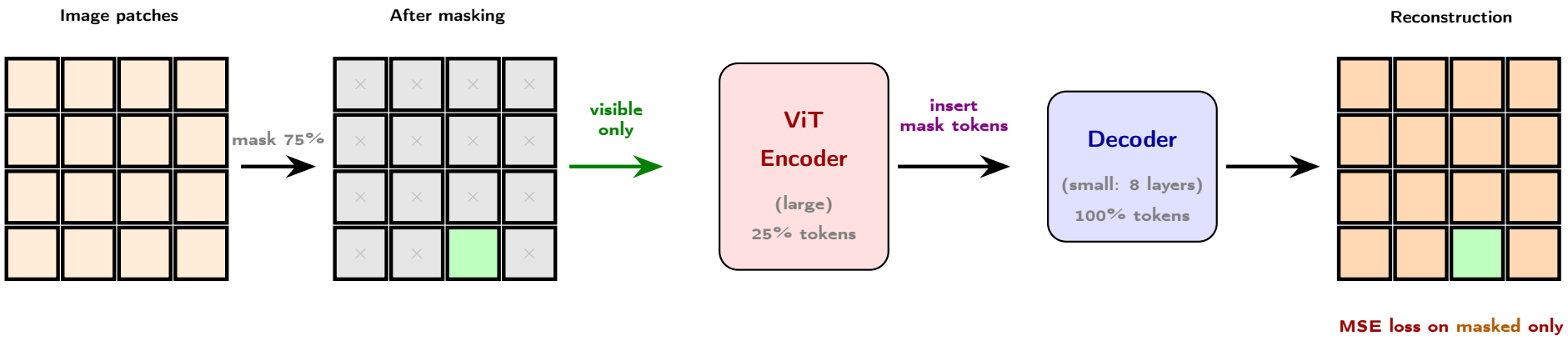
Key finding: CNN teacher (RegNet) outperforms Transformer teacher for distillation.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
   - Vision Transformer (ViT)
   - DeiT: Data-efficient Image Transformers
   - DETR
   - Deformable DETR
   - TrackFormer
   - MaskFormer
   - Mask2Former
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

MAE: Masked Autoencoders Are Scalable Vision Learners

[He, X. Chen, et al., *"Masked Autoencoders Are Scalable Vision Learners"*, 2022]: Self-supervised pre-training by masking **75%** of image patches and reconstructing them. Key insight: only *visible* patches pass through the (expensive) encoder.



- **Encoder**: standard ViT, processes only visible patches (25%) — 3×+ speedup
- **Decoder**: lightweight (8 layers, width 512), operates on all tokens incl. shared learned mask tokens
- **Target**: per-patch normalized pixel values; loss $= \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{x}_i - x_i\|^2$ on masked patches only

MAE: Design Choices & Results

Why 75% masking?

- Images have high spatial redundancy. Low masking allows trivial interpolation
- 75% forces the model to learn *semantic* understanding, not local texture
- Much higher than BERT's 15% for text

Asymmetric encoder-decoder:

- Encoder: large ViT (e.g. 24 layers, 1024 dim)
- Decoder: small (8 layers, 512 dim)
- Decoder **discarded** after pre-training
- Only encoder used for downstream tasks

Downstream usage:

Fine-tuning
Full encoder + classification head; best results

Linear probing
Frozen encoder + linear layer; weaker than contrastive methods but gap narrows at scale

ImageNet-1K results (fine-tuning):

Model	Params	Top-1
ViT-B/16	86M	83.6%
ViT-L/16	307M	85.9%
ViT-H/14	632M	86.9%

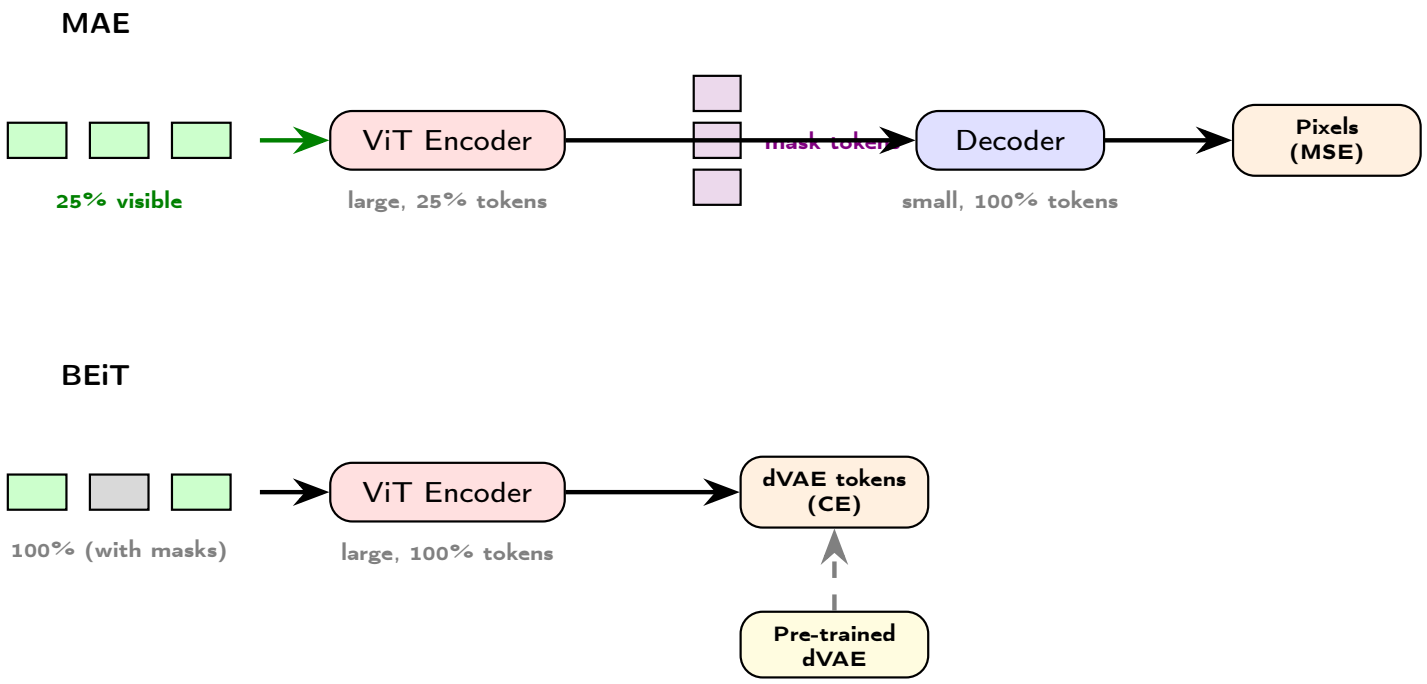
Using ImageNet-1K only. No external data.

vs. other self-supervised methods:

	Fine-tune	Lin. probe
MAE	**best**	good
BEiT	good	good
DINO	good	**best**
MoCo v3	good	good

Key: MAE scales excellently to ViT-H; contrastive methods plateau at larger sizes.

MAE: Comparison with BEiT



	Encoder input	Target	Ext. tokenizer	Mask ratio	Speed
MAE	visible only (25%)	pixels	No	75%	3×+ faster
BEiT	all (incl. masks)	dVAE tokens	Yes	40%	slower
SimMIM	all (incl. masks)	pixels	No	32–60%	slower

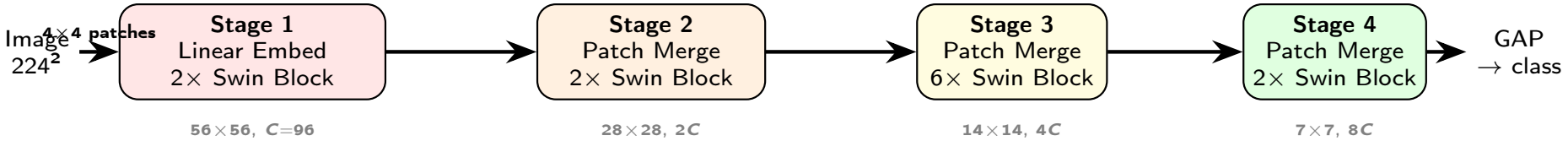
MAE is simpler (no tokenizer), faster (encoder sees fewer tokens), and achieves better fine-tuning accuracy.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
   - Vision Transformer (ViT)
   - DeiT: Data-efficient Image Transformers
   - DETR
   - Deformable DETR
   - TrackFormer
   - MaskFormer
   - Mask2Former
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

Swin Transformer: Hierarchical Vision Transformer

[Ze Liu et al., *"Swin Transformer: Hierarchical Vision Transformer using Shifted Windows"*, 2021]: Build **hierarchical feature maps** (like CNNs) using **shifted window** self-attention for linear complexity.



vs. ViT: flat 14×14 tokens throughout all layers
Swin: hierarchical, like ResNet's feature pyramid

Patch Partition
Split image into 4×4 non-overlapping patches (56×56 tokens)

Patch Merging
Concatenate 2×2 neighbor patches → linear projection to $2C$ dims; halves spatial resolution, doubles channels

Swin Block Window-MSA → Shifted Window-MSA (alternating in consecutive blocks)

Swin: Window & Shifted Window Attention

Window-MSA (W-MSA):

- Partition feature map into non-overlapping $M \times M$ windows (M=7)
- Self-attention computed *within* each window
- Complexity: $\mathcal{O}(M^2 \cdot hw \cdot C)$, **linear** in image size

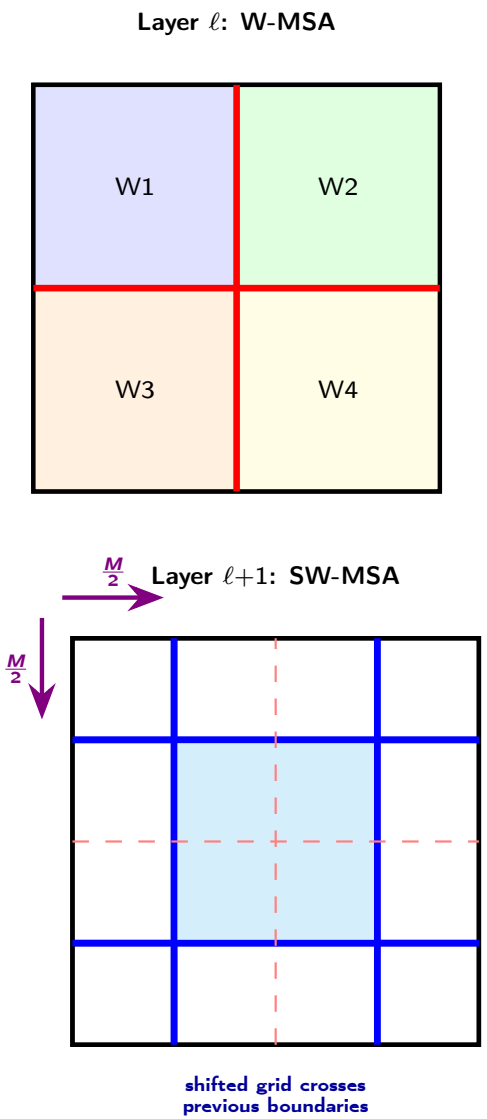
Problem: No cross-window information flow.

Shifted Window-MSA (SW-MSA):

- Shift window grid by $(\lfloor M/2 \rfloor, \lfloor M/2 \rfloor)$
- Windows now straddle previous boundaries
- Consecutive blocks alternate W-MSA / SW-MSA

Efficient cyclic shift:

1. Cyclically shift feature map
2. Regular window partition + attention masks
3. Reverse shift



Relative position bias: $\mathrm{Attn}(Q, K, V) = \mathrm{softmax}(\frac{QK^\top}{\sqrt{d}} + B)\, V$, B from learnable

Swin: Complexity Comparison & Configurations

Swin block equations:

$$\hat{z}^{\ell} = \text{W-MSA}(\text{LN}(z^{\ell-1})) + z^{\ell-1}$$
$$z^{\ell} = \text{MLP}(\text{LN}(\hat{z}^{\ell})) + \hat{z}^{\ell}$$
$$\hat{z}^{\ell+1} = \text{SW-MSA}(\text{LN}(z^{\ell})) + z^{\ell}$$
$$z^{\ell+1} = \text{MLP}(\text{LN}(\hat{z}^{\ell+1})) + \hat{z}^{\ell+1}$$

Complexity ($h \times w$ spatial, M^2 window):

Global MSA $\mathcal{O}(4hwC^2 + 2(hw)^2C)$

W-MSA $\mathcal{O}(4hwC^2 + 2M^2hwC)$

Quadratic $\rightarrow$ **linear** in spatial resolution.

Variant	C	Layers	Params
Swin-T	96	[2,2,6,2]	29M
Swin-S	96	[2,2,18,2]	50M
Swin-B	128	[2,2,18,2]	88M
Swin-L	192	[2,2,18,2]	197M

Key advantages:

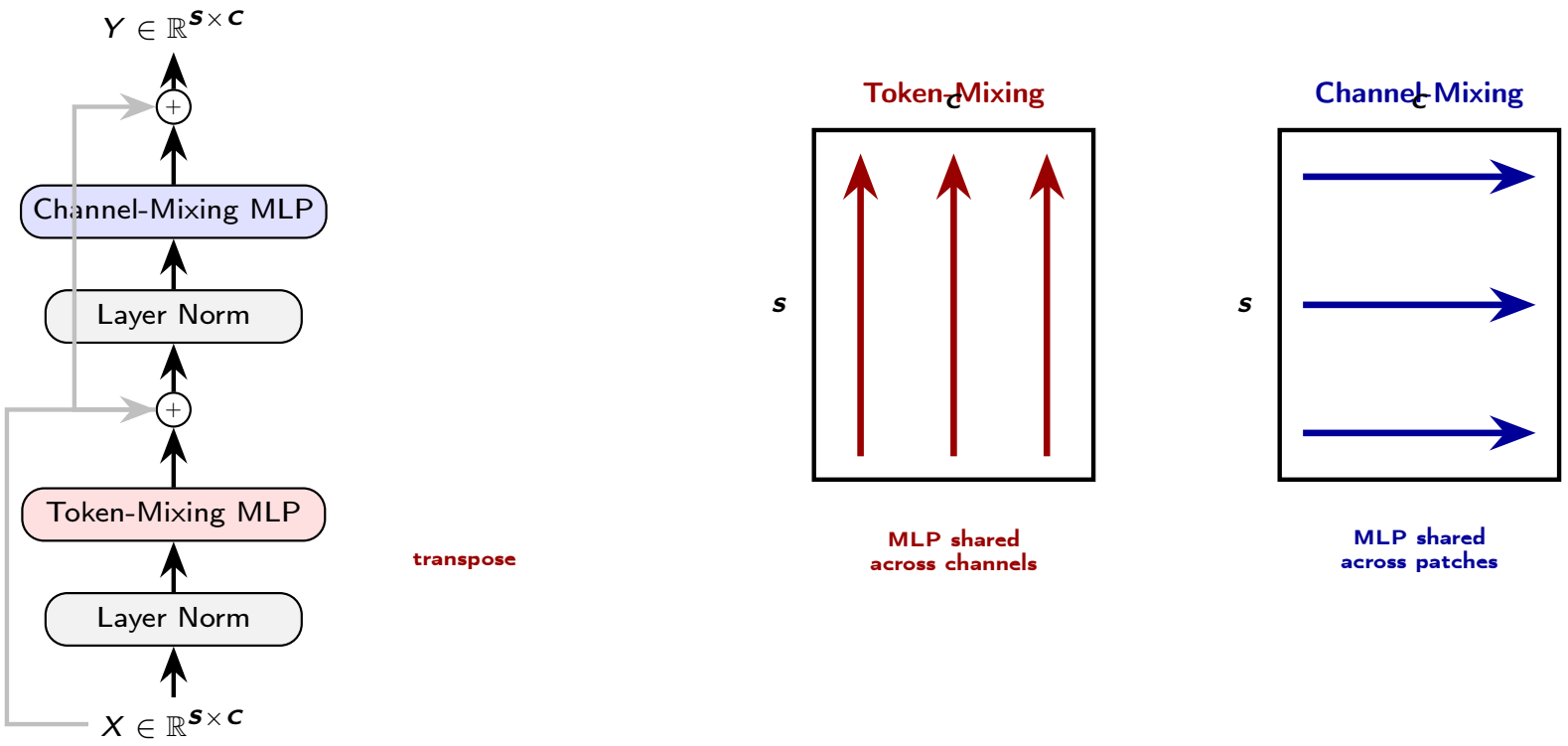
- Hierarchical features $\rightarrow$ natural fit as backbone for detection, segmentation
- Linear complexity $\rightarrow$ scales to high-resolution images
- Outperforms ViT on ImageNet at comparable size (without large-scale pre-training)
- Relative position bias instead of absolute positional embeddings

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
   - Vision Transformer (ViT)
   - DeiT: Data-efficient Image Transformers
   - DETR
   - Deformable DETR
   - TrackFormer
   - MaskFormer
   - Mask2Former
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

MLP-Mixer: No Attention, Only MLPs

[Tolstikhin et al., *"MLP-Mixer: An all-MLP Architecture for Vision"*, 2021]: Replace attention entirely with two types of MLPs: **token-mixing** (across patches) and **channel-mixing** (within patches).



Token-mixing: $U_{*,i} = X_{*,i} + W_2\, \sigma(W_1\, \mathrm{LN}(X)^\top_{*,i})^\top$ (mixes across S patches, shared across C channels)

Channel-mixing: $Y_{j,*} = U_{j,*} + W_4\, \sigma(W_3\, \mathrm{LN}(U)_{j,*})$ (mixes across C channels, shared across S patches)

MLP-Mixer: Architecture Comparison

Full pipeline:

1. Split image into $S = HW/P^2$ patches
2. Per-patch linear projection $\rightarrow X \in \mathbb{R}^{S \times C}$
3. N Mixer layers (token-mix + channel-mix)
4. Global average pooling over $S \rightarrow \mathbb{R}^C$
5. Linear classification head

Why it works:

- Token-mixing MLPs learn *fixed* spatial mixing patterns (no data-dependent attention)
- With enough data, fixed mixing is competitive with attention
- Much simpler architecture

Comparison with Transformer:

	Transformer	Mixer
Spatial mix	Attention	Token MLP
Channel mix	FFN	Channel MLP
Data-dep.	Yes	No
Complexity	$\mathcal{O}(S^2)$	$\mathcal{O}(S)$

Configurations:

Variant	N	C	Params
Mixer-S/16	8	512	18M
Mixer-B/16	12	768	59M
Mixer-L/16	24	1024	207M

Competitive with ViT when pre-trained on large data; degrades more on small datasets.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models
- Vision Transformer (ViT)
- DeiT: Data-efficient Image Transformers
- DETR
- Deformable DETR
- TrackFormer
- MaskFormer
- Mask2Former

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

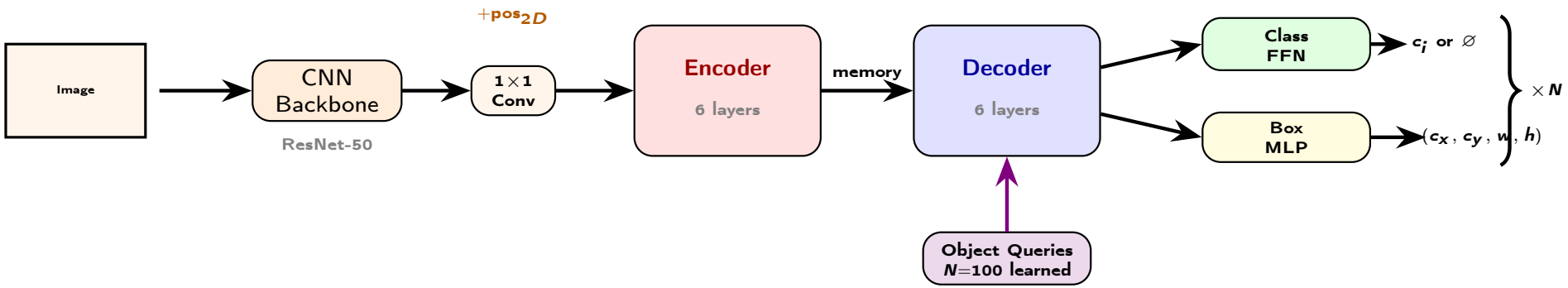
15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG, and

DETR: End-to-End Object Detection with Transformers

[Carion et al., *"End-to-End Object Detection with Transformers"*, 2020]: Reformulate object detection as a **direct set prediction** problem: no anchors, no NMS, no hand-crafted components.



Key components:

- CNN extracts features → flatten to sequence
- Encoder: self-attention over spatial features
- Decoder: N **object queries** cross-attend to encoder memory
- Each query predicts one (class, box) or "no object"

Why it matters:

- No anchors, NMS, or hand-crafted post-processing
- Set prediction via **Hungarian matching**
- Object queries self-attend → reason about inter-object relations
- Simple, elegant, end-to-end trainable

DETR: Bipartite Matching Loss

Training requires matching N predictions to M ground truth objects (padded with $\varnothing$).

Step 1: Hungarian matching. Find optimal assignment $\hat{\sigma}$:

$$\hat{\sigma} = \arg\min_{\sigma \in \mathfrak{S}_N} \sum_{i=1}^{N} \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

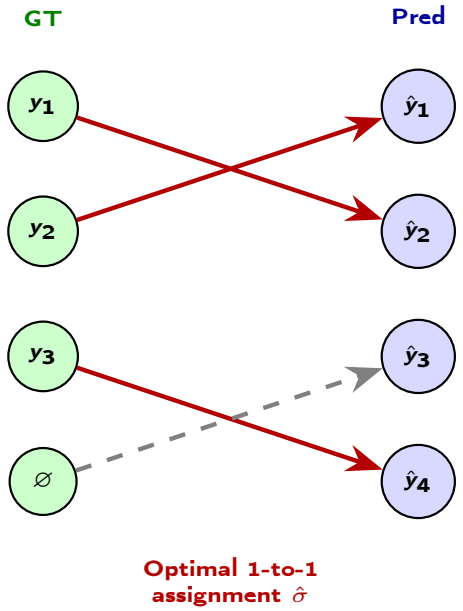
$$\mathcal{L}_{\text{match}} = -\mathbb{1}_{c_i \neq \varnothing}\, \hat{p}_{\sigma(i)}(c_i) + \mathbb{1}_{c_i \neq \varnothing}\, \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

Solved by the Hungarian algorithm in $\mathcal{O}(N^3)$.

Step 2: Compute loss on matched pairs:

$$\mathcal{L} = \sum_{i=1}^{N} \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{c_i \neq \varnothing}\, \mathcal{L}_{\text{box}} \right]$$

$$\mathcal{L}_{\text{box}} = \lambda_{\text{iou}}\, \mathcal{L}_{\text{giou}} + \lambda_{L_1} \|b_i - \hat{b}\|_1$$



- Each GT matched to exactly one prediction
- Unmatched predictions trained to output $\varnothing$
- No NMS needed: queries learn to avoid duplicates

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models
- Vision Transformer (ViT)
- DeiT: Data-efficient Image Transformers
- DETR
- Deformable DETR
- TrackFormer
- MaskFormer
- Mask2Former

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

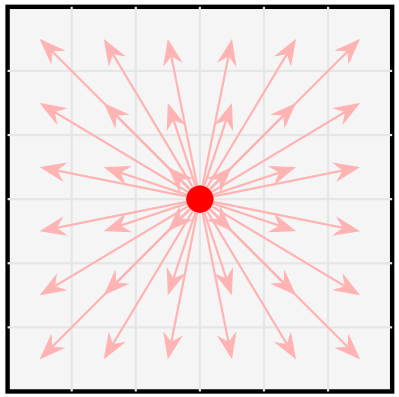
16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG, and

Deformable DETR: Sparse Attention for Detection

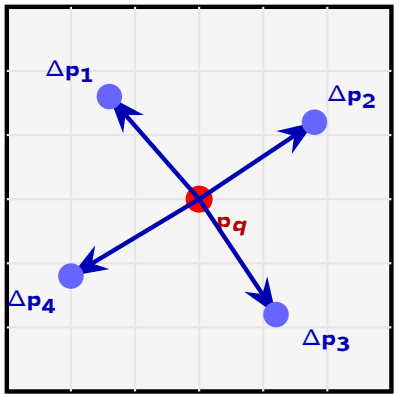
[X. Zhu et al., *"Deformable DETR: Deformable Transformers for End-to-End Object Detection"*, 2021]: Fix DETR's slow convergence (500 epochs) and poor small-object performance via **deformable attention**. Each query attends to only K learned sampling points.

DETR: Global Attention



$\mathcal{O}(H^2 W^2)$

Deformable: K Points



$\mathcal{O}(HW \cdot K)$, K=4

Deformable attention:

$$\mathrm{DeformAttn}(z_q, p_q, x) = \sum_{m=1}^{M} W_m \left[\sum_{k=1}^{K} A_{mqk} \cdot W'_m x(p_q + \Delta p_{mqk}) \right]$$

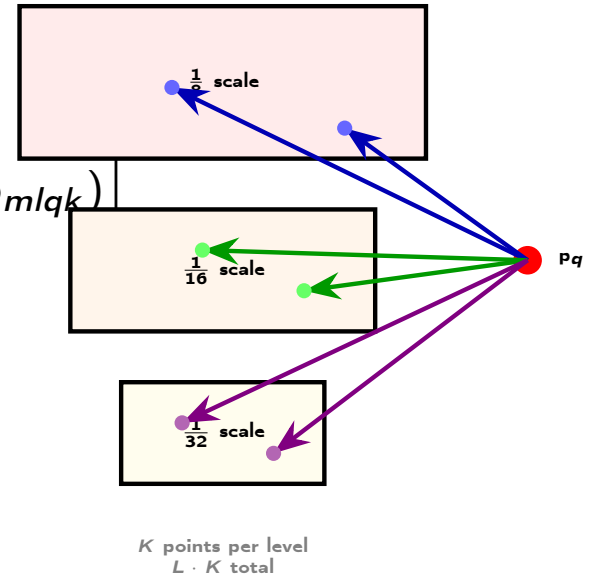
- Offsets Δp_{mqk} and weights A_{mqk} predicted from query z_q via linear layers
- $x(\cdot)$: bilinear interpolation at continuous positions
- K=4 sampling points per head, M heads

Deformable DETR: Multi-Scale Features

Multi-scale deformable attention:

$$\text{MSDeformAttn} = \sum_{m=1}^{M} W_m \left[\sum_{l=1}^{L} \sum_{k=1}^{K} A_{mlqk} \cdot W'_m x^l(\phi_l(\hat{p}_q) + \Delta p_{mlqk}) \right]$$

- Attend across L=4 feature map levels simultaneously
- ϕ_l: rescales reference point to level l
- Weights normalized: $\sum_l \sum_k A_{mlqk} = 1$



Additional features:

- **Iterative box refinement**: each decoder layer refines the box from the previous layer
- **Two-stage variant**: encoder produces proposals as initial reference points

	DETR	Def. DETR
Epochs	500	50
Scales	1	4
Queries	100	300
AP (COCO)	42.0	46.2

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models
- Vision Transformer (ViT)
- DeiT: Data-efficient Image Transformers
- DETR
- Deformable DETR
- TrackFormer
- MaskFormer
- Mask2Former

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

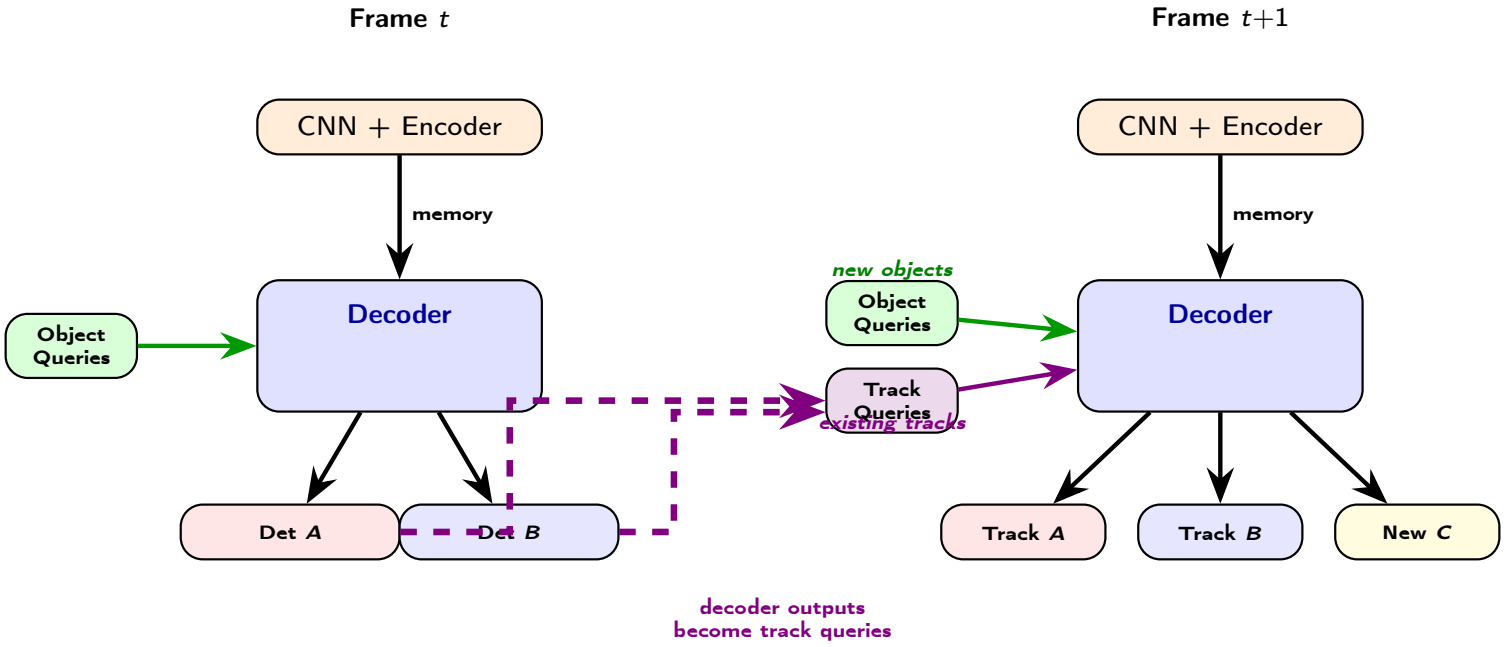
15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG, and

TrackFormer: Multi-Object Tracking with Transformers

[Meinhardt et al., "*TrackFormer: Multi-Object Tracking with Transformers*", 2022]: Extend DETR to video by introducing **track queries** that persist across frames: tracking by attention.



- **Object queries**: detect *new* objects (as in DETR)
- **Track queries**: autoregressive embeddings from previous frame → track *existing* objects
- Decoder self-attention lets track/object queries reason about each other (avoid identity switches)
- Track terminated when classified as ∅; can be revived from stored embedding

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models
- Vision Transformer (ViT)
- DeiT: Data-efficient Image Transformers
- DETR
- Deformable DETR
- TrackFormer
- MaskFormer
- Mask2Former

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

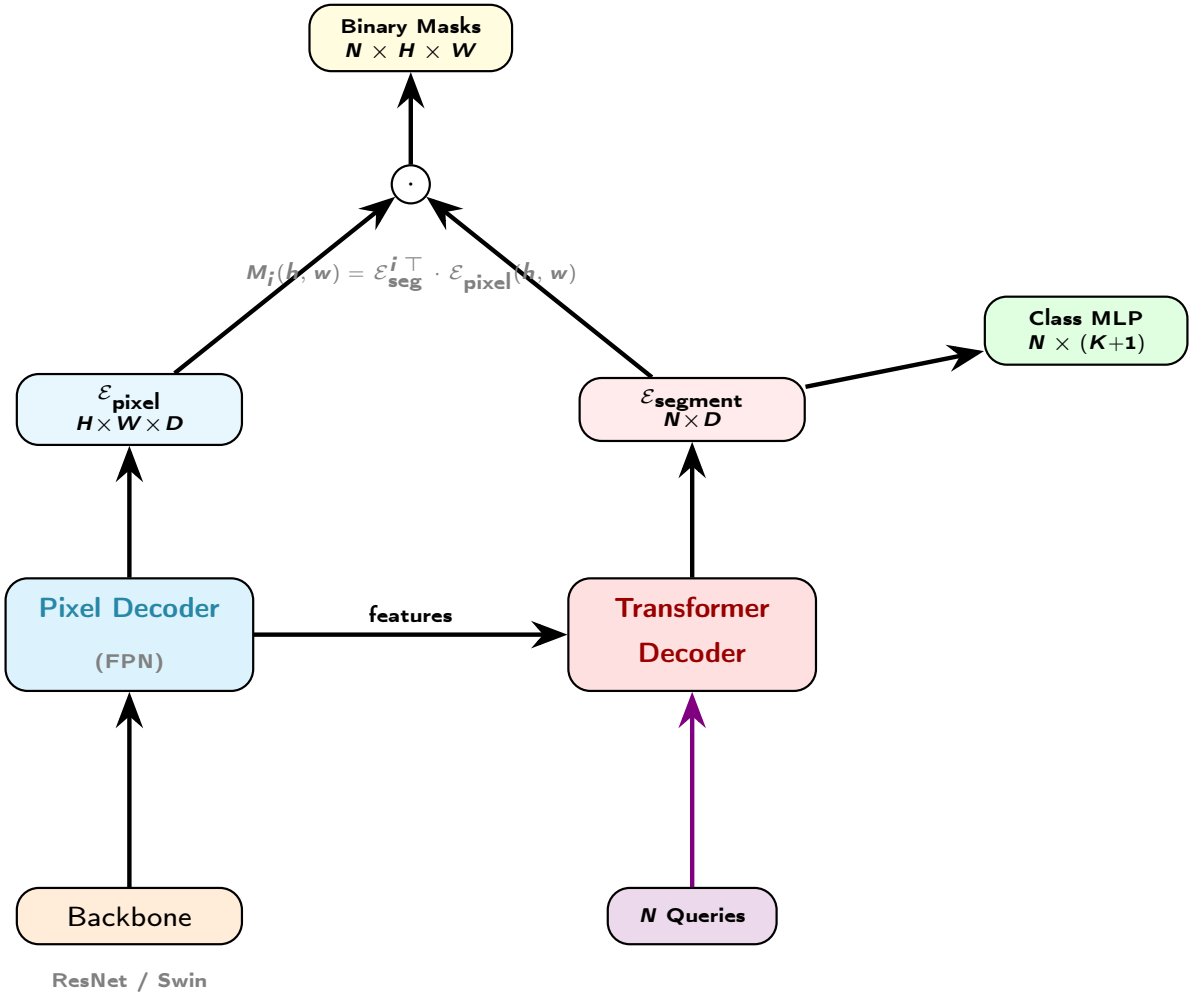
15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG, and

MaskFormer: From Per-Pixel to Mask Classification

[Cheng, Schwing, et al., *"Per-Pixel Classification is Not All You Need for Semantic Segmentation"*, 2021]: Reformulate segmentation as predicting a set of **(class, binary mask)** pairs instead of assigning a class to every pixel independently.



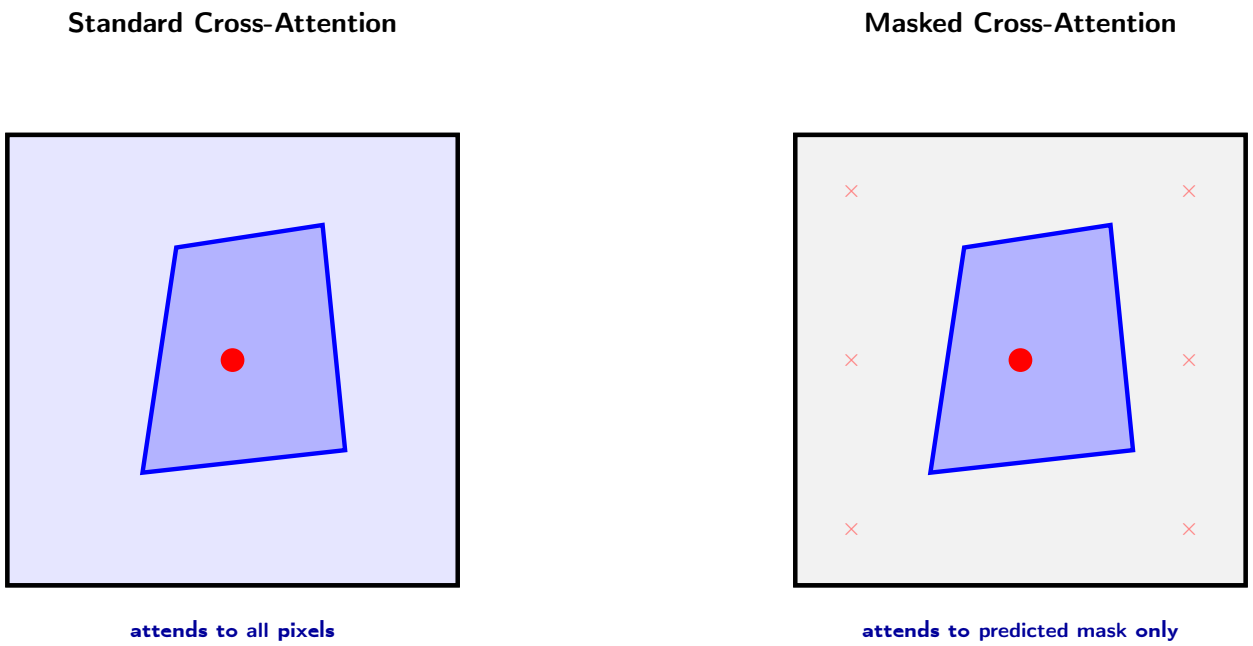
- **Pixel decoder**: produces dense per-pixel embeddings $\mathcal{E}_{\text{pixel}} \in \mathbb{R}^{H \times W \times D}$

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
   - Vision Transformer (ViT)
   - DeiT: Data-efficient Image Transformers
   - DETR
   - Deformable DETR
   - TrackFormer
   - MaskFormer
   - Mask2Former
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

Mask2Former: Masked Attention for Universal Segmentation

[Cheng, Misra, et al., *"Masked-attention Mask Transformer for Universal Image Segmentation"*, 2022]: Improve MaskFormer with **masked cross-attention** (restrict attention to predicted mask region) and **multi-scale** decoder features.



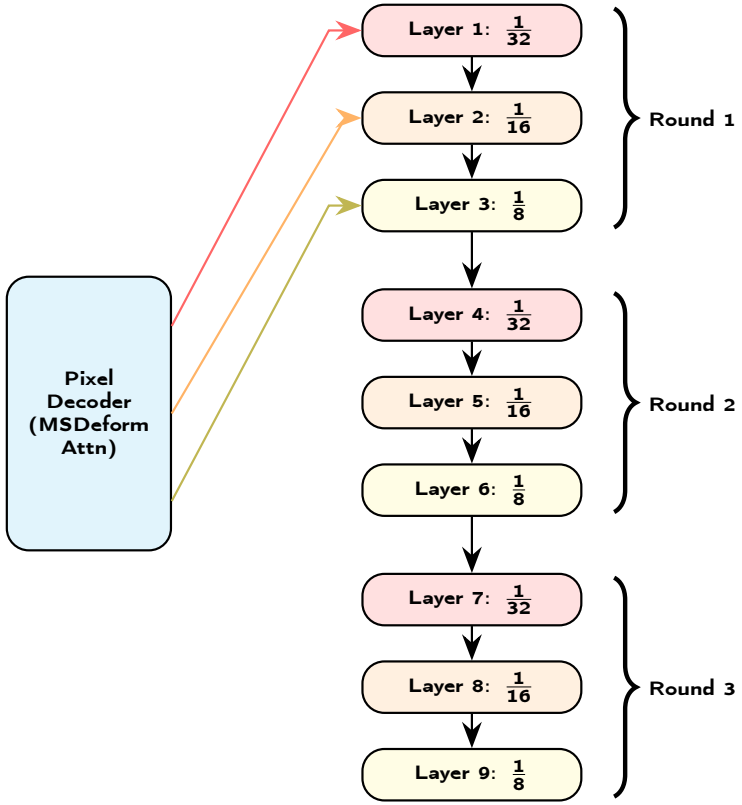
Masked cross-attention:

$$X_\ell = \mathrm{softmax}\Big(\mathcal{M}_{\ell-1} + \frac{Q_\ell K_\ell^\top}{\sqrt{d}}\Big) V_\ell + X_{\ell-1}$$

where $\mathcal{M}_{\ell-1}(i,h,w) = \begin{cases} 0 & \text{if } \sigma(m_{\ell-1}(h,w)) \geq 0.5 \\ -\infty & \text{otherwise} \end{cases}$ ($m_{\ell-1}$: predicted mask from previous layer)

Mask2Former: Multi-Scale Decoder & Results

Multi-scale feature processing:



Key innovations:

- 9 decoder layers in 3 rounds
- Round-robin across 3 resolutions
- Coarse-to-fine within each round
- Pixel decoder uses Deformable DETR encoder (MSDeformAttn)

Universal architecture: same model for

- Semantic segmentation
- Instance segmentation
- Panoptic segmentation

Results (COCO):

Task	MF	M2F
Panoptic (PQ)	52.7	57.8
Instance (AP)	43.7	47.2
Semantic (mIoU)	–	57.7

Masked attention provides faster convergence and better localization.

Vision Models: Summary

Model	Key Innovation	Task	Attention
ViT	Patches + Transformer	Classification	Global
DeiT	Distillation token	Classification	Global
MAE	Masked autoencoding (75%)	Pre-training	Global
Swin	Shifted windows + hierarchy	General backbone	Window-local
MLP-Mixer	MLPs only (no attention)	Classification	None
DETR	Object queries + Hungarian	Detection	Global
Def. DETR	Deformable + multi-scale	Detection	Sparse sampling
TrackFormer	Track queries across frames	Tracking	Global
MaskFormer	Mask classification	Segmentation	Global
Mask2Former	Masked attn + multi-scale	Segmentation	Masked

Evolution:

- **Classification**: ViT → DeiT (data-efficient) → MAE (self-supervised) → Swin (hierarchical) / MLP-Mixer (simplicity)
- **Detection**: DETR (end-to-end) → Deformable DETR (practical) → TrackFormer (video)
- **Segmentation**: MaskFormer (mask classification) → Mask2Former (universal, masked attention)

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models
- CLIP
- SigLIP
- PaliGemma
- Summary

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
    - CLIP
    - SigLIP
    - PaliGemma
    - Summary
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Motivation: Why Multimodal?

Large language models understand text, but the world is *multimodal*: images, video, audio, ...

Key questions:

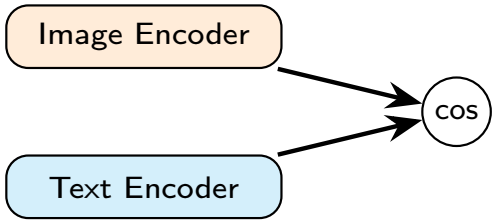
- Can we learn *joint* representations of images and text?
- Can a model *reason* about images using language?
- Can we leverage web-scale image-text data?

Two paradigms:

Contrastive alignment
Learn a shared embedding space for images and text (CLIP, SigLIP)

Generative VLM
Feed visual features into a language model that *generates* text (PaliGemma, LLaVA, ...)

Contrastive (CLIP)



$\Rightarrow$

Generative (PaliGemma)

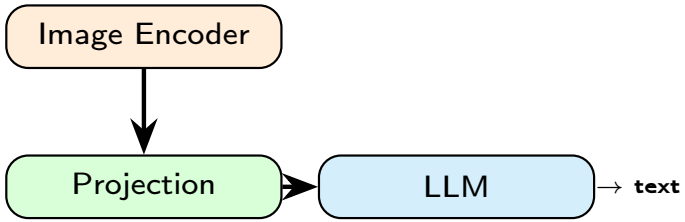


Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models
- CLIP
- SigLIP
- PaliGemma
- Summary

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

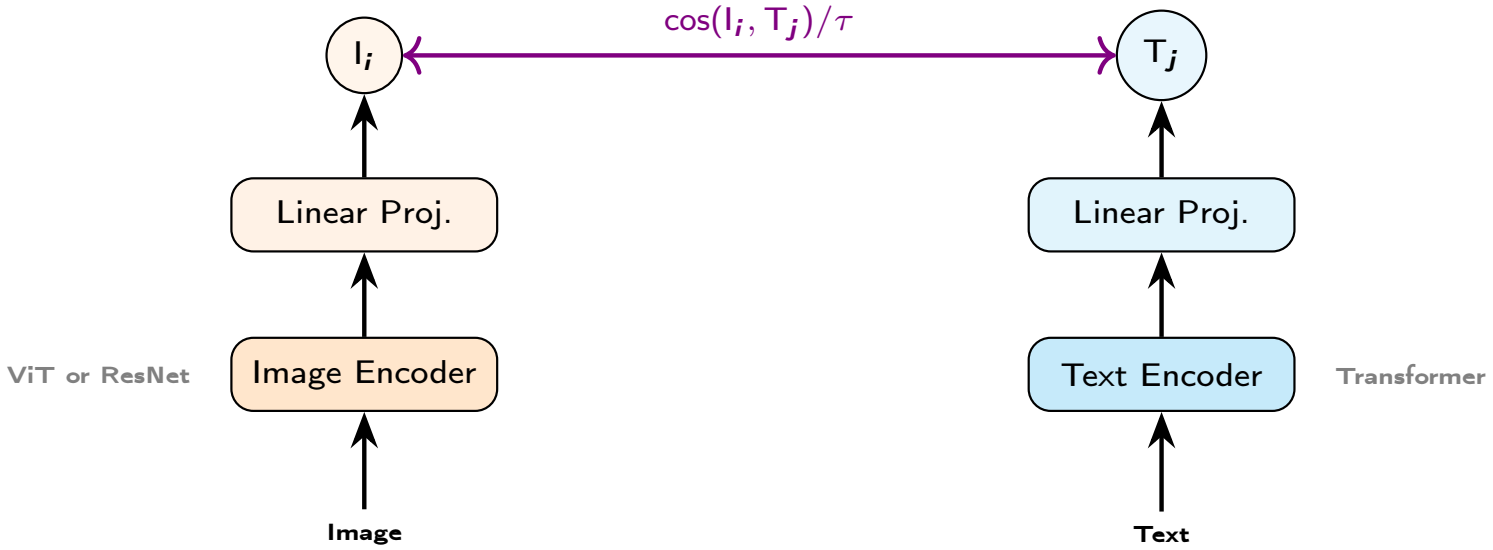
17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

CLIP: Contrastive Language-Image Pre-training

[Radford, Kim, et al., *"Learning Transferable Visual Models From Natural Language Supervision"*, 2021]: Learn visual representations from **natural language supervision** instead of fixed label sets.

Core idea: Train two encoders (image + text) so that matching pairs have high cosine similarity in a shared embedding space.



- **Dataset**: 400M image-text pairs from the web (WebImageText)
- **Encoders**: ViT (best: ViT-L/14) or ResNet for images; Transformer for text
- Both encoders project into a shared d-dimensional space via learned linear projections

CLIP: Contrastive Loss

Given a batch of N image-text pairs, CLIP computes an $N \times N$ matrix of scaled cosine similarities:

$$\ell_{ij} = \frac{\mathsf{I}_i \cdot \mathsf{T}_j}{\tau}$$

where τ is a **learnable temperature** parameter (log-parameterized, initialized to $1/0.07$).
The loss is a **symmetric cross-entropy** over this matrix:

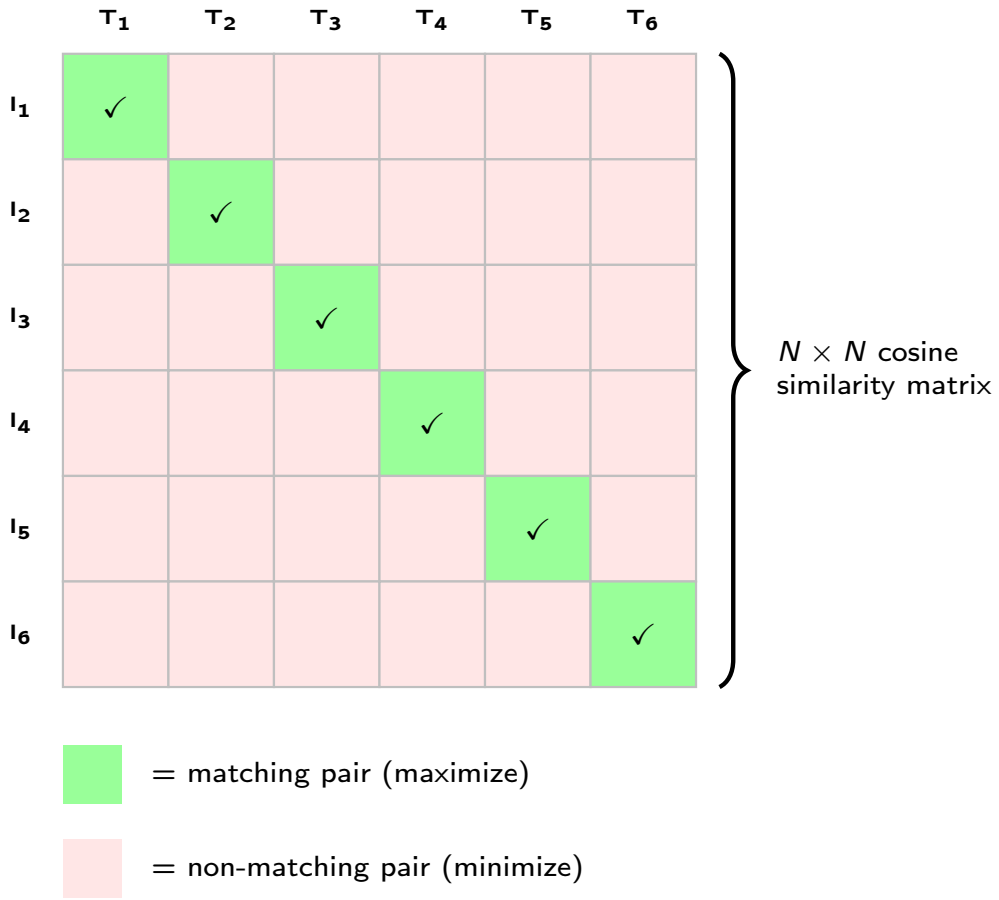
$$\mathcal{L}_{\text{img}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(\ell_{ii})}{\sum_{j=1}^{N}\exp(\ell_{ij})} \qquad \text{(image} \rightarrow \text{text)}$$

$$\mathcal{L}_{\text{txt}} = -\frac{1}{N}\sum_{j=1}^{N} \log \frac{\exp(\ell_{jj})}{\sum_{i=1}^{N}\exp(\ell_{ij})} \qquad \text{(text} \rightarrow \text{image)}$$

$$\mathcal{L}_{\text{CLIP}} = \tfrac{1}{2}\left(\mathcal{L}_{\text{img}} + \mathcal{L}_{\text{txt}}\right)$$

- Ground truth: the **diagonal** of the $N \times N$ matrix (matching pairs)
- Each row = N-way classification: "which text matches this image?"
- Each column = N-way classification: "which image matches this text?"
- Large batch size ($N = 32{,}768$) is critical: more negatives $\Rightarrow$ better representations

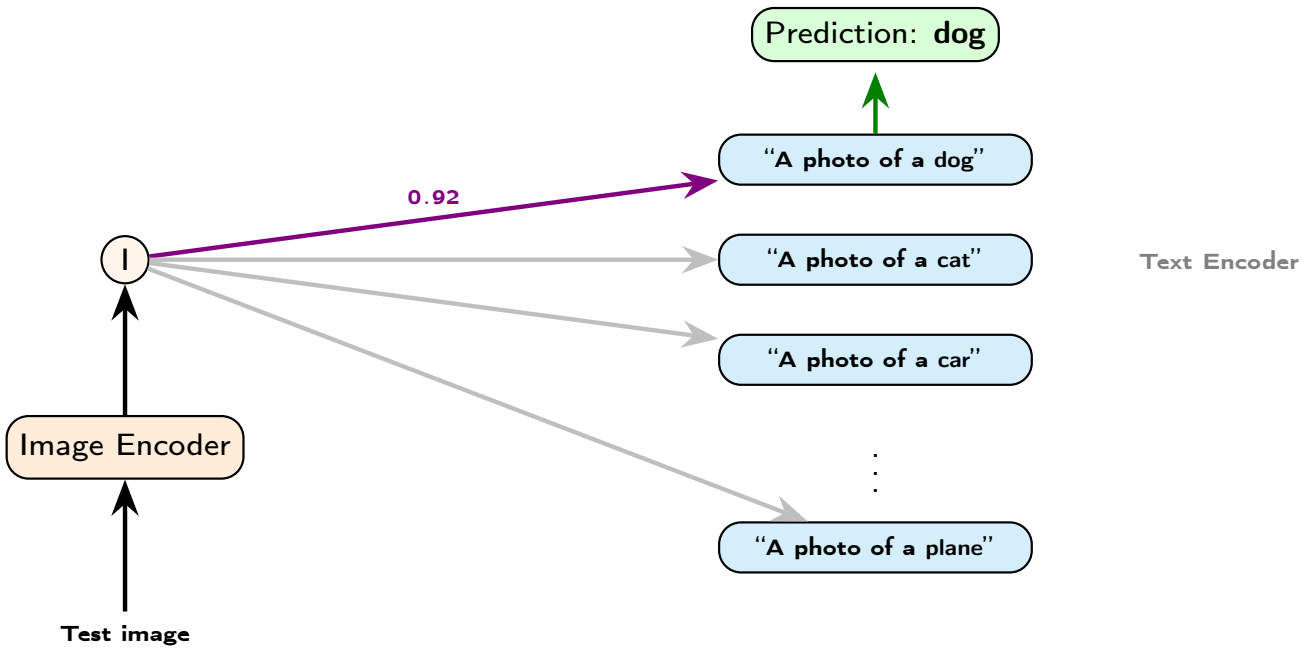
CLIP: Contrastive Loss and Similarity Matrix



Each row and each column is treated as an independent N-way softmax classification. The model learns to push matching pairs (diagonal) together and non-matching pairs apart.

CLIP: Zero-Shot Image Classification

CLIP enables **zero-shot classification** without any task-specific training:



1. Construct text prompts for each class: `"A photo of a {class}."`
2. Encode all prompts with the text encoder → class embeddings
3. Encode the test image → image embedding
4. Predict the class with the highest cosine similarity

Prompt engineering: Ensembling multiple templates (e.g. "A photo of a big {class}", "A bright photo of a {class}") improves accuracy by ∼3–5%.

CLIP: Key Results

Zero-shot performance:

- ImageNet: **76.2%** zero-shot (ViT-L/14@336px)
- Matches supervised ResNet-50 (76.1%) *without seeing a single ImageNet example*
- Competitive on 16 / 27 downstream datasets

Robustness:

- Much more robust to distribution shift than supervised models
- Degrades less on ImageNet-V2, ImageNet-R, ImageNet-Sketch, ObjectNet

Limitations:

- Struggles with fine-grained tasks (car models, flower species)
- Poor at abstract/systematic reasoning (counting objects)
- Cannot *generate* text. Can only compare embeddings

Legacy

CLIP's main impact: the **vision encoder** trained with language supervision became the standard visual backbone for subsequent VLMs (LLaVA, OpenFlamingo, . . .).

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models
- CLIP
- SigLIP
- PaliGemma
- Summary

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

SigLIP: Sigmoid Loss for Language-Image Pre-training

[Zhai et al., *"Sigmoid Loss for Language Image Pre-Training"*, 2023] replaces CLIP's softmax contrastive loss with a **sigmoid loss** that decomposes into independent per-pair terms.

For each pair (i, j) in a batch of N image-text pairs, compute the logit:

$$z_{ij} = \tau \cdot \mathsf{I}_i \cdot \mathsf{T}_j + b$$

where $\tau > 0$ is a learnable temperature (parameterized as $\tau = e^t$ to stay positive) and b is a learnable bias (initialized to -10 to counteract the imbalance between the many negatives and few positives).

$$y_{ij} = \begin{cases} +1 & \text{if } i = j \text{ (matching pair)} \\ -1 & \text{if } i \neq j \text{ (non-matching)} \end{cases}$$

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{N} \log \sigma(y_{ij} \cdot z_{ij})$$

Note: the sum is over all N^2 pairs but divided by N, so each image contributes 1 positive and $N-1$ negative terms.

CLIP (softmax):

- Global normalization over full batch
- Requires all-gather across GPUs
- N-way classification per row/column

SigLIP (sigmoid):

- Independent binary classification per pair
- No global normalization needed
- Scales better to very large batches and distributed training

SigLIP: Architecture and Training

ViT-So400m/14: a *shape-optimized* ViT from [Dehghani et al., *"Getting ViT in Shape: Scaling Laws for Compute-Optimal Model Design"*, 2023]. Instead of uniformly scaling depth/width, a compute-optimal search over depth, width, and MLP dimension finds a better shape for a given parameter budget.

Model	Depth	Width	Heads	MLP	Params
ViT-B/16	12	768	12	3072	86M
ViT-L/16	24	1024	16	4096	307M
So400m/14	27	1152	16	4304	428M
ViT-H/14	32	1280	16	5120	632M

Note: MLP dim $= 3.74\times$ width (not the standard $4\times$). The search finds a non-standard shape: more depth and width than ViT-L but a narrower MLP ratio, achieving better performance than ViT-H at fewer parameters.

Text encoder: same shape as the image encoder (depth 27, width 1152). Fully symmetric two-tower design.

Training:

- **Dataset**: WebLI (Google's web-scraped image-text dataset), $\sim$10B pairs
- **Batch sizes**: from 1K to 1M; sweet spot at $N = 32{,}768$
- Resolution: 224×224, optionally fine-tuned at 384×384
- Shared embedding space dim: 1152 (= hidden width, no extra projection)

SigLIP: Results

Zero-shot ImageNet top-1 accuracy:

Model	Params	IN top-1
ResNet-50 (supervised)	25M	76.1%
CLIP ViT-L/14@336	428M	76.2%
SigLIP B/16@256	86M	76.6%
SigLIP L/16@256	307M	80.5%
SigLIP L/16@384	307M	82.1%
SigLIP So400m/14@224	428M	82.2%
SigLIP So400m/14@384	428M	83.2%

- SigLIP B/16 already matches CLIP ViT-L/14 and supervised ResNet-50
- Shape-optimized So400m/14 pushes to **83.2%** zero-shot
- Higher resolution at fine-tuning consistently helps (+1–2%)

SigLIP: Sigmoid vs. Softmax at Different Batch Sizes

ViT-B/16 on WebLI, zero-shot ImageNet accuracy (%). Same architecture, same data, only the loss changes.

	3B examples seen		9B examples seen	
Batch size	Sigmoid	Softmax	Sigmoid	Softmax
512	51.5	47.7	—	—
1K	57.3	53.2	—	—
4K	65.3	63.8	68.4	66.6
8K	68.6	66.6	70.6	69.4
16K	—	—	72.3	71.7
32K	69.9	69.9	**73.4**	72.9
98K	69.5	69.7	73.0	**73.2**

- **Small batches**: sigmoid clearly better (e.g. +4.1% at batch 1K)
- **Medium batches**: sigmoid peaks earlier (32K vs. 98K for softmax)
- **Large batches**: both losses converge; softmax slightly better at 98K with long training
- Sigmoid does not require cross-device all-gather for the normalizing denominator

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models
- CLIP
- SigLIP
- PaliGemma
- Summary

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

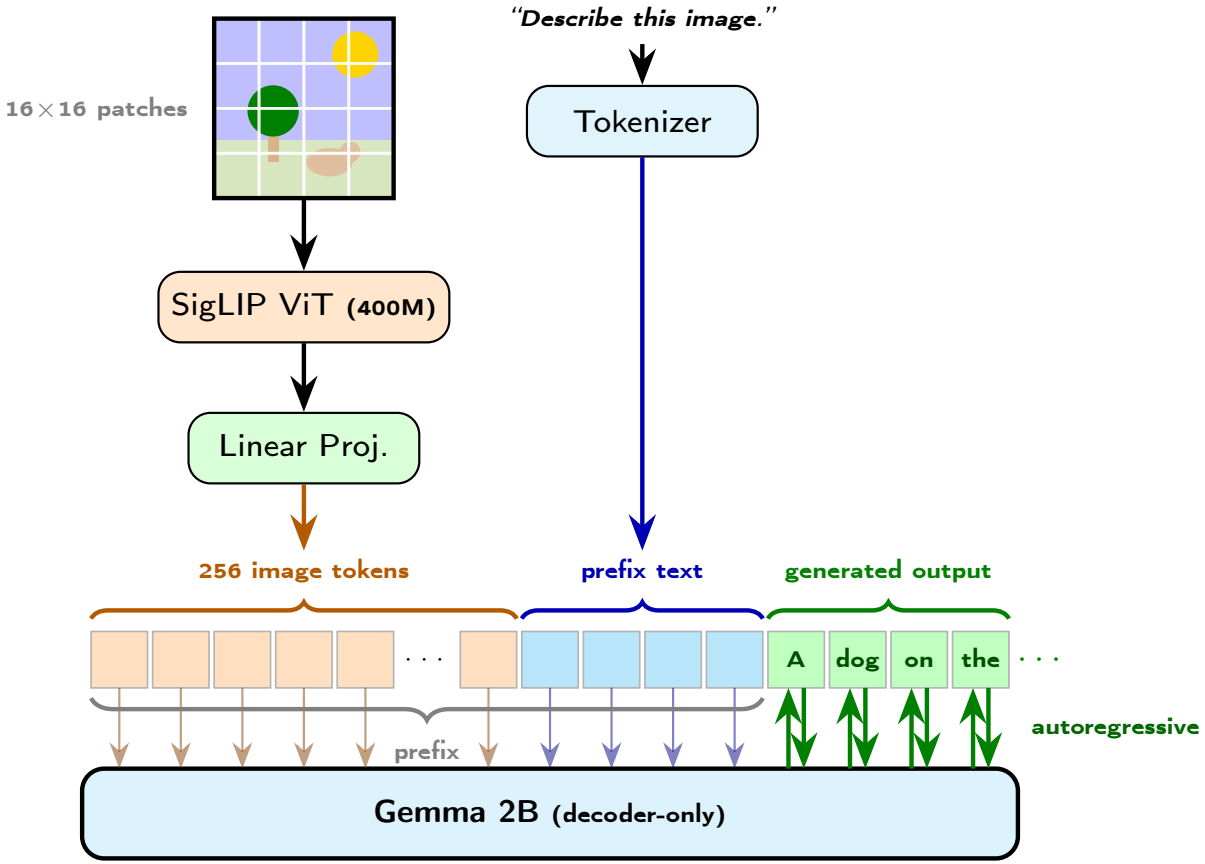
16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

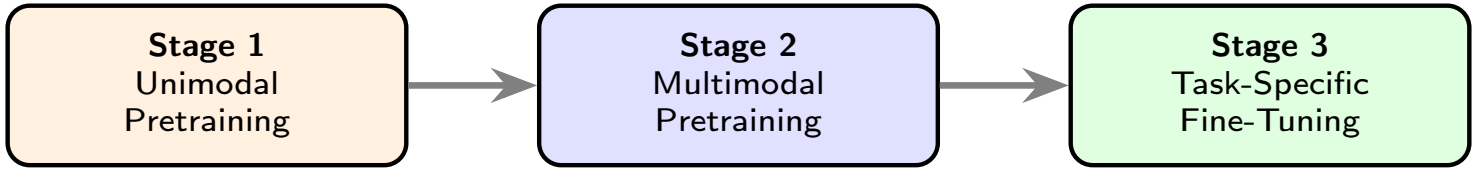
PaliGemma: A Versatile Vision-Language Model

[Beyer et al., *"PaliGemma: A Versatile 3B VLM for Transfer"*, 2024]: Combine a strong vision encoder (SigLIP) with a strong LLM (Gemma) via a **simple linear projection**.



- **Total**: ∼3B parameters (400M vision + 2B language + linear projection)
- **Key design choice**: A simple linear connector suffices when both components are individually strong

PaliGemma: Three-Stage Training



Stage 1: Unimodal pretraining

SigLIP and Gemma are pretrained *separately* on their respective data.

- SigLIP: contrastive (sigmoid) loss on image-text pairs
- Gemma: autoregressive next-token prediction on text

Stage 2: Multimodal pretraining

Full model trained jointly on a large mixture of multimodal tasks (captioning, VQA, OCR, detection, ...). Vision encoder is *unfrozen*. Resolution: 224×224 (256 image tokens).

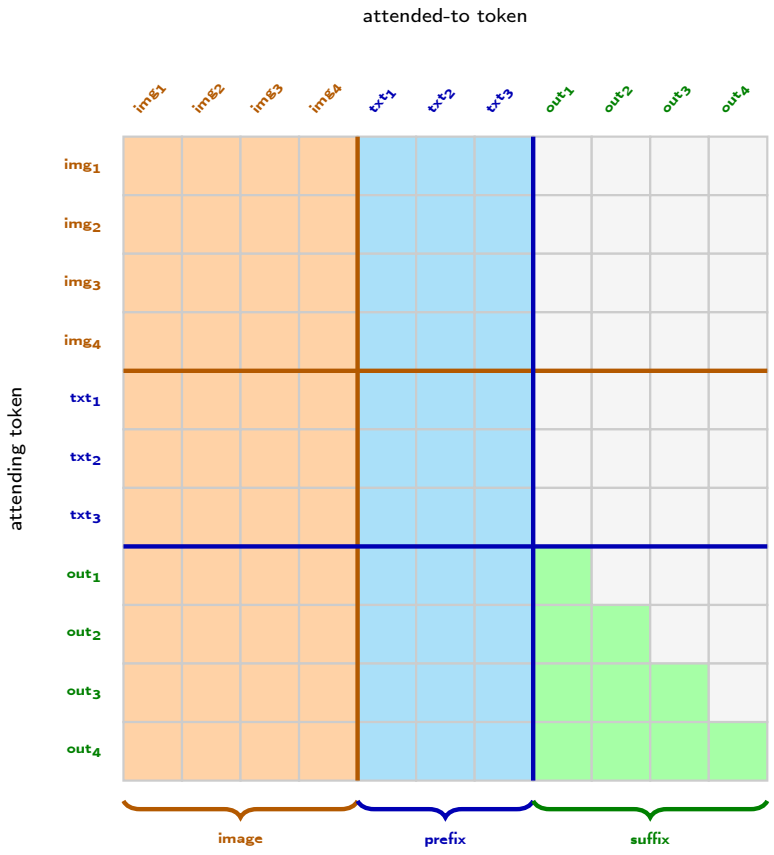
- **Loss**: Purely autoregressive next-token prediction on *suffix (output) tokens only*. No contrastive loss
- Image and prefix text tokens are excluded from the loss (prefix-LM masking)
- Gradients still flow into the vision encoder via backpropagation through the linear projection

Stage 3: Task-specific fine-tuning

Fine-tune on individual downstream benchmarks. Same loss as Stage 2.

- Resolution can be increased: 448×448 ($\rightarrow$ 1024 tokens) or 896×896 ($\rightarrow$ 4096 tokens) for detail-oriented tasks (OCR, documents)

PaliGemma: Prefix-LM Attention Mask



Prefix-LM masking:

- **Image** + **prefix text** tokens: **bidirectional** attention (full block)
- **Suffix (output)** tokens: **causal** attention (lower triangle)
- Suffix tokens attend to all prefix tokens + causally to previous suffix tokens

Loss masking:

- Loss computed *only* on **suffix** tokens
- Image and prefix tokens are excluded
- Gradients still flow into the vision encoder via backpropagation

Colored cells = attention allowed.
Gray cells = masked (no attention).

PaliGemma: Tasks and Capabilities

PaliGemma handles diverse vision-language tasks through **text prompts**:

Task	Prompt	Output
Captioning	`caption en`	"A dog running on the beach"
VQA	`What color is the car?`	"red"
Object detection	`detect dog`	`<loc0123><loc0456>... dog`
OCR	`ocr`	Extracted text from image

Object detection via text:

- Bounding boxes encoded as special `<loc>` tokens
- Normalized coordinates in $[0, 1023]$
- No task-specific detection head needed

Why this works:

- All tasks unified as text generation
- No task-specific architectures
- Same model, different prompts
- Higher resolution for detail-oriented tasks (OCR, documents)

PaliGemma: Benchmark Results

PaliGemma (3B) vs. larger VLMs after task-specific fine-tuning. Bold = best in column.

Model	Size	Res.	VQAv2	TextVQA	DocVQA	ChartQA	AI2D	SciQA	GQA
LLaVA-1.5	7B	336	78.5	58.2	–	–	–	66.8	62.0
LLaVA-1.5	13B	336	80.0	61.3	–	–	–	71.6	63.3
Qwen-VL-Chat	10B	448	78.2	61.5	65.1	65.7	62.3	68.2	57.5
PaliGemma	3B	224	83.2	55.5	43.7	57.1	72.1	95.4	65.6
PaliGemma	3B	448	**85.6**	73.2	78.0	71.4	**73.3**	**95.9**	**67.0**
PaliGemma	3B	896	–	**76.5**	**84.8**	–	–	–	–

SciQA = ScienceQA (image subset). All PaliGemma numbers from [Beyer et al., "*PaliGemma: A Versatile 3B VLM for Transfer*", 2024].

- 3B model outperforms LLaVA-1.5-13B on most benchmarks
- **Resolution is critical**: DocVQA jumps from 43.7 (224px) to 84.8 (896px)
- ScienceQA at 95.9% — remarkable for a 3B model

PaliGemma: OCR, Captioning & Localization

OCR & text-heavy benchmarks:

Model	Res.	OCR-VQA	ST-VQA	InfoVQA	TallyQA$^{s/c}$
PaliGemma-3B	224	73.2	63.3	28.5	81.7 / 69.6
PaliGemma-3B	448	75.6	81.8	40.5	84.9 / 72.3
PaliGemma-3B	896	**75.9**	**84.4**	**47.8**	–

Captioning & referring expression segmentation:

Model	Res.	COCO^{CIDEr}	TextCaps^{CIDEr}	RefCOCO	RefCOCO+	RefCOCOg
PaliGemma-3B	224	141.9	127.5	73.4	68.3	67.7
PaliGemma-3B	448	**144.6**	**153.9**	75.6	69.8	70.2
PaliGemma-3B	896	–	–	**76.9**	**72.2**	**72.2**

RefCOCO metrics: Mean IoU. TallyQA$^{s/c}$ = simple/complex split.

Design Lessons

1. A **simple linear connector** suffices when both vision encoder and LLM are strong
2. **Resolution matters**: cheap to increase at fine-tuning, large gains on detail-oriented tasks
3. **Unified text generation** handles captioning, VQA, detection, OCR without architectural changes

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models
- CLIP
- SigLIP
- PaliGemma
- Summary

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

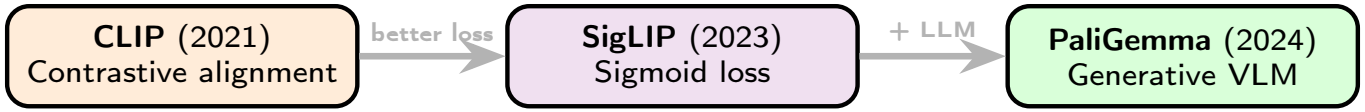
16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Multimodal Models: Summary

Evolution of vision-language models:



CLIP:

- Dual encoder, shared embedding space
- Softmax contrastive loss (InfoNCE)
- Zero-shot classification via text prompts
- Cannot generate text

PaliGemma:

- SigLIP vision encoder + Gemma LLM
- Simple linear projection connector
- Generates text conditioned on images
- Unified interface for captioning, VQA, detection, OCR

Key insight: Contrastive pretraining produces excellent visual features. Pairing them with a language model unlocks generative capabilities far beyond embedding comparison.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
    - Classical Hyperparameter Optimization
    - Learning Rate Schedules
    - Batch Size Selection
    - Weight Decay and AdamW
    - Scaling Laws
    - μP: Hyperparameter Transfer Across Scales
    - LLM Training Recipes
    - Practical Guidelines
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

Why Hyperparameters Matter

Deep learning performance is highly sensitive to hyperparameter choices. Poor choices can lead to divergence, slow convergence, or suboptimal generalization.

Key hyperparameters in LLM training:

- **Learning rate** and its *schedule* (warmup, decay shape)
- **Batch size** (interacts with learning rate)
- **Weight decay** (regularization strength)
- **Adam parameters** ($\beta_1, \beta_2, \epsilon$)
- **Architectural choices**: depth, width, number of heads
- **Data mix** and *training duration* (tokens seen)

Challenge: Each training run takes a long time. We cannot do grid search over hyperparameters.

$\Rightarrow$ Need principled, efficient strategies & insights.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

- Classical Hyperparameter Optimization
- Learning Rate Schedules
- Batch Size Selection
- Weight Decay and AdamW
- Scaling Laws
- μP: Hyperparameter Transfer Across Scales
- LLM Training Recipes
- Practical Guidelines

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG, and

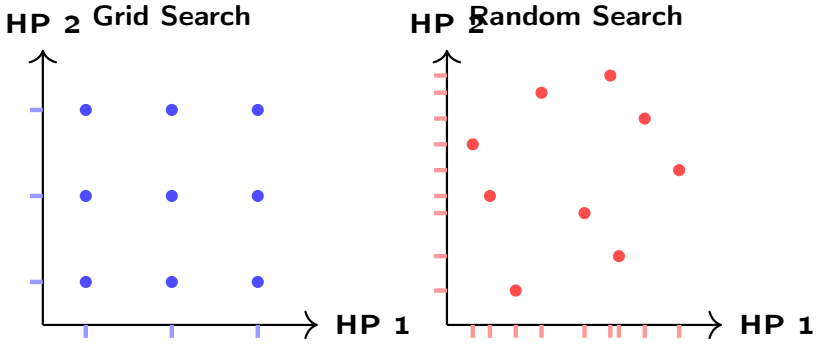
Grid Search vs. Random Search

Grid search:

- Evaluate all combinations on a regular grid
- Cost grows *exponentially* in the number of hyperparameters: $\mathcal{O}(k^d)$
- Wastes budget on unimportant dimensions

Random search [Bergstra and Bengio, *"Random Search for Hyper-Parameter Optimization"*, 2012]:

- Sample configurations uniformly at random
- Each trial explores a *distinct value* for every dimension
- Provably more efficient when only a few hyperparameters matter (low effective dimensionality)
- Matches or beats grid search using a fraction of the budget



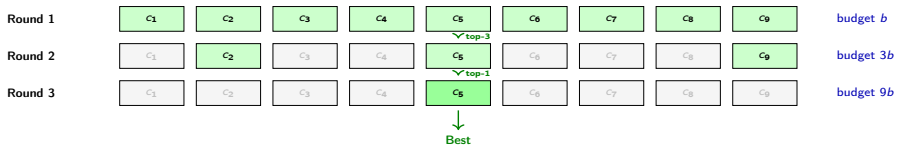
Random search projects onto more distinct values per axis (tick marks). After Bergstra & Bengio (2012).

Successive Halving

Key idea: Most bad configurations can be identified early. Stop them quickly and reallocate budget to promising ones [L. Li et al., *"Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization"*, 2017].

Algorithm (SHA):

1. Sample n random configurations
2. Train each for a small budget b
3. Evaluate; keep the top $1/\eta$ fraction (typically $\eta = 3$)
4. Multiply budget by η, repeat until one remains



Example: n=9, η=3. Each round keeps top $1/\eta$ and multiplies budget by η.

Properties:

- Total compute $\approx n \cdot B_{\max} / \log_\eta(n)$
- Much less than training all n to completion
- Tradeoff: more initial configs $\Rightarrow$ better coverage but more aggressive early stopping

Hyperband

Problem with SHA: Must choose n upfront: trade-off between coverage and early stopping aggressiveness.

Hyperband [L. Li et al., *"Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization"*, 2017] runs *multiple independent SHA instances* (called *brackets*) in parallel, each with a different n-vs-budget trade-off:

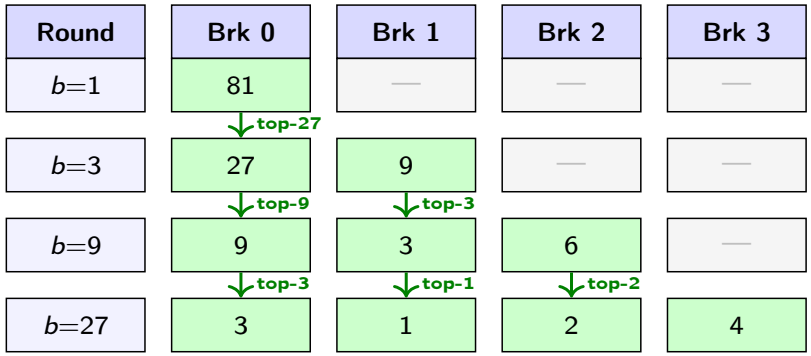
Setup: Given total budget B and η, create $s_{\max}+1$ brackets. Each bracket *independently* samples its own fresh random configs:

Brk 0: sample 81 configs, start at budget b=1

Brk 1: sample 9 configs, start at budget b=3

Brk 2: sample 6 configs, start at budget b=9

Brk 3: sample 4 configs, start at budget b=27



Cells = #configs at each budget level (η=3). Arrows: top-1/η advance to next round.

Within each bracket: run SHA: evaluate, keep best $1/\eta$, multiply budget by η, repeat.

Output: Return the best config across all brackets.

Each bracket uses $\approx$ the same total compute. Provides $\approx \log(B_{\max})$ speedup over random search.

TPE: Tree-structured Parzen Estimator

TPE follows the idea of Bayesian optimization but takes a different approach [Bergstra, Bardenet, et al., *"Algorithms for Hyper-Parameter Optimization"*, 2011].

KDE: Given points $\{x_1, \ldots, x_n\}$, estimate a smooth density: $\hat{p}(x) = \frac{1}{n} \sum_i K_h(x - x_i)$

Core idea: Split trials at threshold y^* (e.g. top 20%). Fit a KDE to each group:

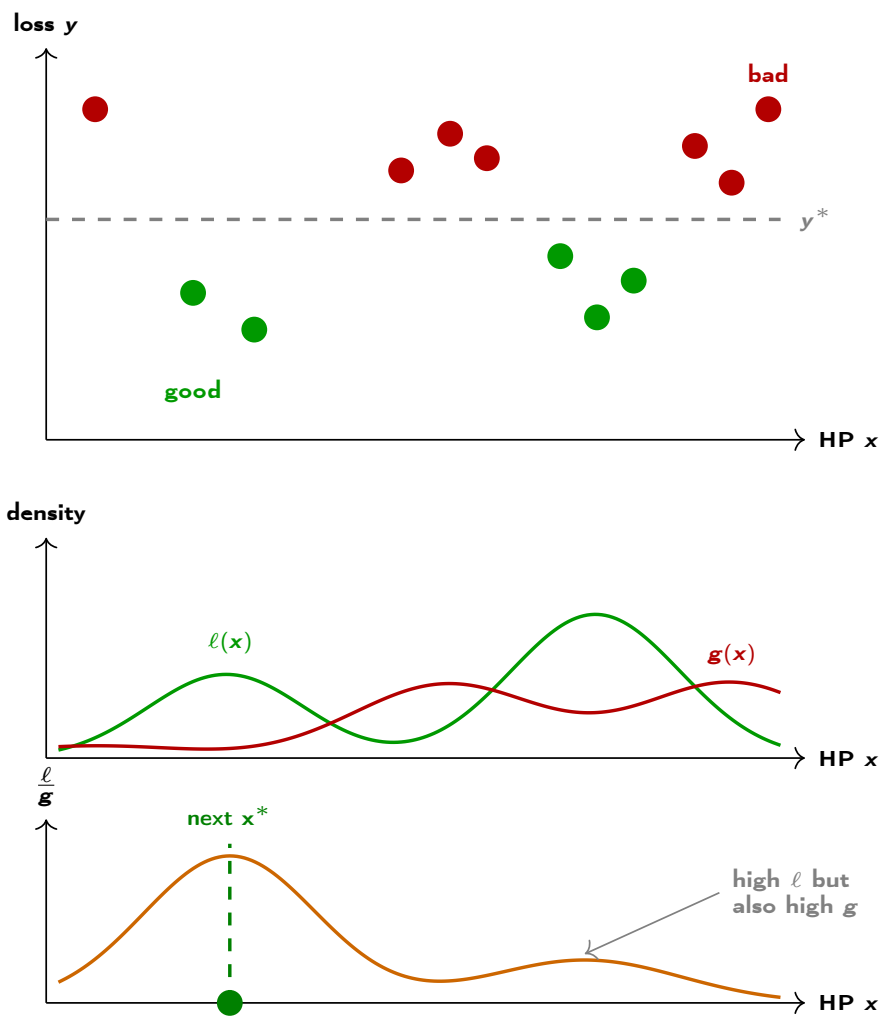
$$p(\mathrm{x} \mid y) = \begin{cases} \ell(\mathrm{x}) & y < y^* \text{ (good)} \\ g(\mathrm{x}) & y \geq y^* \text{ (bad)} \end{cases}$$

Next config: Sample from $\ell(\mathrm{x})$, pick the one maximizing $\ell(\mathrm{x})/g(\mathrm{x})$.

- High ℓ: looks good. Low g: not bad.
- $\ell/g \propto$ Expected Improvement

Each HP modelled independently $\Rightarrow$ scales to many dims. Default in Optuna [Akiba et al., *"Optuna: A Next-generation Hyperparameter Optimization Framework"*, 2019].

Note: TPE can be understood as Bayesian optimization where the density ratio ℓ/g plays the



BOHB: Bayesian Optimization and Hyperband

Problem: Hyperband explores broadly but samples configs *randomly*. TPE samples intelligently but evaluates each config to completion (no early stopping).

BOHB [Falkner et al., *"BOHB: Robust and Efficient Hyperparameter Optimization at Scale"*, 2018] combines both: use TPE to *sample* configs, Hyperband to *schedule* them.

Algorithm:

1. Run Hyperband's bracket structure (multiple SHA instances with different budgets)
2. Instead of sampling configs uniformly at random, use a **TPE model** fitted on completed evaluations at the *current budget level*
3. As more configs finish, the TPE model improves $\Rightarrow$ later configs are better

Key design choices:

- Fit TPE only on results from the *same budget level* (short runs inform short-run sampling, long runs inform long-run sampling)
- Fall back to random sampling until enough observations exist (typically ≥ 10)
- Inherits Hyperband's parallelism: multiple brackets run concurrently

Result: Matches Hyperband's fast early progress *and* TPE's late-stage refinement. Consistently among the best methods across benchmarks.

Comparing HPO Methods

When controlling for total compute budget, there is no single best method [Falkner et al., *"BOHB: Robust and Efficient Hyperparameter Optimization at Scale"*, 2018].

Method	Strengths	Weaknesses	Speedup vs. RS
Random search	Trivially parallel, strong baseline	No learning from past trials	1× (baseline)
Hyperband / SHA	Fast with small budgets; explores broadly	Relies on cheap proxies being informative	5–30×
TPE / Bayesian opt.	Exploits structure; best with few HPs	Sequential; slow start	2–5×
BOHB	Informed sampling + early stopping	More complex to implement	10–50×

Speedups from [L. Li et al., *"Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization"*, 2017] and [Falkner et al., *"BOHB: Robust and Efficient Hyperparameter Optimization at Scale"*, 2018], measured as wall-clock time to reach target validation error on neural network benchmarks.

Key findings:

- **Small compute budget:** Hyperband wins: aggressive early stopping explores more configs
- **Large compute budget:** Bayesian methods (TPE) win: learned surrogate model pays off
- **General purpose:** BOHB (Hyperband + TPE) rarely loses badly to any single method
- **Random search** is competitive in high dimensions and embarrassingly parallel settings

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
    - Classical Hyperparameter Optimization
    - Learning Rate Schedules
    - Batch Size Selection
    - Weight Decay and AdamW
    - Scaling Laws
    - μP: Hyperparameter Transfer Across Scales
    - LLM Training Recipes
    - Practical Guidelines
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

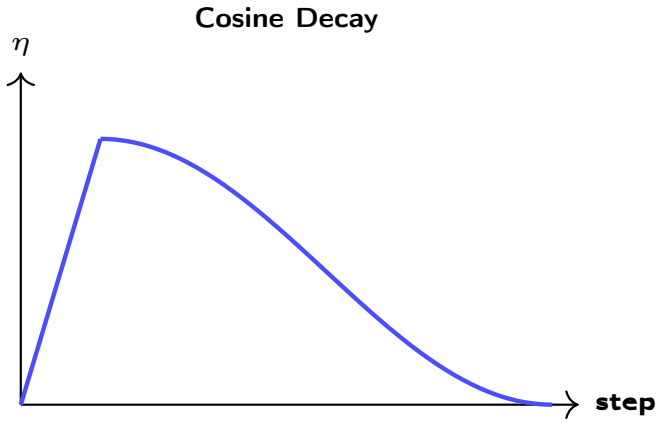
Learning Rate: The Most Important Hyperparameter

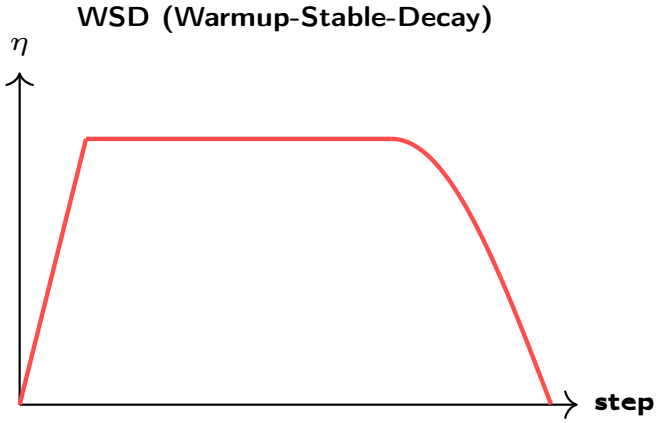
The learning rate η controls the step size of gradient updates. Too high $\Rightarrow$ divergence. Too low $\Rightarrow$ slow convergence / poor optima.

LR Range Test [Smith, *"Cyclical Learning Rates for Training Neural Networks"*, 2017]:

- Increase η exponentially from very small to very large over one epoch
- Plot loss vs. η; the optimal range is where loss decreases fastest
- Quick diagnostic before a full training run

Common LR schedules for LLM pretraining:





Cosine decay [Loshchilov and Hutter, *"SGDR: Stochastic Gradient Descent with Warm Restarts"*, 2017]: Standard for GPT-3, LLaMA, Chinchilla. **WSD**: Used in MiniCPM, recent practice. Allows extending training without restarting.

Learning Rate Warmup

Problem: Adam's moving-average estimates m_t, v_t are initialized to zero. Early in training they are heavily biased, so the effective step sizes $m_t/\sqrt{v_t}$ can be unreliable and cause divergent updates.

Solution: Start with $\eta \approx 0$ and linearly increase to the target $\eta_{\max}$ over a *warmup period*, giving the moment estimates time to stabilize [P. Goyal et al., *"Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour"*, 2017].

$$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}} & \text{if } t \leq T_{\text{warmup}} \\ \text{schedule}(\eta_{\max}, t) & \text{otherwise} \end{cases}$$

Typical warmup durations:

- GPT-3: 375M tokens ($\approx$ 0.1% of training) [Brown et al., *"Language Models are Few-Shot Learners"*, 2020]
- LLaMA: 2000 steps [Touvron, Lavril, et al., *"LLaMA: Open and Efficient Foundation Language Models"*, 2023]
- Chinchilla: 5B tokens [Hoffmann et al., *"Training Compute-Optimal Large Language Models"*, 2022]
- Rule of thumb: $\sim$0.1–1% of total training steps

Why it works:

- Gives Adam's moment estimates (m_t, v_t) enough samples to become reliable
- Also helps with large early gradients (complementary to gradient clipping)
- Especially important with large batch sizes: linear scaling rule $\Rightarrow$ larger LR $\Rightarrow$ unreliable early steps are amplified

Choosing a Good Learning Rate

Safe LR regimes by task:

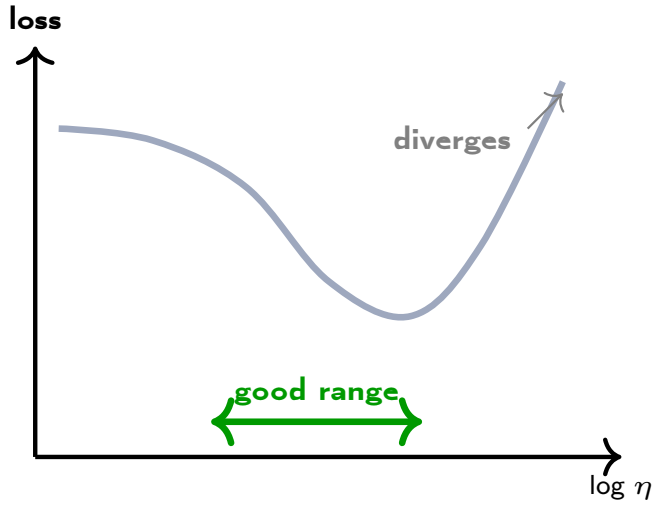
From scratch 10^{-4}–10^{-3} (Adam/AdamW). Larger models ⇒ lower end. Use LR range test or μP to narrow down.

Fine-tune 10^{-5}–5×10^{-5} (full fine-tuning). 10–50× smaller than pretraining LR to avoid catastrophic forgetting.

LoRA/PEFT 10^{-4}–3×10^{-4}. Adapters have few parameters ⇒ can tolerate higher LR than full fine-tuning.

SGD (vision) 10^{-2}–10^{-1}. SGD needs ∼100× higher LR than Adam since it lacks adaptive scaling.

LR range test [Smith, *"Cyclical Learning Rates for Training Neural Networks"*, 2017]:



Sweep η exponentially over one epoch. Pick the range where loss drops fastest (steepest descent), not the minimum.

Rules of thumb:

- If loss spikes early: reduce LR or increase warmup
- If loss plateaus: try 2–3× higher LR
- When scaling batch size $B \to kB$: scale $\eta \to k\eta$ (linear) or $\eta \to \sqrt{k}\,\eta$ (square

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
    - Classical Hyperparameter Optimization
    - Learning Rate Schedules
    - Batch Size Selection
    - Weight Decay and AdamW
    - Scaling Laws
    - μP: Hyperparameter Transfer Across Scales
    - LLM Training Recipes
    - Practical Guidelines
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

Batch Size and the Linear Scaling Rule

The mini-batch gradient *averages* over B random samples:

$$\hat{g}_B = \frac{1}{B}\sum_{i=1}^{B} \nabla L_i(\mathrm{w}) \qquad \Rightarrow \qquad \mathrm{w}_{t+1} = \mathrm{w}_t - \eta\, \hat{g}_B$$

Linear scaling rule [P. Goyal et al., *"Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour"*, 2017]: With $k\times$ larger batch we take $k\times$ fewer steps per epoch. To cover the same distance in weight space, scale $\eta \to k\eta$:

1 step, batch kB, rate $k\eta$

$$\mathrm{w} - k\eta\, \hat{g}_{kB}$$

$$\approx$$

k steps, batch B, rate η

$$\mathrm{w} - \eta \sum_{j=1}^{k} \hat{g}_B^{(j)}$$

Both sides have the same *expected* update: $\mathbb{E}[\hat{g}_{kB}] = \mathbb{E}[\hat{g}_B]$ because both are sample means of i.i.d. draws (batch size only affects variance, not expectation).

Limitations:

- Assumes $\mathrm{Var}[\hat{g}]$ is small enough that linear approximation holds, but breaks for very large B
- Does not account for Adam's adaptive scaling (empirically still useful)
- Requires LR warmup to work at large batch sizes

Square Root Learning Rate Scaling

Alternative: Scale $\eta \to \sqrt{k}\,\eta$ when $B \to kB$ [Hoffer et al., *"Train longer, generalize better: closing the generalization gap in large batch training"*, 2017].

Motivation from gradient noise:

- The gradient estimate has variance $\mathrm{Var}[\hat{g}_B] = \sigma^2/B$, so its std scales as $1/\sqrt{B}$
- One SGD step moves weights by $\Delta \mathrm{w} = -\eta\,\hat{g}_B = -\eta\,(g + \varepsilon/\sqrt{B})$
- Scaling $\eta \to \sqrt{k}\,\eta$ with $B \to kB$ keeps the *noise contribution* $\eta \cdot \sigma/\sqrt{B}$ constant
- Linear scaling ($\eta \to k\eta$) would amplify noise by $\sqrt{k}$

Linear vs. square root scaling:

Linear Preserves expected update magnitude. Works well for moderate k ($\leq$ 8–16) with LR warmup. Default in most recipes.

Square root Preserves signal-to-noise ratio. More robust for very large B (above B_{crit}) where linear scaling causes instability.

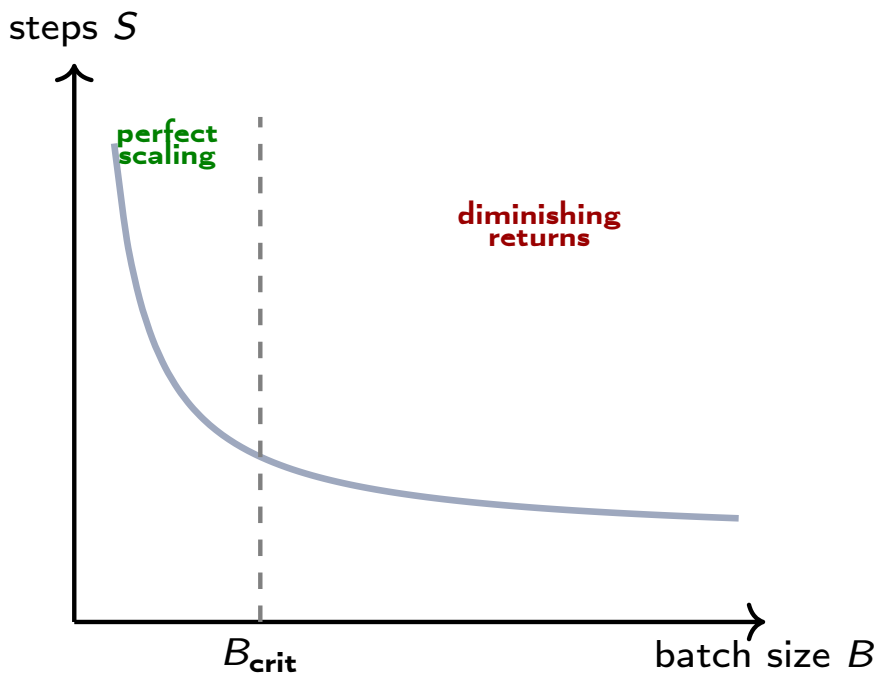
In practice, the optimal exponent α in $\eta \to k^{\alpha}\eta$ is often between 0.5 and 1.0, depending on optimizer, model, and training stage.

Critical Batch Size

Why maximize batch size? Larger B means fewer gradient steps per epoch $\Rightarrow$ fewer synchronization rounds in distributed training, better GPU utilization, and faster wall-clock time. But there is a limit:

Critical batch size [McCandlish et al., *"An Empirical Model of Large-Batch Training"*, 2018]:

- Below B_{crit}: doubling B halves training time (perfect scaling)
- Above B_{crit}: diminishing returns
- $B_{\text{crit}} \propto 1/L$: smaller loss $\Rightarrow$ larger B_{crit}



Typical batch sizes:

LLM GPT-3: 3.2M tokens, LLaMA-2: 4M, Chinchilla: 1.5M, DeepSeek-V3: 15.6M

Vision ViT-L: 4096 images, CLIP: 32k image-text pairs, MAE: 4096

RL PPO collects trajectories (e.g. 2048 steps $\times$ 32 envs $=$ 65k state-action pairs), then trains for multiple epochs on random mini-batches (e.g. 4096 pairs) subsampled from the buffer

Gradient Noise Scale and Batch Size Ramp-Up

Gradient noise scale [McCandlish et al., *"An Empirical Model of Large-Batch Training"*, 2018]:

$$G = \mathbb{E}[\nabla L_i]$$

$$\Sigma = \mathrm{Cov}(\nabla L_i) = \mathbb{E}[(\nabla L_i - G)(\nabla L_i - G)^\top]$$

$$\mathcal{B}_{\text{noise}} = \frac{\mathrm{tr}(\Sigma)}{|G|^2}$$

The mini-batch gradient has variance $\mathrm{Var}[\hat{g}_B] = \Sigma/B$, so the noise-to-signal ratio of a step is:

$$\frac{\mathrm{tr}(\Sigma/B)}{|G|^2} = \frac{\mathcal{B}_{\text{noise}}}{B}$$

- $B \ll \mathcal{B}_{\text{noise}}$: ratio $\gg 1$, step is mostly noise $\Rightarrow$ but cheap, so efficient
- $B \approx \mathcal{B}_{\text{noise}}$: ratio ≈ 1, noise $\approx$ signal $\Rightarrow$ sweet spot
- $B \gg \mathcal{B}_{\text{noise}}$: ratio $\ll 1$, very accurate but extra samples wasted

Batch size ramp-up:
Since $B_{\text{crit}} \propto 1/L$, it grows during training as loss decreases. Ramp up B to stay in the efficient regime:

Algorithm 8: Batch Size Schedule

Input: $B_{\min}$, $B_{\max}$, ramp tokens T_{ramp}
$B \leftarrow B_{\min}$
for *each training step* t **do**
  if *tokens seen* $< T_{\text{ramp}}$ **then**
    $B \leftarrow B_{\min} + (B_{\max} - B_{\min}) \cdot \frac{\text{tokens seen}}{T_{\text{ramp}}}$
  else
    $B \leftarrow B_{\max}$
  Train one step with batch size B

GPT-3: 32k $\rightarrow$ 3.2M tokens over first 4–12B tokens [Brown et al., *"Language Models are Few-Shot Learners"*, 2020]

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
    - Classical Hyperparameter Optimization
    - Learning Rate Schedules
    - Batch Size Selection
    - Weight Decay and AdamW
    - Scaling Laws
    - μP: Hyperparameter Transfer Across Scales
    - LLM Training Recipes
    - Practical Guidelines
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

Weight Decay: L2 Regularization Done Right

Problem with L2 regularization in Adam:
In standard Adam + L2, the regularization gradient λw gets divided by the adaptive second moment. This *decouples* the effective regularization from λ [Loshchilov and Hutter, *"Decoupled Weight Decay Regularization"*, 2019].

Adam + L2 (wrong):

$$w_{t+1} = w_t - \eta \cdot \frac{\hat{m}_t + \lambda w_t}{\sqrt{\hat{v}_t} + \epsilon}$$

The λw_t term is scaled by $1/\sqrt{\hat{v}_t}$, giving inconsistent regularization.

AdamW (correct):

$$w_{t+1} = (1 - \eta\lambda)\, w_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Weight decay is applied *directly* to the weights, independent of gradients.

Practical defaults for LLMs:

- Weight decay $\lambda = 0.1$ (nearly universal: GPT-3, LLaMA, Chinchilla, DeepSeek)
- Apply weight decay to *all weight matrices*, but **not** to biases, LayerNorm parameters, or embeddings
- Adam betas: $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$
- **Dropout:** Modern LLMs use $p = 0$ during pretraining (single-epoch $\Rightarrow$ no overfitting). Use $p = 0.1$–0.3 for fine-tuning on small datasets.

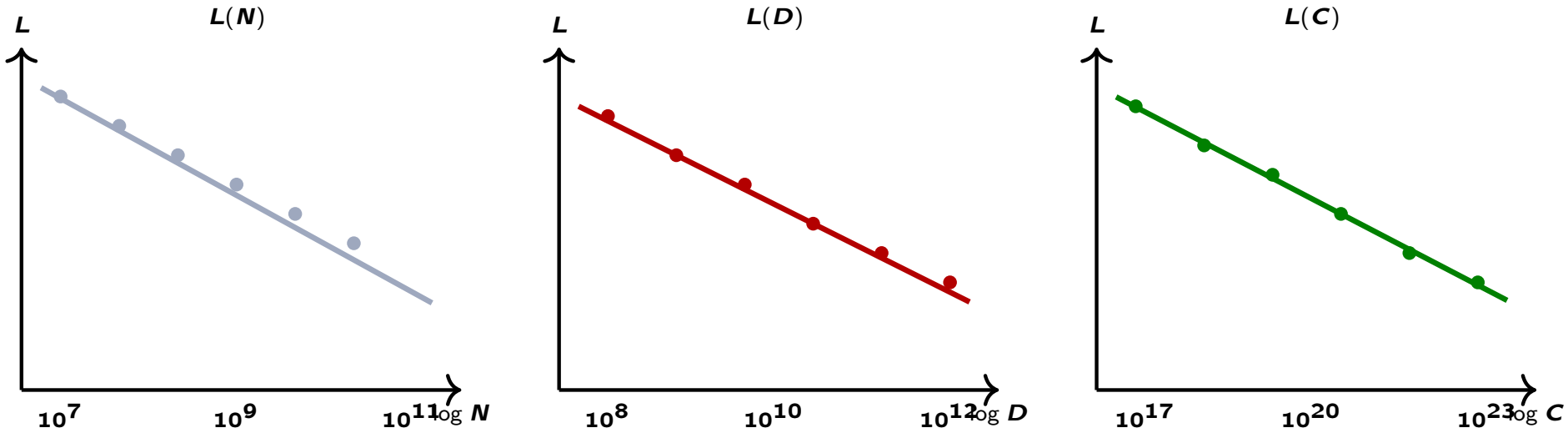
Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
    - Classical Hyperparameter Optimization
    - Learning Rate Schedules
    - Batch Size Selection
    - Weight Decay and AdamW
    - Scaling Laws
    - μP: Hyperparameter Transfer Across Scales
    - LLM Training Recipes
    - Practical Guidelines
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

Neural Scaling Laws [J. Kaplan et al., *"Scaling Laws for Neural Language Models"*, 2020]

Key discovery: LLM test loss follows *power laws* in model size N (non-embedding params), dataset size D (tokens), and compute C (FLOPs):

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \quad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \quad L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C}$$



	α	Scale constant	Optimal alloc. $\propto C^?$
Parameters N	0.076	$N_c = 8.8 \times 10^{13}$	$N \propto C^{0.73}$
Data D	0.095	$D_c = 5.4 \times 10^{13}$	$D \propto C^{0.27}$
Compute C	0.050	$C_c = 3.1 \times 10^8$ PF-days	—

Straight lines on log-log plots $\Rightarrow$ power laws. Relationship: $C \approx 6ND$ (FLOPs).

Chinchilla Scaling Laws [Hoffmann et al., *"Training Compute-Optimal Large Language Models"*, 2022]

Chinchilla correction: Kaplan et al. recommended scaling N faster than D. Hoffmann et al. showed N and D should be scaled *equally*:

Joint scaling law:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

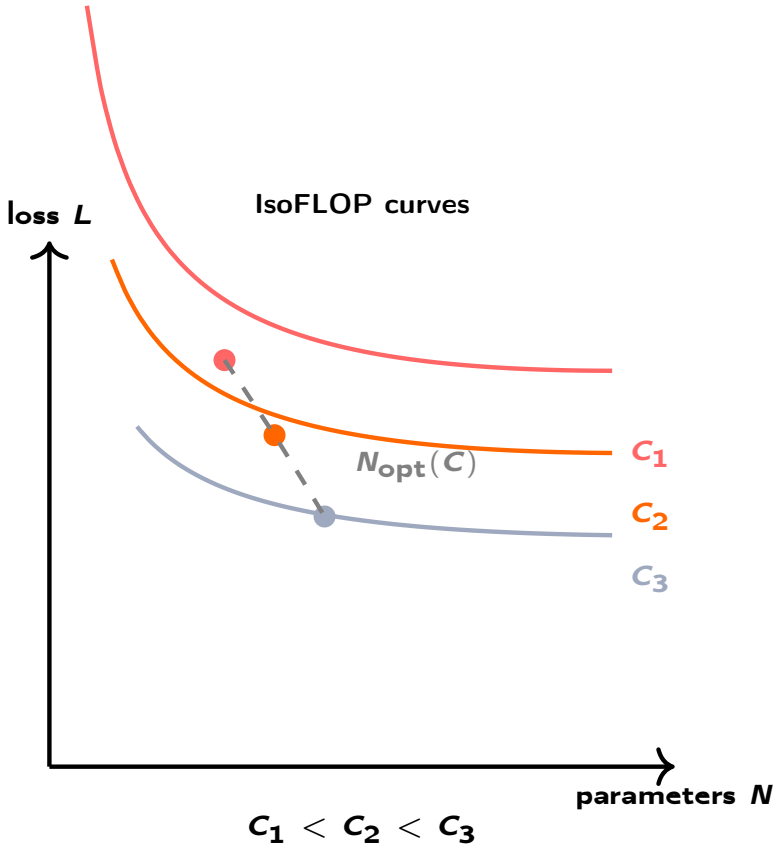
Irreducible loss	$E = 1.69$
$A = 406.4$	$\alpha = 0.34$
$B = 410.7$	$\beta = 0.28$

Optimal allocation:

$$N_{\text{opt}} \propto C^{0.50}, \quad D_{\text{opt}} \propto C^{0.50}$$

Optimal ratio: ≈ 20 tokens per parameter.

Chinchilla (70B, 1.4T tokens) outperformed Gopher (280B, 300B tokens) with *same compute*.



Each curve: fixed compute C, varying N (with $D = C/6N$). Minima shift right with more compute.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
    - Classical Hyperparameter Optimization
    - Learning Rate Schedules
    - Batch Size Selection
    - Weight Decay and AdamW
    - Scaling Laws
    - μP: Hyperparameter Transfer Across Scales
    - LLM Training Recipes
    - Practical Guidelines
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

μP: Maximal Update Parameterization

Problem: Optimal learning rate, initialization scale, etc. change with model width. Tuning on large models is prohibitively expensive.

μP [Yang et al., *"Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer"*, 2022]: A parameterization that ensures hyperparameters *transfer* from small to large models.

Standard Parameterization (SP):

- Init: $W_{ij} \sim \mathcal{N}(0, 1/n)$ (fan-in)
- All layers use the *same* learning rate
- Optimal LR *shifts* as width n changes

μP:

- Input layer: LR $\sim \eta$
- Hidden layers: LR $\sim \eta/n$
- Output layer: LR $\sim \eta/n$
- Init: hidden weights $\sim 1/n$ (not $1/\sqrt{n}$)

Key result:

- Under μP, the optimal LR for a 13B model can be found by tuning a **40M proxy**
- Verified on GPT-3 scale models
- Also transfers: init scale, weight decay, other Adam HPs

Practical recipe:

1. Implement μP parameterization
2. Grid-search LR on a small proxy model ($\sim$100$\times$ cheaper)
3. Use the same LR for the full-scale model

Used in Cerebras-GPT and adopted by several LLM labs.

μP: Comparison with Standard Parameterization

Intuition: In SP, hidden layer updates grow with width, causing the optimal LR to shrink. μP rescales so that layer-wise update magnitudes stay $\mathcal{O}(1)$ regardless of width.

	Standard (SP)	**μP**
Hidden weight init variance	$1/n$	$1/n$
Output weight init variance	$1/n$	$1/n$
Hidden weight learning rate	η	η/n
Output weight learning rate	η	η/n
Embedding learning rate	η	η
Hidden layer update size	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(1)$
Optimal LR transfers?	No	**Yes**

n = model width (hidden dimension). Under μP, the optimal η found for a narrow model also works when n is increased by orders of magnitude.

Note: μP transfers *width*-dependent HPs. Depth scaling and the interplay with other dimensions (batch size, sequence length) remain active research areas.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
    - Classical Hyperparameter Optimization
    - Learning Rate Schedules
    - Batch Size Selection
    - Weight Decay and AdamW
    - Scaling Laws
    - μP: Hyperparameter Transfer Across Scales
    - LLM Training Recipes
    - Practical Guidelines
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

LLM Hyperparameter Recipes

	GPT-3 175B	Chinchilla 70B	LLaMA 65B	LLaMA-2 70B	LLaMA-3 405B	DeepSeek-V3 671B (MoE)
Peak LR	6×10^{-5}	1×10^{-4}	1.5×10^{-4}	1.5×10^{-4}	8×10^{-5}	2.2×10^{-4}
LR schedule	cosine	cosine	cosine	cosine	cosine	WSD
Min LR	10% of peak	10% of peak	10% of peak	10% of peak	$\frac{1}{80}$ peak	$\frac{1}{10}$ peak
Warmup	375M tok	5B tok	2000 steps	2000 steps	8000 steps	2000 steps
Batch size (tok)	3.2M	1.5M	4M	4M	16M	15.6M
Weight decay	0.1	0.1	0.1	0.1	0.1	0.1
β_1, β_2	0.9, 0.95	0.9, 0.95	0.9, 0.95	0.9, 0.95	0.9, 0.95	0.9, 0.95
Grad clip	1.0	1.0	1.0	1.0	1.0	1.0
Tokens trained	300B	1.4T	1.4T	2T	15T	14.8T
Tok / param ratio	1.7	20	21.5	28.6	37	22.1

Sources: [Brown et al., *"Language Models are Few-Shot Learners"*, 2020], [Hoffmann et al., *"Training Compute-Optimal Large Language Models"*, 2022], [Touvron, Lavril, et al., *"LLaMA: Open and Efficient Foundation Language Models"*, 2023], [Touvron, Martin, et al., *"Llama 2: Open Foundation and Fine-Tuned Chat Models"*, 2023], [Grattafiori et al., *"The Llama 3 Herd of Models"*, 2024], [DeepSeek-AI, *"DeepSeek-V3 Technical Report"*, 2024].

Common Patterns in LLM Recipes

Remarkably, despite different model sizes and labs, most choices converge:

Near-universal defaults:

- **Optimizer:** AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$
- **Weight decay:** $\lambda = 0.1$
- **Gradient clipping:** global norm $= 1.0$
- **LR schedule:** cosine decay (WSD emerging as alternative)
- **Precision:** BF16 mixed precision

Model-size-dependent:

- **Peak learning rate:** Decreases with model size. Roughly $\eta \propto N^{-0.1}$ to $N^{-0.2}$
- **Batch size:** Increases with model size (to match B_{crit})
- **Warmup steps:** Relatively short ($\sim$0.1–1% of training)

Trend: Labs are now training *far beyond* Chinchilla-optimal token counts. Inference cost matters: a smaller model trained on more data is cheaper to *serve*.

Vision Model Hyperparameter Recipes

	ResNet-50	ResNet-50	ResNeXt-101	ViT-B/16	ViT-L/16	MAE	DINOv2
	orig. [He, K. Zhang, et al., "Deep Residual Learning for Image Recognition", 2016]	**improved** [Wightman et al., "ResNet Strikes Back: An Improved Training Procedure in timm", 2021]	[S. Xie et al., "Aggregated Residual Transformations for Deep Neural Networks", 2017]	[Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", 2021]	[Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", 2021]	**ViT-L** [He, K. Chen, et al., "Masked Autoencoders Are Scalable Vision Learners", 2022]	**ViT-L/14** [Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision", 2023]
Optimizer	SGD+mom	LAMB	SGD+mom	Adam	Adam	AdamW	AdamW
Peak LR	0.1	5×10^{-3}	0.1	8×10^{-4}	4×10^{-4}	1.5×10^{-4}	2×10^{-4}
LR schedule	step	cosine	step	lin. decay	lin. decay	cosine	cosine
Warmup	—	5 ep	—	10k steps	10k steps	40 ep	80 ep
Batch size	256	2048	256	4096	4096	4096	~1024
Weight decay	10^{-4}	0.02	10^{-4}	0.1	0.1	0.05	0.04→0.4
β_1, β_2	0.9, –	–	0.9, –	.9, .999	.9, .999	.9, .95	.9, .999
Grad clip	—	—	—	1.0	1.0	—	3.0
Epochs	90	600	100	7 (JFT)	7 (JFT)	800	500
Resolution	224	224	224	224	224	224	224

Trends: Vision moved from SGD (ResNet era) → AdamW (ViT era). Modern recipes use cosine LR, longer training, heavier augmentation, and much larger batches.

Contrastive & Multimodal Model Recipes

	CLIP ViT-L/14 OpenCLIP repro. [Radford, Kim, et al., *"Learning Transferable Visual Models From Natural Language Supervision"*, 2021]	SigLIP [Zhai et al., *"Sigmoid Loss for Language Image Pre-Training"*, 2023]	PaliGemma [Beyer et al., *"PaliGemma: A Versatile 3B VLM for Transfer"*, 2024]
Optimizer	AdamW	Adafactor	Adafactor
Peak LR	5×10^{-4}	10^{-3}	rsqrt schedule
LR schedule	cosine	cosine	rsqrt (continuous)
Warmup	2000 steps	200M examples	slow linear
Batch size	32,768	32,768	—
Weight decay	0.2	10^{-4}	—
β_1, β_2	.9, .98	.9, .95	—
Training data	400M pairs	40B examples	1B examples
Resolution	224	224	224→448/896

Key differences from LLMs:

- Contrastive models need very large batch sizes (32k+) for enough negatives
- SigLIP replaces softmax cross-entropy with sigmoid loss ⇒ scales better to huge batches
- Weight decay varies widely: 0.2 (CLIP) vs. 10^{-4} (SigLIP). Less converged than LLM recipes
- Multimodal models often use multi-stage training with increasing resolution

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
    - Classical Hyperparameter Optimization
    - Learning Rate Schedules
    - Batch Size Selection
    - Weight Decay and AdamW
    - Scaling Laws
    - μP: Hyperparameter Transfer Across Scales
    - LLM Training Recipes
    - Practical Guidelines
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG, and

Practical Hyperparameter Selection Workflow

Recipe for practitioners:

1. **Start from published recipes.** Use HP settings from a similar published model as a baseline (see previous table).
2. **Tune learning rate first.** It has the largest impact. Use:
   - LR range test for a quick estimate
   - Small grid: $\{1, 3, 5\} \times 10^k$ for 2–3 orders of magnitude
3. **Use μP for large models.** Tune on a small proxy ($\sim$10–100$\times$ cheaper), transfer LR and other HPs to the target scale.
4. **Scale batch size with compute.** Use the linear or square root scaling rule; start small, ramp up. Stay near B_{crit}.
5. **Keep everything else at defaults.** Weight decay $= 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.95$, grad clip $= 1.0$.
6. **Use short runs for exploration.** Train for 1–5% of total budget to compare configurations. Scaling laws can extrapolate final performance.
7. **Monitor loss curves.** Watch for spikes (reduce LR or increase warmup), slow convergence (increase LR), or late divergence (add grad clipping).

Recent Advances in Hyperparameter Selection

Schedule-Free optimization [Defazio et al., *"The Road Less Scheduled"*, 2024]:

- Eliminates the need for specifying a LR schedule or total training steps
- Replaces momentum with interpolation and averaging; no extra hyperparameters
- Won the MLCommons 2024 AlgoPerf Algorithmic Efficiency Challenge

Step Law – optimal HP scaling [H. Li et al., *"Predictable Scale: Part I, Step Law — Optimal Hyperparameter Scaling Law in Large Language Model Pretraining"*, 2025]:

- Trained 3700+ LLMs to derive power-law relationships for optimal HPs
- Optimal LR follows a power law in N and D; optimal batch size scales with D (invariant to N)
- Unifies dense transformers and MoE under one scaling law for HPs

Power Lines – weight decay and batch size scaling [Bernstein et al., *"Power Lines: Scaling Laws for Weight Decay and Batch Size in LLM Pre-training"*, 2025]:

- Introduces the *AdamW timescale*: $\tau = B/(\eta \cdot \lambda \cdot D)$
- Optimal τ decreases as a power law with tokens-per-parameter ratio D/N
- When changing batch size, adjust weight decay (not LR) for best results

Summary

Key Takeaways

- **Random search** beats grid search; Hyperband and tools like Optuna improve further
- **Learning rate** is the most critical HP; use cosine or WSD schedule with warmup
- **Scaling laws** predict performance from small experiments
- μ**P** enables *hyperparameter transfer* from small to large models
- **LLM recipes** have converged: AdamW, λ=0.1, β_2=0.95, cosine LR, grad clip 1.0
- **Batch size** interacts with LR via scaling rules (linear or $\sqrt{k}$); respect B_{crit}
- **Recent:** schedule-free optimizers and HP scaling laws make tuning more principled

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
    - The Adam Optimizer
    - The Muon Optimizer
    - Newton-Schulz Iteration
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
    - The Adam Optimizer
    - The Muon Optimizer
    - Newton-Schulz Iteration
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Adam: Adaptive Moment Estimation [Kingma and Ba, *"Adam: A Method for Stochastic Optimization"*, 2015]

Key idea: Maintain per-parameter running estimates of gradient mean and variance, then scale updates adaptively.

Algorithm 9: Adam

Input: LR η, decay rates β_1, β_2, ϵ
Initialize $m_0 = 0, v_0 = 0$
for $t = 1, 2, \ldots$ **do**
$g_t \leftarrow \nabla_w L(w_{t-1})$
$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
$\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$
$\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$
$w_t \leftarrow w_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$

Components:

m_t First moment (mean): momentum-smoothed gradient

v_t Second moment (variance): tracks gradient magnitude per parameter

Bias corr. $\hat{m}_t, \hat{v}_t$ correct the zero-initialization bias

$1/\sqrt{\hat{v}_t}$ Rescales each parameter's update inversely to its gradient magnitude

Defaults: β_1=0.9, β_2=0.999, ϵ=10^{-8}

LLM defaults: β_2=0.95, weight decay via AdamW [Loshchilov and Hutter, *"Decoupled Weight Decay Regularization"*, 2019]

Adam's Per-Parameter Adaptivity

Adam normalizes each gradient coordinate independently:

$$\Delta w_i \propto \frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \epsilon} \qquad \Rightarrow \qquad |\Delta w_i| \approx \pm\eta$$

Advantages:

- Parameters with sparse/small gradients still get meaningful updates
- Relatively insensitive to gradient scale $\Rightarrow$ less LR tuning
- Works well across different parameter types (embeddings, attention, MLP)

Limitation for matrix parameters:

- Adam treats each entry W_{ij} independently
- Ignores the *matrix structure* of weight updates
- The gradient $G = \nabla_W L$ often has high condition number: a few singular values dominate
- Adam's element-wise rescaling does not fix this directional imbalance

$\Rightarrow$ Can we design an optimizer that respects the matrix structure of 2D parameters?

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
    - The Adam Optimizer
    - The Muon Optimizer
    - Newton-Schulz Iteration
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Muon: Momentum + Orthogonalization [Jordan, *Muon: An optimizer for hidden layers in neural networks*, 2024]

Key insight: For 2D weight matrices, *orthogonalize* the momentum buffer before applying the update. This ensures all singular directions are updated equally.

Algorithm overview:

1. Compute gradient $G_t = \nabla_W L$
2. Accumulate with **Nesterov momentum**:

$$M_t = \beta \cdot M_{t-1} + G_t$$

3. **Orthogonalize** the momentum via Newton-Schulz iteration:

$$U_t = \mathrm{NS}_5(M_t) \approx \mathrm{Ortho}(M_t)$$

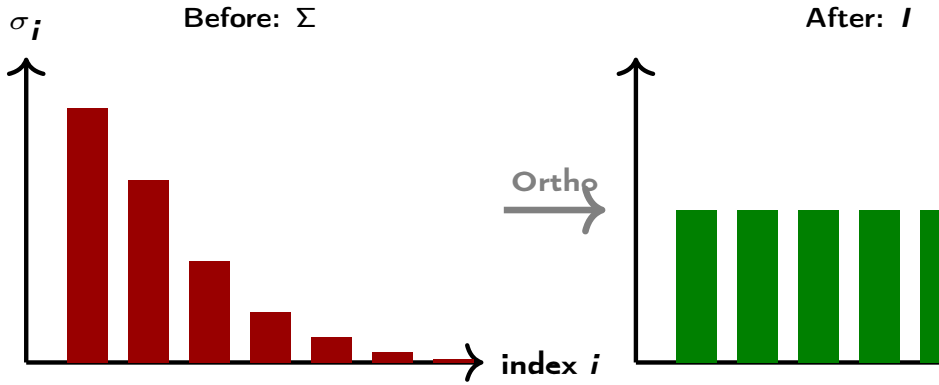
4. Update: $W_t = W_{t-1} - \eta \cdot U_t$

Apply Muon only to 2D (matrix) parameters.
Use AdamW for embeddings, classifier head, biases, and all 1D parameters.

What does orthogonalization do?
Given $G = U\Sigma V^\top$ (SVD), the nearest semi-orthogonal matrix is:

$$\mathrm{Ortho}(G) = UV^\top$$

This *replaces all singular values with 1*:



Rare gradient directions (small σ_i) get amplified to the same scale as dominant ones.

Why Orthogonalize? The Condition Number Problem

Observation: Gradient matrices for 2D transformer parameters typically have *very high condition numbers*.

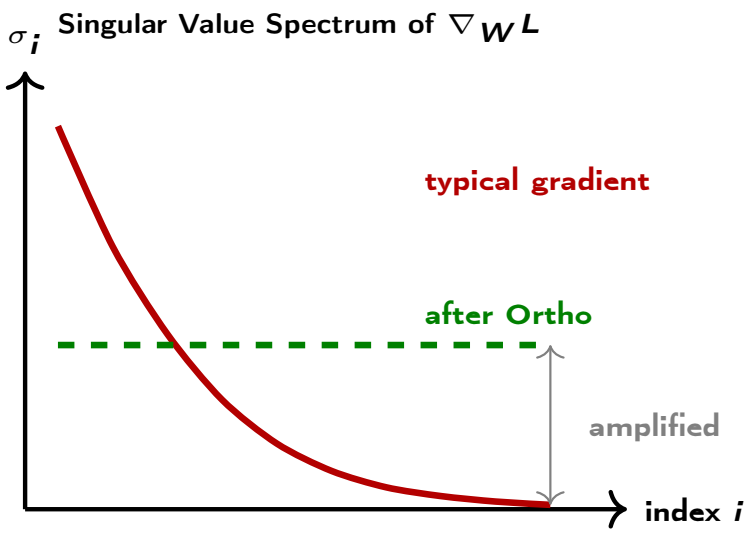
A few singular directions dominate the update, while most directions receive negligible signal.

SGD/Adam behavior:

- SGD: all neuron updates dominated by the top few singular vectors
- Adam: element-wise rescaling partially helps, but does not fix the directional imbalance

Orthogonalization:

- Sets all singular values to 1
- Every direction in the update gets equal weight
- "Amplifies rare directions" that have small magnitude but are important for learning



Connection to steepest descent: Orthogonalized gradient $UV^\top$ is the *steepest descent direction* under the **spectral norm** [Bernstein and Newhouse, *Old Optimizer, New Norm: An Anthology*, 2024]:

$$UV^\top = \arg\min_{\|D\|_{\mathrm{op}} \leq 1} \langle G, D \rangle$$

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon
- The Adam Optimizer
- The Muon Optimizer
- Newton-Schulz Iteration

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Newton-Schulz Iteration for Fast Orthogonalization

Computing SVD is expensive ($O(nm^2)$ for $n \times m$). The **Newton-Schulz (NS) iteration** approximates orthogonalization using only matrix multiplications.

One NS step for matrix X with SVD $X = USV^\top$:

$$X' = aX + b(XX^\top)X + c(XX^\top)^2 X$$

In terms of singular values, this applies a quintic polynomial:

$$\phi(s) = a \cdot s + b \cdot s^3 + c \cdot s^5$$

After N steps: $X_N = U\,\phi^{(N)}(S)\,V^\top$, where $\phi^{(N)}$ is the N-fold composition.

Goal: Choose a, b, c so that $\phi^{(N)}(s) \to 1$ for all $s \in (0, 1]$ as fast as possible.

Requirements:

- $\phi(1) = 1$ (fixed point): $a + b + c = 1$
- $\phi'(1) = 0$ (stable fixed point): $a + 3b + 5c = 0$
- $\phi(s) \to 1$ quickly for s near 0 (fast convergence for small singular values)

Tuning Newton-Schulz Coefficients

Baseline coefficients (standard in literature):

$$(a, b, c) = (2,\ -1.5,\ 0.5)$$

Slow convergence near $s = 0$: many iterations needed.

Optimized coefficients [Jordan, *Muon: An optimizer for hidden layers in neural networks*, 2024]:

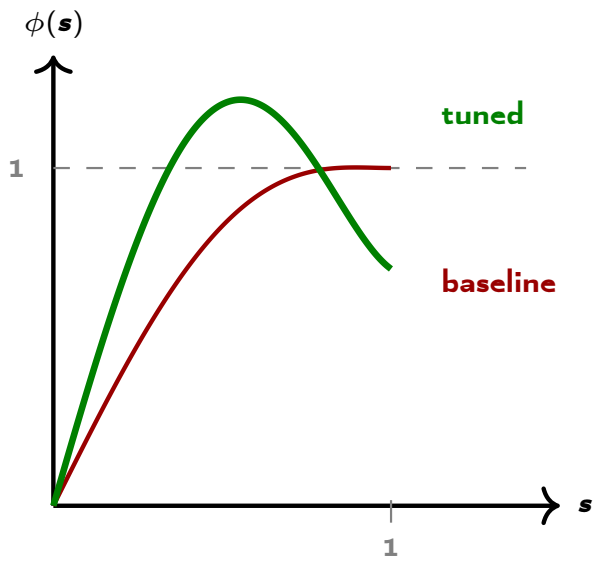
$$(a, b, c) = (3.4445,\ -4.7750,\ 2.0315)$$

Much steeper growth near $s = 0 \Rightarrow$ **only 5 iterations suffice** for good approximation.

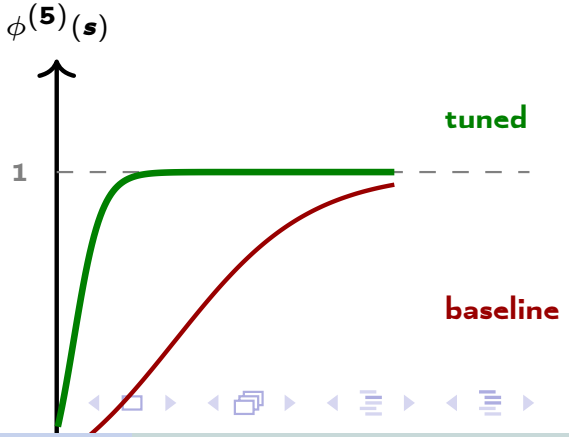
Implementation detail:

- Normalize $X \leftarrow X/\|X\|_F$ so singular values lie in $[0, 1]$
- Runs entirely in **bfloat16**
- If matrix is tall ($n > m$), transpose before iterating

Single NS polynomial $\phi(s)$:



After 5 iterations $\phi^{(5)}(s)$:



Newton-Schulz Implementation

The complete NS orthogonalization in PyTorch:

```
def newtonschulz5(G, steps=5, eps=1e-7):
    a, b, c = (3.4445, -4.7750, 2.0315)
    X = G.bfloat16()
    X /= (X.norm() + eps)  # normalize to [0,1]
    if G.size(0) > G.size(1):
        X = X.T              # work on smaller dim
    for _ in range(steps):
        A = X @ X.T
        B = b * A + c * A @ A
        X = a * X + B @ X   # quintic polynomial
    if G.size(0) > G.size(1):
        X = X.T
    return X
```

Computational cost per NS step (for $n \times m$ matrix, $m \leq n$):

- $XX^\top$: $2nm^2$ FLOPs, then $A \cdot A$: $2m^3$ FLOPs, then $B \cdot X$: $2nm^2$ FLOPs
- Total per step: $\leq 6nm^2$ FLOPs. For 5 steps: $30nm^2$ FLOPs
- **Relative overhead:** $\leq 5m/B$ of the forward pass (where B = batch size in tokens)
- Negligible for typical LLM training batch sizes ($B \gg m$)

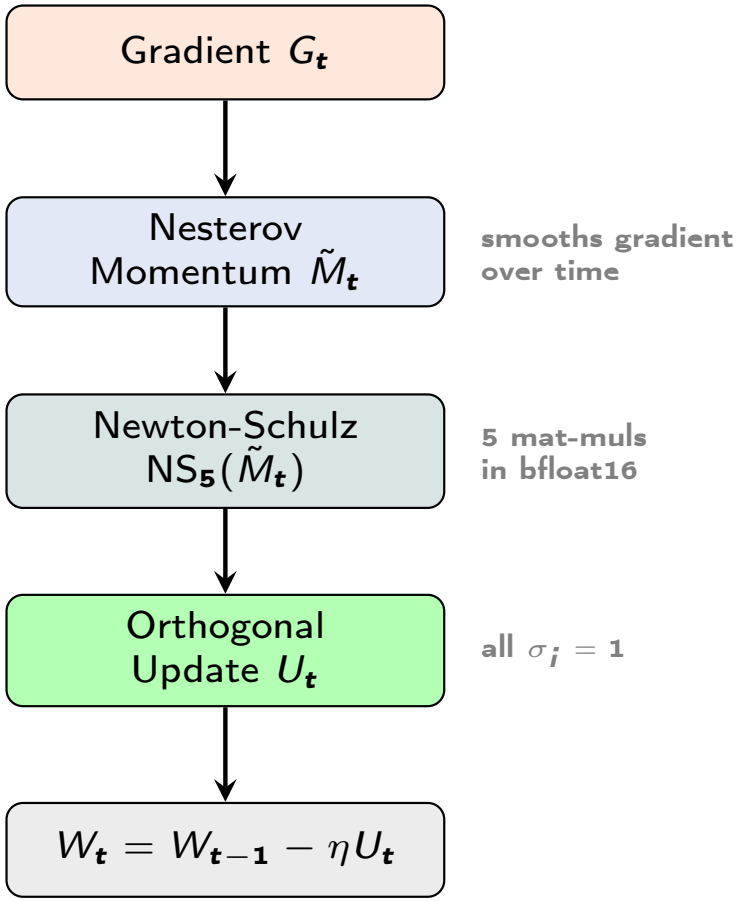
Full Muon Algorithm

Algorithm 10: Muon Optimizer

Input: LR η, momentum β, NS steps T=5
Initialize $M_0 = 0$ for each 2D param W
for $t = 1, 2, \ldots$ **do**
  $G_t \leftarrow \nabla_W L(\mathrm{w}_{t-1})$
  `// Nesterov momentum`
  $M_t \leftarrow \beta \cdot M_{t-1} + G_t$
  $\tilde{M}_t \leftarrow \beta \cdot M_t + G_t$
  `// Orthogonalize via NS`
  $U_t \leftarrow \mathrm{NewtonSchulz}_T(\tilde{M}_t)$
  `// Update weights`
  $W_t \leftarrow W_{t-1} - \eta \cdot U_t$

Practical details:

- Muon: 2D weight matrices (Q, K, V, MLP)
- AdamW: embeddings, output head, biases, LayerNorm
- Apply to Q, K, V *separately*, not concatenated



Connection to Shampoo

Shampoo [Gupta et al., *"Shampoo: Preconditioned Stochastic Tensor Optimization"*, 2018] is a second-order optimizer that preconditions gradients using accumulated outer products.

Instantaneous Shampoo update (without accumulation):

$$W_{t+1} = W_t - \eta \cdot (G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4}$$

Substituting SVD $G_t = U\Sigma V^\top$:

$$(GG^\top)^{-1/4} = (U\Sigma^2 U^\top)^{-1/4} = U\Sigma^{-1/2} U^\top$$
$$(G^\top G)^{-1/4} = (V\Sigma^2 V^\top)^{-1/4} = V\Sigma^{-1/2} V^\top$$

Therefore:

$$(GG^\top)^{-1/4} G (G^\top G)^{-1/4} = U\Sigma^{-1/2} \cdot \Sigma \cdot \Sigma^{-1/2} V^\top = UV^\top$$

Key equivalence

Muon (without momentum) = instantaneous Shampoo.
Both reduce to the orthogonalized gradient $UV^\top$.
Muon replaces Shampoo's expensive preconditioner accumulation with a lightweight NS iteration.

Muon Results

NanoGPT speedrunning benchmark:

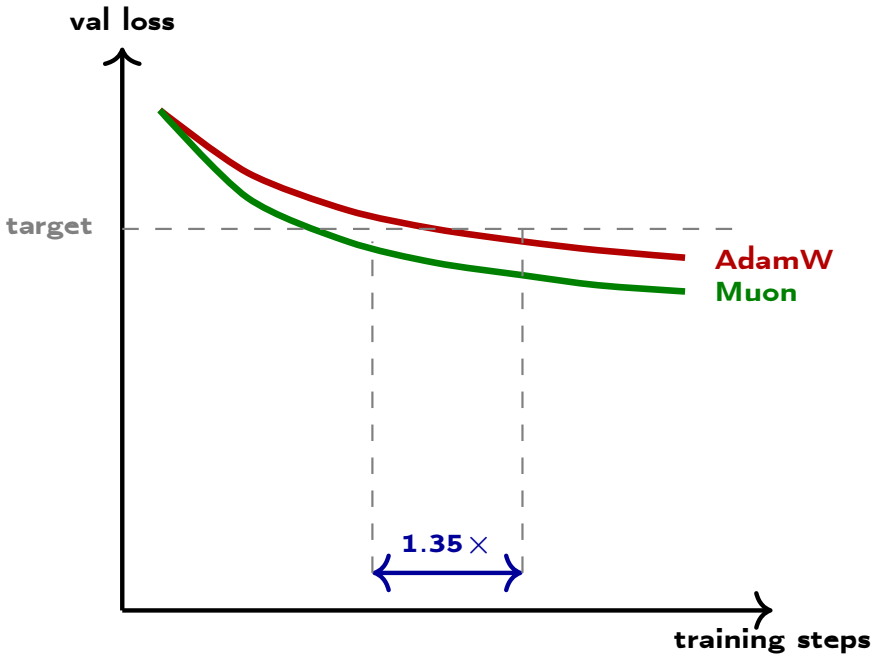
- Muon achieves **1.35× faster training** to target validation loss on FineWeb
- After introduction (Oct 2024), Muon was used in **all 12 subsequent records** set by 7 different researchers
- Muon has slower per-step wallclock time than AdamW (due to NS overhead) ⇒ its persistent adoption is strong evidence of genuine sample-efficiency gains

1.5B parameter scale:

- GPT-2 XL-level HellaSwag in **10 8xH100-hours** vs. 13.3 hours for AdamW

CIFAR-10:

- Speed record: 3.3 → 2.6 A100-seconds for 94% accuracy



Schematic of Muon vs AdamW training curves. Muon reaches the same validation loss with fewer steps.

Why trust these results?
The NanoGPT speedrunning task is a *competitive benchmark*: researchers are incentivized to find the best optimizer. If Muon were worse, competitors would discard it. Its persistent adoption across records by independent researchers is strong evidence

Muon: Summary and Practical Advice

Key Takeaways

- **Muon = Momentum + Orthogonalization** via Newton-Schulz iteration
- Orthogonalization ensures all singular directions get equal-magnitude updates
- Equivalent to steepest descent under the spectral norm
- Mathematically equivalent to instantaneous (accumulation-free) Shampoo
- Newton-Schulz iteration: 5 steps of matrix multiplications in bfloat16, negligible overhead
- 1.35× sample-efficiency improvement over AdamW on NanoGPT benchmark

When to use:

Muon	All 2D weight matrices: Q, K, V projections, MLP layers (apply separately, not concatenated)
AdamW	Embeddings, output head, biases, LayerNorm/RMSNorm parameters, all 1D tensors

Prior work: Orthogonal-SGDM [Tuddenham et al., *"Orthogonalising gradients to speed up neural network optimisation"*, 2022], Stochastic Spectral Descent [Carlson et al., *"Stochastic Spectral Descent for Restricted Boltzmann Machines"*, 2015]

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs
- Motivation
- RL Introduction
- Reward Model
- Policy Gradient
- Reducing Variance
- Off-Policy Learning and PPO
- GRPO
- DPO
- Summary and Open Problems

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

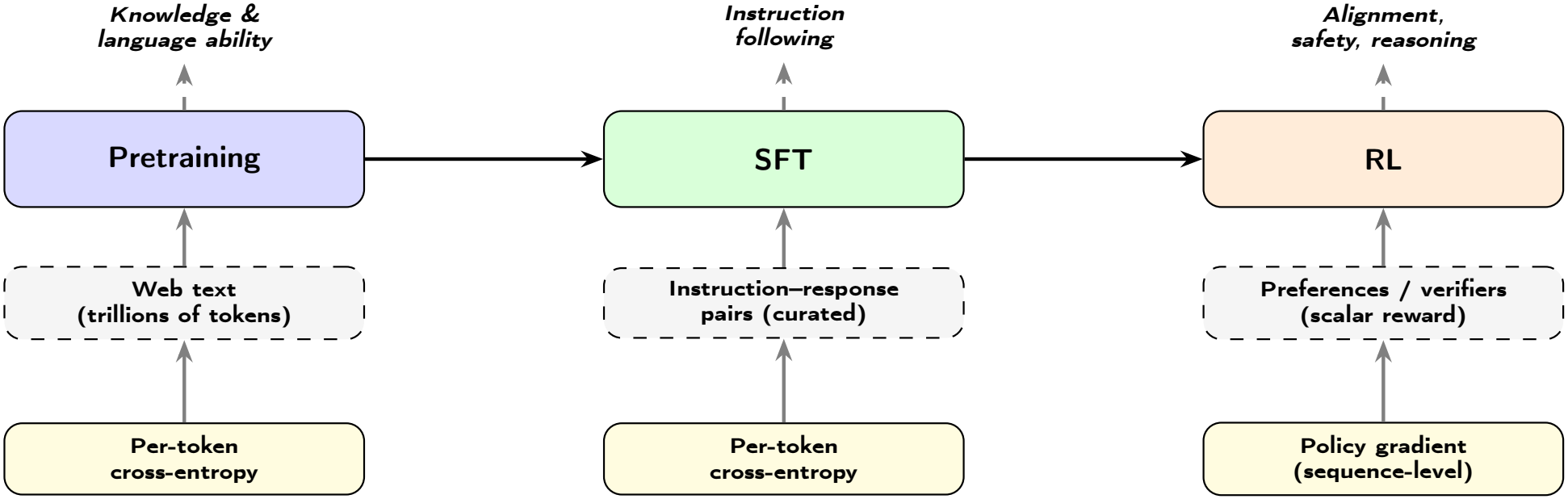
16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
    - Motivation
    - RL Introduction
    - Reward Model
    - Policy Gradient
    - Reducing Variance
    - Off-Policy Learning and PPO
    - GRPO
    - DPO
    - Summary and Open Problems
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

LLM Training Stages



Pretraining

- 1–15T+ tokens
- 1000s of GPUs
- Weeks to months
- Most expensive stage (right now, might change)

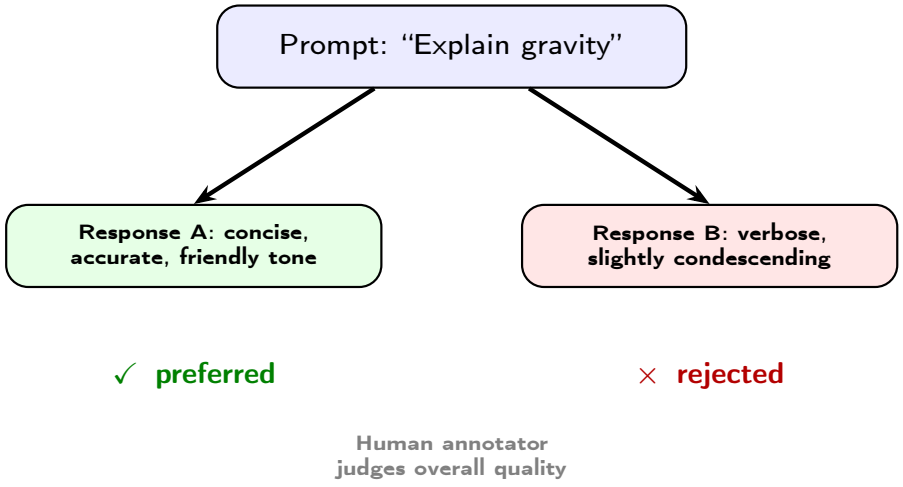
SFT

- 10k–1M+ pairs
- 10s of GPUs
- Hours to days
- Relatively cheap

RL

- **Alignment:**
  - Preferences / RLHF
  - 10k–100k prompts
  - 100s of GPUs, days
- **Reasoning:**
  - Verifier rewards
  - 100k+ prompts
  - 1000s of GPUs, weeks

Human Preference Alignment: Why Not Supervised Learning?



Why not SFT on the preferred response?

- SFT requires per-token ground truth
- Which tokens made A better than B?
  - Was it word choice? Sentence order? Tone? All of them?
- Preference is **holistic**: correctness, helpfulness, safety, style. It emerges from **interaction of all tokens**
- Cannot decompose "A $\succ$ B" into per-token labels

What SFT on A misses:

- Ignores the rejected response entirely
- Does not learn *what to avoid*
- May copy surface patterns of A without understanding *why* it was preferred

Verifiers: Automated Non-Per-Token Supervision

Beyond human preferences, many tasks have **automated verifiers** that check outputs without per-token labels:

Math: "Solve $\int_0^1 x^2\,dx$"
Verifier: Is the answer $\frac{1}{3}$? ✓ / ×

Code: "Write a function to sort a list"
Verifier: Run test suite
`assert sort([3,1,2]) == [1,2,3]`

Logic: "Solve this Sudoku"
Verifier: Do all rows, columns, boxes contain 1–9?

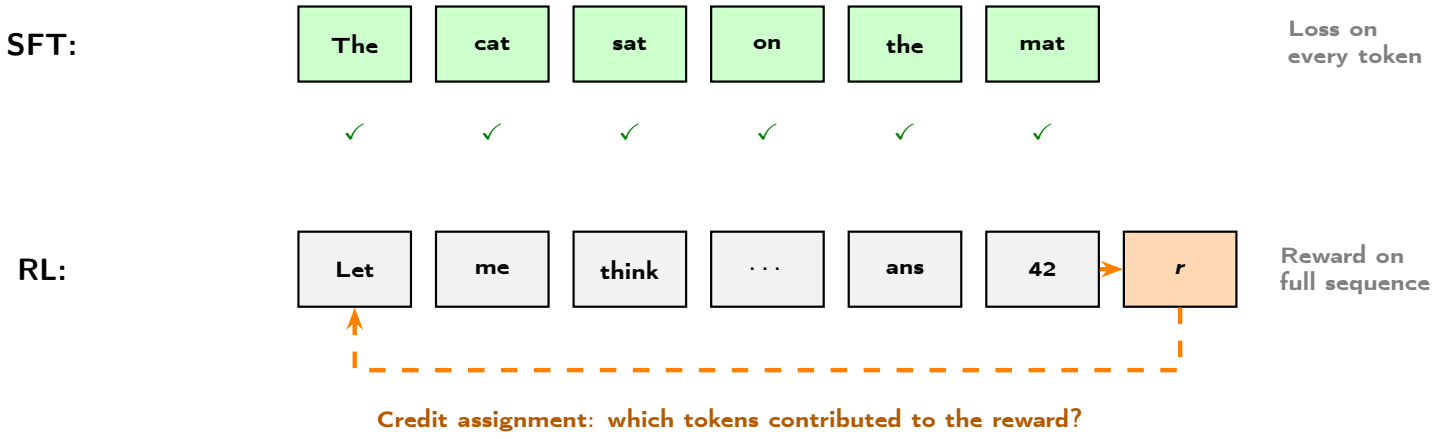
Games: Play a game of Go
Verifier: Did the agent win?
$r \in \{-1, +1\}$

Proof: "Prove this theorem"
Verifier: Formal proof checker (e.g. Lean)

Chemistry: "Design a molecule with property X"
Verifier: Simulation / wet lab confirms property

- All cases: **binary or scalar signal** on the full output. No indication which tokens were good or bad.
- Verifiers are cheap, scalable, and do not require human annotation ⇒ ideal for RL at scale.

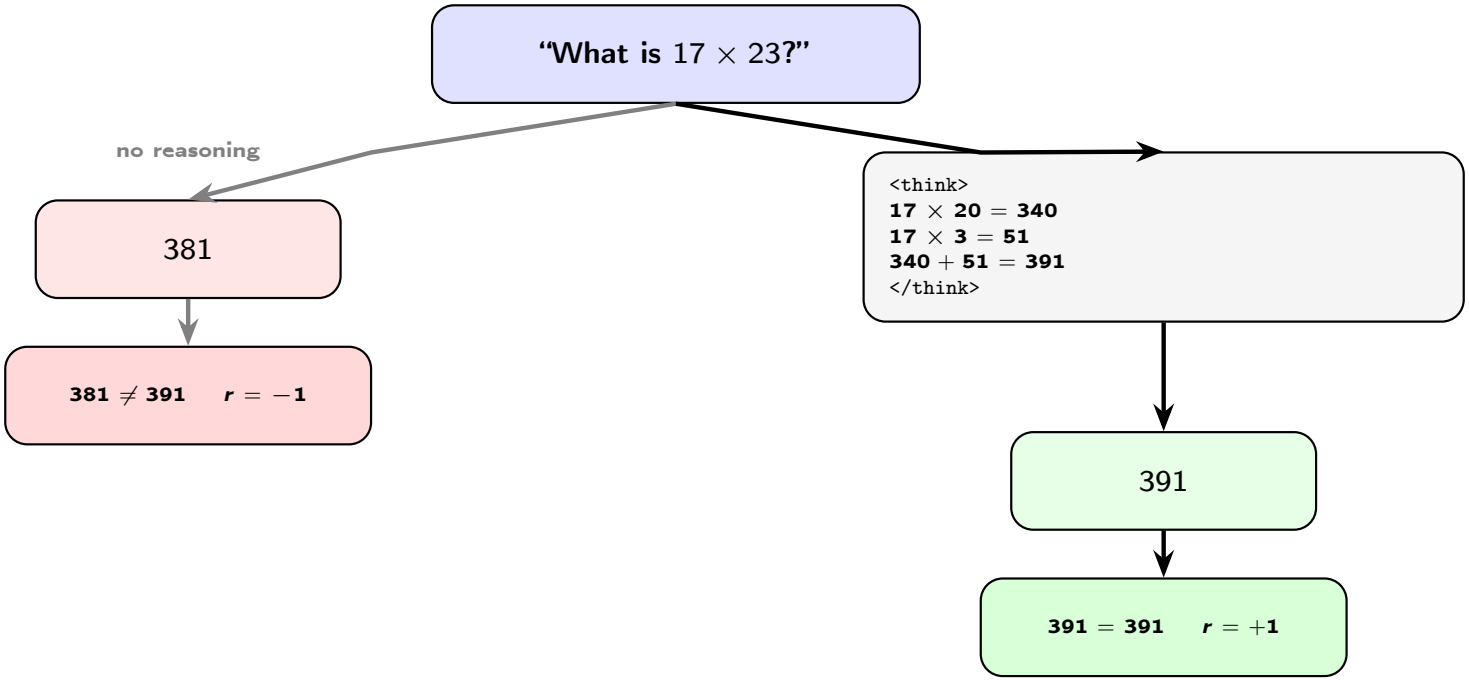
Technical Differences: Dense vs. Sparse Supervision



Dense (SFT): Cross-entropy loss on each token. Easy optimization, but needs per-token ground truth.

Sparse (RL): Single reward after full generation. Harder optimization, but works with any sequence-level signal.

RL for Reasoning: Emergent Chain-of-Thought



- The verifier only checks the **final answer**. It does not supervise the reasoning trace
- The model **discovers on its own** that reasoning leads to correct answers
- No human-written chain-of-thought labels needed

Why Reasoning Emerges

Without RL:

- SFT model mimics training data
- May or may not produce reasoning
- No incentive to reason correctly
- Reasoning style fixed by data

With RL + verifier:

- Correct final answer $\Rightarrow$ positive reward
- Responses with reasoning traces tend to be correct more often
- RL **amplifies** these responses
- Reasoning emerges as a **byproduct** of reward maximization

Key insight: We never explicitly train "think step by step." The model learns that **reasoning is instrumentally useful** for maximizing reward.

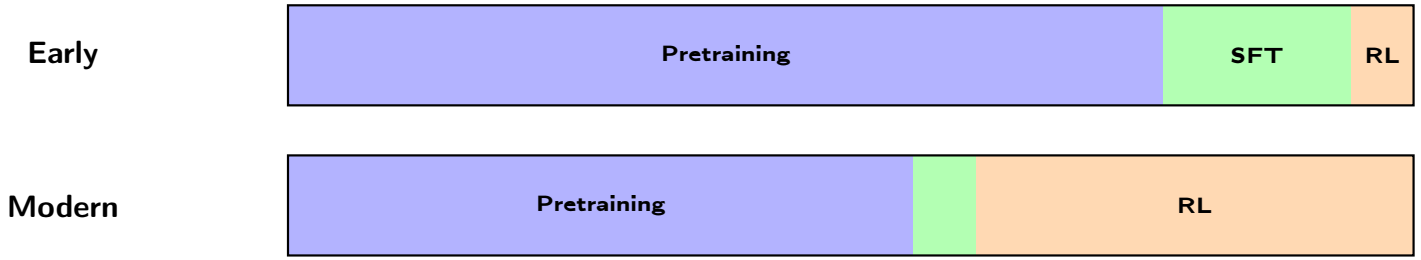
Historical Shift: RL's Growing Role

Early RLHF (2022–2023):

- RL as thin alignment layer
- Small fraction of total compute
- Mainly for safety and style
- InstructGPT, ChatGPT, Llama 2

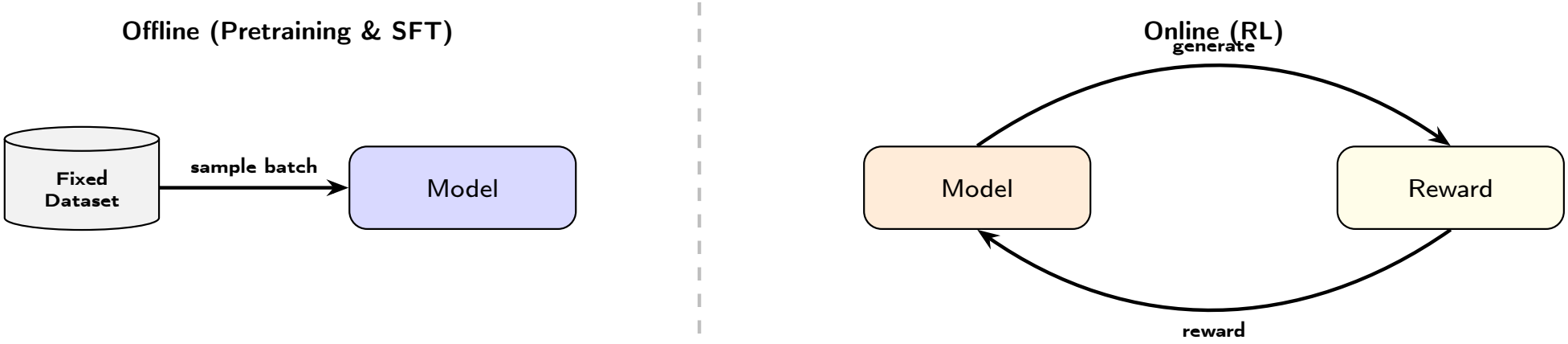
Modern RL-heavy training (2024+):

- RL as **primary capability driver**
- Significant fraction of total compute
- Reasoning, math, code via verifiers
- DeepSeek-R1, OpenAI o1/o3



Relative compute allocation (illustrative)

Offline vs. Online Training



- Data collected once, training iterates over it
- Model does not influence training data
- Dense supervision (per-token loss)
- **Distribution mismatch**: model trains on states the *teacher/data* visits, not states the *model* will encounter
- Compounding error: early mistakes lead to unseen states $\Rightarrow$ model diverges

- Model generates its own training data
- Learns from states **it actually visits** at inference time
- Sparse supervision (reward on full sequence)
- No distribution mismatch
- Can **explore** and discover novel solutions
- Non-stationary, harder to stabilize

Offline vs. Online: Chess Analogy

Offline (SFT / distillation):

- Like **watching a grandmaster** play chess
- You see brilliant moves, but in board positions *you* would never reach
- Dense signal (learn from every move) but in the **wrong distribution**

Online (RL):

- Like **playing chess yourself** with no coach
- You experience your own board positions
- Sparse signal (only win/loss at the end) but in the **right distribution**

Ideal: A grandmaster watches *you* play and comments on each move = **on-policy distillation** (dense supervision on student-generated data).

Remarks

- Attending a full RL course is advised
- We will distill a big field into a short excursion
- Wonderful theory and intricate algorithms; we scratch the surface

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs
- Motivation
- RL Introduction
- Reward Model
- Policy Gradient
- Reducing Variance
- Off-Policy Learning and PPO
- GRPO
- DPO
- Summary and Open Problems

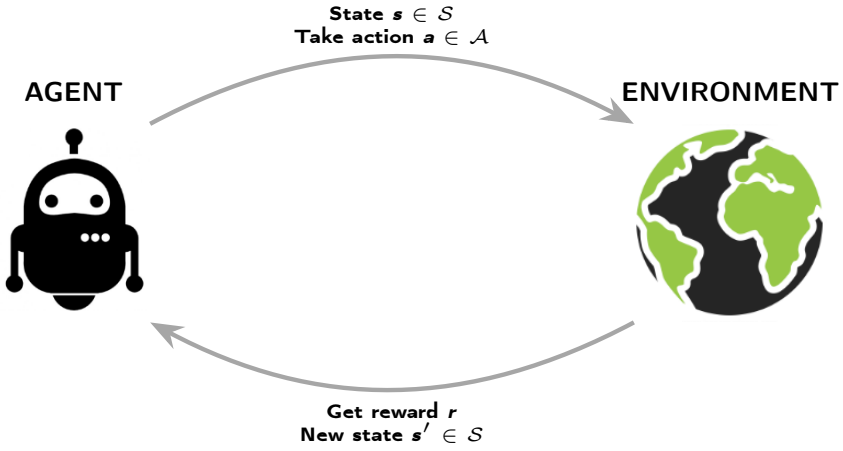
14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

RL Intro



Environment:

- State $s \in \mathcal{S}$
- Transition: $P(s' \mid s, a)$
- Reward: $R(s, a, s')$

Agent:

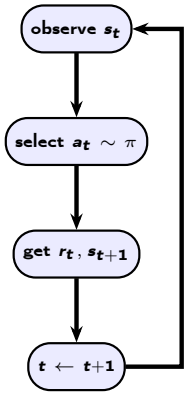
- Policy: $a \sim \pi(\cdot \mid s)$

Objective:

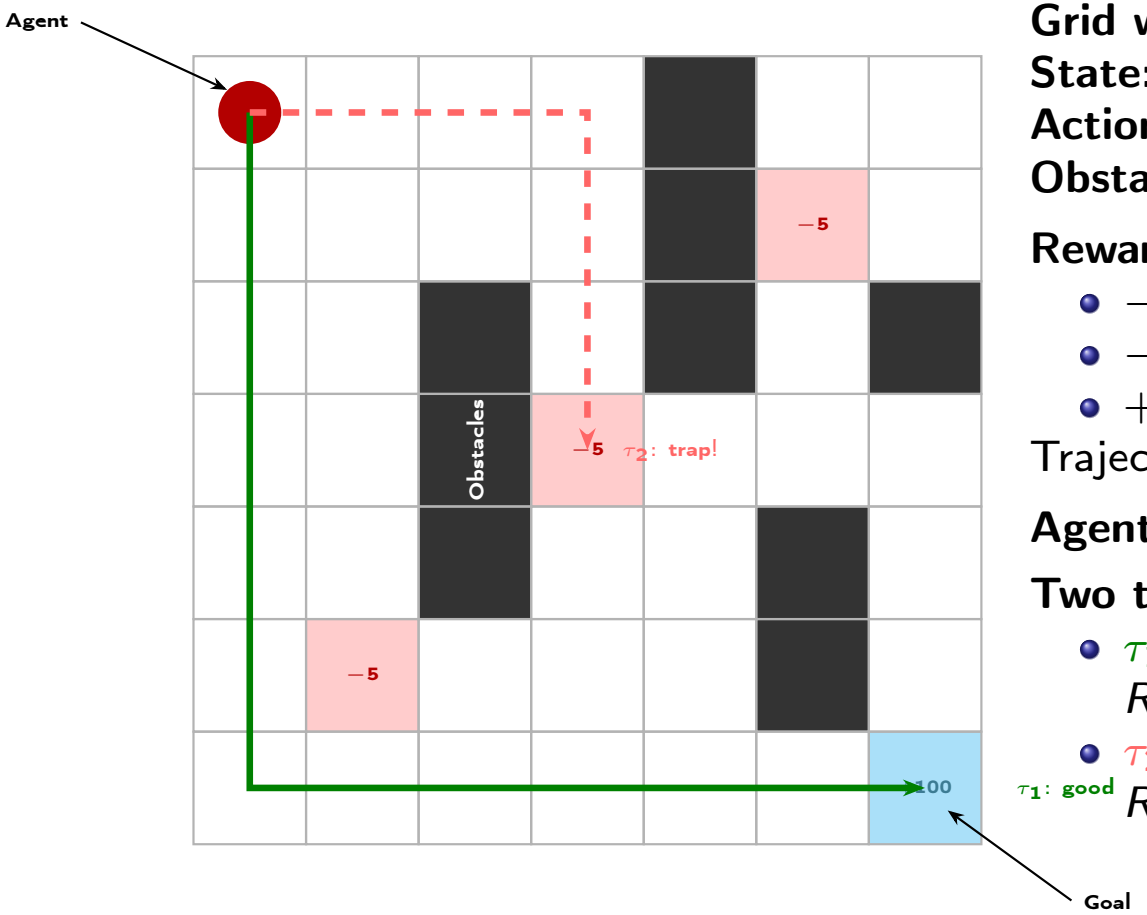
$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

$\gamma \in [0, 1]$: discount factor

Trajectory loop:



RL Toy Example



Grid world:
State: Cell position (x, y)
Actions: $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$
Obstacles: Cannot be entered (black)

Rewards:

- -1 per step (empty cell)
- -5 for trap cells (red)
- $+100$ for reaching the goal (cyan)

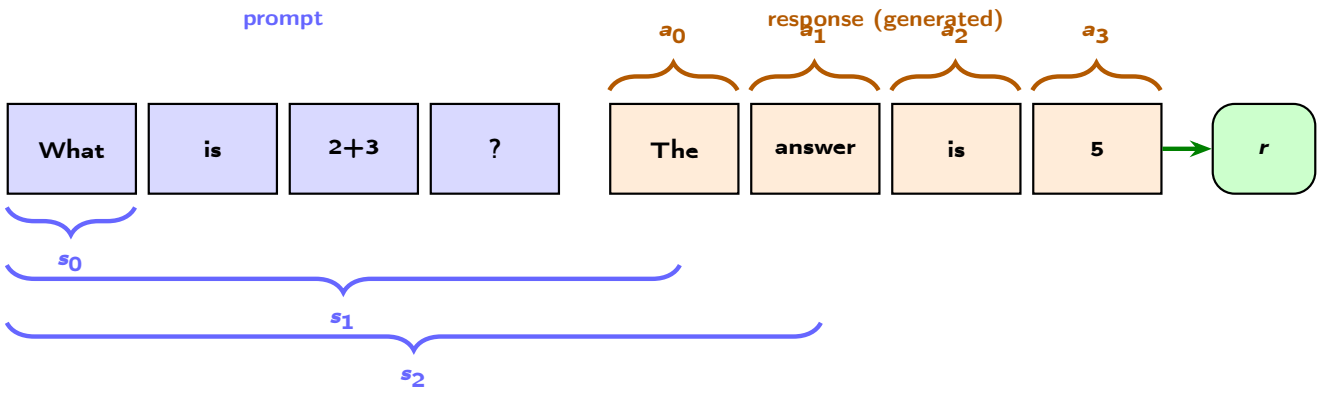
Trajectory ends upon reaching goal or trap.

Agent (•): Follows policy $a_t \sim \pi(\cdot \mid s_t)$

Two trajectories ($\gamma = 1$):

- τ_1: 12 steps + goal
  $R(\tau_1) = 12 \cdot (-1) + 100 = 88$
- τ_2: 5 steps + trap
  $R(\tau_2) = 5 \cdot (-1) + (-5) = -10$

RL Setting for LLMs



Agent: The LLM itself

State s_t: All tokens so far (prompt + response until now)

Action a_t: Next token to generate

Policy: $a_t \sim \pi_\theta(\cdot \mid s_t)$: softmax distribution over vocabulary

Reward r: Scalar signal after full response (verifier or preference model)

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs
- Motivation
- RL Introduction
- Reward Model
- Policy Gradient
- Reducing Variance
- Off-Policy Learning and PPO
- GRPO
- DPO
- Summary and Open Problems

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

Reward for LLMs

- Assigning exact reward to text is hard, but **comparing two responses** is often easy
- Train reward model on pairs: given $(x, y_w)/(x, y_l)$, enforce $r(x, y_w) > r(x, y_l)$

Bradley-Terry Model

Probability that y_w is preferred over y_l:

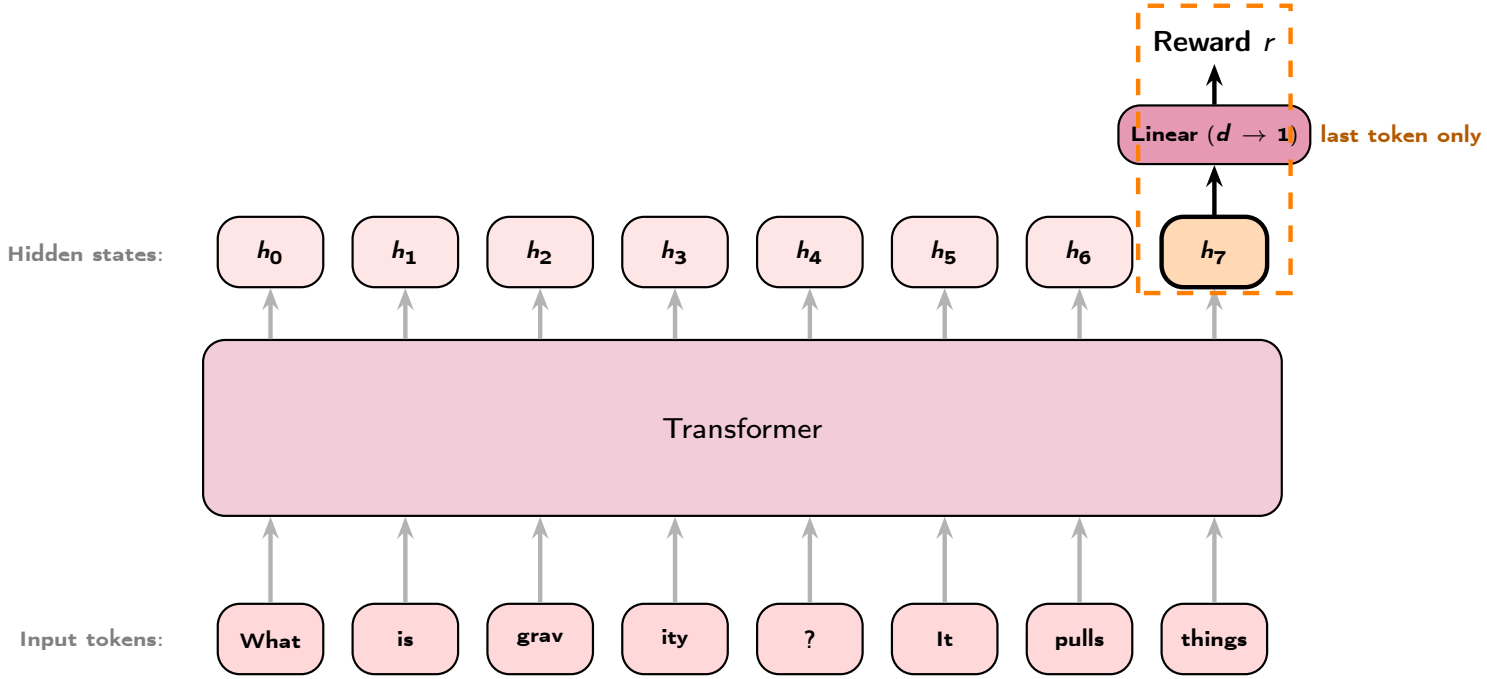
$$P(y_w \succ y_l) = \frac{\exp r(x, y_w)}{\exp r(x, y_w) + \exp r(x, y_l)} = \sigma\big(r(x, y_w) - r(x, y_l)\big)$$

MLE on comparison data gives the **reward loss**:

$$\mathcal{L} = -\sum_{(y_w, y_l)} \log \sigma\big(r(x, y_w) - r(x, y_l)\big)$$

- $r(x, y_w) > r(x, y_l) \Rightarrow$ sigmoid $> 0.5 \Rightarrow$ loss is small
- $r(x, y_w) < r(x, y_l) \Rightarrow$ sigmoid $< 0.5 \Rightarrow$ loss is large

Reward Model for LLMs



- Start from a pretrained LLM, replace the output head with a **scalar linear layer**
- Feed prompt + response, take the **last token's hidden state** → linear → reward $r \in \mathbb{R}$
- Train on human comparison data using the Bradley-Terry loss

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs
- Motivation
- RL Introduction
- Reward Model
- Policy Gradient
- Reducing Variance
- Off-Policy Learning and PPO
- GRPO
- DPO
- Summary and Open Problems

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

Loss on Trajectories

Notation:

- Trajectory $\tau = (s_0, a_0, r_0, \ldots, s_T, a_T, r_T)$, return $R(\tau) = \sum_{t=0}^{T} \gamma^t r_t$
- $\pi_\theta(a_t \mid s_t)$: policy: the LLM with parameters θ, outputs next-token distribution
- $\rho_0(s_0)$: distribution over initial states (e.g. prompt distribution)
- $P(s_{t+1} \mid s_t, a_t)$: environment transition: deterministic for LLMs (append token)

Goal: Find parameters θ that maximize the expected return:

$$\theta^* = \arg\max_\theta J(\theta), \qquad J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \int P(\tau \mid \theta)\, R(\tau)\, d\tau$$

$J(\theta)$: expected return over all trajectories sampled from the current policy π_θ.

Probability of trajectory:

$$P(\tau \mid \theta) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} \mid s_t, a_t)\, \pi_\theta(a_t \mid s_t)$$

Policy Gradient Optimization

Parametrize π_θ via weights of LLM. Update: $\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_\theta J(\theta)\big|_{\theta_k}$

Why is optimizing $J(\theta)$ hard?

- **Integral over trajectories:** $J(\theta) = \int P(\tau \mid \theta) R(\tau)\, d\tau$. Cannot enumerate; must **sample**
- **Non-differentiable environment:** $P(s_{t+1} \mid s_t, a_t)$ and $R(\tau)$ are black boxes. Cannot backpropagate through them
- **Non-stationarity:** Data distribution $P(\tau \mid \theta)$ **depends on the policy** being optimized, changing with every update, old samples become stale
- **High variance:** Small policy changes can lead to very different trajectories $\Rightarrow$ gradient estimates are noisy
- **Temporal correlation:** Tokens within a trajectory are highly correlated, violating i.i.d. assumptions of standard SGD
- **Sparse reward:** Often only a single reward at the end of a full sequence. Credit assignment across hundreds of tokens

$\Rightarrow$ Need specialized techniques: policy gradient theorem, variance reduction, importance sampling.

Policy Gradient Theorem: Derivation

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] \\
&= \nabla_\theta \int P(\tau \mid \theta)\, R(\tau)\, d\tau && \text{expand expectation} \\
&= \int \nabla_\theta P(\tau \mid \theta)\, R(\tau)\, d\tau && \text{gradient under integral} \\
&= \int P(\tau \mid \theta)\, \nabla_\theta \log P(\tau \mid \theta)\, R(\tau)\, d\tau && \text{log-derivative trick} \\
&= \mathbb{E}_{\tau\sim\pi_\theta}[\nabla_\theta \log P(\tau \mid \theta)\, R(\tau)] && \text{back to expectation}
\end{aligned}
$$

Log-derivative trick: $\nabla_\theta P(\tau \mid \theta) = P(\tau \mid \theta)\, \nabla_\theta \log P(\tau \mid \theta)$

Key insight: We no longer need to differentiate through the environment, only through $\log P(\tau \mid \theta)$.

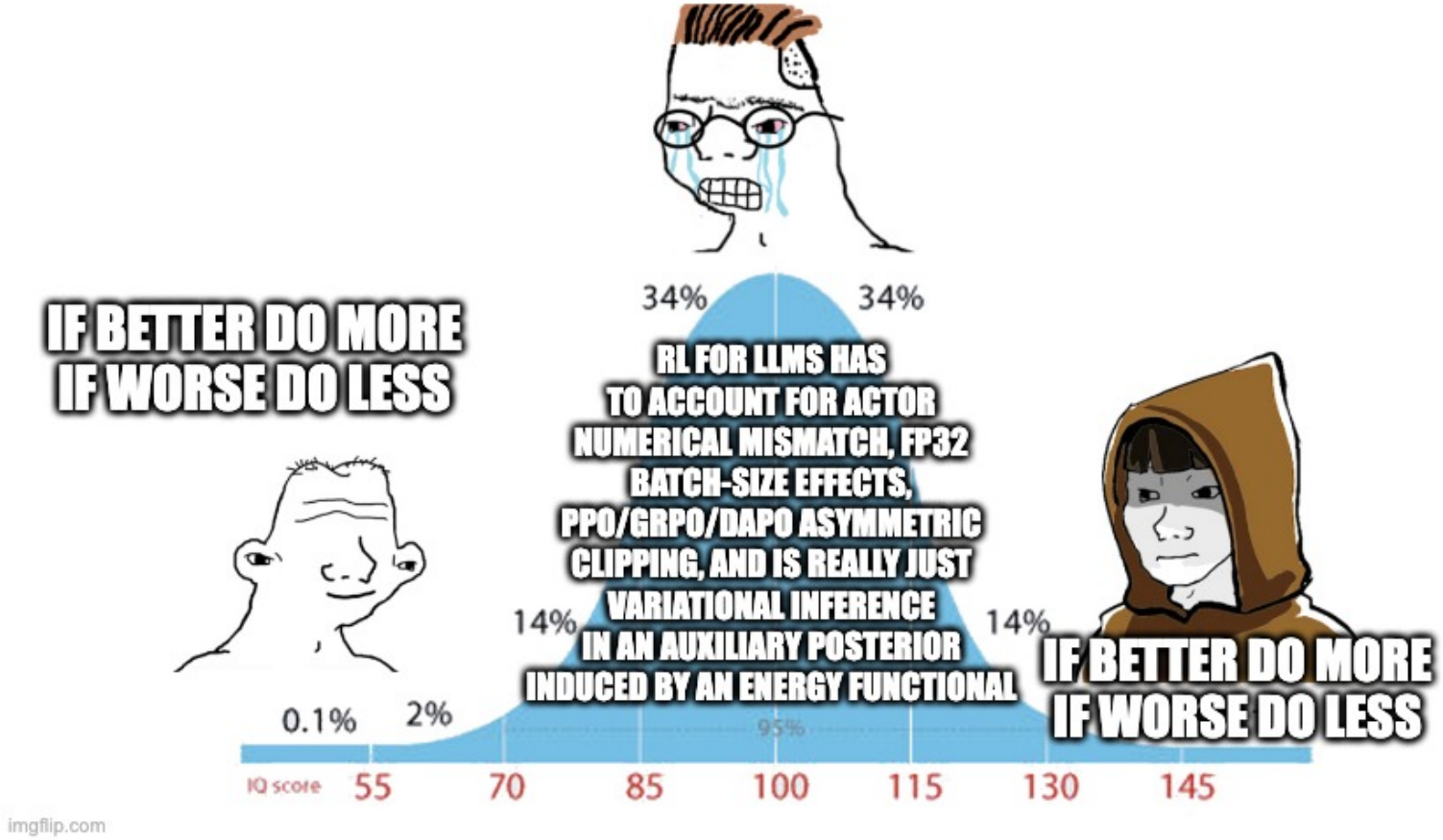
Policy Gradient Theorem: Final Form

Expanding $\log P(\tau \mid \theta) = \log \rho_0(s_0) + \sum_t \log P(s_{t+1} \mid s_t, a_t) + \sum_t \log \pi_\theta(a_t \mid s_t)$
Only $\log \pi_\theta$ depends on θ, so environment terms vanish under ∇_θ:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)\right]}$$

- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$: gradient of log-probability of action taken, computable via backprop through the LLM
- $R(\tau)$: return of the trajectory (scalar, no gradient needed)
- Approximate expectation by **sampling trajectories**:
  $\nabla_\theta J(\theta) \approx \frac{1}{|D|} \sum_{\tau \in D} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)$

Policy Gradient Intuition



REINFORCE Algorithm

Algorithm 11: REINFORCE

Input: Policy network π_θ, reward model R, learning rate α, prompt dataset $\mathcal{P}$
Initialize policy parameters θ
for $k = 0$ **to** $K - 1$ **do**
  Sample trajectories: For each prompt $x \in \mathcal{P}$, generate response $y \sim \pi_\theta(\cdot \mid x)$
  Compute rewards: $R(\tau_i)$ for each trajectory τ_i using reward model
  Estimate policy gradient:
    $\hat{g} = \frac{1}{|D|} \sum_{\tau \in D} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, R(\tau)$
  Update: $\theta \leftarrow \theta + \alpha\, \hat{g}$

Trajectories: Generate text with LLM. Prompts are questions/user requests.
Rewards: Given by trained reward model (or verifier).

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs
- Motivation
- RL Introduction
- Reward Model
- Policy Gradient
- Reducing Variance
- Off-Policy Learning and PPO
- GRPO
- DPO
- Summary and Open Problems

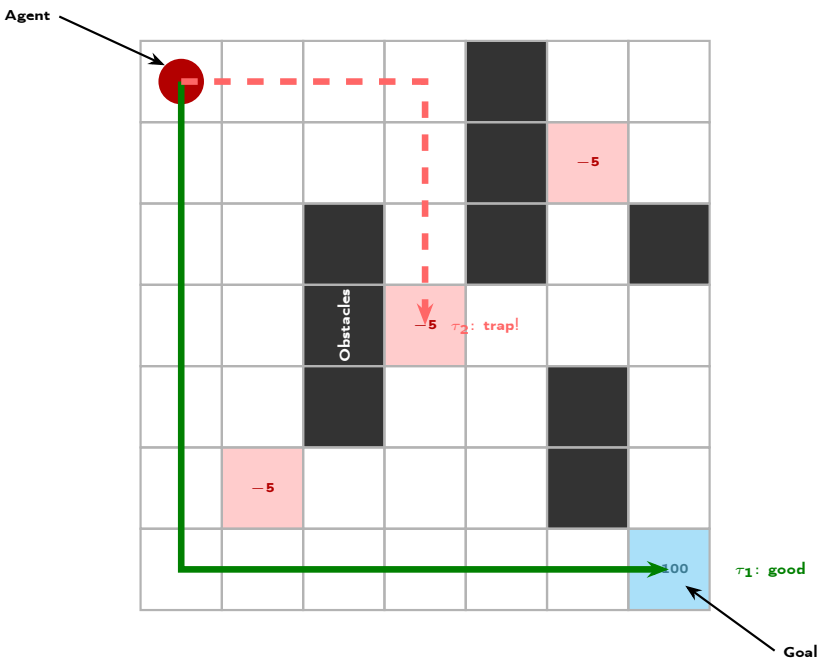
14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

Why REINFORCE Has High Variance



τ_1 (green): 12 steps to goal, $R = 88$

- REINFORCE: all 12 actions get gradient $\times 88$
- But step 1 ($\downarrow$) was trivial, step 7 ($\rightarrow$ turning toward goal) was the key decision

τ_2 (red): 6 steps into trap, $R = -10$

- All 6 actions get gradient $\times (-10)$
- Only the last step ($\downarrow$ into trap) was bad

Problem:

- Same start, two rollouts $\Rightarrow$ gradients differ by factor $\approx -9\times$
- Good/bad actions credited equally
- Gradient estimate is **unbiased but very noisy**

REINFORCE Gradient: Reducing Variance

Fix 1: Reward-to-go. Only credit future rewards, not past ones:

$$\nabla_\theta J(\theta) = \frac{1}{|D|} \sum_{\tau \in D} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \underbrace{\left(\sum_{t'=t}^{T} r(s_{t'}, a_{t'}) \right)}_{\text{reward-to-go}}$$

Why valid? Past rewards $r_{t'}$ for $t' < t$ are independent of a_t:

$$\mathbb{E}_{a_t}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, r_{t'}] = r_{t'} \underbrace{\sum_{a_t} \nabla_\theta \pi_\theta(a_t \mid s_t)}_{=\nabla_\theta 1 = 0} = 0 \quad \text{for } t' < t$$

Past terms vanish in expectation $\Rightarrow$ removing them preserves unbiasedness but reduces variance.

Reducing Variance: Baseline

Fix 2: Baseline. Subtract a state-dependent baseline $b(s_t)$:

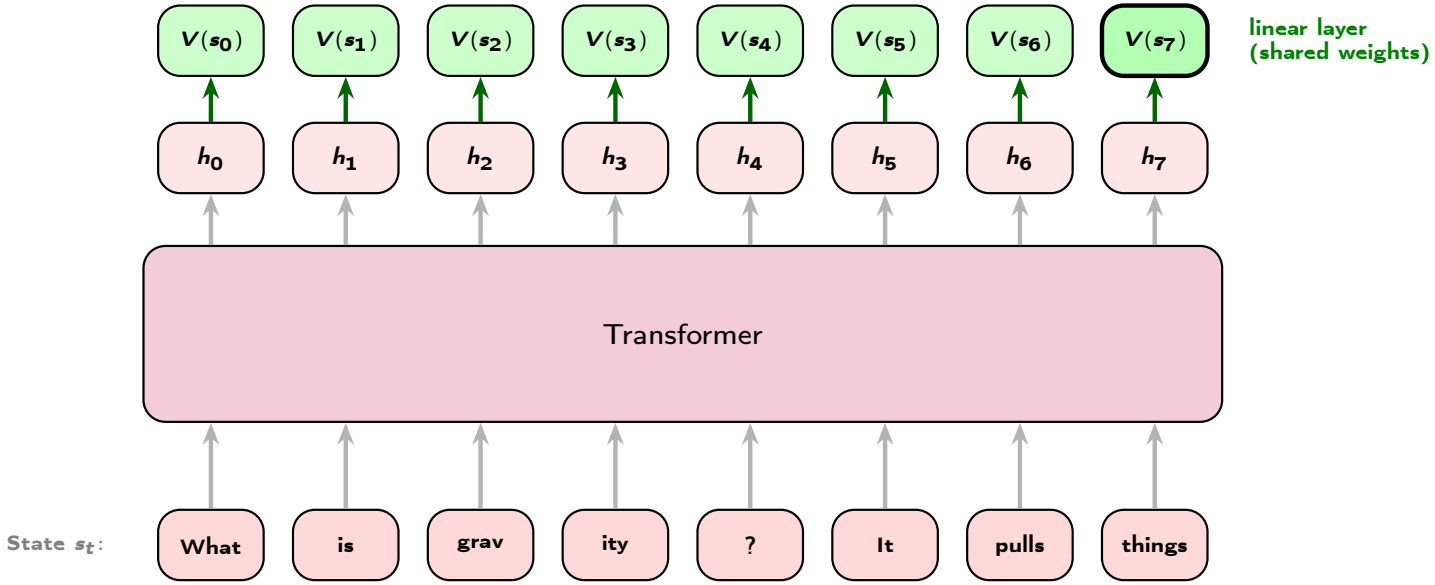
$$\nabla_\theta J(\theta) = \frac{1}{|D|} \sum_{\tau \in D} \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \left(\sum_{t'=t}^{T} r(s_{t'}, a_{t'}) - b(s_t) \right)$$

Why valid? $b(s_t)$ does not depend on a_t, so its contribution vanishes:

$$\begin{aligned}
\mathbb{E}_{a_t \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, b(s_t)] &= b(s_t) \sum_{a_t} \pi_\theta(a_t \mid s_t) \frac{\nabla_\theta \pi_\theta(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)} \\
&= b(s_t) \sum_{a_t} \nabla_\theta \pi_\theta(a_t \mid s_t) = b(s_t)\, \nabla_\theta \underbrace{\sum_{a_t} \pi_\theta(a_t \mid s_t)}_{=1} = 0
\end{aligned}$$

$\Rightarrow$ Any $b(s_t)$ independent of actions preserves unbiasedness but **reduces variance** by centering the reward around a reference value.

$V(s)$ Baseline Estimator



- **Value function** $V(s) = \mathbb{E}_{\tau:s_0=s}[R(\tau)]$: expected return from state s
- Linear layer on each token's hidden state. Trained via regression: $\min_\phi (V_\phi(s_t) - R(\tau))^2$
- Backbone can be **shared** with policy or **separate**:
  - Shared: cheaper, but value gradients may interfere with policy representations
  - Separate: used by InstructGPT/PPO. Doubles memory, but cleaner optimization

$A(s_t, a_t)$ Advantage Function

$$A(s, a) = \mathbb{E}_{\tau:s_0=s,a_0=a}[R(\tau)] - V(s)$$

Interpretation:

- How much better is it to take action a in state s over the average action of policy π
- When used in policy gradient: Increase/decrease logits of actions with positive/negative advantage

Recursively rewriting Advantage Function:

$$A(s_t, a_t) = \mathbb{E}[r(s_t, a_t) + \gamma V(s_{t+1})] - V(s_t)$$
$$A(s_t, a_t) = \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})] - V(s_t)$$
$$\vdots$$

Generalized Advantage Estimation (GAE): Temporal difference (TD) residual:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

GAE advantage estimate (hyperparameter $\lambda \in [0, 1]$):

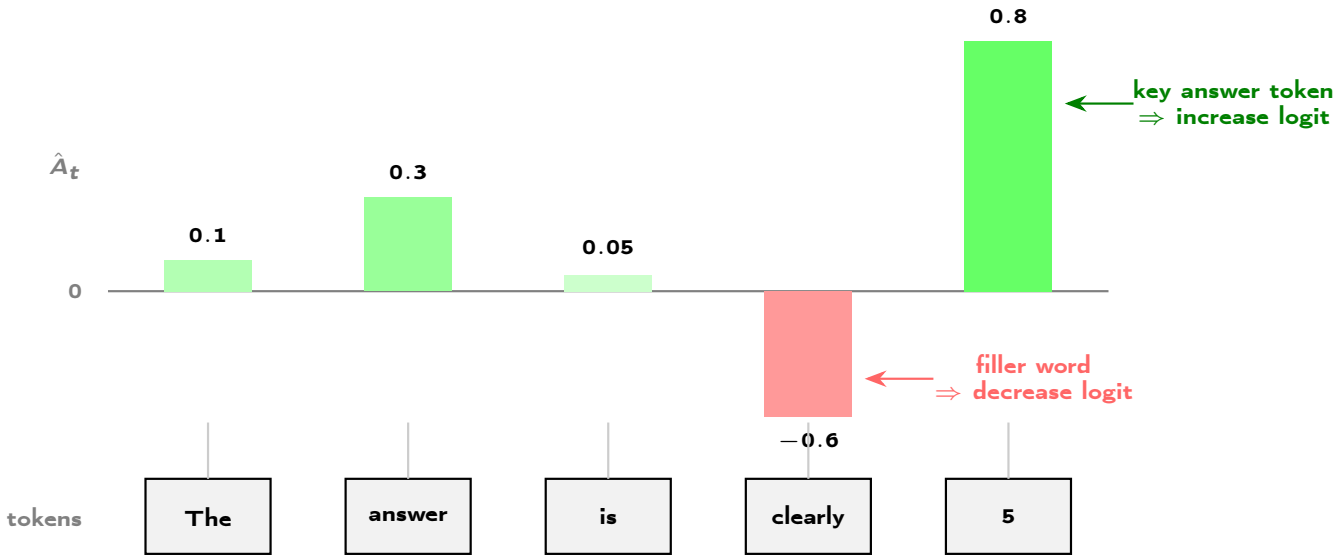
$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \, \delta_{t+l}$$

- $\lambda = 0$: $\hat{A}_t = \delta_t$ (one-step TD, low variance, high bias)
- $\lambda = 1$: $\hat{A}_t = R_t - V(s_t)$ (Monte Carlo, high variance, no bias)

Advantage Function: Policy Gradient Update

Policy Gradient Update with Advantage Function:

$$\nabla_\theta J(\theta) \approx \frac{1}{|D|} \sum_{\tau \in D} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}(s_t, a_t)$$



- **Positive advantage**: action was better than average ⇒ increase its logit
- **Negative advantage**: action was worse than average ⇒ decrease its logit

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs
- Motivation
- RL Introduction
- Reward Model
- Policy Gradient
- Reducing Variance
- Off-Policy Learning and PPO
- GRPO
- DPO
- Summary and Open Problems

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

(Off-Policy) Sampling Trajectories

We need to sample trajectories τ to estimate $\nabla_\theta J(\theta)$ and make a gradient step.

On-Policy: Sample from current policy π_θ before every update.

- Inefficient: trajectories discarded after each gradient step.

Off-Policy: Reuse trajectories from an older policy $\pi_{\theta_{\text{old}}}$ via **Importance Sampling**.

Importance Sampling: Derivation

$$\mathbb{E}_{x\sim p}[f(x)] = \int p(x)f(x)\,dx = \int q(x)\frac{p(x)}{q(x)}f(x)\,dx = \mathbb{E}_{x\sim q}\left[\frac{p(x)}{q(x)}f(x)\right]$$

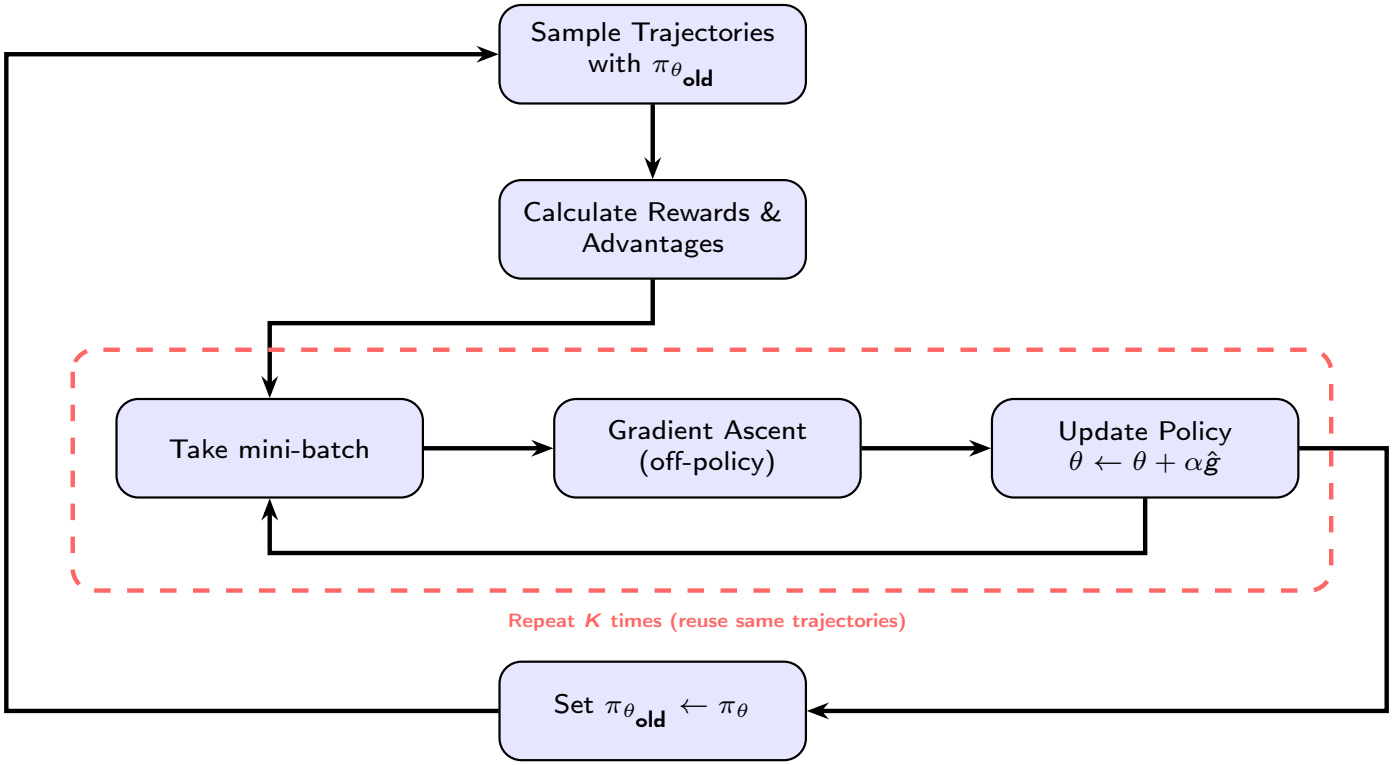
Applied to RL:

- $x = (s_t, a_t)$: state-action pairs from a trajectory
- $p = \pi_\theta$: current policy (what we want to optimize)
- $q = \pi_{\theta_{\text{old}}}$: old policy (what we actually sampled trajectories from)
- f: the quantity we want to estimate, e.g. $\nabla_\theta \log \pi_\theta(a_t \mid s_t)\,\hat{A}_t$
- Ratio $\frac{p}{q} = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$: corrects for the mismatch between old and current policy

Why higher variance?

- If π_θ assigns high probability where $\pi_{\theta_{\text{old}}}$ assigns low $\Rightarrow$ ratio explodes
- A few samples with extreme ratios dominate the estimate $\Rightarrow$ noisy gradients
- The more π_θ diverges from $\pi_{\theta_{\text{old}}}$, the worse it gets

Off-Policy Learning

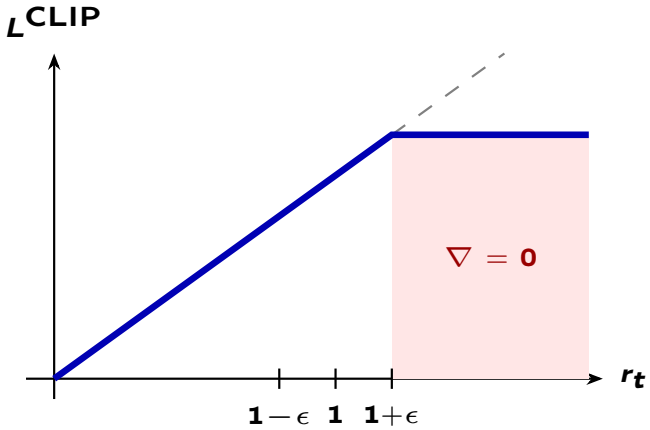


- Sample trajectories once with $\pi_{\theta_{\text{old}}}$, store them
- Perform K gradient updates reusing the same trajectories (with importance sampling correction)
- Then set $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$ and resample

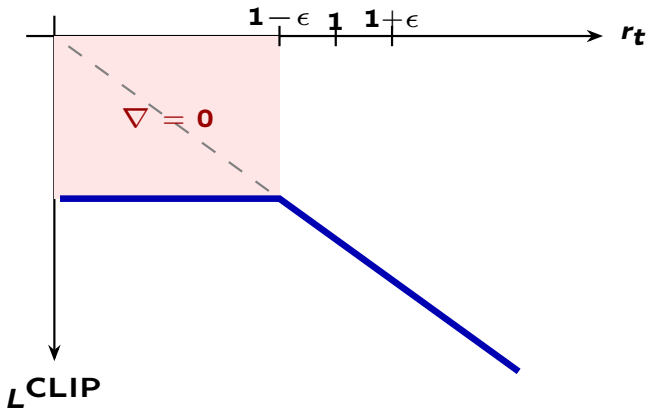
PPO Clipping Loss: Illustration

$$L^{\text{CLIP}} = \min\big(r_t\, A_t,\ \text{clip}(r_t,\ 1-\epsilon,\ 1+\epsilon)\, A_t\big), \qquad r_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

$A_t > 0$ (good action)



$A_t < 0$ (bad action)



- $A_t > 0$: no incentive to increase r_t beyond $1+\epsilon$. This prevents making good actions *too much* more likely
- $A_t < 0$: no incentive to decrease r_t below $1-\epsilon$. This prevents making bad actions *too much* less likely
- Gradient is zero in shaded region $\Rightarrow$ policy stays close to $\pi_{\theta_{\text{old}}}$

Proximal Policy Optimization (PPO) Loss

PPO [Schulman et al., *"Proximal Policy Optimization Algorithms"*, 2017] = REINFORCE + Offline Sampling + Clipped Policy Loss + Regularization

$\mathcal{B}$: mini-batch of (s_t, a_t) pairs sampled from trajectories collected with π_{offline}

Clipped Loss:

$$L^{\text{policy}} = \frac{1}{|\mathcal{B}|} \sum_{(s_t, a_t) \in \mathcal{B}} \min\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{offline}}(a_t \mid s_t)} A_t,\ \text{clip}\left(\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{offline}}(a_t \mid s_t)}, 1-\epsilon, 1+\epsilon \right) A_t \right)$$

Value Function Loss:

$$L^{\text{VF}} = \frac{1}{|\mathcal{B}|} \sum_{(s_t, a_t) \in \mathcal{B}} \frac{1}{2} \left\| V_\phi(s_t) - \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'} \Big|_{s_t} \right\|_2^2$$

Entropy Loss: $L^{\text{entropy}} = \frac{1}{|\mathcal{B}|} \sum_{(s_t, a_t) \in \mathcal{B}} \left(-\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t) \right)$

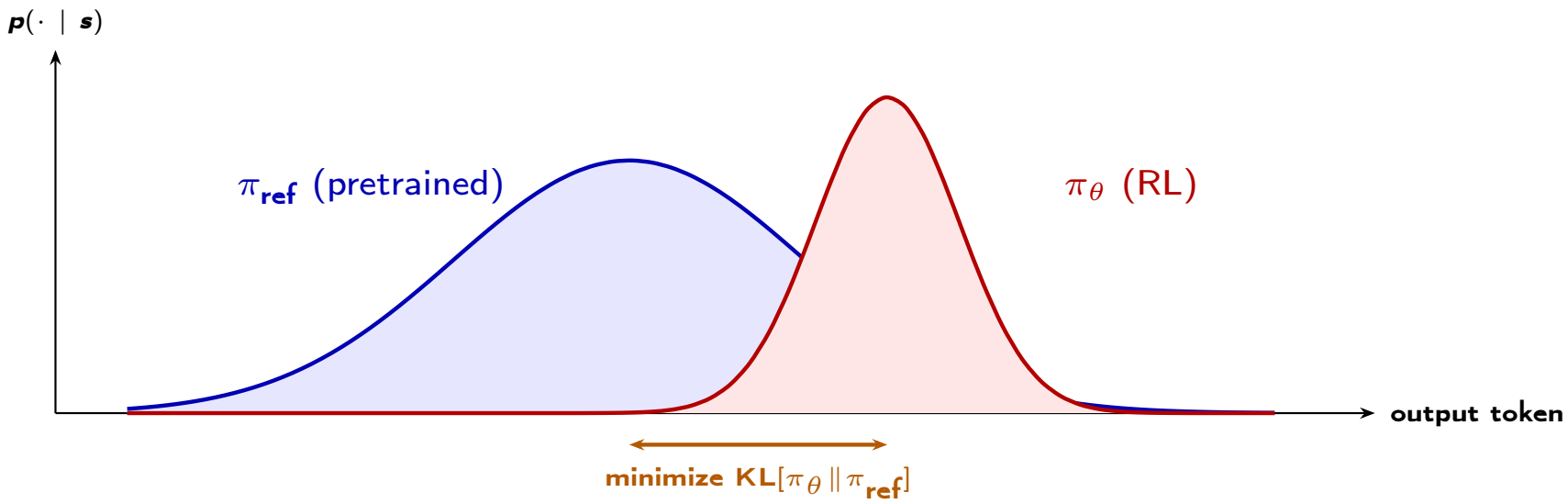
Overall:

$$\boxed{L^{\text{PPO}} = -L^{\text{policy}} + c_1 L^{\text{VF}} - c_2 L^{\text{entropy}}}$$

Regularization via KL Divergence

- Without regularization, PPO makes the LLM **degrade**: collapses to outputs the reward model likes
- Add penalty for deviating from frozen pretrained model π_{ref}:

$$L^{\text{KL}} = \beta \cdot \text{KL}[\pi_\theta \| \pi_{\text{ref}}], \qquad \text{KL}[q\|p] = \mathbb{E}_{x\sim q}\left[\log \frac{q(x)}{p(x)}\right] \geq 0$$



$\Rightarrow$ Keeps π_θ close to π_{ref}: retains language capabilities while optimizing reward.

KL Divergence Estimators

Goal Estimate $\mathrm{KL}[\pi_\theta \| \pi_{\mathrm{ref}}]$ during training

Problem Exact computation requires summing over full vocabulary at every token, which is intractable

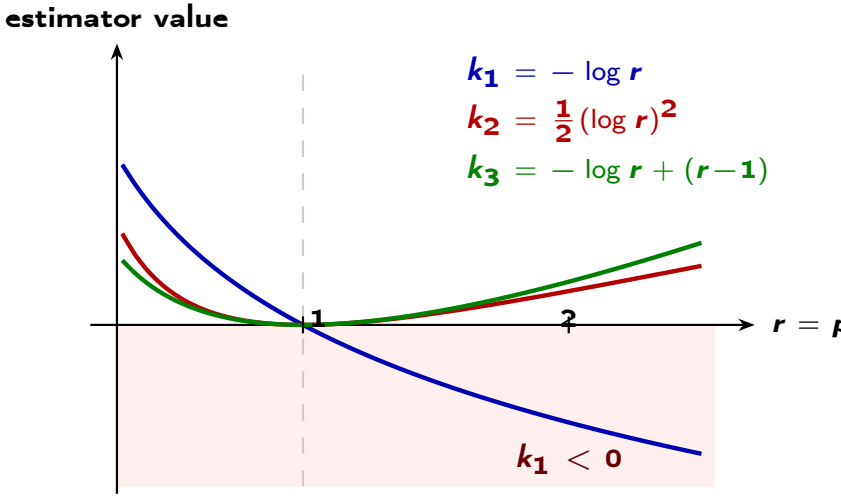
Solution Use **single-sample estimators** with $r = \frac{\pi_{\mathrm{ref}}(x)}{\pi_\theta(x)}$

- $k_1 = -\log r$
  - Unbiased: $\mathbb{E}_q[k_1] = \mathrm{KL}$
  - Can be **negative**, high variance
- $k_2 = \frac{1}{2}(\log r)^2$
  - $\mathbb{E}_q[k_2] = \frac{1}{2}\left(\mathrm{Var}_q[\log r] + \mathrm{KL}^2\right)$
  - Biased (second-order), vanishes as $q \to p$
  - Always ≥ 0
- $k_3 = -\log r + (r-1)$ (preferred)
  - Unbiased:

$$\mathbb{E}_q[k_3] = \mathbb{E}_q[-\log r] + \underbrace{\mathbb{E}_q[r-1]}_{=\sum_x p(x)-1=0} = \mathrm{KL}$$

  - Non-negative: $\log x \leq x - 1$ (concavity)

$$k_3 = \underbrace{-\log r}_{\geq 1-r} + (r-1) \geq 0$$



PPO Overall Algorithm

Algorithm 12: Proximal Policy Optimization (PPO)

for $k = 0$ **to** $K - 1$ **do**

- **Collect Trajectories:** Run π_{θ_k} to collect trajectories $\mathcal{D}_k = \{\tau_i\}$
- **Compute Returns and Advantages:**
  - Compute rewards-to-go $\hat{R}_t$
  - Compute advantage estimates $\hat{A}_t$ using current value function V_{ϕ_k}
- **Optimize Policy: for** $e = 1$ **to** E **do**
  - Shuffle and partition $\mathcal{D}_k$ into minibatches
  - Compute ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$
  - Compute surrogate loss: $L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$
  - Compute KL penalty: $L^{\text{KL}}(\theta) = \widehat{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$
  - Update θ via gradient ascent on $L^{\text{CLIP}} - \beta L^{\text{KL}}$
- **Optimize Value Function:** $L^{\text{VF}}(\phi) = \mathbb{E}_t[(V_\phi(s_t) - \hat{R}_t)^2]$

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs
- Motivation
- RL Introduction
- Reward Model
- Policy Gradient
- Reducing Variance
- Off-Policy Learning and PPO
- GRPO
- DPO
- Summary and Open Problems

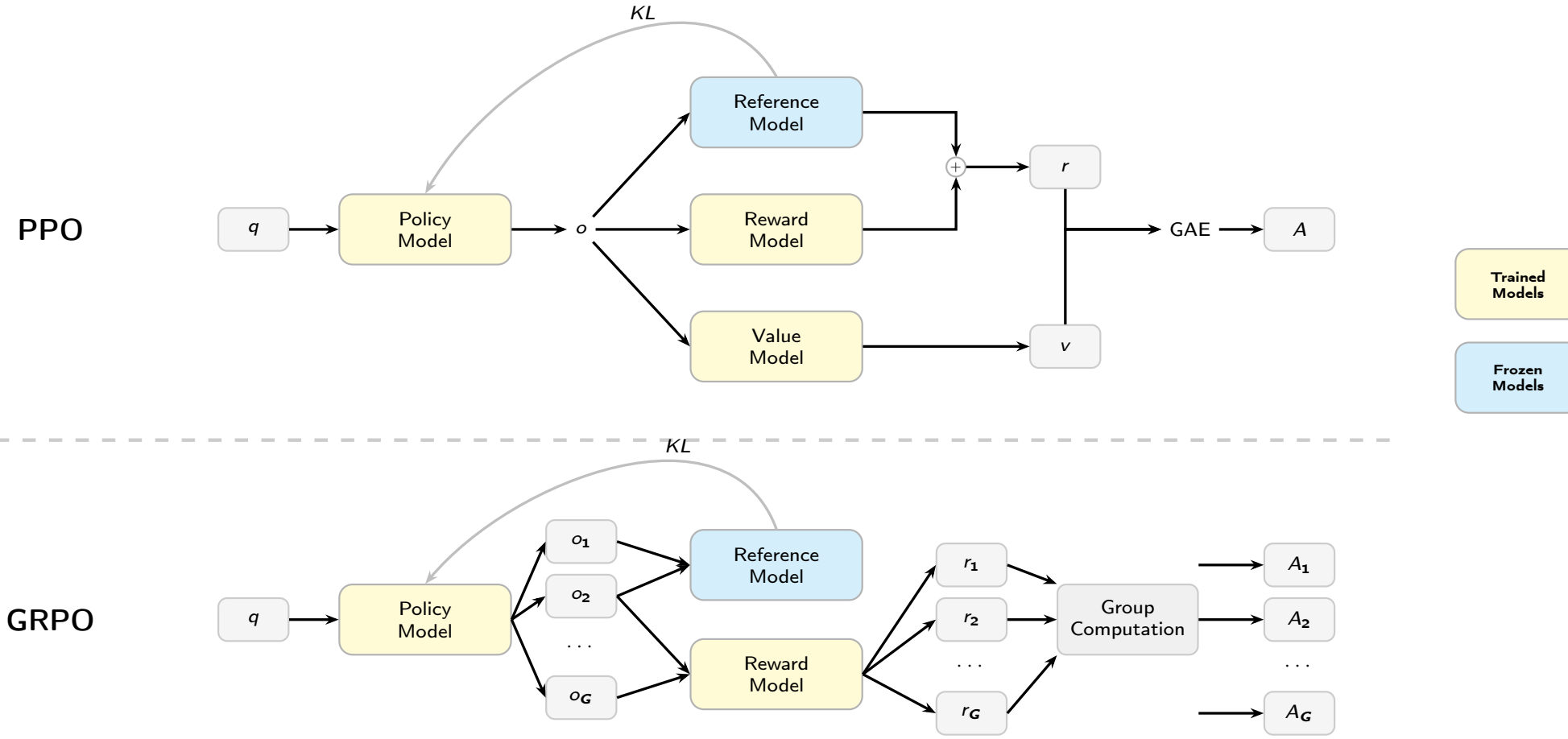
14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

GRPO: Group Relative Policy Optimization



- GRPO [Shao et al., *"DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models"*, 2024] drops the **value model**: estimates advantages from group reward statistics
- Saves $\approx \frac{1}{3}$ of GPU memory (no critic to store or train)

GRPO: Advantage Function

For a given prompt q, sample G responses: $o_1, \ldots, o_G \sim \pi_\theta(\cdot \mid q)$.

Compute **return** for each response: $R_i = \sum_{t'=t}^{T} r_{i,t'}$ (reward-to-go for trajectory i)

Group statistics across the G responses:

$$\mu = \frac{1}{G}\sum_{j=1}^{G} R_j, \qquad \sigma = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(R_j - \mu)^2}$$

GRPO Advantage for response i at token t:

$$\boxed{A_{i,t} = \frac{R_i - \mu}{\sigma}}$$

- Replaces GAE: no value model needed, only **relative ranking** within the group
- $A_{i,t} > 0$ if response i is better than group average, < 0 if worse

GRPO Algorithm

Algorithm 13: Group Relative Policy Optimization (GRPO)

for $k = 0$ **to** $K - 1$ **do**

Data Collection: Sample prompts $\{s_j\}_{j=1}^{M}$. For each s_j, generate K_j responses using π_{θ_k}

Reward Evaluation: For each response a_{jk}, compute $R_{jk} = R(s_j, a_{jk})$

Advantage Computation:

Mean reward: $\bar{R}_j = \frac{1}{K_j} \sum_{k=1}^{K_j} R_{jk}$

Advantage: $A_{jk} = \frac{R_{jk} - \bar{R}_j}{\mathrm{std}(R_{jk})_{k=1,\ldots,K_j}}$

Policy Update:

Ratio: $r_{jk} = \frac{\pi_\theta(a_{jk} | s_j)}{\pi_{\theta_k}(a_{jk} | s_j)}$

$L = -\sum_{j,k} \min(r_{jk} A_{jk}, \mathrm{clip}(r_{jk}, 1-\epsilon, 1+\epsilon) A_{jk}) + \beta \sum_j \mathrm{KL}[\pi_\theta(\cdot \mid s_j) \| \pi_{\theta_k}(\cdot \mid s_j)]$

- GRPO is specific to RL with LLMs (not general RL)
- Saves typically somewhat less than half the RAM compared to PPO

GRPO vs. PPO: Key Differences

No Value Model GRPO replaces the learned critic $V_\phi(s)$ with group statistics. Saves ≈30–50% GPU memory (no extra LLM-sized model).

Per-Sequence Reward Standard GRPO assigns a **single reward per sequence** (e.g. from a reward model or verifier). All tokens in a response share the same advantage $\hat{A}_i$. PPO with GAE can assign per-token advantages via temporal difference.

Multiple Samples Required GRPO needs $G \geq 2$ responses per prompt (typically $G = 4$–16) to compute group statistics. PPO can work with a single response per prompt.

Simpler Hyperparameters GRPO removes GAE parameters (λ, value loss coefficient c_1). Key hyperparameters: group size G, clipping $\epsilon = 0.2$, KL coefficient β.

Trade-off GRPO needs G forward passes per prompt for generation (more than PPO's single sample), but saves forward/backward passes of the value model. Net effect: typically faster.

Empirically GRPO matches or exceeds PPO on math/code reasoning tasks. Works especially well when reward is **binary** (correct/incorrect) or **sparse** (one score per response).

Dr. GRPO: GRPO Done Right

Standard GRPO has two subtle biases [Zichen Liu et al., *"Understanding R1-Zero-Like Training: A Critical Perspective"*, 2025]:

Length Bias: Normalizes by sequence length $\Rightarrow$ penalizes longer correct responses more, incentivizes verbose wrong answers.

Response	Tokens	Per-token loss	Effect
"42" (correct)	1	$R/1 = R$	strong $\uparrow$
"The answer is 42" (correct)	5	$R/5 = 0.2R$	diluted $\uparrow$
"7" (wrong)	1	$\lvert R_{\text{bad}}\rvert/1$	strong $\downarrow$
"Well, I think it's 7" (wrong)	6	$\lvert R_{\text{bad}}\rvert/6$	weak $\downarrow$

Std. Normalization Bias: Dividing by σ_G overweights nearly-solved problems (low variance).

Dr. GRPO: Algorithm

Algorithm 14: Dr. GRPO (modifications in red)

for $k = 0$ **to** $K - 1$ **do**

Data Collection: Sample prompts $\{s_j\}_{j=1}^{M}$. For each s_j, generate K_j responses using π_{θ_k}

Reward Evaluation: For each response a_{jk}, compute $R_{jk} = R(s_j, a_{jk})$

Advantage Computation:

Mean reward: $\bar{R}_j = \frac{1}{K_j} \sum_{k=1}^{K_j} R_{jk}$

Advantage: $A_{jk} = R_{jk} - \bar{R}_j$ **(no σ division)**

Policy Update:

Ratio: $r_{jk,t} = \frac{\pi_\theta(a_{jk,t} | s_j, a_{jk,<t})}{\pi_{\theta_k}(a_{jk,t} | s_j, a_{jk,<t})}$

$L = -\frac{1}{T_{\max}} \sum_{j,k} \sum_t \min(r_{jk,t} A_{jk},\ \text{clip}(r_{jk,t}, 1-\epsilon, 1+\epsilon) A_{jk})$ **(fixed normalization)**

- No σ_G division: removes disproportionate updates on nearly-solved problems
- $T_{\max}$ normalization: divides by max token count, not actual length $\Rightarrow$ no length bias

DAPO: Decoupled Alignment Policy Optimization

DAPO [Q. Yu et al., *"DAPO: An Open-Source LLM Reinforcement Learning System at Scale"*, 2025] addresses four issues in GRPO:

Decoupled Clipping: Symmetric clip $[1-\epsilon, 1+\epsilon]$ prevents exploration. DAPO uses **asymmetric** bounds:

$$\text{clip}(r_t,\ 1-\epsilon_{\text{low}},\ 1+\epsilon_{\text{high}}), \quad \epsilon_{\text{low}} = 0.2,\ \epsilon_{\text{high}} = 0.28$$

Higher upper bound $\Rightarrow$ low-probability tokens can increase more freely.

Dynamic Sampling: Skip prompts where all G responses are all-correct or all-wrong (advantage $= 0$, wasted compute). Resample until batch has non-trivial signal.

Token-Level Averaging: GRPO averages per-sequence ($\frac{1}{|o_i|}$), then across sequences ($\frac{1}{G}$). A 1-token wrong answer gets the **same weight** as a 100-token reasoning chain. DAPO weights every token equally: $\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_i\sum_t(\cdots)$. Longer sequences contribute proportionally more gradient. (Compare Dr. GRPO's $T_{\max}$ fix: different approach to the same length bias.)

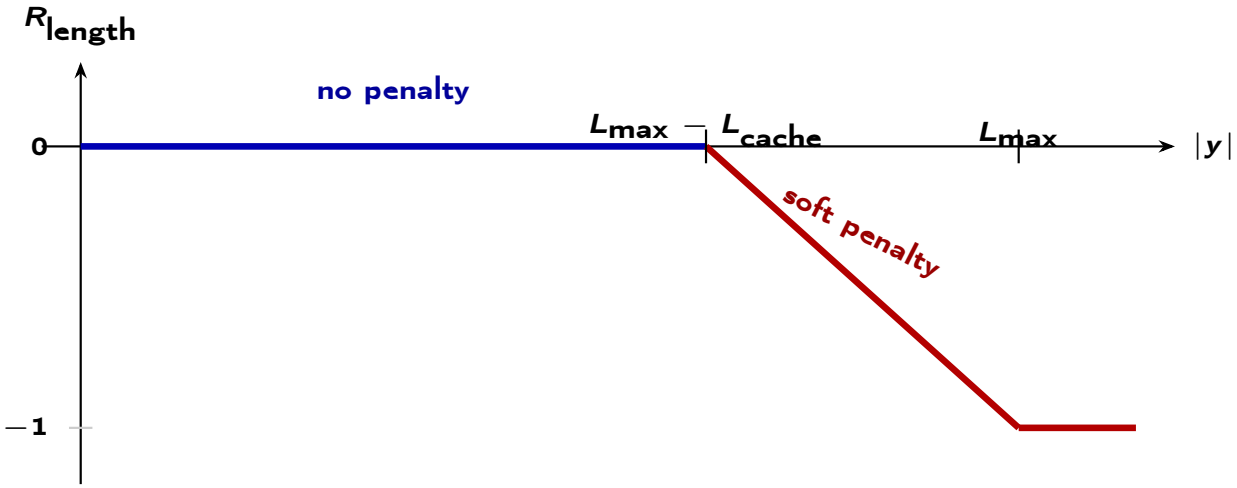
No KL Penalty: For long chain-of-thought reasoning, KL to π_{ref} is counterproductive. DAPO drops it entirely.

DAPO: Overlong Reward Shaping

Hard truncation at $L_{\max}$ tokens corrupts the reward signal (correct but long $\Rightarrow$ truncated $\Rightarrow$ wrong reward).

Soft overlong punishment:

$$R_{\text{length}}(y) = \begin{cases} 0, & |y| \leq L_{\max} - L_{\text{cache}} \\ \dfrac{(L_{\max} - L_{\text{cache}}) - |y|}{L_{\text{cache}}}, & L_{\max} - L_{\text{cache}} < |y| \leq L_{\max} \\ -1, & |y| > L_{\max} \end{cases}$$



Typical values: $L_{\max} = 20{,}480$, $L_{\text{cache}} = 4{,}096$ tokens.

DAPO: Algorithm

Algorithm 15: DAPO (modifications vs. GRPO in red)

for $k = 0$ **to** $K - 1$ **do**

Data Collection: Sample prompts $\{s_j\}$. For each s_j, generate G responses using π_{θ_k}

Dynamic Sampling: Discard prompts where all responses are all-correct or all-wrong. Resample to fill batch.

Reward: $R_{jk} = R(s_j, a_{jk}) + R_{\text{length}}(a_{jk})$

Advantage: $A_{jk} = \frac{R_{jk} - \bar{R}_j}{\text{std}(\{R_{jk}\}_k)}$

Policy Update:

$$r_{jk,t} = \frac{\pi_\theta(a_{jk,t} | s_j, a_{jk,<t})}{\pi_{\theta_k}(a_{jk,t} | s_j, a_{jk,<t})}$$

$$L = -\frac{1}{\sum_{j,k} |a_{jk}|} \sum_{j,k} \sum_t \min\left(r_{jk,t} A_{jk},\ \text{clip}(r_{jk,t},\ 1-\epsilon_{\text{low}},\ 1+\epsilon_{\text{high}}) A_{jk}\right)$$

(No KL penalty)

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs
- Motivation
- RL Introduction
- Reward Model
- Policy Gradient
- Reducing Variance
- Off-Policy Learning and PPO
- GRPO
- DPO
- Summary and Open Problems

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

DPO: Direct Preference Optimization

Recall the **Bradley-Terry model** [Rafailov et al., *"Direct Preference Optimization: Your Language Model is Secretly a Reward Model"*, 2024]:

$$P(y_w \succ y_l) = \frac{\exp r(x, y_w)}{\exp r(x, y_w) + \exp r(x, y_l)}$$

The RLHF overall objective:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim D, y \sim \pi_\theta(\cdot|x)}[r(x, y)] - \beta \, \mathrm{KL}[\pi_\theta(x, y) \| \pi_{\mathrm{ref}}(y \mid x)]$$

- Difficult: Non-differentiable, many moving parts, RL setting
- **DPO Idea:** Put away with reward model and RL-loop, directly optimize a preference based loss formulated as maximum likelihood

The Optimal Policy

RLHF objective:

$$\max_{\pi} \mathbb{E}_{x,y\sim\pi(\cdot|x)}[r(x,y)] - \beta\,\mathrm{KL}[\pi(y \mid x)\|\pi_{\mathsf{ref}}(y \mid x)]$$

Optimal policy:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\pi_{\mathsf{ref}}(y \mid x)\exp\left(\tfrac{1}{\beta}\, r(x,y)\right)$$

where $Z(x) = \sum_y \pi_{\mathsf{ref}}(y \mid x)\exp\left(\frac{1}{\beta}\, r(x,y)\right)$ is the partition function.

Since KL is nonnegative and achieves its unique minimum when both distributions are equal, the optimal policy π^* uniquely minimizes the objective.

The Optimal Policy: Reward in Terms of Policy

Rearrange to express the reward:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

Plug into the Bradley-Terry model:

$$P(y_w \succ y_l) = \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

Note: the partition functions $Z(x)$ cancel out!

DPO Loss:

$$\boxed{\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim D}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]}$$

Optimizing $\mathcal{L}_{\text{DPO}}$ w.r.t. $\theta \Rightarrow$ obtaining optimal policy π^*.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
    - Motivation
    - RL Introduction
    - Reward Model
    - Policy Gradient
    - Reducing Variance
    - Off-Policy Learning and PPO
    - GRPO
    - DPO
    - Summary and Open Problems
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents

State of RL for Reasoning LLMs: Summary

Major patterns across recent methods:

Critic Elimination: Every post-PPO method found that simpler baselines match or exceed learned value functions while saving $\sim$50% memory.

- LLM fine-tuning from strong pretrained checkpoints makes PPO's variance reduction largely unnecessary.

Std. Normalization Harmful: Dr. GRPO, DAPO and others show that dividing advantages by σ adds too much weight on nearly-solved problems.

Loss Aggregation Criticality: Sequence-level rewards combined with sample averaging introduce subtle token-level biases that fundamentally affect per-token learning signals.

Trust Region Refinement: PPO's $\epsilon = 0.2$ works well, yet recent methods explore asymmetric clipping (DAPO), weight-based approaches (CISPO), or divergence-based constraints (DPPO).

Open Research Challenges

Credit Assignment: Current outcome-based methods assign essentially the same reward to all tokens. This works but is inefficient. Process reward models and step-level verifiers remain unsolved.

Sample Efficiency: Methods require 8–64 rollouts per prompt despite information being just a single bit (correct/incorrect).

Very Hard Problems: No gradient signal when models never produce correct rollouts. Curriculum learning provides only workarounds.

Domain Generalization: Progress concentrates in domains with cheap and unambiguous verification (math and code). Extending to noisy or subjective evaluation remains difficult.

Empirical Reliability: Evidence remains relatively narrow and expensive to reproduce.

New Capabilities vs. Distribution Narrowing: Does RL teach fundamentally new skills, or does it merely **sharpen the output distribution** toward abilities already latent from pretraining? Evidence suggests RL primarily surfaces and focuses existing capabilities rather than creating novel ones. The ceiling may be set by pre- and mid-training.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
    - Motivation and Classical Distillation
    - KL on Logits vs Cross-Entropy on Labels
    - Self-Distillation
    - On-Policy Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
    - Motivation and Classical Distillation
    - KL on Logits vs Cross-Entropy on Labels
    - Self-Distillation
    - On-Policy Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Why Distillation?

Large models perform best, but are expensive to deploy:

- Inference latency, memory footprint, energy cost
- Edge devices, mobile, real-time agents
- Multi-tenant serving at scale

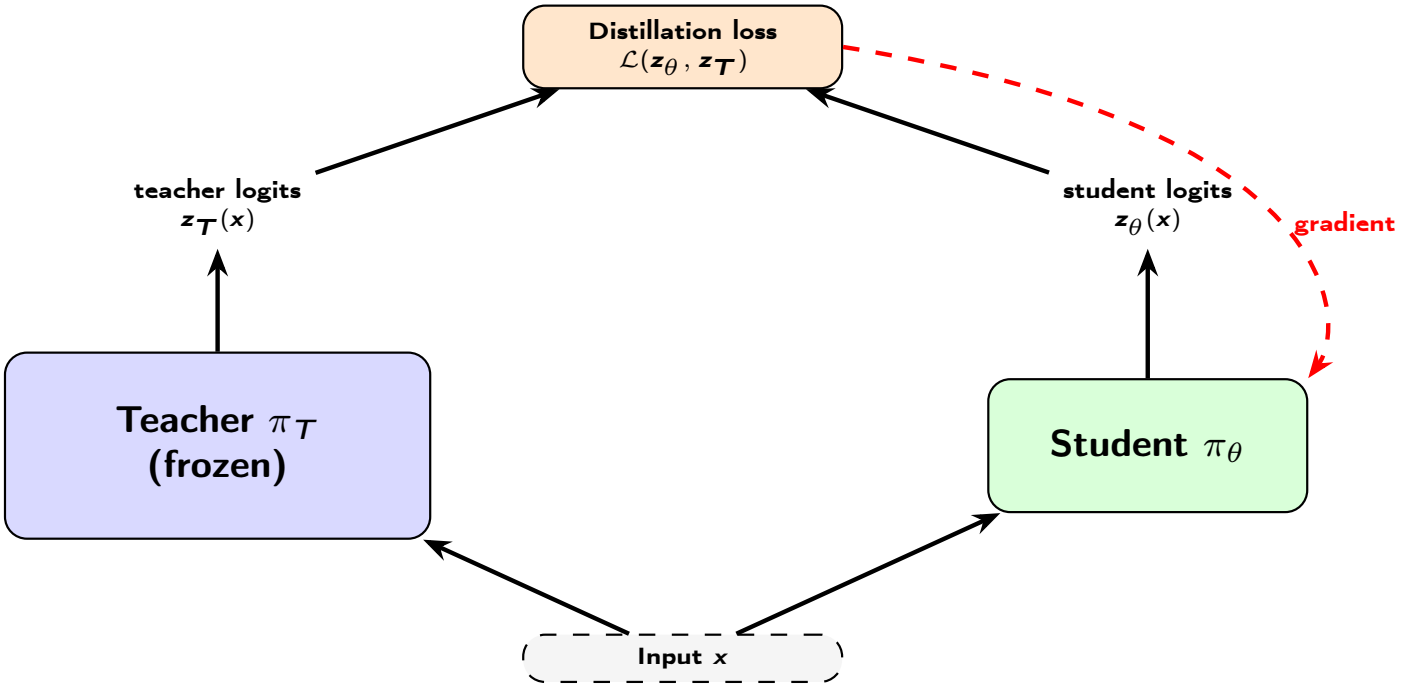
Knowledge distillation [Bucilă et al., *"Model Compression"*, 2006; Hinton, Vinyals, et al., *"Distilling the Knowledge in a Neural Network"*, 2015]: train a small **student** model to mimic a large **teacher**.

Teacher: Large, accurate, expensive (ensemble, frontier LLM, oracle).
Student: Small, fast, cheap (deployable model).
Goal: Student matches teacher quality at a fraction of the parameters.

Uses beyond compression:

- Transfer reasoning skills from a frontier LLM to a small open model (e.g. DeepSeek-R1 distill, DistilBERT [Sanh et al., *"DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter"*, 2019])
- Domain adaptation and personalization
- Recover capabilities lost during fine-tuning or mid-training

Distillation Setup



Key idea: Replace (or augment) hard labels with the teacher's full predictive distribution.
Teacher signal: $\pi_T(\cdot \mid x) = \mathrm{softmax}(z_T(x))$, a vector over all classes / vocabulary tokens.
Student trains: To make $\pi_\theta(\cdot \mid x)$ close to $\pi_T(\cdot \mid x)$.

Hinton Distillation: Soft Targets and Temperature

Plain softmax of confident teachers is almost one-hot, hiding most information.

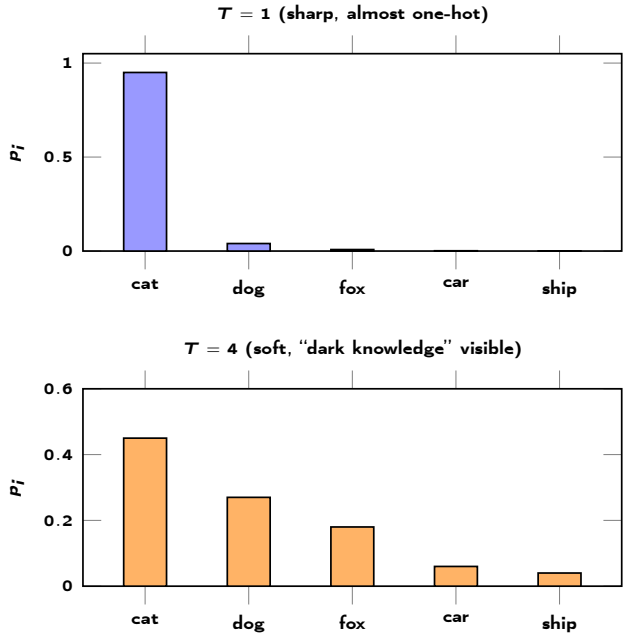
Temperature $T \geq 1$ flattens the distribution and exposes **relative** probabilities of non-target classes:

$$p_i^T = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Distillation loss [Hinton, Vinyals, et al., *"Distilling the Knowledge in a Neural Network"*, 2015]:

$$\mathcal{L} = \alpha\, T^2\, \mathrm{KL}\Big(p_T^T \,\|\, p_\theta^T\Big) + (1-\alpha)\,\mathrm{CE}(y, p_\theta^{T=1})$$

- First term: match teacher distribution at temperature T
- Second term: optional ground-truth supervision
- Factor T^2 keeps gradient magnitudes comparable



At $T = 4$ the student sees: ***cat is much more like dog and fox than like car and ship.***

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation
- Motivation and Classical Distillation
- KL on Logits vs Cross-Entropy on Labels
- Self-Distillation
- On-Policy Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Why KL on Logits Beats CE on Hard Labels

Hard labels carry only $\log_2 K$ bits per example. Teacher distributions carry the **full predictive vector**, encoding several non-trivial signals at once.

Dark knowledge: Relative scores of *wrong* classes encode the teacher's similarity structure (cat∼dog, truck∼car). One-hot labels destroy this.

Dense gradient signal: CE on a one-hot label provides a single nonzero target component; KL with a soft teacher provides gradient on every output dimension.

Implicit regularization: Soft targets are a label-smoothing prior derived from a model that has actually seen the data, so they smooth in the right directions.

Calibrated confidence: On ambiguous inputs the teacher hedges; the student inherits well-calibrated uncertainty instead of over-confident one-hots.

Noise robustness: Wrong or ambiguous labels matter less because the teacher's distribution dominates the loss.

Information per Example: Bits Argument

One-hot label:

- $\log_2 K$ bits per token
- Vocab $K = 32{,}000 \Rightarrow \approx 15$ bits
- All "other" classes look identical (zero)

Teacher distribution:

- Real number per class, $\sim K$ values
- Encodes ranking, magnitudes, similarities
- Bits per example bounded only by precision and entropy

RL reward (for contrast):

- One scalar per *trajectory*
- $\mathcal{O}(1)$ bits per episode

Rule of thumb [Goodfire et al., *On-Policy Distillation*, 2025]:

- RL teaches $\mathcal{O}(1)$ bits per episode
- Distillation teaches $\mathcal{O}(N)$ bits per episode, where N is the number of tokens
- SFT teaches $\mathcal{O}(N \log_2 K)$ bits, but along a single sample only

Consequence: A distilled gradient step is much more informative than an RL step or an SFT step on a one-hot target. In practice this translates to many fewer optimizer steps and much smaller datasets needed to reach a given quality level.

Dark Knowledge: A Concrete Example

Teacher prediction on an image of a BMW:

Class	Teacher p_T	Hard label
car	0.90	1
truck	0.07	0
bus	0.025	0
garbage truck	0.004	0
cat	3×10^{-7}	0
dog	2×10^{-7}	0

- The teacher tells the student: this looks "a bit like a truck or bus, almost nothing like a cat".
- One-hot loss treats truck and cat equivalently as "not the answer".
- Distillation loss penalizes confusing a BMW with a cat much more than confusing it with a truck, mirroring how a good model *should* fail.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
    - Motivation and Classical Distillation
    - KL on Logits vs Cross-Entropy on Labels
    - Self-Distillation
    - On-Policy Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Self-Distillation: Student = Teacher Architecture

Self-distillation: distill into a model with the *same* (or even smaller) architecture as the teacher. Surprisingly, this often *improves* accuracy.

Born-Again Networks [Furlanello et al., *"Born-Again Neural Networks"*, 2018]**:** Train net M_0 normally, then train M_1 with the same architecture using M_0 as teacher. Repeat. Final ensemble or M_k outperforms M_0.

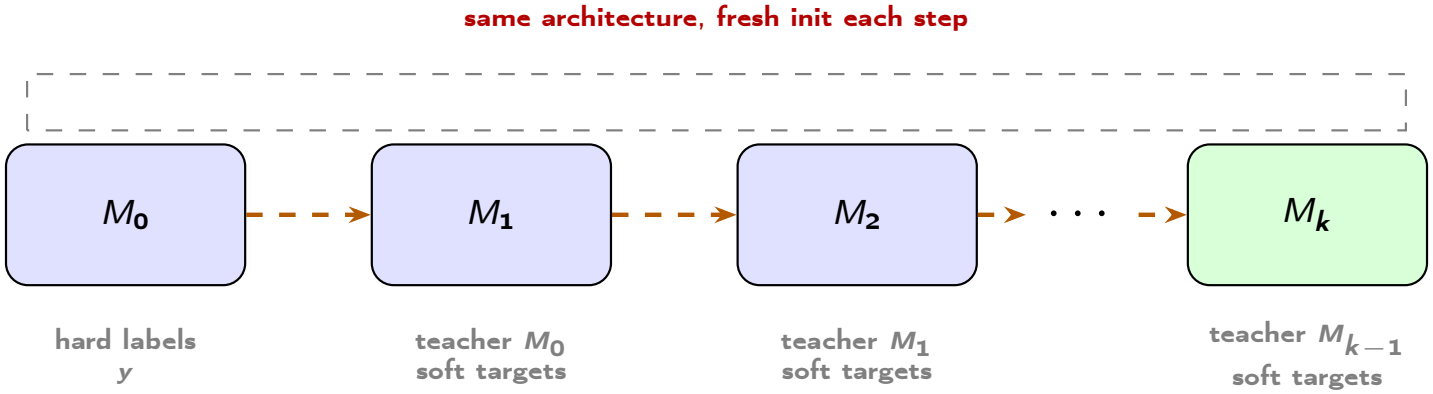
Be Your Own Teacher [L. Zhang et al., *"Be Your Own Teacher: Improve the Performance of*
Within a single CNN, deeper sections supervise shallower sections via auxiliary classifiers. No external teacher.

Online self-distillation: Use an EMA of the student's own weights as a moving teacher (cf. BYOL, mean teacher).

Why does it work? Several plausible explanations:

- Soft targets act as data-dependent label smoothing, reducing overconfidence
- Teacher logits encode similarity structure that hard labels destroy
- The teacher passes a model whose classes align with the student's inductive bias
- Gradient signal is denser and easier to optimize

Born-Again Networks



Empirical observation: M_k keeps improving over M_0 for several generations, then saturates. The trick is essentially free supervision from a model that has already digested the dataset.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation
- Motivation and Classical Distillation
- KL on Logits vs Cross-Entropy on Labels
- Self-Distillation
- On-Policy Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

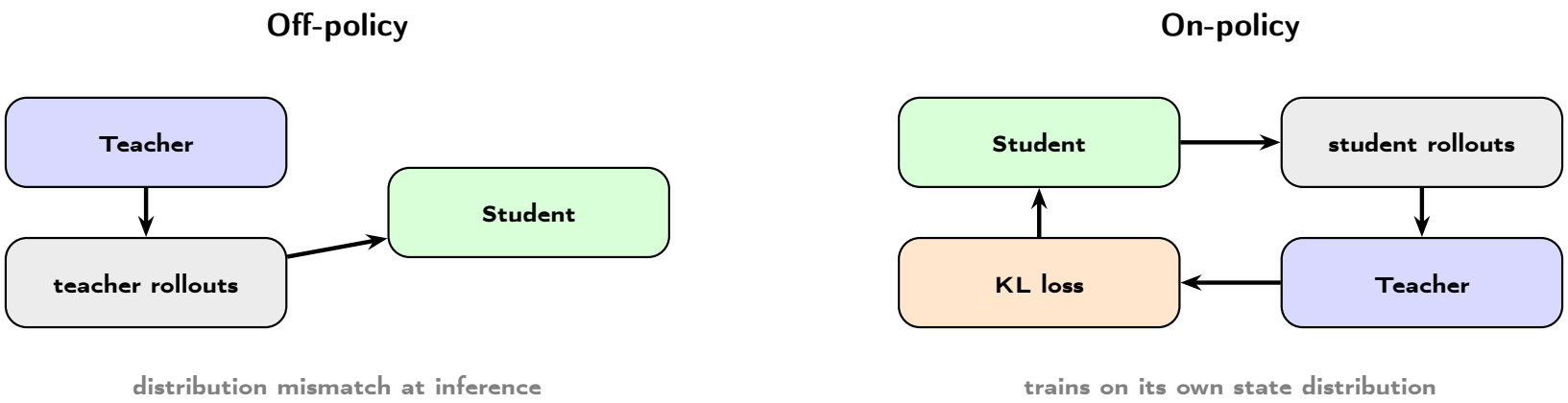
18 References

Off-Policy vs On-Policy Distillation

Two ways to source the inputs the student trains on:

Off-policy (classical): Use sequences sampled from the *teacher* (or a fixed dataset). The student is trained to imitate teacher tokens given teacher prefixes.

On-policy [Agarwal et al., *"On-Policy Distillation of Language Models: Learning from Self-Ger*
Sample sequences from the *student* itself, then ask the teacher to score them. The student trains on *its own* mistakes.



Exposure bias: Off-policy training never shows the student its own error states, so small mistakes at inference time compound. On-policy fixes this by definition.

Forward vs Reverse KL

Forward KL (used by SFT, off-policy KD):

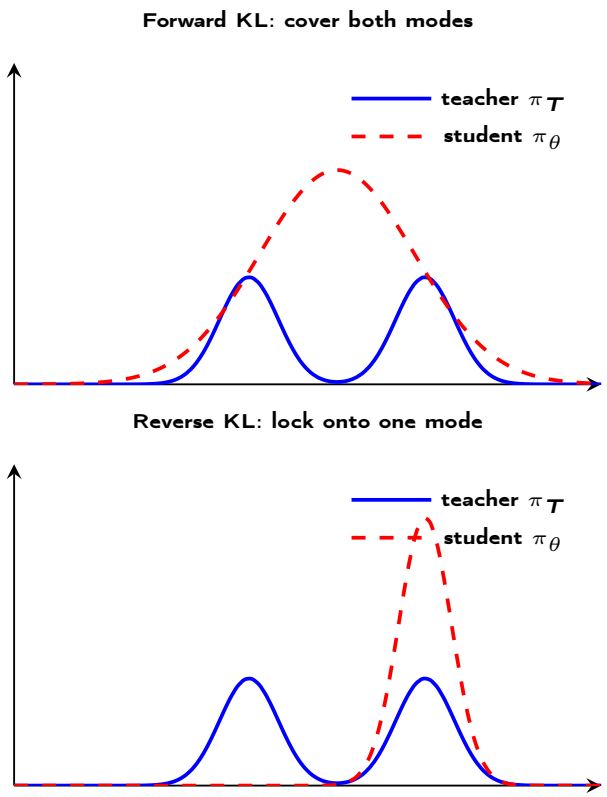
$$\mathrm{KL}(\pi_{\boldsymbol{T}} \,\|\, \pi_\theta) = \mathbb{E}_{x\sim\pi_{\boldsymbol{T}}}\left[\log \frac{\pi_{\boldsymbol{T}}(x)}{\pi_\theta(x)}\right]$$

- Samples drawn from *teacher*
- **Mode-covering**: penalizes any x where $\pi_{\boldsymbol{T}} > 0$ but $\pi_\theta \approx 0$
- Student spreads mass to cover all teacher modes, including bad ones
- Can be "hacked": π_θ assigns a tiny probability to garbage to keep KL low

Reverse KL (used by on-policy distillation):

$$\mathrm{KL}(\pi_\theta \,\|\, \pi_{\boldsymbol{T}}) = \mathbb{E}_{x\sim\pi_\theta}\left[\log \frac{\pi_\theta(x)}{\pi_{\boldsymbol{T}}(x)}\right]$$

- Samples drawn from *student* (on-policy!)
- **Mode-seeking**: forces student mass to land on teacher-approved regions
- Low reverse KL $\Rightarrow$ student behavior is provably teacher-like



On-Policy Distillation Algorithm

Algorithm 16: On-Policy Distillation [Agarwal et al., *"On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes"*, 2024; Goodfire et al., *On-Policy Distillation*, 2025]

Input: Student π_θ, frozen teacher $\pi_{\boldsymbol{T}}$, prompt set $\mathcal{P}$, learning rate α
for $k = 0$ **to** $K - 1$ **do**
  Sample prompts $x \sim \mathcal{P}$
  Student rollout: generate $y = (y_{\boldsymbol{1}}, \ldots, y_{\boldsymbol{N}}) \sim \pi_\theta(\cdot \mid x)$
  Teacher scoring: query $\log \pi_{\boldsymbol{T}}(y_{\boldsymbol{t}} \mid x, y_{<\boldsymbol{t}})$ for every token (single forward pass)
  Per-token reward: $r_{\boldsymbol{t}} = -\big(\log \pi_\theta(y_{\boldsymbol{t}} \mid \cdot) - \log \pi_{\boldsymbol{T}}(y_{\boldsymbol{t}} \mid \cdot)\big)$ `// negative reverse KL`
  Loss: $\mathcal{L}(\theta) = -\mathbb{E}\big[\sum_{\boldsymbol{t}} r_{\boldsymbol{t}}\big]$ (with importance sampling for off-policy steps)
  Update $\theta \leftarrow \theta + \alpha \nabla_\theta(-\mathcal{L})$

- Looks like RL with REINFORCE / GRPO, but the reward is *dense* and *differentiable through the teacher*.
- No reward model, no human labels, no verifier needed: the teacher is the reward.
- Same data can be reused for many epochs (low memorization risk because the student keeps generating new rollouts).

Why On-Policy Distillation Wins

Dense reward: Per-token reverse KL gives a learning signal on *every* token, not just the final answer. $\mathcal{O}(N)$ bits per episode vs RL's $\mathcal{O}(1)$.

On-policy: The student trains on the exact state distribution it will encounter at inference time, eliminating exposure bias and teacher/student mismatch.

Mode-seeking: Reverse KL is "unhackable". Low KL implies the student's actual rollouts are teacher-approved, not just that they have nonzero probability under the teacher.

Differentiable critic: The teacher acts as a frozen, perfect dense critic; no need to train a reward model.

Data efficiency: Multi-epoch reuse is safe because each epoch generates fresh student rollouts, unlike SFT which can memorize.

Slogan [Goodfire et al., *On-Policy Distillation*, 2025]: "On-policy training, off-policy reward signal".

Empirical Results: Math Reasoning

Distilling Qwen3-32B into Qwen3-8B on AIME'24 [Goodfire et al., *On-Policy Distillation*, 2025]:

Method	**AIME'24**	**GPU-hours**
SFT on 400k teacher traces	60.0%	(baseline)
RL from scratch (verifier)	67.6%	17,920
On-policy distillation	74.4%	1,800

- On-policy distillation *beats* RL while using $\sim$10$\times$ less compute.
- Including the SFT data amortization, end-to-end savings are 50–100$\times$ vs running RL alone.
- Reaches teacher performance in 7–10$\times$ fewer gradient steps than RL, thanks to dense per-token rewards.

Recovering Capabilities Lost to Mid-Training

Setting: Take a strong post-trained model (Qwen3-8B post-trained), then mid-train it on internal company documents to inject domain knowledge. Instruction-following collapses.

Stage	Knowledge	Instruction following
Post-trained baseline	$\sim 18\%$	$\sim 85\%$
+ mid-train on internal docs	43%	45%
+ on-policy distillation from baseline	41%	83%

- The teacher is the model's own pre-mid-training self.
- The student keeps the new domain knowledge and recovers almost all of the lost instruction-following ability.
- On-policy distillation acts as a targeted "unforgetting" mechanism: the LM itself is the reward model.

Summary

Distillation: Train a student to match a teacher's full distribution. Compresses models, transfers skills, and recovers capabilities.

Why logits not labels? Soft targets carry dark knowledge, give dense gradients, smooth labels in data-aware ways, and preserve calibration.

Self-distillation: Even when teacher = student architecture, soft-target supervision improves accuracy (born-again networks, be-your-own-teacher).

On-policy distillation: Sample from the student, score with a frozen teacher, minimize per-token reverse KL. Combines RL's on-policy state distribution with SFT's dense supervision.

In practice: Fewer GPU-hours than RL, no reward model, robust to multi-epoch reuse, and applicable to compression, reasoning transfer, and capability recovery.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Motivation: The Memory Wall

Recall: Training with Adam requires $\sim 16\Psi$ bytes of model state *plus* activations. Fine-tuning a 7B model naively needs $\sim$112 GB, more than any single GPU.

On modern GPUs, **memory is the bottleneck, not compute**:

- An H100 delivers 990 TFLOPS (BF16) but only has 80 GB HBM
- Compute is abundant; memory is the hard constraint
- It is therefore worthwhile to *trade extra compute for reduced memory*

Key question: How can we reduce memory usage?

Reduce activation memory Gradient checkpointing: discard intermediate activations during forward pass, recompute them during backward. Trades $\sim$33% extra compute for $O(\sqrt{L})$ instead of $O(L)$ activation memory.

Reduce parameter memory Parameter-Efficient Fine-Tuning (PEFT): freeze the pretrained weights, only train a small number of additional parameters ($\sim$0.1–1% of Ψ). Gradients and optimizer states are only needed for the small trainable set $\Rightarrow$ massive memory savings with minimal quality loss.

Gradient Checkpointing (Activation Recomputation)

Problem: Standard backpropagation stores all intermediate activations. Memory grows linearly with depth.

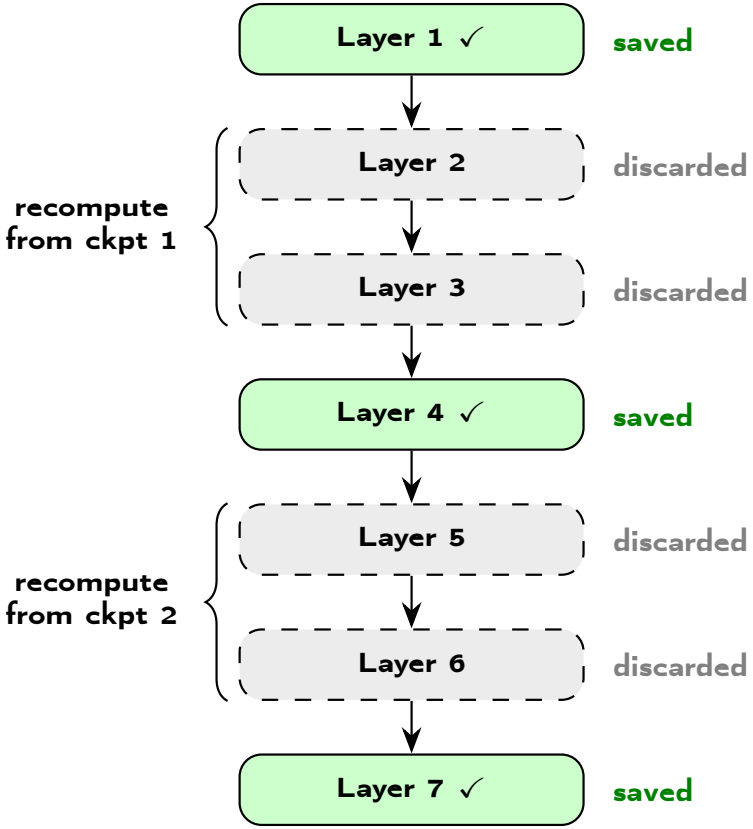
Idea [T. Chen et al., *"Training Deep Nets with Sublinear Memory Cost"*, 2016]:

- During forward pass, only save activations at selected **checkpoint** layers
- Discard all other intermediate activations
- During backward pass, *recompute* missing activations from the nearest checkpoint

Trade-off:

- Memory: $O(\sqrt{L})$ instead of $O(L)$ for L layers (with $\sqrt{L}$ evenly spaced checkpoints)
- Compute: ∼33% extra forward-pass compute

Selective recomputation [Korthikanti et al., *"Reducing Activation Recomputation in Large Transformer Models"*, 2023]: Only recompute attention (most memory-intensive). Saves ∼70% activation memory at only ∼2.7% compute overhead.



Demo: See `benchmark_checkpointing.py` for a script that measures memory and speed with/without checkpointing on GPT-2.

Parameter-Efficient Fine-Tuning (PEFT): Overview

Full fine-tuning updates *all* parameters. For a 7B model, this means storing 7B gradients + optimizer states.

PEFT idea: Freeze the pretrained weights, only train a small number of *additional* or *selected* parameters.

Method
Full fine-tuning
LoRA [Hu et al., *"LoRA: Low-Rank Adaptation of Large Language Models"*, 2022]
Adapters [Houlsby et al., *"Parameter-Efficient Transfer Learning for NLP"*, 2019]
Prefix Tuning [X. L. Li and Liang, *"Prefix-Tuning: Optimizing Continuous Prompts for Generation"*
Prompt Tuning [Lester et al., *"The Power of Scale for Parameter-Efficient Prompt Tuning"*, 2021]

All PEFT methods: gradients and optimizer states only for the small trainable set $\Rightarrow$ massive memory savings.

LoRA: Low-Rank Adaptation

Key insight [Hu et al., *"LoRA: Low-Rank Adaptation of Large Language Models"*, 2022]: Weight updates during fine-tuning have low intrinsic rank.

Instead of updating $W \in \mathbb{R}^{d \times k}$ directly, decompose the update:

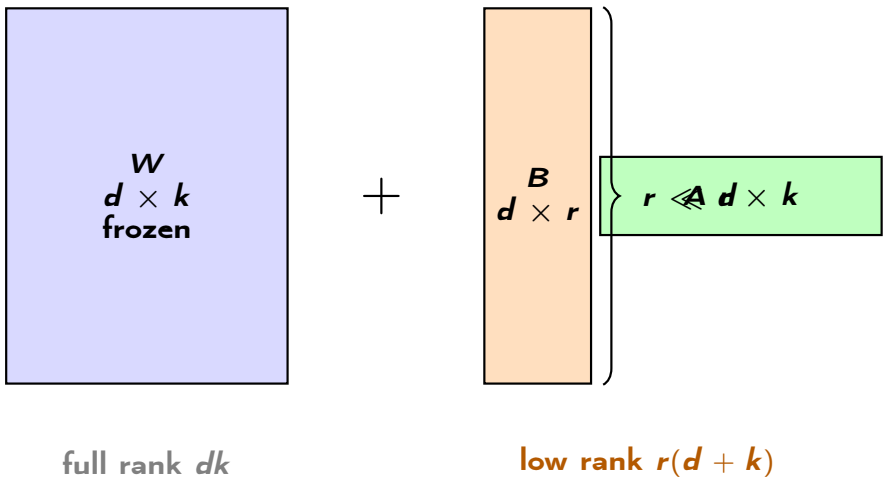
$$W' = W + \Delta W = W + BA$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$.

Training:

- Freeze W (no gradients, no optimizer state)
- Only train A and B ($r(d + k)$ params instead of dk)
- Initialize: $A \sim \mathcal{N}(0, \sigma^2)$, $B = 0$ (so $\Delta W = 0$ at start)
- Scale by $\frac{\alpha}{r}$ (hyperparameter α controls update magnitude)

Inference: Merge $W' = W + BA$: *no extra latency*.



Example: $d = k = 4096$, $r = 16$
Full: 16.8M params per matrix
LoRA: 131K params (∼0.8%)

LoRA: Practical Details

Which layers?

- Typically applied to attention projections (W_Q, W_K, W_V, W_O)
- Can also apply to MLP layers
- More layers with smaller r often beats fewer layers with larger r

Typical hyperparameters:

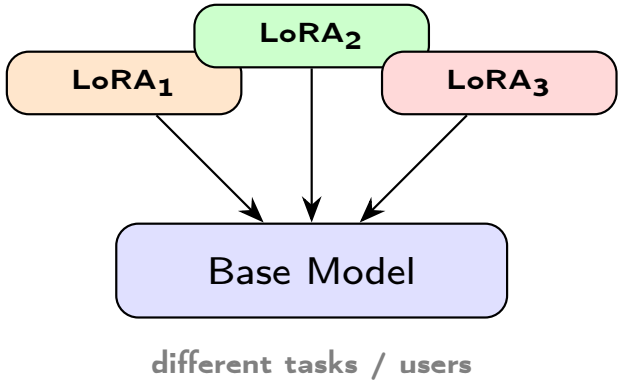
- Rank $r \in \{4, 8, 16, 32, 64\}$
- α often set to $2r$ or tuned
- Learning rate $\sim$3–10$\times$ higher than full fine-tuning

Memory savings:

- No gradient/optimizer for frozen weights
- 7B model: full FT needs $\sim$112 GB, LoRA $\sim$16–18 GB

Serving multiple LoRAs:

- One base model in memory
- Swap/add small LoRA adapters per task or user
- Enables multi-tenant serving



Adapters

Idea [Houlsby et al., *"Parameter-Efficient Transfer Learning for NLP"*, 2019]: Insert small trainable **bottleneck modules** between existing layers.

Architecture:

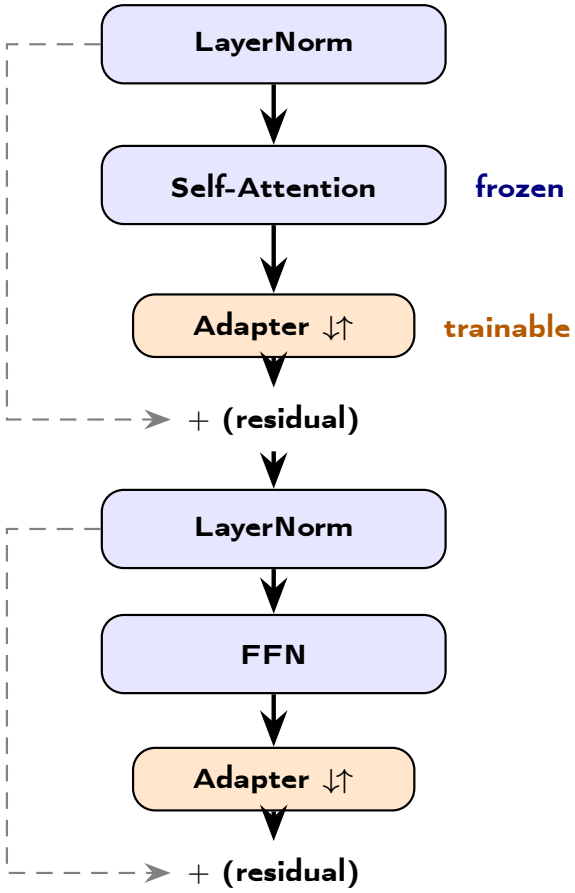
- Project down: $h \mapsto \mathbb{R}^m$ where $m \ll h$
- Nonlinearity (e.g. ReLU/GELU)
- Project up: $\mathbb{R}^m \mapsto \mathbb{R}^h$
- Residual connection around the adapter

$$\text{Adapter}(x) = x + f(xW_{\text{down}})W_{\text{up}}$$

Placement: After attention and/or after FFN sublayer, before the residual add.

Comparison to LoRA:

- Adapters add *sequential* computation $\Rightarrow$ small inference overhead
- LoRA can be merged into weights $\Rightarrow$ zero overhead
- Both achieve similar quality at $\sim$1% params



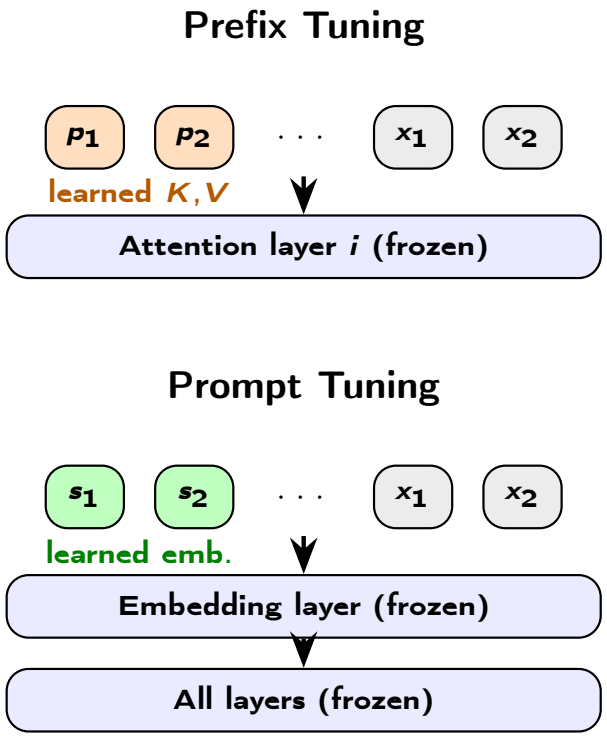
Prefix Tuning and Prompt Tuning

Prefix Tuning [X. L. Li and Liang, *"Prefix-Tuning: Optimizing Continuous Prompts for Generation"*, 2021]:

- Prepend learnable **prefix vectors** to keys and values in every attention layer
- Prefix length l (e.g. 10–200 tokens)
- Parameters: $l \times L \times 2 \times d$ (prefix per layer, for K and V)
- Typically <0.1% of model parameters

Prompt Tuning [Lester et al., *"The Power of Scale for Parameter-Efficient Prompt Tuning"*, 2021]:

- Even simpler: learnable **soft tokens** prepended at the *input embedding* level only
- No changes to internal layers
- Parameters: $l \times d$ (one embedding per soft token)
- Scales well: approaches full FT quality for large models (>10B)



Summary and Comparison

Method	Memory saving	Trainable params	Inference overhead	Quality
Grad. checkpointing	Activations	100%	None	Same
LoRA	Large	$\sim$0.5%	None (merged)	Near full FT
Adapters	Large	$\sim$2%	Small	Near full FT
Prefix Tuning	Very large	$<$0.1%	Prefix KV	Good ($\geq$10B)
Prompt Tuning	Very large	$<$0.1%	Extra tokens	Good ($\geq$10B)

In practice:

- **Pretraining:** Gradient checkpointing + mixed precision + parallelism (always)
- **Fine-tuning on limited hardware:** LoRA (most popular PEFT method)
- **Multi-task serving:** LoRA adapters swapped per task on shared base model

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
    - Data Parallelism
    - ZeRO Optimization
    - Pipeline Parallelism
    - Tensor Parallelism
    - Context Parallelism
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Why Parallelism?

Modern LLMs far exceed the memory of a single GPU.

The scaling reality

- GPT-3: 175B parameters
- Llama-3 405B: 405B parameters
- DeepSeek-V3: 671B parameters (MoE)
- Training requires storing weights, gradients, optimizer states, and activations

Single GPU is not enough

- H100 has 80 GB HBM
- A 7B model in BF16 training already needs ∼112 GB
- A 70B model needs ∼1120 GB

Three fundamental challenges [Tazi et al., *The Ultra-Scale Model Training Playbook*, 2025]

1. **Memory**: Hard limit: if model state doesn't fit, training cannot begin
2. **Compute efficiency**: Minimize GPU idle time, maximize FLOP utilization (MFU)
3. **Communication overhead**: Minimize data transfer between GPUs, overlap with computation

Goal

Distribute work across many GPUs such that training is both *feasible* and *efficient*.

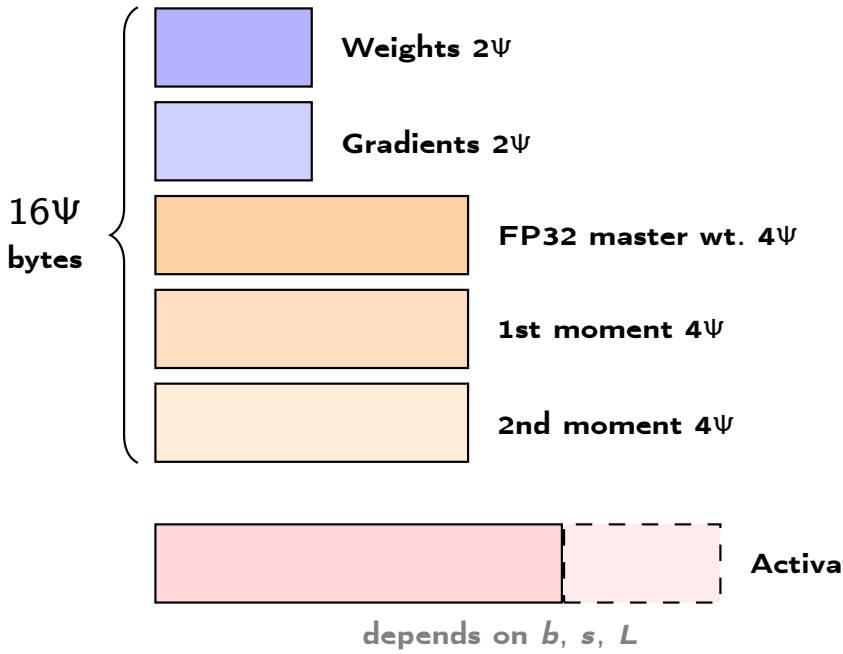
References

- Material based on the Ultra-Scale Playbook [Tazi et al., *The Ultra-Scale Model Training Playbook*, 2025]
- Tensor & pipeline parallelism at scale (Megatron-LM) [Shoeybi et al., *"Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism"*, 2020]
- Memory-efficient data parallelism via sharding (ZeRO) [Rajbhandari et al., *"ZeRO: Memory Optimizations Toward Training Trillion Parameter Models"*, 2020]
- Pipeline parallelism with micro-batching (GPipe) [Y. Huang et al., *"GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism"*, 2019]
- Interleaved pipeline schedules, 3D parallelism [Narayanan, Shoeybi, et al., *"Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM"*, 2021]
- Selective activation recomputation [Korthikanti et al., *"Reducing Activation Recomputation in Large Transformer Models"*, 2023]
- Mixed-precision training [Micikevicius et al., *"Mixed Precision Training"*, 2018]
- FlashAttention [Dao, Fu, et al., *"FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness"*, 2022]
- 4D parallelism at scale (Llama 3) [Grattafiori et al., *"The Llama 3 Herd of Models"*, 2024]
- FP8, DualPipe, expert parallelism (DeepSeek-V3) [DeepSeek-AI, *"DeepSeek-V3 Technical Report"*, 2024]
- Pathways system (PaLM) [Chowdhery et al., *"PaLM: Scaling Language Modeling with Pathways"*, 2023]
- 10k+ GPU training (MegaScale) [Z. Jiang et al., *"MegaScale: Scaling Model Training to More Than 10,000 GPUs"*, 2024]
- PyTorch Distributed (FSDP) [S. Li et al., *"PyTorch Distributed: Experiences on Accelerating Data Parallel Training"*, 2020]

Memory Breakdown: What Needs to Fit on a GPU?

Training a Transformer with Ψ parameters in **mixed precision** (BF16 compute + FP32 Adam):

- **Weights** (BF16, 2Ψ bytes): Parameters used in forward/backward pass
- **Gradients** (BF16, 2Ψ bytes): One gradient per parameter, computed in backward pass
- **Master weights** (FP32, 4Ψ bytes): Full-precision copy of weights. Needed because weight updates $\theta \leftarrow \theta - \eta \cdot \Delta$ can be too small for BF16's limited mantissa (8-bit exponent, 7-bit mantissa)
- **1st moment** (FP32, 4Ψ bytes): Adam's running mean of gradients $m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$
- **2nd moment** (FP32, 4Ψ bytes): Adam's running mean of squared gradients $v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$
- **Activations**: Intermediate values saved for backward pass; depends on batch size, sequence length, number of layers



Total (excl. activations): 16^{-} bytes per parameter

Concrete Memory Requirements

	1B	7B	70B	405B
Weights (BF16)	2 GB	14 GB	140 GB	810 GB
+ Gradients (BF16)	4 GB	28 GB	280 GB	1620 GB
+ Optimizer (FP32)	16 GB	112 GB	1120 GB	6480 GB
+ Activations*	∼2 GB	∼8 GB	∼50 GB	∼200 GB
H100 GPUs needed	1	2	15	84

Assuming 80 GB per H100, mixed-precision training with Adam. *Activations for mbs $= 1$, $s = 4096$ with gradient checkpointing (see Section 15); scales with batch size and sequence length.

Key takeaway:

- Even a 7B model doesn't fit on a single GPU for training
- We need strategies to *distribute* model state across GPUs
- This is where the *five dimensions* of parallelism come in

Overview of Parallelism Strategies

Five complementary dimensions, each splitting along a different axis:

Batch dim	**Data Parallelism**	Split *batch* across GPUs
Hidden dim	**Tensor Parallelism**	Split *weight matrices* (hidden dim) within a layer
Layer dim	**Pipeline Parallelism**	Split *layers* across GPUs
Seq. dim	**Context Parallelism**	Split *sequence* across GPUs (long contexts)
Expert dim	**Expert Parallelism**	Split *experts* across GPUs (MoE models)

5D Parallelism

State-of-the-art training (Llama-3, DeepSeek-V3) combines *all five* dimensions simultaneously. The global batch size becomes: $\text{GBS} = \text{mbs} \times \text{grad_acc} \times \text{DP}$.

GPU Interconnects and Topology

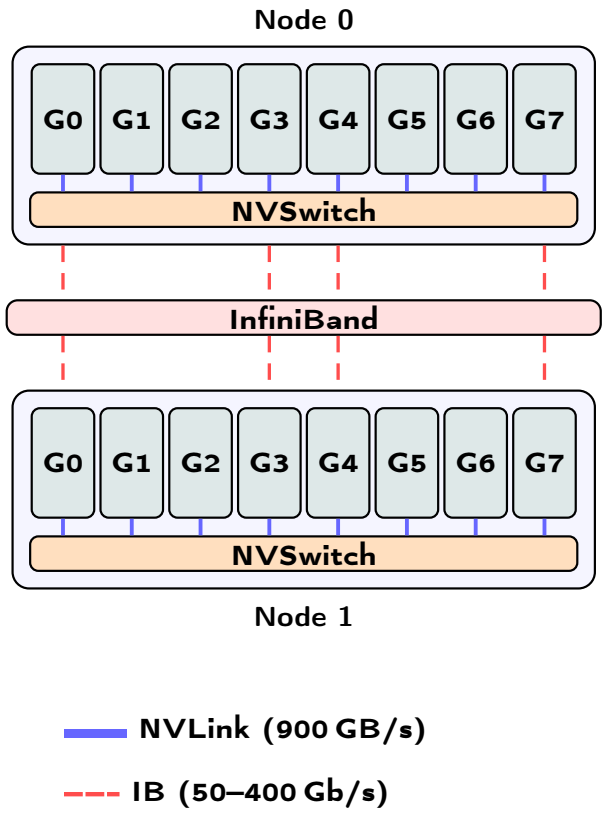
Communication speed between GPUs critically determines which parallelism strategies are practical.

Intra-node (within a server):

- **NVLink / NVSwitch**: Up to 900 GB/s (H100)
- 8 GPUs fully connected within one node
- Used for *Tensor Parallelism* (frequent, latency-sensitive communication)

Inter-node (across servers):

- **InfiniBand / RoCE**: ∼50–400 Gb/s per link
- Much slower than NVLink (∼5–20×)
- Used for *Data & Pipeline Parallelism* (less frequent, larger messages)



Rule of thumb: Use TP within a node (degree ≤ 8), DP and PP across nodes.

Collective Communication Operations

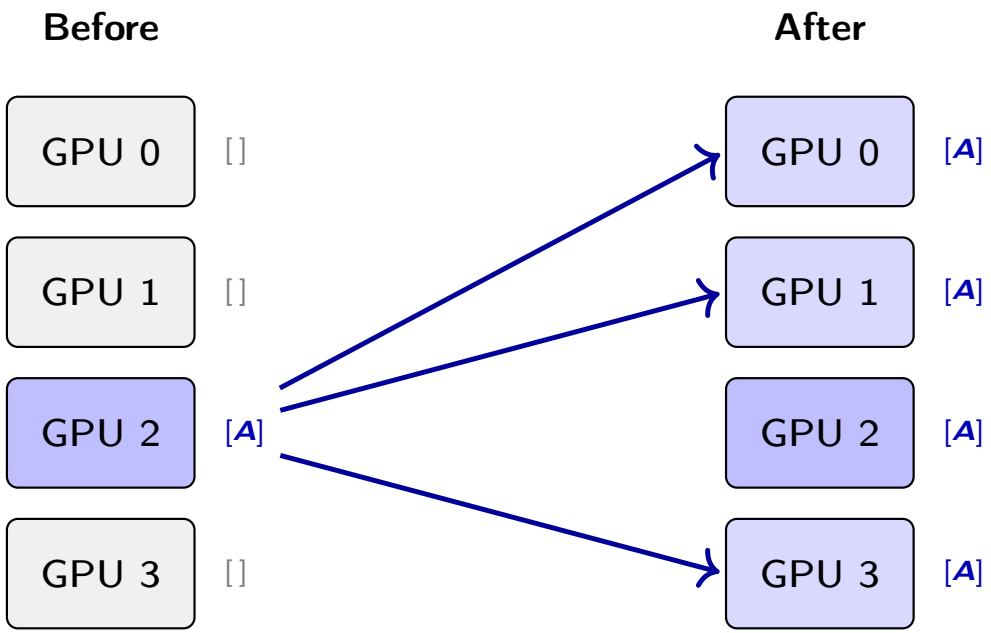
Parallelism strategies rely on a small set of **collective operations**:

Operation	Description
Broadcast	One GPU sends data to all others
Reduce	Combine (e.g. sum) data from all GPUs to one
AllReduce	Reduce + broadcast result to all GPUs
Scatter	Split data from one GPU, send chunks to each
Gather	Collect chunks from all GPUs to one
AllGather	Gather + broadcast: every GPU gets the full result
ReduceScatter	Reduce + scatter: each GPU gets a reduced chunk
AllToAll	Each GPU sends a different chunk to each other GPU

- Implemented in **NCCL** (NVIDIA Collective Communications Library)
- **AllReduce** is the workhorse of data parallelism (gradient sync)
- **AllGather** + **ReduceScatter** are key for ZeRO / FSDP

Broadcast

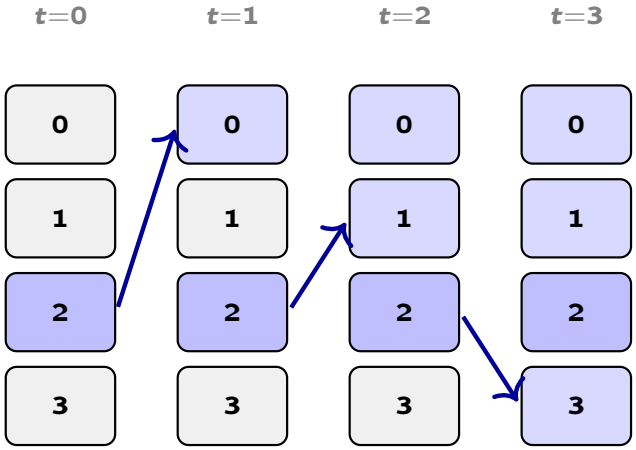
One GPU sends the same data to all other GPUs.



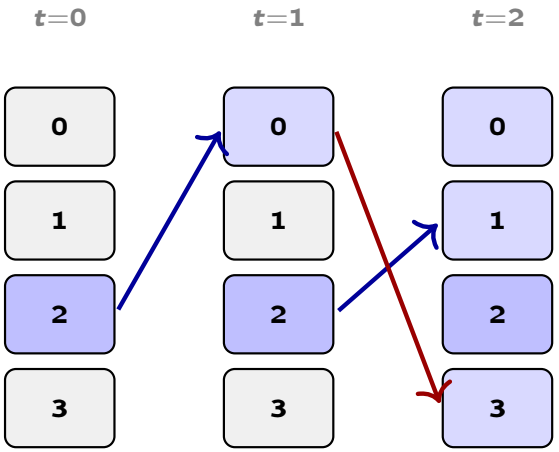
- **Root** GPU holds the data, all others receive a copy
- Data volume transferred: $(N-1) \cdot |A|$ in the naive case; tree-based broadcast reduces latency to $O(\log N)$ steps
- **Use cases:**
  - Distributing initial model weights to all GPUs at startup
  - Sharing hyperparameters or random seeds

Naive vs. Tree-Based Broadcast

Naive: $N-1$ sequential steps



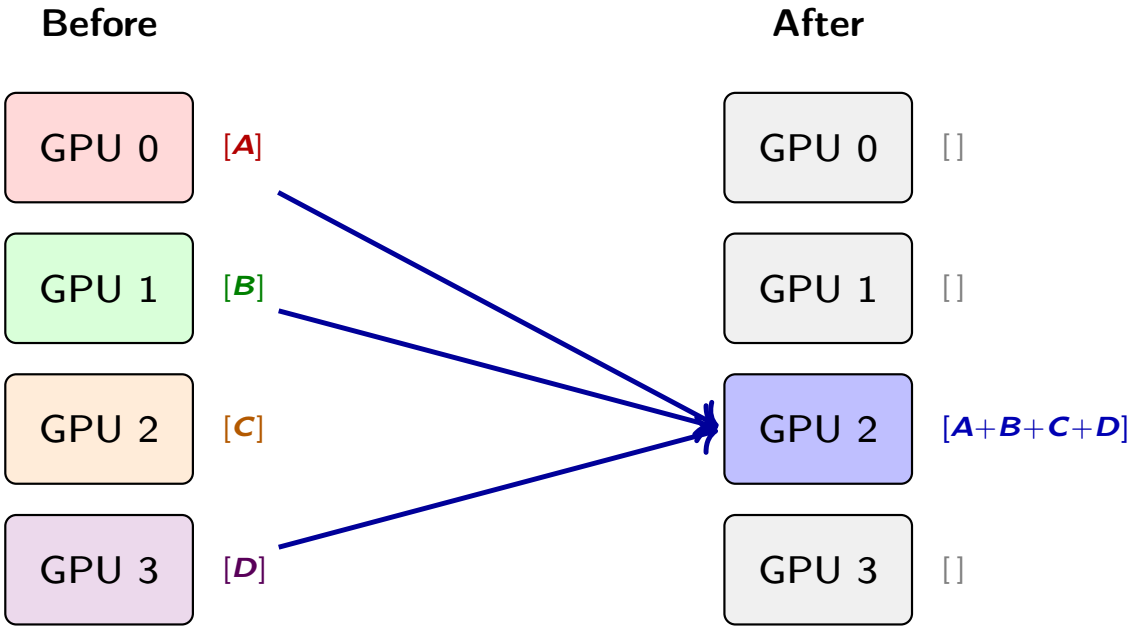
Tree-based: $\lceil \log_2 N \rceil$ steps



- Tree-based: GPUs that already have the data **help forward** it
- At step t=2, GPU 2 and GPU 0 send *in parallel* (red arrow = forwarded by receiver)
- With N=1024 GPUs: 10 steps instead of 1023
- NCCL uses ring and tree topologies under the hood

Reduce

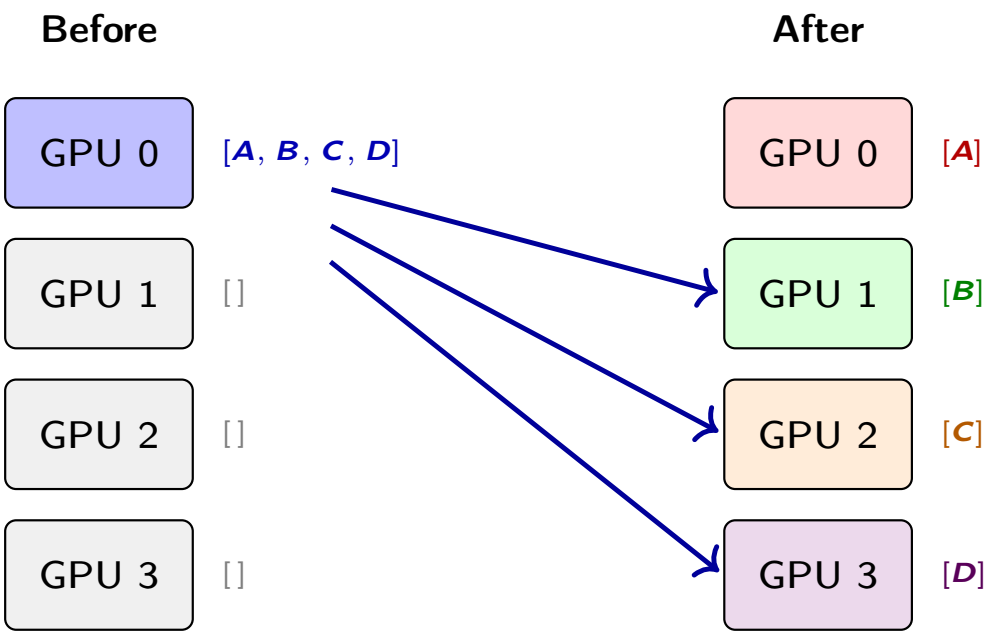
Combine data from all GPUs onto one root GPU (e.g. sum, max).



- Each GPU holds local data; the **root** receives the combined result
- Reduction operator is typically *sum*, but can be max, min, product, etc.
- Inverse of broadcast: many $\rightarrow$ one
- **Implementation:** Tree-based reduce (mirror of tree broadcast): partial sums flow up a binary tree in $O(\log N)$ steps
- **Use cases:**
  - Summing partial loss values across GPUs
  - Aggregating metrics (accuracy, counts)

Scatter

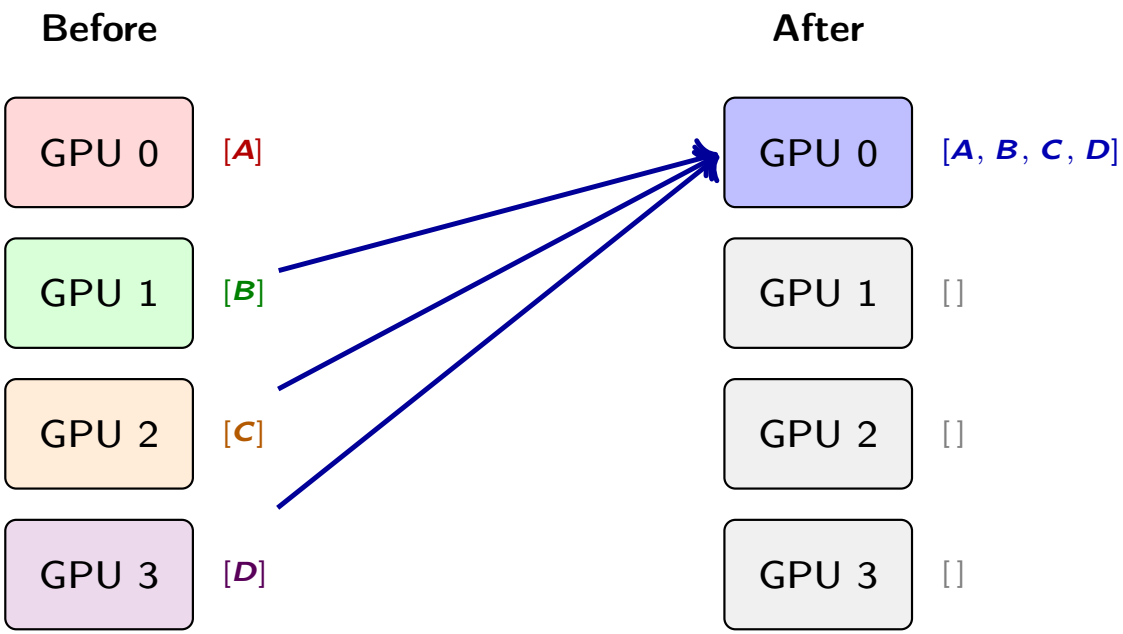
Split data from one GPU and distribute chunks to each GPU.



- One GPU holds the *full* data, which is split into N equal chunks
- Each GPU receives exactly one chunk
- Inverse of gather: one $\rightarrow$ many (partitioned)
- Unlike broadcast, each GPU gets a *different* piece
- **Implementation:** Direct point-to-point sends from root; can also use tree-based scatter in $O(\log N)$ steps
- **Use cases:**
  - Distributing sharded optimizer states (ZeRO)
  - Partitioning input batches

Gather

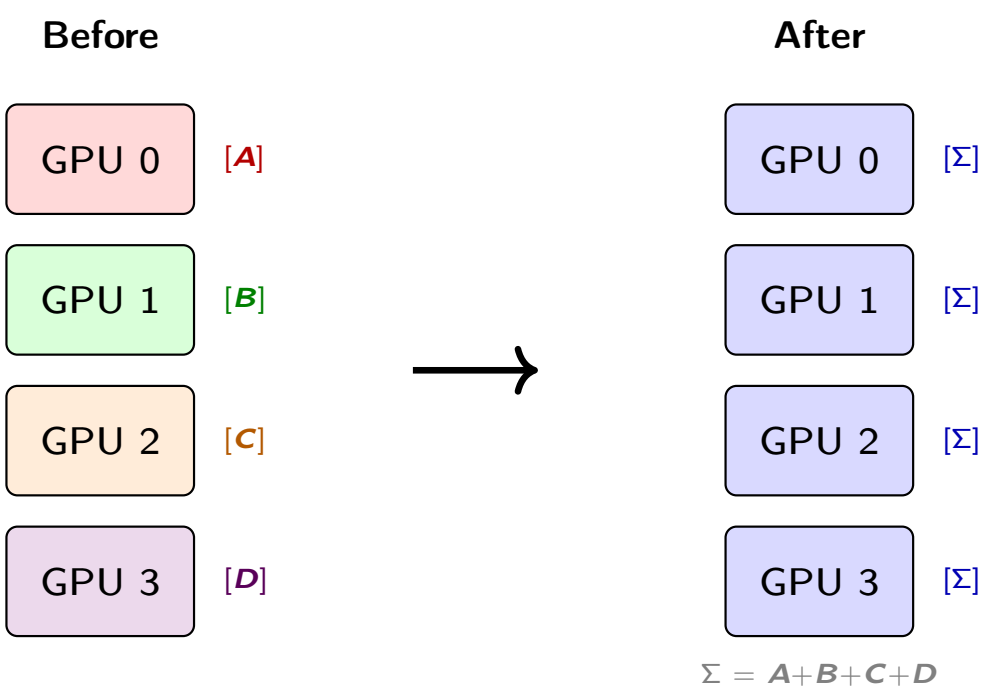
Collect chunks from all GPUs onto one root GPU.



- Each GPU holds one chunk; the **root** collects and concatenates all chunks
- Inverse of scatter: many $\rightarrow$ one (concatenated)
- Unlike reduce, chunks are *concatenated*, not combined with an operator
- **Implementation:** Direct point-to-point sends to root; mirror of scatter
- **Use cases:**
  - Collecting partial results (e.g. activations from pipeline stages)
  - Reassembling sharded tensors on one GPU for checkpointing

AllReduce

Reduce data across all GPUs, then distribute the result to every GPU. Equivalent to Reduce + Broadcast, but implemented more efficiently.

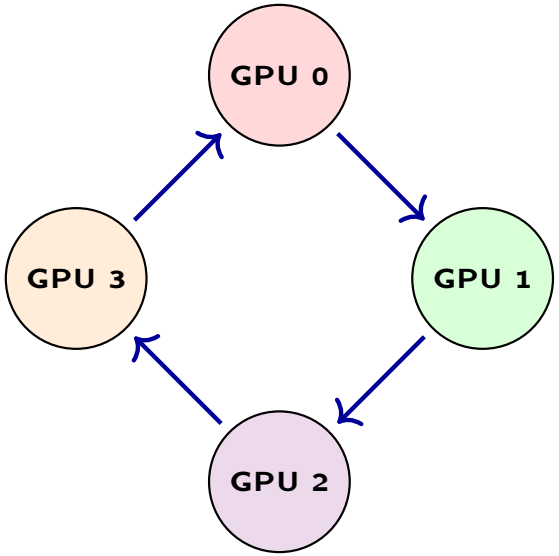


- Every GPU starts with local data and ends with the *same* reduced result
- **The** workhorse of data parallelism: synchronize gradients across all replicas
- **Implementation:** Ring AllReduce (see next slide), bandwidth-optimal
- **Use cases:**
  - Gradient synchronization in data parallelism
  - Averaging batch statistics (e.g. BatchNorm)

Ring AllReduce

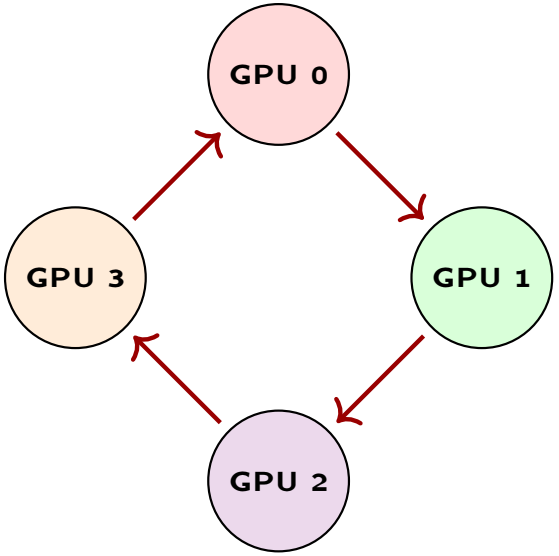
Bandwidth-optimal AllReduce: GPUs arranged in a logical ring, two phases.

Phase 1: ReduceScatter



Each GPU sends a chunk to its neighbor, which adds it to its own. After $N-1$ steps, each GPU holds one fully reduced chunk.

Phase 2: AllGather



Reduced chunks circulate around the ring. After $N-1$ steps, every GPU has the complete result.

- Each GPU sends and receives exactly $\frac{2(N-1)}{N} \cdot |D|$ data: **bandwidth-optimal**
- All links are utilized simultaneously. Scales well to large N
- **Tree vs. ring:** Tree minimizes *latency* ($O(\log N)$ steps); ring minimizes *bandwidth* usage. AllReduce operates on large gradient tensors, so bandwidth dominates, so ring wins

Roadmap

In the following slides we will cover each parallelism strategy in depth:

1. **Data Parallelism (DP):** replicate model, split data
2. **ZeRO / FSDP:** memory-efficient data parallelism
3. **Tensor Parallelism (TP):** shard individual layers
4. **Sequence / Context Parallelism (SP/CP):** split along sequence dimension
5. **Pipeline Parallelism (PP):** split model into stages
6. **Expert Parallelism (EP):** distribute MoE experts
7. **Putting it all together:** 5D parallelism and choosing configurations

For each strategy we will cover:

- Core idea and what dimension is split
- Memory savings and communication costs
- Practical considerations and limitations

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
    - Data Parallelism
    - ZeRO Optimization
    - Pipeline Parallelism
    - Tensor Parallelism
    - Context Parallelism
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Data Parallelism (DP): Recap

We covered DDP in the GPT-2 chapter. Brief recap and new details:

Core idea

- Each GPU holds a *full model replica*
- Each GPU processes a different micro-batch
- Gradients synchronized via **AllReduce** after backward pass
- Every GPU performs the same optimizer step $\Rightarrow$ models stay in sync

Batch size with DP:

$$\text{GBS} = \text{mbs} \times \text{grad_acc} \times N_{\text{DP}}$$

Communication cost

- AllReduce transfers 2Ψ bytes (ring AllReduce)
- Independent of number of GPUs N_{DP} !
- But: latency grows with more GPUs

Memory per GPU

- No memory savings: each GPU stores full weights, gradients, optimizer states (16Ψ)
- Only activation memory is reduced (smaller micro-batch per GPU)

DDP Training Algorithm

Algorithm 17: Distributed Data Parallel (DDP) Training

Input: Model θ (replicated on each of N GPUs), learning rate η, gradient accumulation steps G, micro-batch size b

for *each training step* **do**
  $\nabla\theta \leftarrow 0$ `// zero gradients`
  for $g = 1, \ldots, G$ **do**
    Sample micro-batch $\mathcal{B}_g$ (different on each GPU)
    $\mathcal{L}_g \leftarrow \frac{1}{b}\texttt{forward}(\theta, \mathcal{B}_g)$
    if $g = G$ **then** `// last micro-step: sync gradients`
      $\nabla\theta \leftarrow \nabla\theta + \frac{1}{G}\texttt{backward}(\mathcal{L}_g)$
      $\nabla\theta \leftarrow \frac{1}{N}\textsc{AllReduce}(\nabla\theta)$ `// average across GPUs`
    else `// accumulate locally, no communication`
      $\nabla\theta \leftarrow \nabla\theta + \frac{1}{G}\texttt{backward}(\mathcal{L}_g)$
  $\theta \leftarrow \textsc{OptimizerStep}(\theta, \nabla\theta, \eta)$

- Effective batch size per step: $\text{GBS} = b \times G \times N$
- Communication happens only once per training step (line 7)

DP: Overlapping Communication with Compute

Key optimizations that make DP scale well in practice:

1. **Overlap AllReduce with backward pass:**
   - Gradients for early layers are ready while later layers are still computing
   - Start AllReduce for each layer's gradients *as soon as they are computed*
   - In PyTorch DDP: automatic via gradient hooks
2. **Gradient bucketing:**
   - Launching one AllReduce per parameter is inefficient (high latency overhead)
   - Group parameters into *buckets* ($\sim$25 MB each in PyTorch)
   - One AllReduce per bucket amortizes latency
3. **Skip sync during gradient accumulation:**
   - Only AllReduce on the *last* micro-step: `model.no_sync()` context manager
   - Reduces communication by factor of `grad_acc`

Limitation

DP alone cannot train models that don't fit on a single GPU ($16\Psi >$ GPU memory). This motivates ZeRO and model parallelism.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference
- Data Parallelism
- ZeRO Optimization
- Pipeline Parallelism
- Tensor Parallelism
- Context Parallelism

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

ZeRO: Motivation

ZeRO (Zero Redundancy Optimizer) eliminates memory redundancy in data-parallel training [Rajbhandari et al., *"ZeRO: Memory Optimizations Toward Training Trillion Parameter Models"*, 2020].

Problem with vanilla DP:

- Every GPU stores a *full copy* of weights, gradients, and optimizer states
- All GPUs compute identical optimizer steps $\Rightarrow$ redundant work and memory
- Memory per GPU is 16Ψ (mixed-precision Adam) regardless of N_{DP}

Key insight:

- Not every GPU *needs* every parameter/gradient/optimizer state at all times
- *Partition* these across the DP dimension
- Gather on demand via collective communication
- Same mathematical result as vanilla DP

PyTorch implementation:
ZeRO-3 = **FSDP** (Fully Sharded Data Parallelism)

Memory Baseline: Mixed-Precision Training with Adam

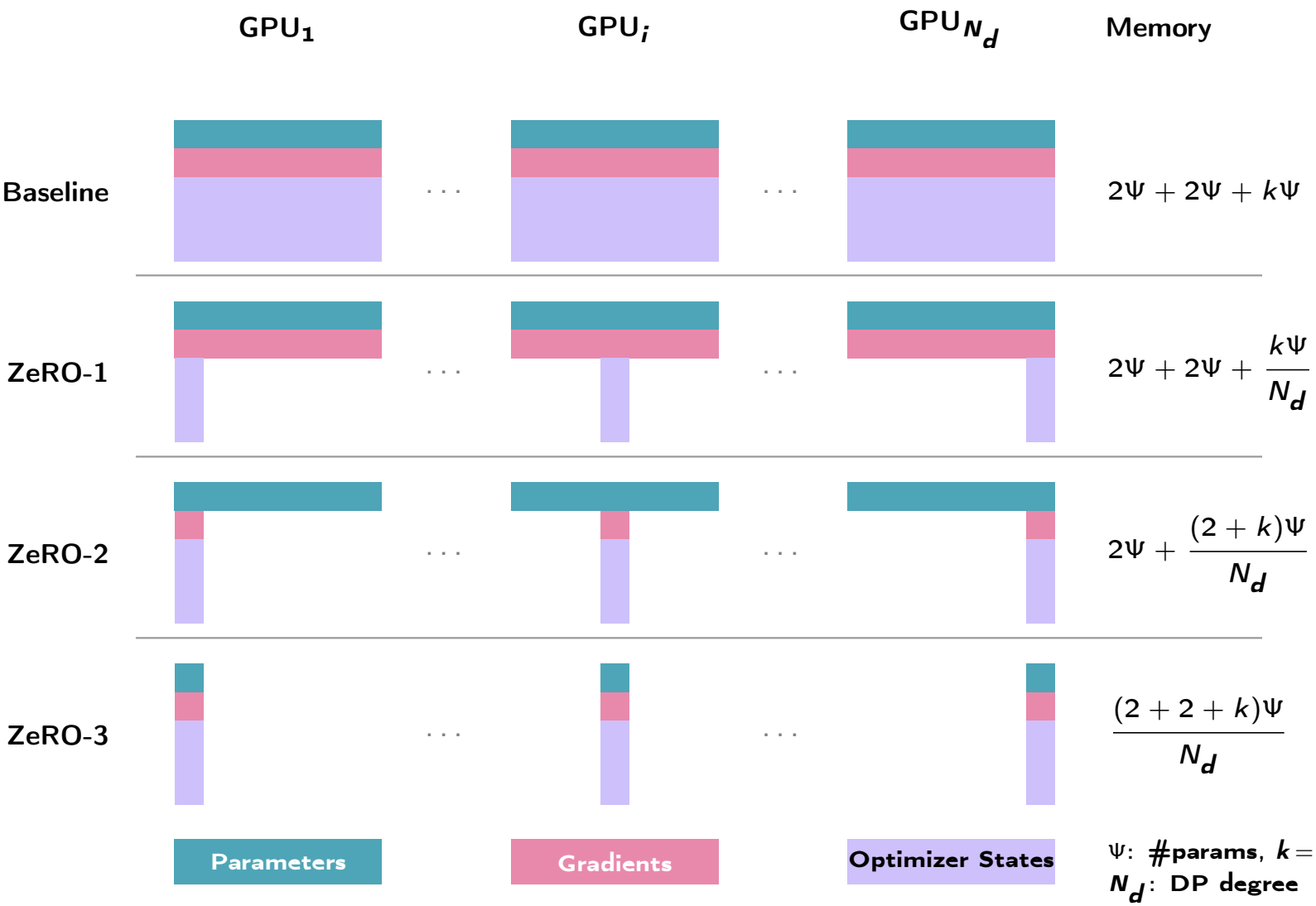
For a model with Ψ parameters using BF16 mixed-precision training:

Component	Precision	Memory
Model parameters	BF16	2Ψ
Gradients	BF16	2Ψ
Master weights (FP32 copy)	FP32	4Ψ
Adam momentum	FP32	4Ψ
Adam variance	FP32	4Ψ
Total		16Ψ

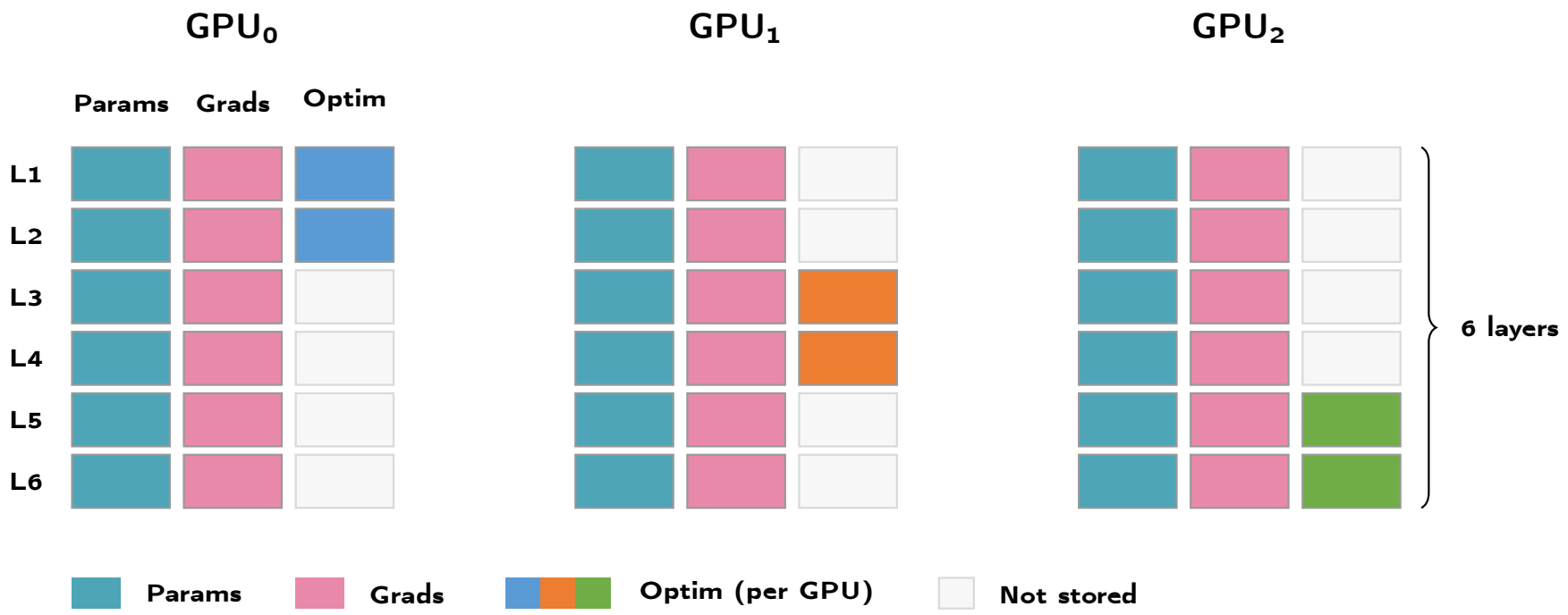
- **Optimizer states:** $k\Psi = 12\Psi$ bytes (master weights + momentum + variance)
- **Activations:** not included, depend on batch/seq. length, *different* per DP replica

Model	1B	7B	70B	405B
Memory	16 GB	112 GB	1120 GB	6480 GB

ZeRO Stages: Memory per GPU



ZeRO-1: Illustrated ($N_d = 3$)

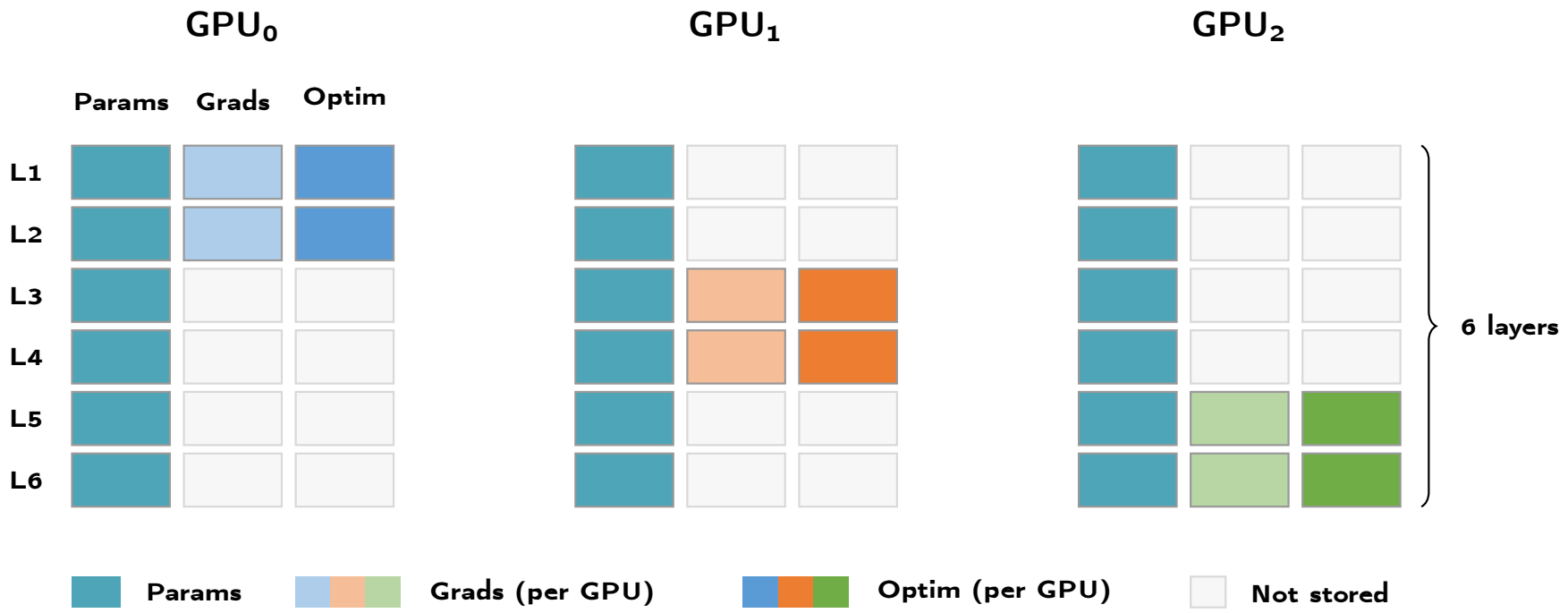


Algorithm:

1. Fwd+Bwd as in vanilla DP $\rightarrow$ full grads on each GPU
2. ReduceScatter grads: each GPU receives its $\frac{1}{N_d}$ slice (summed)
3. Each GPU runs optimizer on its slice only
4. AllGather updated params to reconstruct full model

Memory: $4\Psi + \frac{12\Psi}{N_d}$ per GPU. **Comm:** same as DP AllReduce (2Ψ).

ZeRO-2: Illustrated ($N_d = 3$)

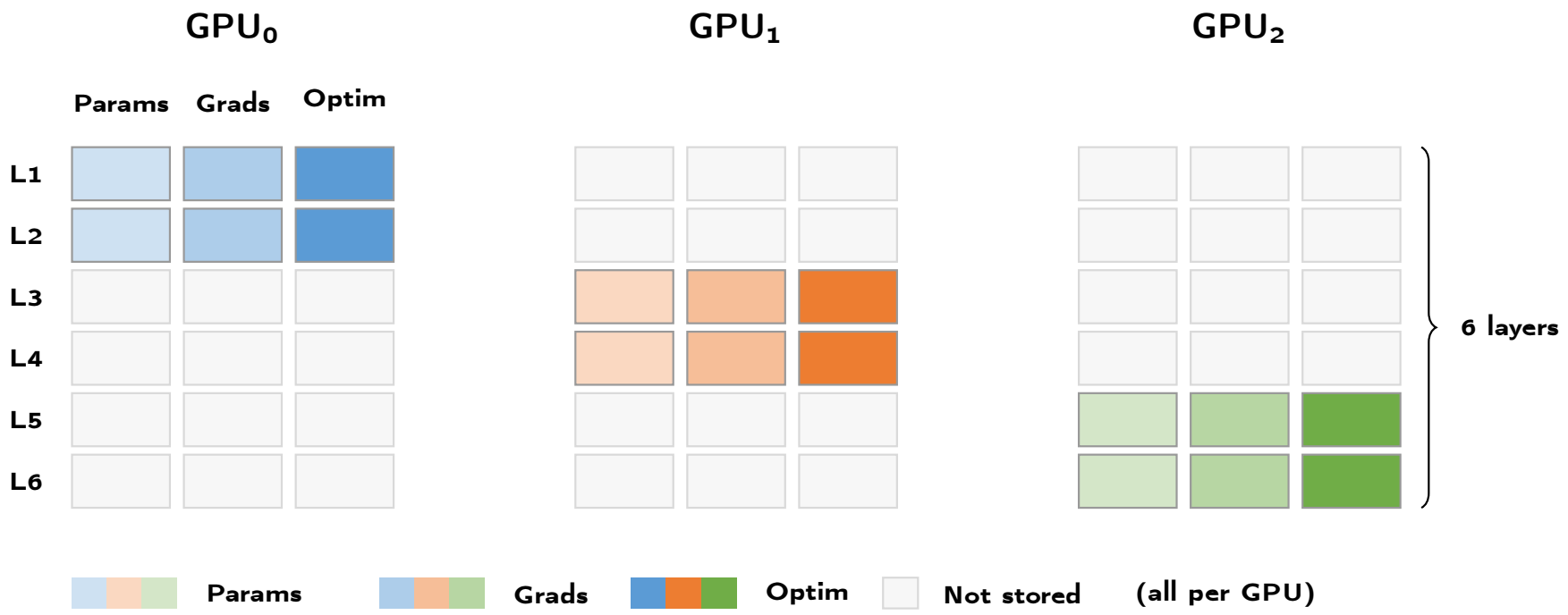


Algorithm:

1. Fwd+Bwd: ReduceScatter grads *during* backward → each GPU keeps only its $\frac{1}{N_d}$ slice
2. Each GPU runs optimizer on its slice only
3. AllGather updated params to reconstruct full model

Memory: $2\Psi + \frac{(2+12)\Psi}{N_d}$ per GPU. **Comm:** same as ZeRO-1 (2Ψ), strictly better, no overhead.

ZeRO-3 / FSDP: Illustrated ($N_d = 3$)



Key insight: each layer's params are only needed *during* its fwd/bwd computation. So params can be partitioned and gathered *on demand* per layer, then discarded.

Algorithm:

1. **Fwd:** for each layer ℓ: AllGather full params from all GPUs → compute → *discard* non-local params
2. **Bwd:** for each layer ℓ (reverse): AllGather params again → compute grads → ReduceScatter grads → discard
3. Each GPU runs optimizer on its $\frac{1}{N_d}$ slice only

Memory: $\frac{16\Psi}{N_d}$ per GPU. **Comm:** 3Ψ (extra AllGather in fwd). PyTorch: **FSDP**.

Communication Volume Comparison

Method	Comm. Volume	Operations	Memory
Vanilla DP	2Ψ	AllReduce	16Ψ
ZeRO-1	2Ψ	RS + AG	$4\Psi + \frac{12\Psi}{N_d}$
ZeRO-2	2Ψ	RS + AG	$2\Psi + \frac{14\Psi}{N_d}$
ZeRO-3	3Ψ	$2\times$AG + RS	$\frac{16\Psi}{N_d}$

RS = ReduceScatter, AG = AllGather. AllReduce $\equiv$ ReduceScatter + AllGather, so ZeRO-1/2 have the *same* communication volume as vanilla DP.

When to use which:

- **ZeRO-1/2:** Model fits on one GPU but optimizer states don't. Combines easily with PP and TP.
- **ZeRO-3:** Model doesn't fit on one GPU. Higher communication cost ($+\Psi$).

Practical guidelines:

- ZeRO-2 $\geq$ ZeRO-1 (no downside)
- ZeRO-3 prefetching effective up to $N_d \approx 512$
- ZeRO-3 + PP is uncommon (both transfer data between GPUs); prefer one or the other
- DeepSeek-V3: PP + ZeRO-1

ZeRO-3 vs. Other Model-Sharding Strategies

	ZeRO-3 / FSDP	**Tensor Parallelism**
Each GPU stores	fraction of each layer	fraction of each layer
Compute with	*gathered* full params	local shard only
Comm. pattern	AG before compute	AllReduce after compute
Shards activations?	No	Yes
Interconnect need	Moderate	High (NVLink)

	ZeRO-3 / FSDP	**Pipeline Parallelism**
Each GPU stores	fraction of each layer	full layers (stage)
Transfers	weights	activations
Prefers large ...	mbs, seq_len	grad_acc
Idle time	No bubble	Bubble overhead

ZeRO does **not** reduce activation memory. Combine with activation checkpointing, TP, or CP for that.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

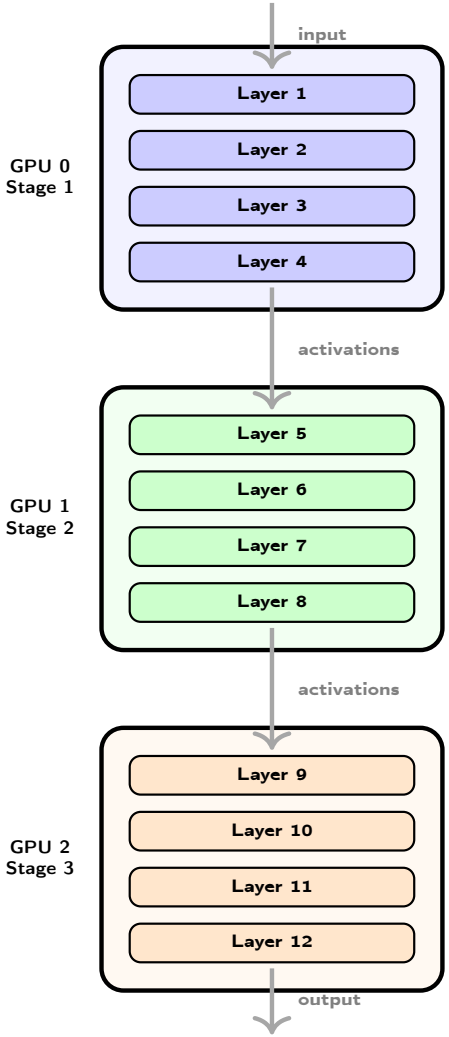
16 Parallelism for LLM Training and Inference
- Data Parallelism
- ZeRO Optimization
- Pipeline Parallelism
- Tensor Parallelism
- Context Parallelism

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

Pipeline Parallelism: Core Idea

Split the model's *layers* across GPUs, forming a sequential **pipeline**.



How it works

- Partition L layers into p **stages**
- Stage i sits on GPU i
- Activations passed *P2P* between stages

Parameter memory

- Each GPU stores $\frac{1}{p}$ of weights, gradients, optimizer states
- $\frac{16\Psi}{p}$ per GPU

Activations NOT reduced!

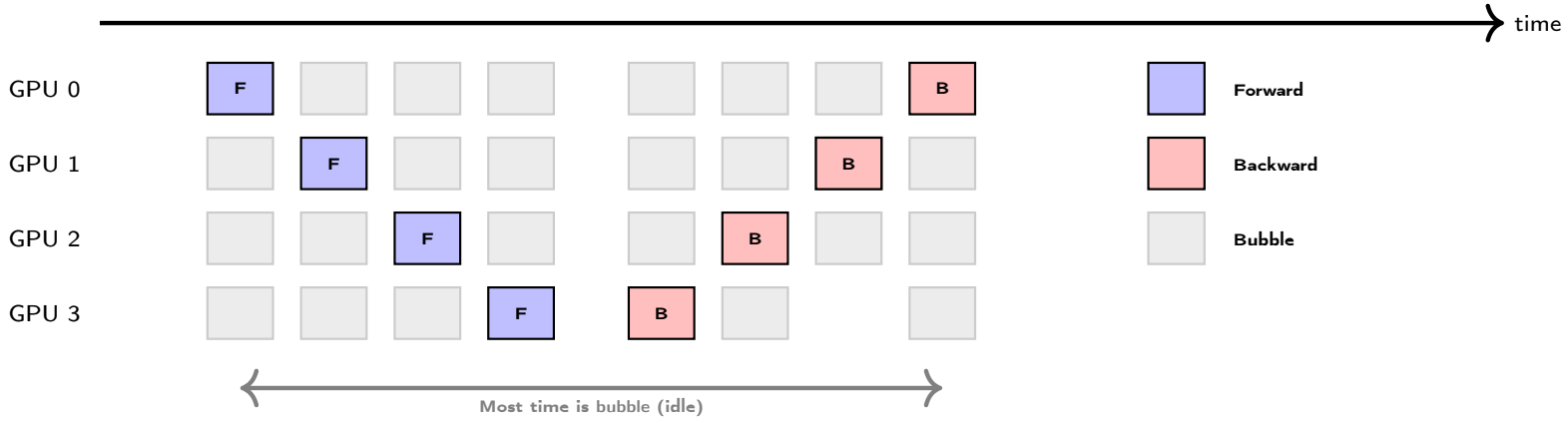
- $\frac{1}{p}$ layers per GPU, but p in-flight micro-batches before backward
- $p \times \frac{\text{activs}}{p} = \text{activs}$
- Pipeline schedule artifact; details below

Communication per micro-batch:

$4 \cdot \text{seq} \cdot \text{mbs} \cdot h$ bytes (P2P)

The Pipeline Bubble

The fundamental challenge: GPUs sit *idle* while waiting for activations from upstream stages.



With a single micro-batch and p stages, the **bubble ratio** is:

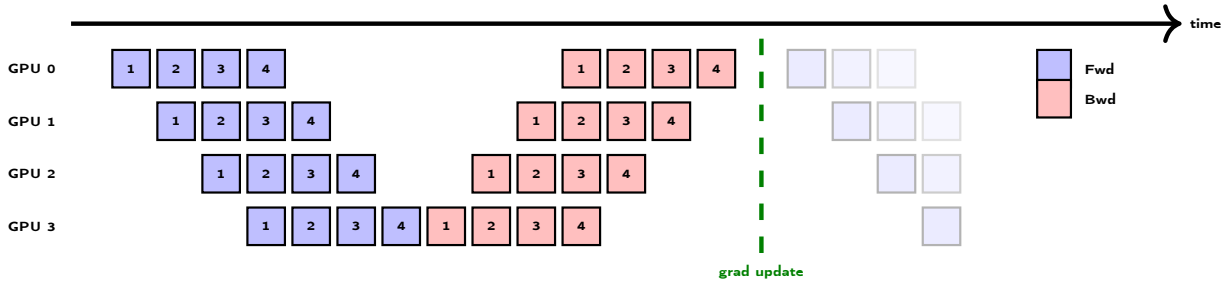
$$r_{\text{bubble}} = \frac{(p-1)(t_f + t_b)}{t_f + t_b} = p - 1$$

where t_f, t_b are forward/backward times per stage. **Catastrophic** for large p.

Solution: Split each batch into m **micro-batches** and pipeline them $\Rightarrow$ multiple schedules.

Schedule 1: All-Forward-All-Backward (AFAB)

Also known as **GPipe** schedule [Y. Huang et al., *"GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism"*, 2019]. Simplest pipeline schedule.



Algorithm:

1. Run all m forward passes through the pipeline
2. Then run all m backward passes
3. Optimizer step

Bubble ratio with m micro-batches:

$$r_{\text{bubble}} = \frac{p-1}{m}$$

Increasing m shrinks the bubble.

Pros:

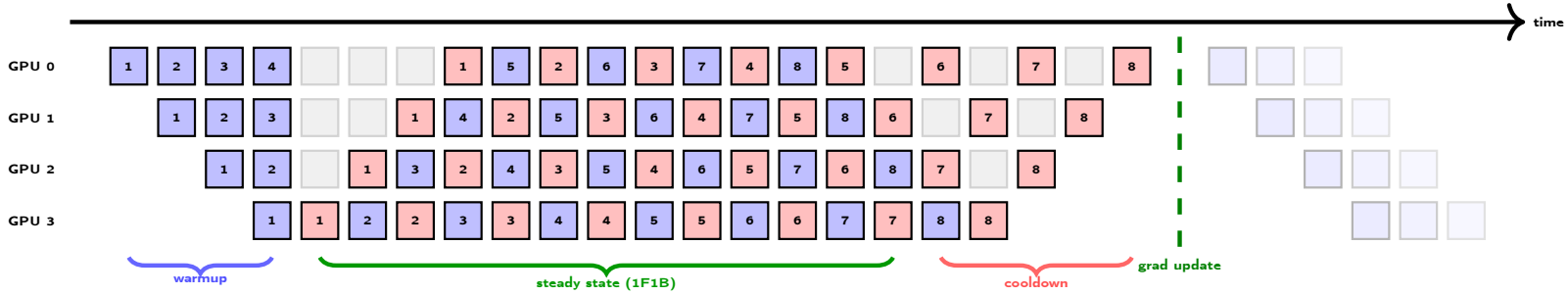
- Simplest to implement
- Clean separation of fwd/bwd phases

Cons:

- Must store activations for *all* m micro-batches until backward phase
- Activation memory scales as $O(m)$: **memory explosion**
- Cannot increase m freely to reduce bubble

Schedule 2: One-Forward-One-Backward (1F1B)

Key insight [Narayanan, Harlap, et al., *"PipeDream: Generalized Pipeline Parallelism for DNN Training"*, 2019]: start backward passes *as soon as possible* to free activation memory.



Three phases per GPU:

1. **Warmup**: only forward passes fill the pipeline
2. **Steady state**: alternate 1 forward, 1 backward
3. **Cooldown**: only backward passes drain the pipeline

Bubble: Same as AFAB: $r_{\text{bubble}} = \frac{p-1}{m}$

Key advantage (memory):

- Only need to store activations for p micro-batches (not m!)
- Can now safely increase $m \gg p$ to shrink the bubble

1F1B: Scaling Behavior

Bubble ratio: $\frac{p-1}{m}$

- Need $m \gg p$ to keep bubble small
- But m is constrained by global batch size: $\text{GBS} = \text{mbs} \times m \times N_{\text{DP}}$
- Cannot increase m arbitrarily without growing the batch size (which can hurt convergence)

Cross-node scaling:

- PP communicates only *activations* between adjacent stages (P2P)
- Much less bandwidth-sensitive than TP
- Throughput drops only ∼14% going from 1 to 2 nodes, vs. ∼43% for TP [Tazi et al., *The Ultra-Scale Model Training Playbook*, 2025]

Bubble ratio for varying m and p:

$p \backslash m$	4	8	16	32
4	75%	38%	19%	9%
8	–	88%	44%	22%
16	–	–	94%	47%

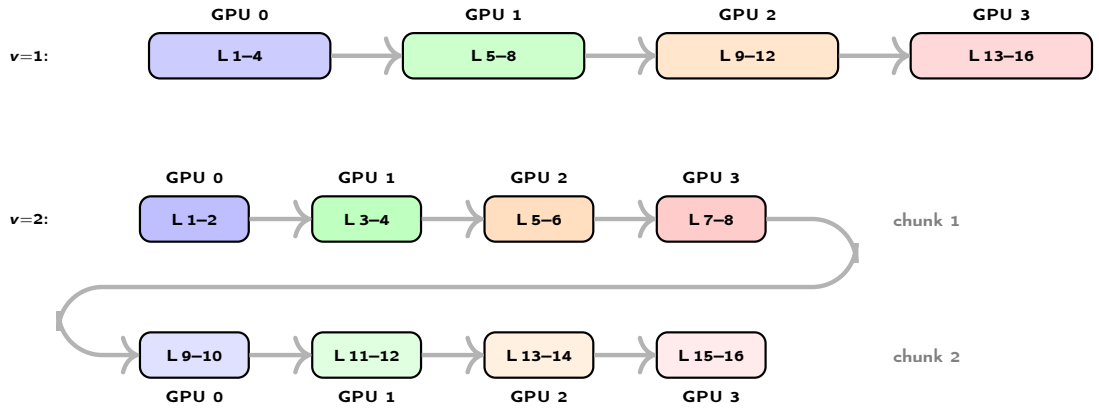
"–" means $m < p$ (not enough micro-batches).

Rule of thumb

Aim for $m \geq 4p$ for reasonable efficiency ($r_{\text{bubble}} \leq 25\%$).

Schedule 3: Interleaved 1F1B Layer Distribution

Interleaved stages [Narayanan, Shoeybi, et al., *"Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM"*, 2021]: assign v *non-contiguous chunks* per GPU. A micro-batch loops through all GPUs v times [Narayanan, Shoeybi, et al., *"Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM"*, 2021].



Bubble ratio: $r_{\text{bubble}} = \frac{p-1}{v \cdot m}$: shrinks by factor v!
Trade-off: $v\times$ more P2P communication (micro-batch traverses pipeline v times).

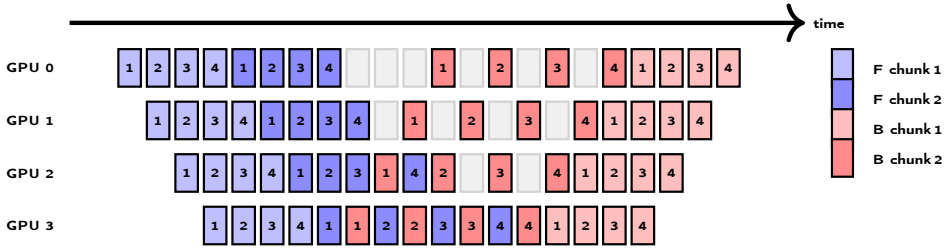
Llama 3.1 uses interleaved 1F1B [Grattafiori et al., *"The Llama 3 Herd of Models"*, 2024].

v	m	r_{bubble}
1	8	88%
2	8	44%
4	8	22%
2	32	11%
4	32	5.5%

(example p=8)

Interleaved 1F1B: Schedule

Each F/B block is $v\times$ smaller $\Rightarrow$ pipeline fills/drains faster $\Rightarrow$ smaller bubbles.



With v=2, p=4, m=4:

- Each block is $\frac{1}{v}$ the size
- Virtual pipeline depth: $v \cdot p = 8$
- Bubble ratio: $\frac{p-1}{v \cdot m} = \frac{3}{8} = 37.5\%$

Compare to standard 1F1B:

- Same m=4: bubble $\frac{3}{4} = 75\%$
- v=2 **halves** the bubble
- But 2$\times$ P2P communication volume

Interleaved 1F1B: Depth-First vs. Breadth-First

With interleaved stages, a scheduling choice arises: which micro-batch/chunk to run next on a given GPU?

Depth-first [Lamy-Poirier, *"Breadth-First Pipeline Parallelism"*, 2023]

- Prioritize advancing *earlier* micro-batches through *later* chunks
- Gets micro-batches out of the pipeline faster
- Backward starts sooner ⇒ *less activation memory*
- Can reduce peak memory significantly

Breadth-first [Lamy-Poirier, *"Breadth-First Pipeline Parallelism"*, 2023]

- Prioritize injecting *new* micro-batches into *early* chunks
- Fills the pipeline more aggressively
- Can achieve slightly *better throughput*
- But keeps more activations alive simultaneously

In practice

The optimal choice depends on memory constraints and pipeline depth. Llama 3.1 uses a tunable priority between depth-first and breadth-first scheduling [Grattafiori et al., *"The Llama 3 Herd of Models"*, 2024].

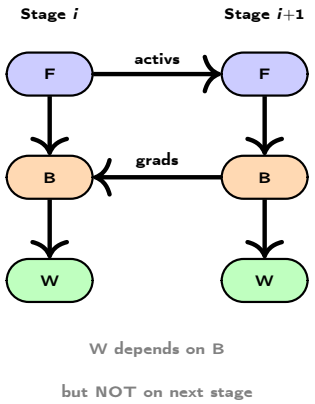
Schedule 4: Zero Bubble

Can we *eliminate* the bubble entirely? Key insight from [Qi et al., *"Zero Bubble Pipeline Parallelism"*, 2024]: decompose the backward pass.

Backward pass has two independent parts:

B Backward for *inputs* (compute $\frac{\partial \mathcal{L}}{\partial x}$)
Needed by the previous stage immediately

W Backward for *weights* (compute $\frac{\partial \mathcal{L}}{\partial \theta}$)
Only needed before the optimizer step

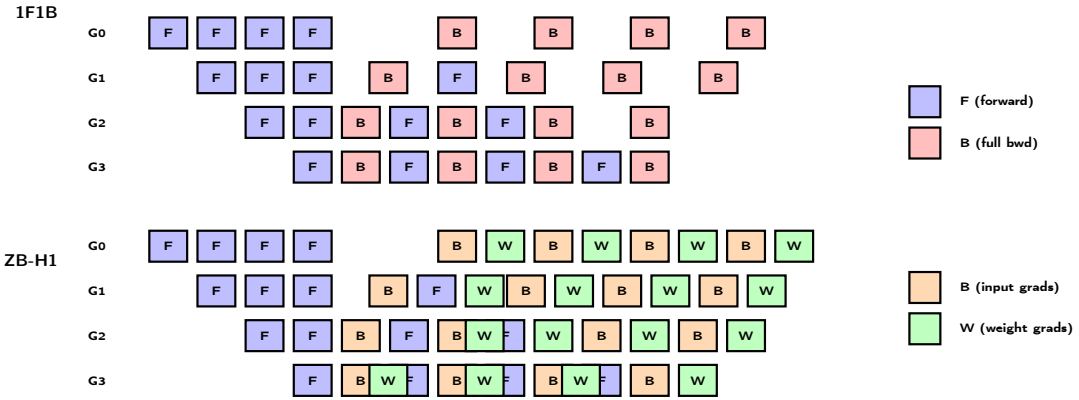


$\Rightarrow$ **W can be deferred** and scheduled flexibly to *fill bubble slots*!

ZB-H1: Schedule W operations in cooldown bubbles.
ZB-H2: Use ILP solver to find a schedule with *zero bubble* (in theory).

Optimization: Measure t_F, t_B, t_W for each stage, then solve an Integer Linear Program to minimize total idle time [Qi et al., *"Zero Bubble Pipeline Parallelism"*, 2024].

Zero Bubble: Schedules Comparison



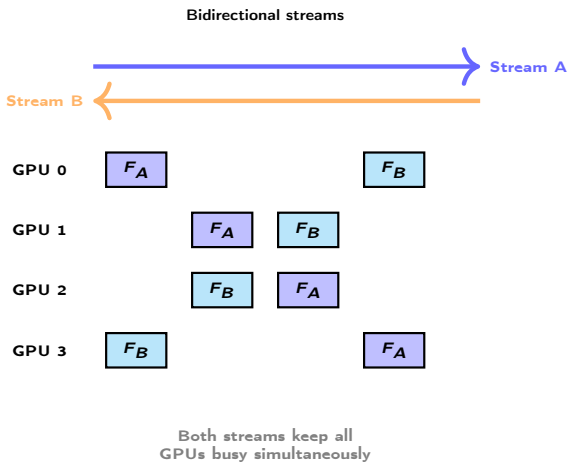
- **1F1B**: Backward is monolithic ⇒ rigid scheduling, bubbles in warmup/cooldown
- **ZB-H1**: Split into B + W, schedule W flexibly ⇒ fills cooldown bubbles
- **ZB-H2**: ILP-optimized placement of F, B, W ⇒ *theoretically zero bubble*

DualPipe (DeepSeek-V3)

DualPipe [DeepSeek-AI, *"DeepSeek-V3 Technical Report"*, 2024]: extends zero-bubble ideas with *bidirectional* pipeline streams.

Key ideas:

1. Split backward into B (input grads) and W (weight grads), as in Zero Bubble
2. Run **two streams** propagating from *both ends* of the pipeline simultaneously
3. Interleave the two streams to keep all GPUs busy
4. Overlap computation and all-to-all communication



Result: Near-zero pipeline bubble with minimal additional communication overhead.

DeepSeek-V3 reports *"near-zero all-to-all communication overhead"* with this schedule.

Complexity

DualPipe requires careful measurement of per-operation timings and sophisticated scheduling. Practical implementations use heuristics or ILP solvers to find near-optimal schedules.

PP Schedules: Summary

Schedule	Bubble ratio	Peak activation memory	Extra comm.
AFAB (GPipe)	$\frac{p-1}{m}$	$O(m)$ micro-batches	$1\times$
1F1B	$\frac{p-1}{m}$	$O(p)$ micro-batches	$1\times$
Interleaved 1F1B	$\frac{p-1}{v \cdot m}$	$O(p)$ micro-batches	$v\times$
Zero Bubble	$\to 0$ (ILP)	$O(p)$ micro-batches	$1\times$
DualPipe	≈ 0	$O(p)$ micro-batches	$\sim 1\times$

Historical progression:

- GPipe $\to$ 1F1B / PipeDream $\to$ Interleaved $\to$ Zero Bubble $\to$ DualPipe
- Each step either reduces the bubble or reduces memory, at increasing implementation complexity

PP vs. ZeRO-3

Both solve "model doesn't fit on one GPU" but differently: PP transfers *activations* between stages; ZeRO-3 transfers *weights*. PP prefers large `grad_acc` to hide the bubble; ZeRO-3 prefers large `mbs` and `seq_len` to hide communication. In practice, PP combines well with ZeRO-1/2 (e.g. DeepSeek-V3 uses PP + ZeRO-1).

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
    - Data Parallelism
    - ZeRO Optimization
    - Pipeline Parallelism
    - Tensor Parallelism
    - Context Parallelism
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Tensor Parallelism: Overview

Tensor Parallelism (TP) shards individual weight matrices *within* each layer across GPUs [Shoeybi et al., *"Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism"*, 2020].

Core idea:

- A Transformer layer is dominated by large matrix multiplications $Y = X \cdot W$
- Split W into N shards, one per GPU
- Each GPU computes with its shard *only*. Partial results are combined via collective communication
- Each GPU stores and computes with only $\frac{1}{N}$ of each weight matrix

What is sharded?

- Weights, gradients, optimizer states ✓
- **Activations** ✓ (unlike DP/ZeRO)

Key difference to Data Parallelism / ZeRO:

- DP: each GPU has a *full copy* of the model, processes different data
- ZeRO: shards parameters but must *gather* full parameters before each computation
- TP: each GPU computes with its *shard only*: no parameter gathering needed!

Communication:

- Requires AllReduce / AllGather *after each layer* (on the critical path)
- Must use fast interconnect (NVLink)
- Practical limit: TP degree $\leq$ GPUs per node (typically ≤ 8)

Linear Layers: Sharding Potential

A Transformer is dominated by linear layers: $Y = X \cdot W$.

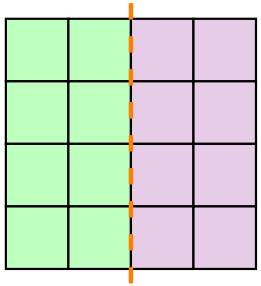
$$\underset{(2\times 4)}{\overset{X}{\begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 2 \end{bmatrix}}} \cdot \underset{(4\times 4)}{\overset{W}{\begin{bmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}}} = \underset{(2\times 4)}{\overset{Y}{\begin{bmatrix} 6 & 3 & 2 & 2 \\ 4 & 3 & 2 & 4 \end{bmatrix}}}$$

Problem: In large models, W has shape $d_{\text{model}} \times d_{\text{model}}$ (e.g. 12288×12288 in GPT-3 $\rightarrow$ 600M params per layer). Weights, gradients, and optimizer states of a single layer can exceed one GPU's memory.

Solution: Shard W across N GPUs. Two natural ways to split a matrix:

Column split: output dim

$$W = [W_0 \mid W_1]$$

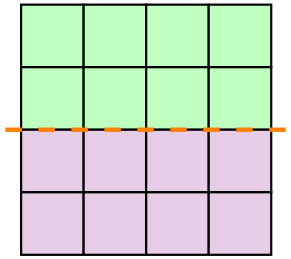


GPU 0 GPU 1

$$X[W_0 \mid W_1] = [XW_0 \mid XW_1]$$

Row split: input dim

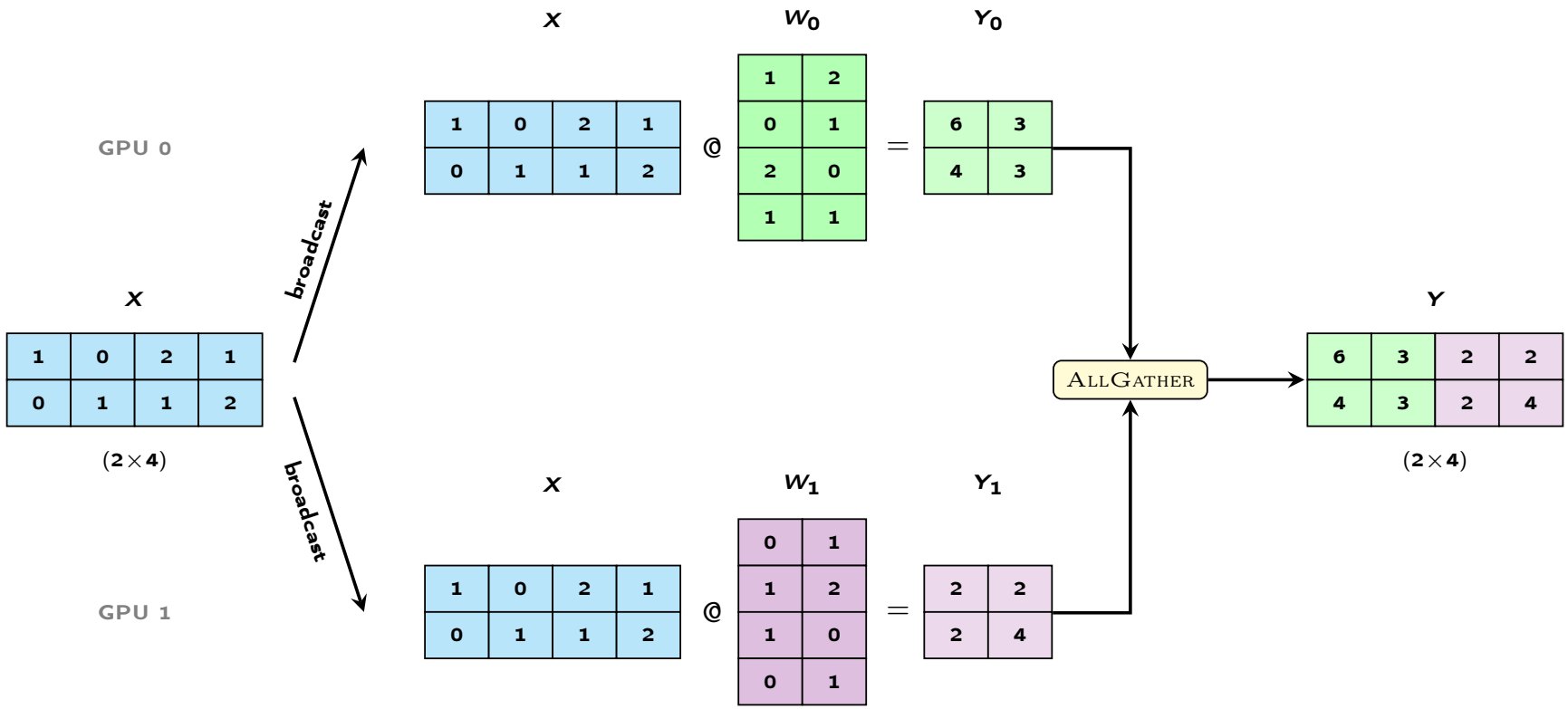
$$W = \begin{bmatrix} W_0 \\ W_1 \end{bmatrix}$$



$$[X_0 \mid X_1] \begin{bmatrix} W_0 \\ W_1 \end{bmatrix} = X_0 W_0 + X_1 W_1$$

Column-Parallel Linear Layer

Split the weight matrix W by **columns**, keep the full input X on every GPU.



Forward:

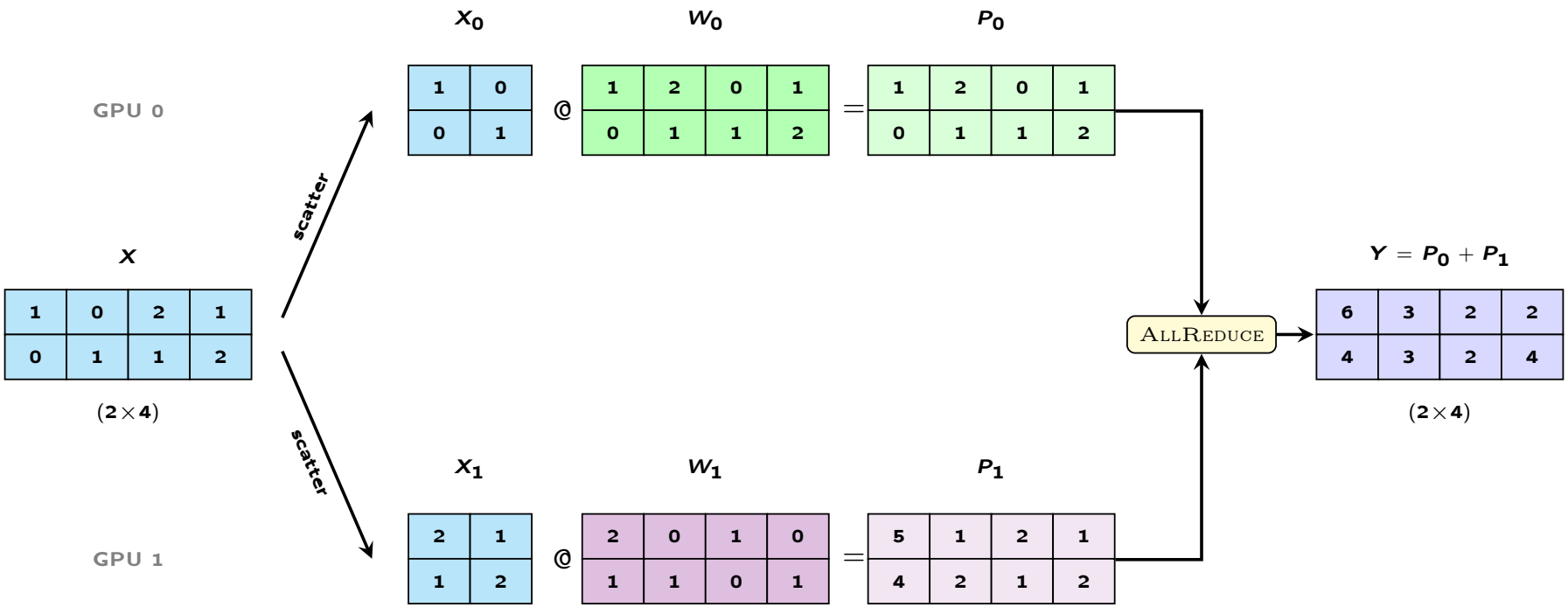
1. Broadcast X to all GPUs (or it is already replicated)
2. Each GPU i: $Y_i = X \cdot W_i$
3. AllGather to get $Y = [Y_0 \mid \cdots \mid Y_{N-1}]$

Properties:

- Output Y is split along the *output feature* dimension
- Each GPU stores only $\frac{1}{N}$ of W
- Useful when the next operation can work on the split output (e.g. GeLU)

Row-Parallel Linear Layer

Split the weight matrix W by **rows** and the input X by **columns**.



Forward:

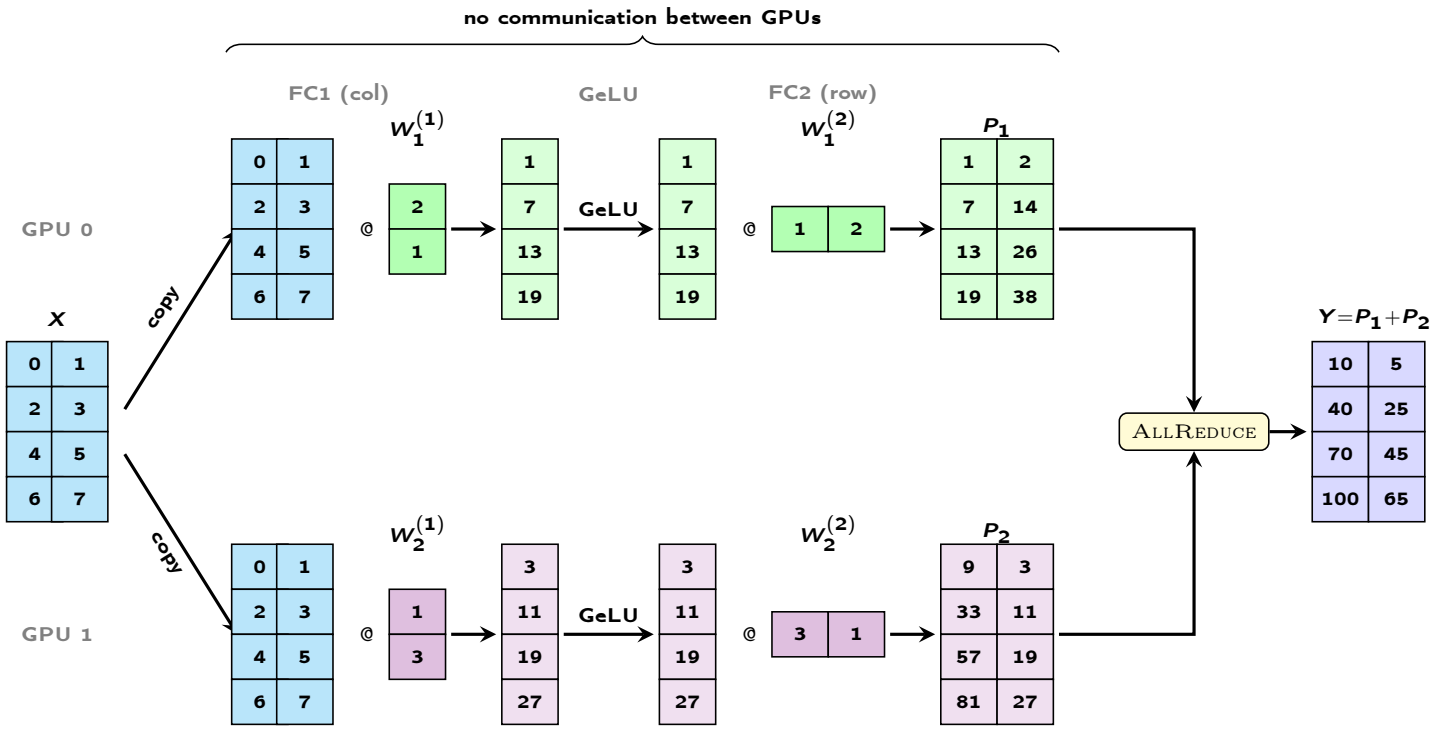
1. Scatter input: GPU i gets X_i (split along feature dim)
2. Each GPU: partial result $P_i = X_i \cdot W_i$
3. AllReduce (sum) to get $Y = \sum_i P_i$

Properties:

- Output Y is *replicated* on all GPUs
- Complementary to column-parallel: the split output of a column-parallel layer can directly serve as the split input here

TP in the Transformer: MLP Block

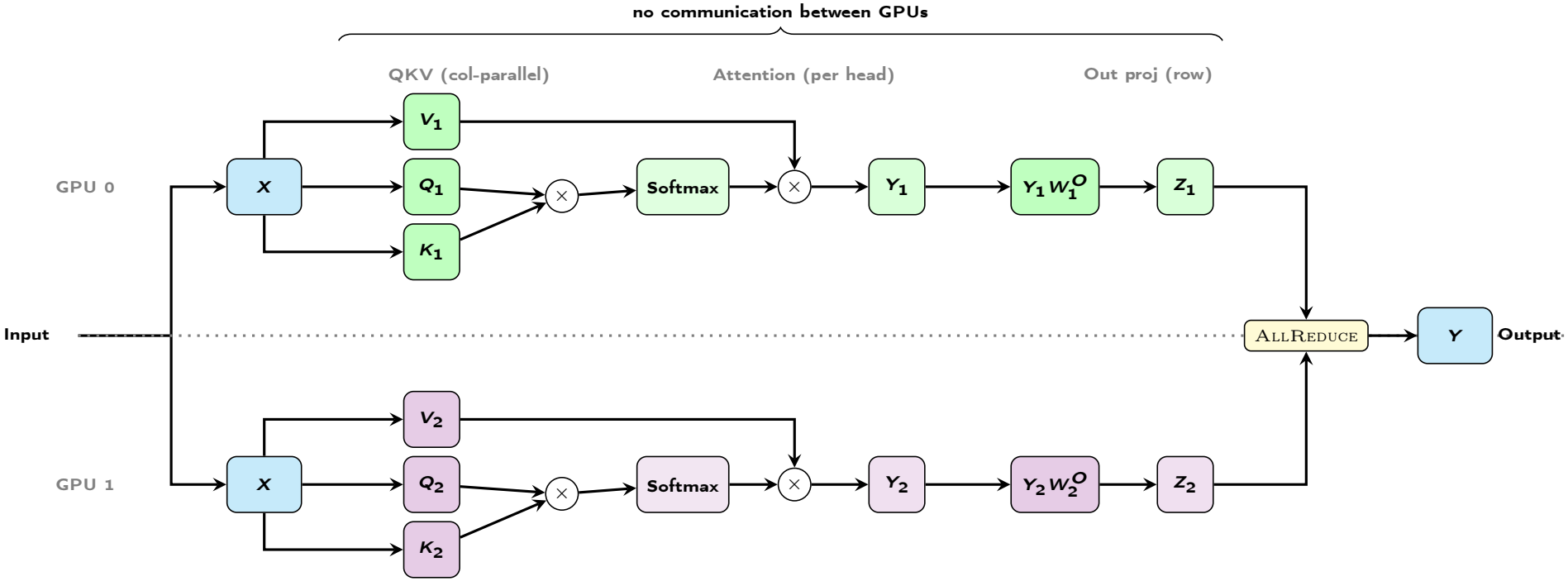
Key insight: pair **column-parallel** → **row-parallel** to minimize communication [Shoeybi et al., *"Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism"*, 2020].



- **FC1** (column-parallel): X replicated, each GPU computes with its column shard of $W^{(1)}$
- **GeLU**: element-wise, applied independently on each shard (no communication)
- **FC2** (row-parallel): split GeLU output feeds directly as split input → partial sums → AllReduce
- Only **one AllReduce** per MLP block (column→row pairing avoids extra communication)

TP in the Transformer: Attention Block

Same column→row pattern: QKV projections (col-parallel) → attention per head → output proj (row-parallel).



- **QKV projections** (col-parallel): each GPU computes a *subset of heads*, a natural fit since heads are independent
- **Attention**: $\text{Softmax}(Q_i K_i^T) \cdot V_i$ computed independently per head (no communication)
- **Output projection** (row-parallel): partial sums → one AllReduce per attention block
- Constraint: TP degree $\leq$ number of attention heads

Communication in Tensor Parallelism

Each transformer layer requires communication in the **critical path**:

Sub-block	Forward	Backward
MLP	1 AllReduce	1 AllReduce
Attention	1 AllReduce	1 AllReduce
Per layer	**2 AllReduce**	**2 AllReduce**

Communication volume

Each AllReduce sends $2 \cdot b \cdot s \cdot h$ bytes (ring AllReduce) where b=batch, s=seq len, h=hidden dim.

Per layer (fwd+bwd):

$$V_{\text{layer}} = 4 \times 2 \cdot b \cdot s \cdot h = 8\,bsh$$

Why this limits scaling:

- Communication is *synchronous*. Cannot be overlapped with compute
- Within node (NVLink): fast enough for TP ≤ 8
- Across nodes: throughput drops sharply at TP > 8

Rule of thumb

Keep TP degree $\leq$ GPUs per node (typically 8). Use DP/PP for scaling beyond a node.

TP: Memory Savings

TP distributes model state *and* activations across GPUs:

Model state (per GPU with TP=N):

- Weights: $\frac{2\Psi}{N}$ bytes
- Gradients: $\frac{2\Psi}{N}$ bytes
- Optimizer: $\frac{12\Psi}{N}$ bytes
- Total: $\frac{16\Psi}{N}$ bytes

Same reduction as ZeRO-3, but *without* gathering parameters before compute.

Activations (partially sharded):

- MatMul activations: $\frac{1}{N}$ per GPU
- LayerNorm, Dropout: still *full* (need complete hidden dim)

Example: 70B model

	TP=1	TP=8
Model state	1120 GB	140 GB
Per GPU	–	17.5 GB

Fits on a single 8-GPU node!

Limitation

LayerNorm and Dropout still need full activations ⇒ not *fully* sharded. This motivates *Sequence Parallelism*.

Sequence Parallelism (SP)

Sequence Parallelism complements TP by sharding the *non-TP* operations (LayerNorm, Dropout) along the **sequence dimension** [Korthikanti et al., *"Reducing Activation Recomputation in Large Transformer Models"*, 2023].

Problem: LayerNorm needs the full hidden dim h:

$$\mathrm{LN}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where μ, σ^2 are computed across h.

$\Rightarrow$ Cannot split along h (that's what TP does).

$\Rightarrow$ Split along the *sequence* dimension instead!

Transitions between TP and SP regions:

- **SP $\rightarrow$ TP:** AllGather (restore full sequence for column-linear)
- **TP $\rightarrow$ SP:** ReduceScatter (sum partial results + scatter along seq.)

Max activation size with TP+SP:

$$\frac{b \cdot s \cdot h}{N_{\mathrm{TP}}}$$

Always split along *some* dimension!

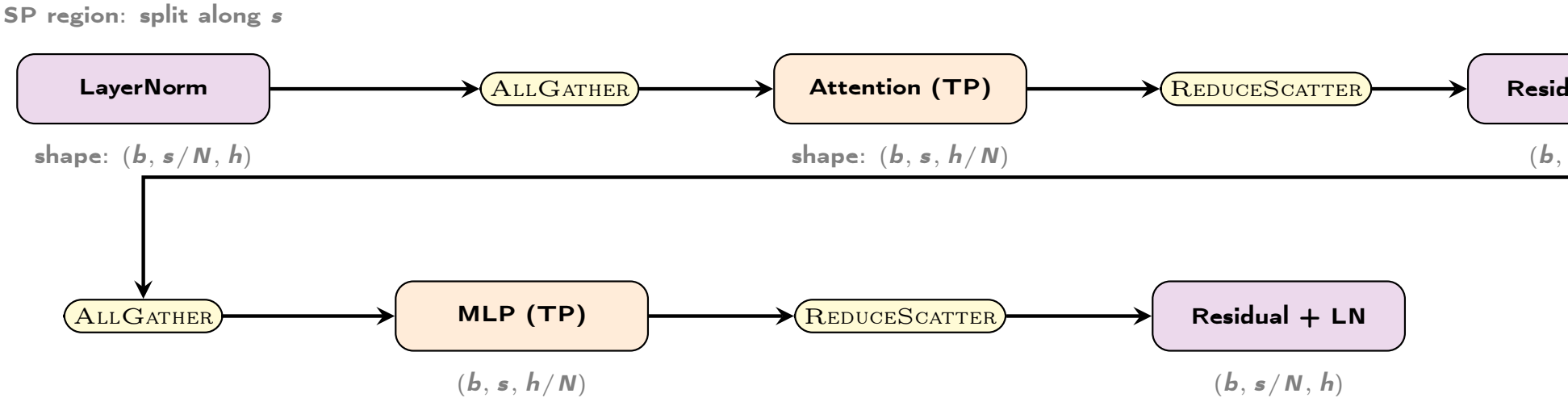
Communication cost:

- TP only: 2 AllReduce / layer
- TP+SP: 2 AllGather + 2 ReduceScatter / layer
- Since AllReduce = AllGather + ReduceScatter, the *total volume is identical*!

Takeaway

SP is essentially free: same communication cost, but better memory savings.

TP+SP: Data Flow Through a Transformer Layer



Region	TP only	TP + SP
Enter TP (col-linear)	h sharded, s full	h sharded, AllGather s to full
Inside TP	h sharded, s full	h sharded, s full
Exit TP (row-linear)	h full, AllReduce	h full, ReduceScatter s to sharded
SP region (LN, Dropout)	h full, s full	h full, s sharded

TP+SP: Summary and Trade-offs

Benefits

- Shards weights, gradients, optimizer states *and* activations
- With SP: maximum activation tensor is $\frac{bsh}{N_{\mathbf{TP}}}$
- Enables training models that are too large for a single GPU
- No parameter gathering before compute (unlike ZeRO)

Limitations

- Communication on the critical path (cannot fully overlap)
- Practical limit: TP $\leq$ GPUs/node (typically 8)
- TP degree $\leq$ num attention heads
- Beyond TP=8: severe throughput degradation

Communication per layer:

Method	Fwd ops
TP only	2 AllReduce
TP + SP	2 AllGather + 2 ReduceScatter

Same total volume (AllReduce = AllGather + ReduceScatter).

When to use TP:

- Model doesn't fit on one GPU even with ZeRO
- Sequence length is moderate ($\leq$8k)
- Fast intra-node interconnect (NVLink)

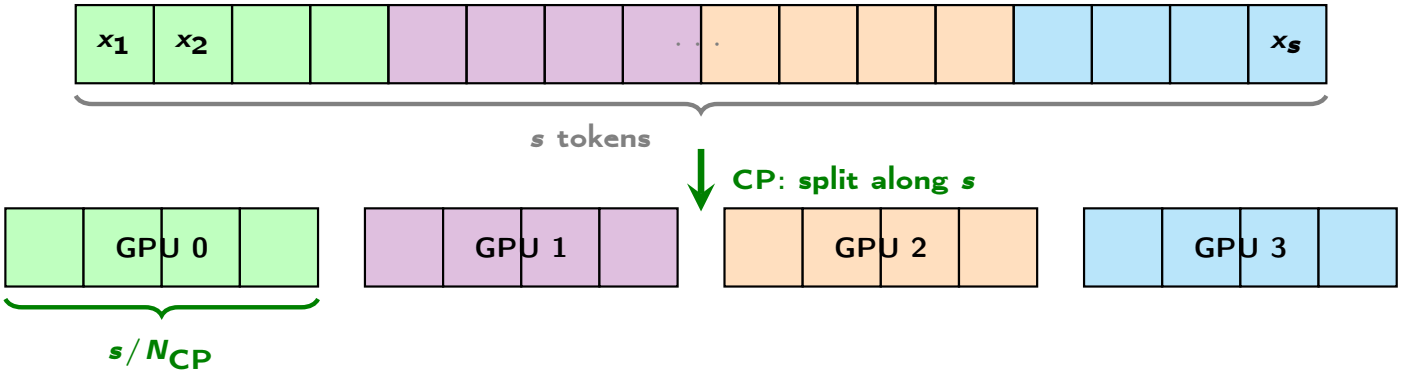
For longer sequences $\Rightarrow$ add *Context Parallelism*.
For larger models $\Rightarrow$ add *Pipeline Parallelism*.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
11. Hyperparameter Selection
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
    - Data Parallelism
    - ZeRO Optimization
    - Pipeline Parallelism
    - Tensor Parallelism
    - Context Parallelism
17. Using LLMs: Prompting, Tools, RAG and Agents
18. References

Context Parallelism: Motivation

Context Parallelism (CP) splits the **input sequence** across GPUs to handle long contexts [H. Liu et al., *"Ring Attention with Blockwise Transformers for Near-Infinite Context"*, 2023].



Problem: Activation memory

- TP shards along h, but s remains full
- Sequence Parallelism (SP) only shards *element-wise* ops. Attention requires every token to attend to every other, so SP cannot help there

Key insight:

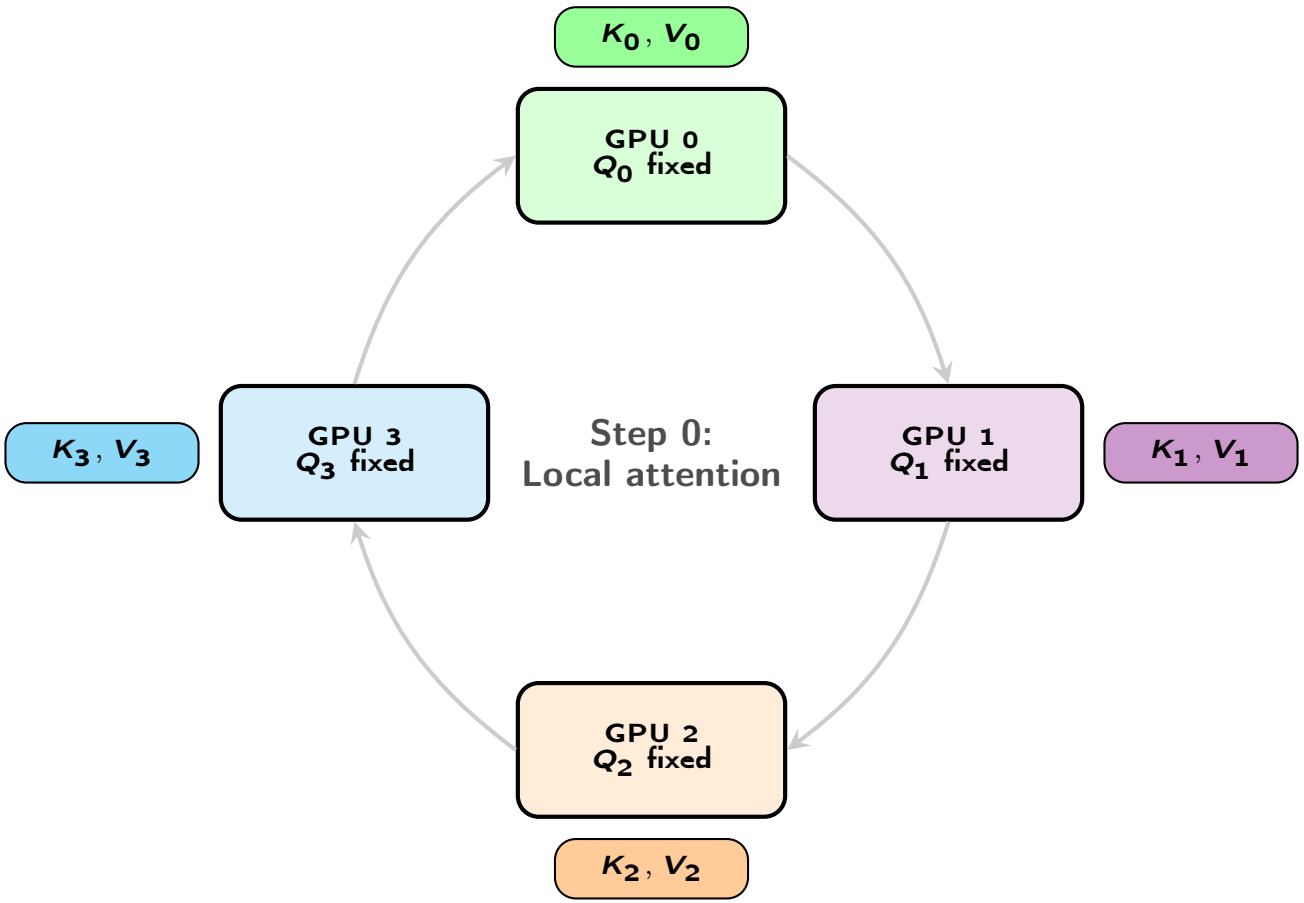
- Flash Attention computes attention *block by block* via online softmax
- The blocks don't need to be on the same GPU!
- Each GPU holds s/N_{CP} tokens and streams K, V from other GPUs

What is sharded?

- **Activations** (along sequence dim) ✓
- Weights replicated (or via TP/ZeRO)

Ring Attention: Core Idea

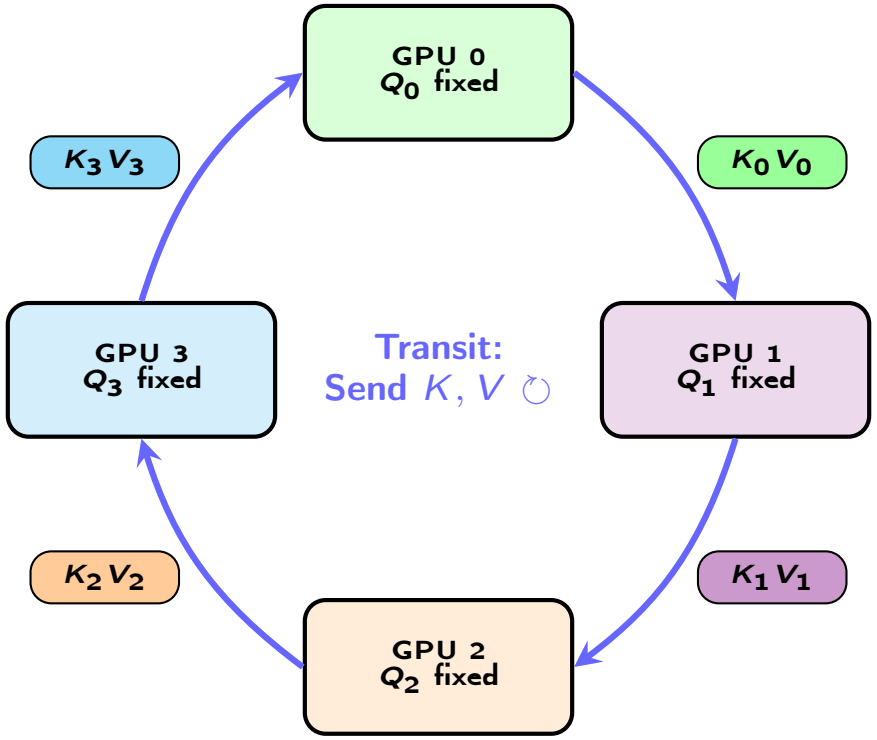
Ring Attention: arrange CP group in a **ring**, pipeline K, V transfers with local FlashAttn.



- Each GPU keeps Q_i fixed; receives K_j, V_j from neighbor each step
- After N_{CP} steps every GPU has attended to the full sequence
- Communication is **overlapped** with FlashAttn computation

Ring Attention: Core Idea

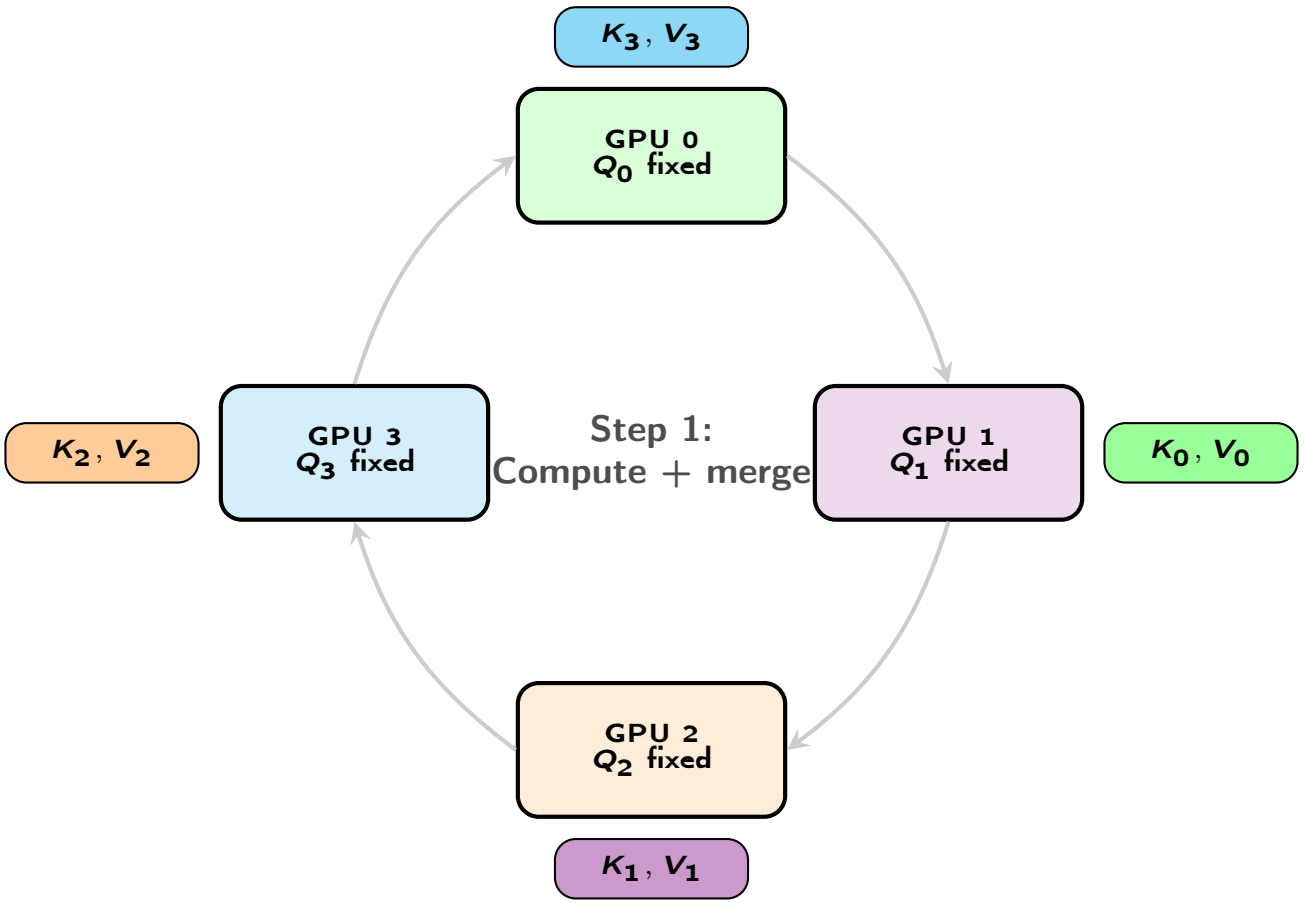
Ring Attention: arrange CP group in a **ring**, pipeline K, V transfers with local FlashAttn.



- Each GPU keeps Q_i fixed; receives K_j, V_j from neighbor each step
- After $N_{\mathbf{CP}}$ steps every GPU has attended to the full sequence
- Communication is **overlapped** with FlashAttn computation

Ring Attention: Core Idea

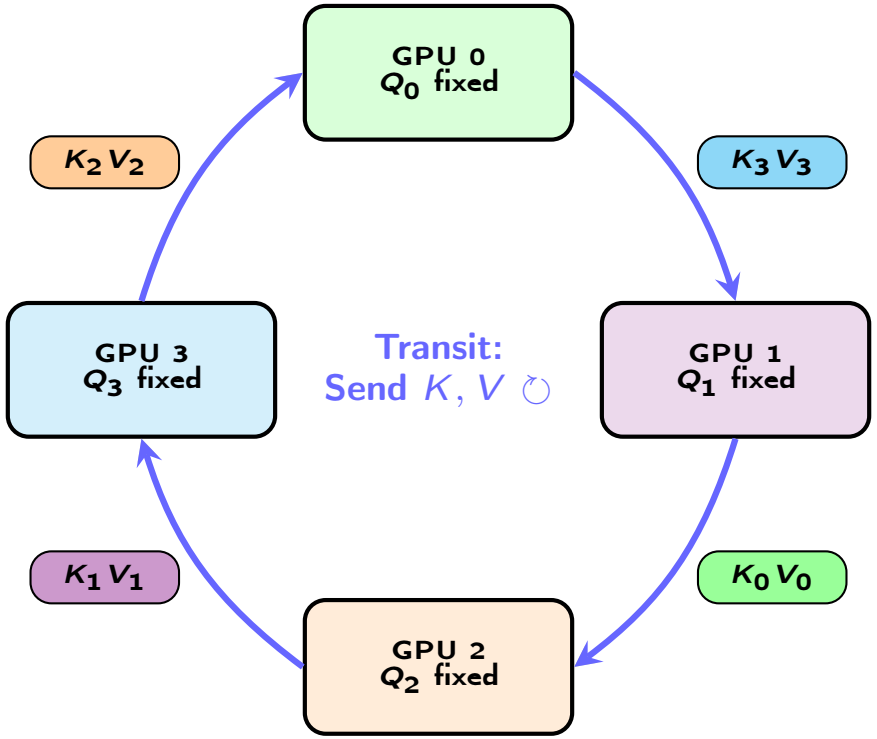
Ring Attention: arrange CP group in a **ring**, pipeline K, V transfers with local FlashAttn.



- Each GPU keeps Q_i fixed; receives K_j, V_j from neighbor each step
- After N_{CP} steps every GPU has attended to the full sequence
- Communication is **overlapped** with FlashAttn computation

Ring Attention: Core Idea

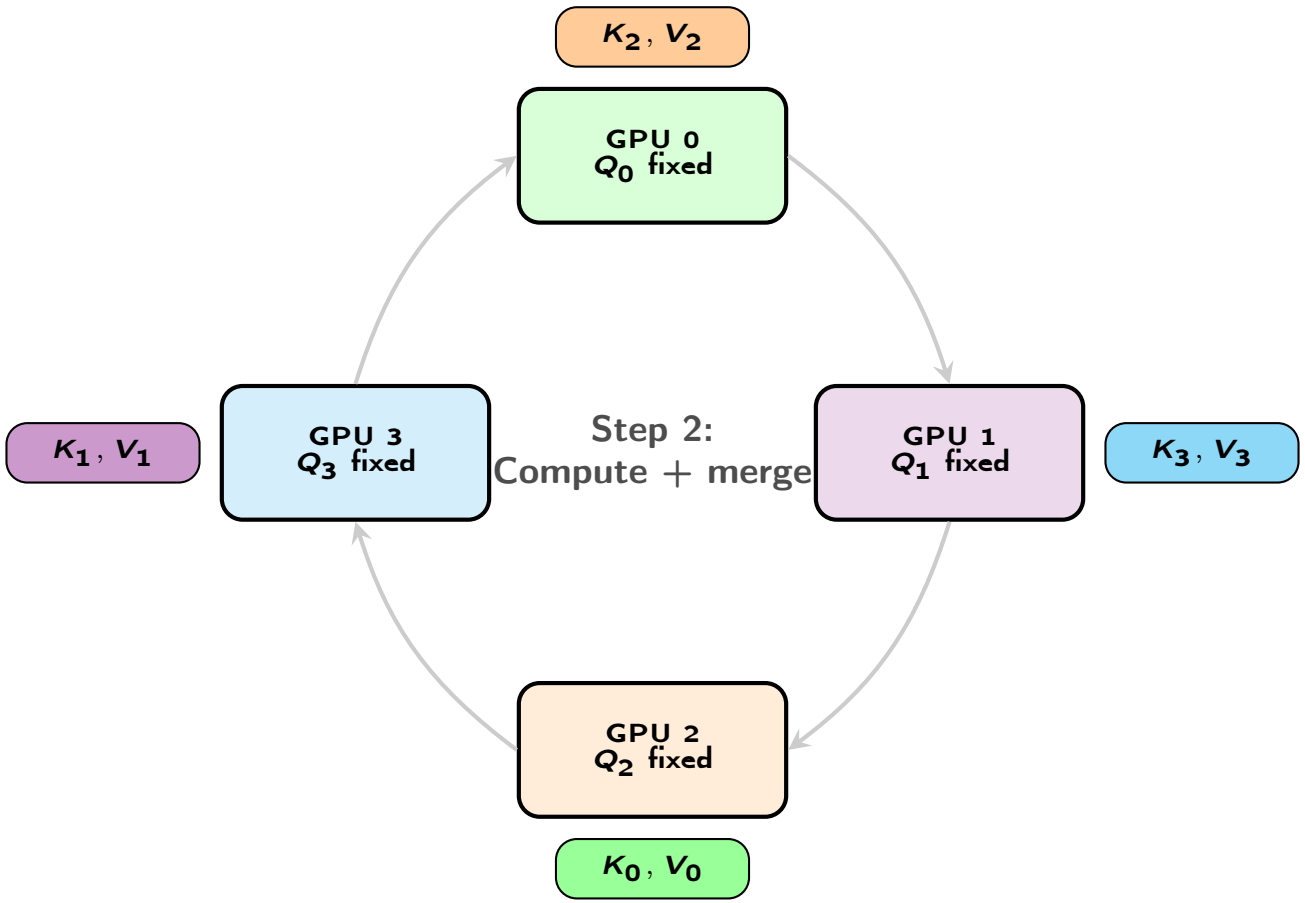
Ring Attention: arrange CP group in a **ring**, pipeline K, V transfers with local FlashAttn.



- Each GPU keeps Q_i fixed; receives K_j, V_j from neighbor each step
- After N_{CP} steps every GPU has attended to the full sequence
- Communication is **overlapped** with FlashAttn computation

Ring Attention: Core Idea

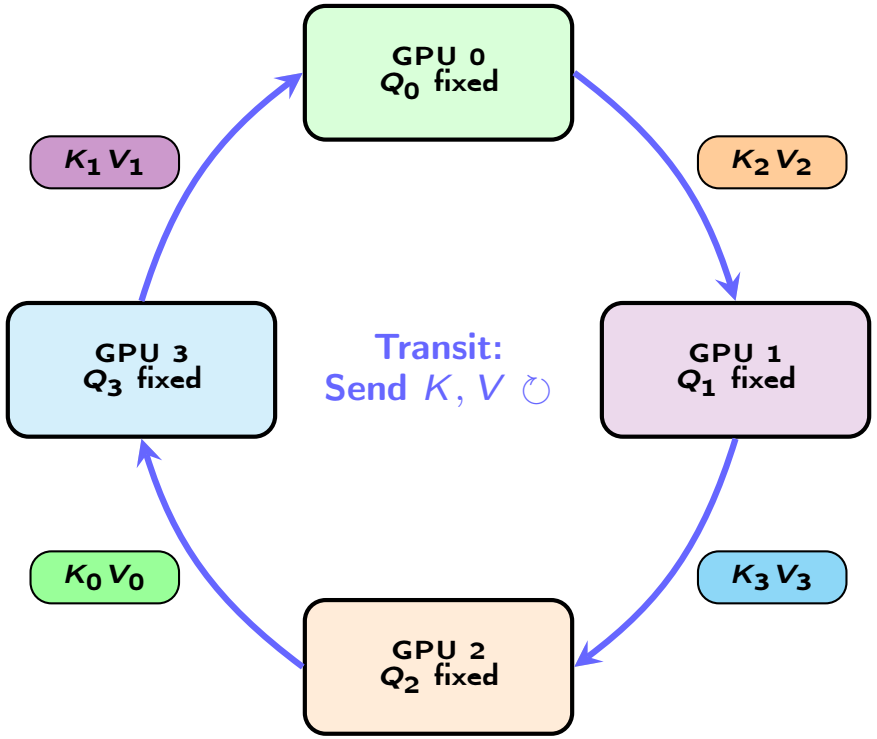
Ring Attention: arrange CP group in a **ring**, pipeline K, V transfers with local FlashAttn.



- Each GPU keeps Q_i fixed; receives K_j, V_j from neighbor each step
- After N_{CP} steps every GPU has attended to the full sequence
- Communication is **overlapped** with FlashAttn computation

Ring Attention: Core Idea

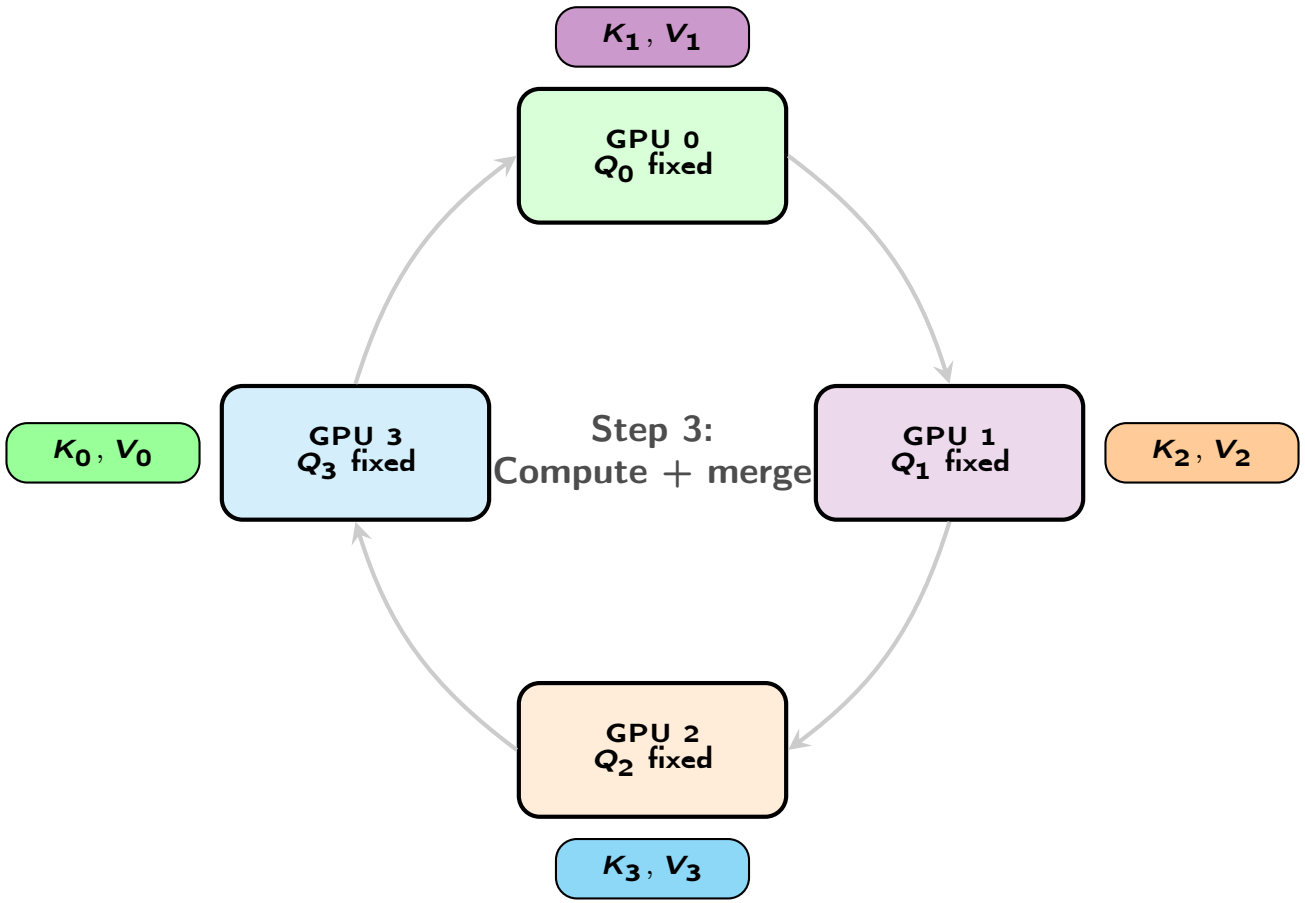
Ring Attention: arrange CP group in a **ring**, pipeline K, V transfers with local FlashAttn.



- Each GPU keeps Q_i fixed; receives K_j, V_j from neighbor each step
- After N_{CP} steps every GPU has attended to the full sequence
- Communication is **overlapped** with FlashAttn computation

Ring Attention: Core Idea

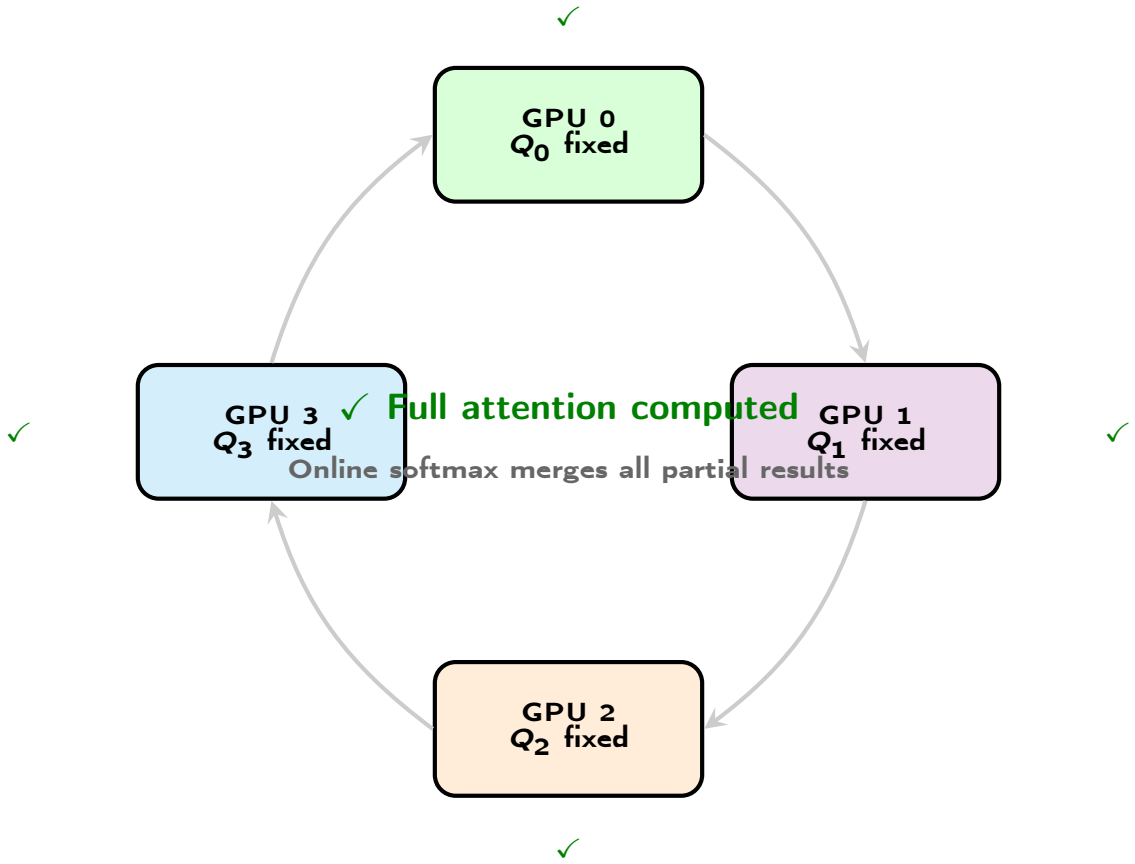
Ring Attention: arrange CP group in a **ring**, pipeline K, V transfers with local FlashAttn.



- Each GPU keeps Q_i fixed; receives K_j, V_j from neighbor each step
- After N_{CP} steps every GPU has attended to the full sequence
- Communication is **overlapped** with FlashAttn computation

Ring Attention: Core Idea

Ring Attention: arrange CP group in a **ring**, pipeline K, V transfers with local FlashAttn.



- Each GPU keeps Q_i fixed; receives K_j, V_j from neighbor each step
- After N_{CP} steps every GPU has attended to the full sequence
- Communication is **overlapped** with FlashAttn computation

Ring Attention: Step-by-Step

Step	GPU 0	GPU 1	GPU 2	GPU 3
0	$Q_0 \times K_0, V_0$	$Q_1 \times K_1, V_1$	$Q_2 \times K_2, V_2$	$Q_3 \times K_3, V_3$
1	$Q_0 \times K_3, V_3$	$Q_1 \times K_0, V_0$	$Q_2 \times K_1, V_1$	$Q_3 \times K_2, V_2$
2	$Q_0 \times K_2, V_2$	$Q_1 \times K_3, V_3$	$Q_2 \times K_0, V_0$	$Q_3 \times K_1, V_1$
3	$Q_0 \times K_1, V_1$	$Q_1 \times K_2, V_2$	$Q_2 \times K_3, V_3$	$Q_3 \times K_0, V_0$

Result Each GPU i has computed $\mathrm{Attn}(Q_i, K_{\text{full}}, V_{\text{full}})$ using online softmax to merge partial results

Communication–computation overlap: While computing $\mathrm{FlashAttn}(Q_i, K_j, V_j)$, the GPU simultaneously sends K_j, V_j to its neighbor and receives K_{j-1}, V_{j-1} from the other neighbor.

Combining Partial Attention Results

Each ring step produces a partial attention output. These are merged using the **online softmax** trick from Flash Attention.

After processing block j, GPU i maintains:

O_i	Running weighted output accumulator
ℓ_i	Running log-sum-exp (logsumexp of attention scores seen so far)

Online merge of two partial results

Given partial outputs (O_a, ℓ_a) and (O_b, ℓ_b):

$$\ell_{\text{new}} = \log\left(e^{\ell_a} + e^{\ell_b}\right) = \text{logaddexp}(\ell_a, \ell_b)$$

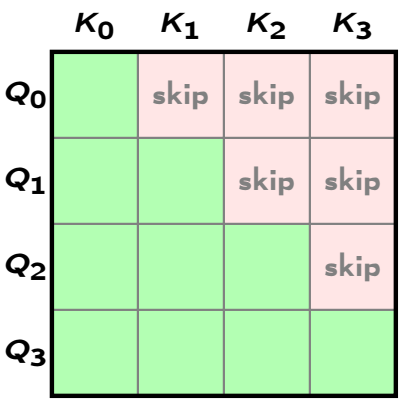
$$O_{\text{new}} = \frac{e^{\ell_a} \cdot O_a + e^{\ell_b} \cdot O_b}{e^{\ell_{\text{new}}}}$$

- This is **numerically stable** and produces the **exact same result** as computing attention over the full sequence
- The merge is $O(s/N_{\text{CP}} \cdot d)$ per step, negligible compared to the attention computation
- Same principle used inside Flash Attention across SRAM tiles, here applied across GPUs

Causal Masking and Load Imbalance

Problem with naïve ring attention:

With causal masking, Q_i can only attend to K_j where $j \leq i$ (earlier tokens).

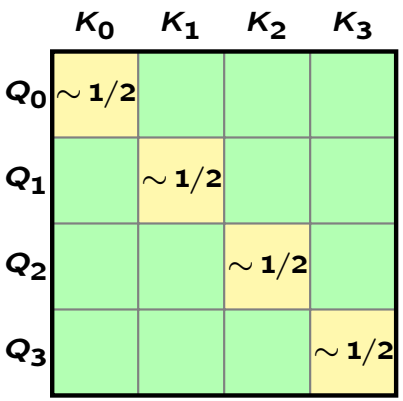


GPU 0 computes 1 block, GPU 3 computes 4 blocks $\Rightarrow$ **load imbalance!**

Solution: Striped Attention [Brandon, A. Mishra, et al., *"Striped Attention: Faster Ring Attention for Causal Transformers"*, 2023]

Instead of contiguous chunks, assign tokens in a *round-robin* (striped) pattern:

GPU 0:	tokens $0, 4, 8, 12, \ldots$
GPU 1:	tokens $1, 5, 9, 13, \ldots$
GPU 2:	tokens $2, 6, 10, 14, \ldots$
GPU 3:	tokens $3, 7, 11, 15, \ldots$



Each GPU pair now has $\approx$ equal work $\Rightarrow$ balanced.

Context Parallelism: Communication Analysis

Consider a single attention layer with N_{CP} GPUs, sequence length s, hidden dim d, batch b.

Ring Attention:

- Each step: send K_j, V_j of shape $(b, s/N_{\mathrm{CP}}, d)$ each $\Rightarrow 2 \cdot b \cdot \frac{s}{N_{\mathrm{CP}}} \cdot d$ per step
- Total steps: $N_{\mathrm{CP}} - 1$ (first block is local)
- Total volume per GPU: $2 \cdot b \cdot s \cdot d \cdot \frac{N_{\mathrm{CP}}-1}{N_{\mathrm{CP}}}$
- **Overlapped** with $O\left(\frac{b \cdot s^2 \cdot d}{N_{\mathrm{CP}}^2}\right)$ FLOPs of FlashAttn per step

When does overlap work?

Ring attention overlap is effective when computation per step $\gg$ communication per step, i.e. when s/N_{CP} is large enough that FlashAttn time exceeds transfer time.

Context Parallelism: Summary and Placement

Benefits

- Enables training with very long sequences (100k–10M+ tokens)
- Linear memory reduction: activations per GPU $\propto s/N_{\mathrm{CP}}$
- Orthogonal to TP, DP, PP. Can be combined freely
- Ring variant: communication overlaps with compute

Limitations

- Only helps when sequence length is the bottleneck (not model size)
- Ring: load imbalance with causal masking (needs striped attention)
- Additional implementation complexity

Typical 5D parallelism config:

Dimension	Scope
TP (+SP)	Intra-node (NVLink)
CP	Intra- or inter-node
PP	Inter-node
DP/ZeRO	Remaining GPUs

Total GPUs $= N_{\mathbf{TP}} \times N_{\mathbf{CP}} \times N_{\mathbf{PP}} \times N_{\mathbf{DP}}$

When to use CP:

- Sequence length $>$ 8k tokens
- Activation memory is the bottleneck
- After TP+SP is already applied within the node

Context Parallelism vs. Sequence Parallelism

	Sequence Parallelism (SP)	Context Parallelism (CP)
Scope	Shards *non-TP regions* (LayerNorm, Dropout) along *s*	Shards *attention and the full forward pass* along *s*
Tied to TP?	Yes, always paired with TP	No, orthogonal to TP
Communication	AllGather / ReduceScatter within TP group	Point-to-point (ring) or AllToAll across CP group
Scales to	Same node (TP limit)	Across nodes
Goal	Reduce activation memory in non-TP ops	Enable very long sequences (>100k tokens)

Complementary, not competing

Use TP+SP inside a node and CP across nodes. Both shard along the sequence dimension, but at different levels of the computation.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents
- From Chat Completion to Agentic Systems
- Prompting
- In-Context Learning
- Chain of Thought
- Tool Use
- Retrieval-Augmented Generation (RAG)
- Agents and the Agent Loop
- Memory
- Putting It Together

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

Motivation: LLMs as Building Blocks

Pretrained + instruction-tuned LLMs [Ouyang et al., *"Training Language Models to Follow Instructions with Human Feedback"*, 2022] are powerful general-purpose text predictors, but on their own they suffer from well-known limitations:

Knowledge cutoff Training data is frozen; cannot answer questions about recent events.

Hallucination Confidently produce wrong facts when knowledge is missing.

No grounding Cannot access private documents, databases, files, or the user's machine.

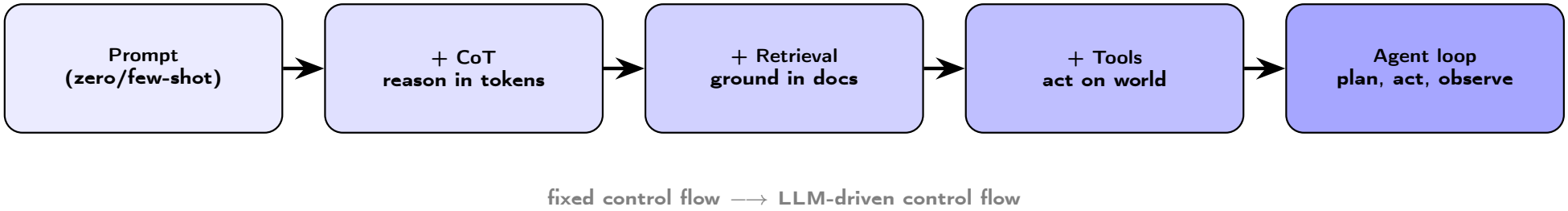
Closed world Cannot perform actions: send mails, run code, query APIs, control a browser.

Limited reasoning Single forward pass is not enough for problems that require search or backtracking.

Key idea of agentic AI: treat the LLM as a *component* inside a larger system that augments it with

- structured **prompts** and **in-context examples** (steer behavior)
- **chain-of-thought** (use extra tokens for reasoning)
- **tools** (read files, call APIs, execute code)
- **retrieval** (inject relevant knowledge into context)
- **memory** (state that persists across turns)
- a **control loop** (decide when to stop)

Spectrum: From Single Call to Autonomous Agent



Workflows vs. Agents [Anthropic, *Building Effective Agents*, 2024]

Workflow Pre-defined chain of LLM calls and tool calls. Predictable, debuggable, cheap. Use when the task decomposes into known steps.

Agent LLM *itself* decides at each step whether to call a tool, retrieve, reflect, or answer. Flexible but expensive and harder to control. Use only when the path is not knowable up-front.

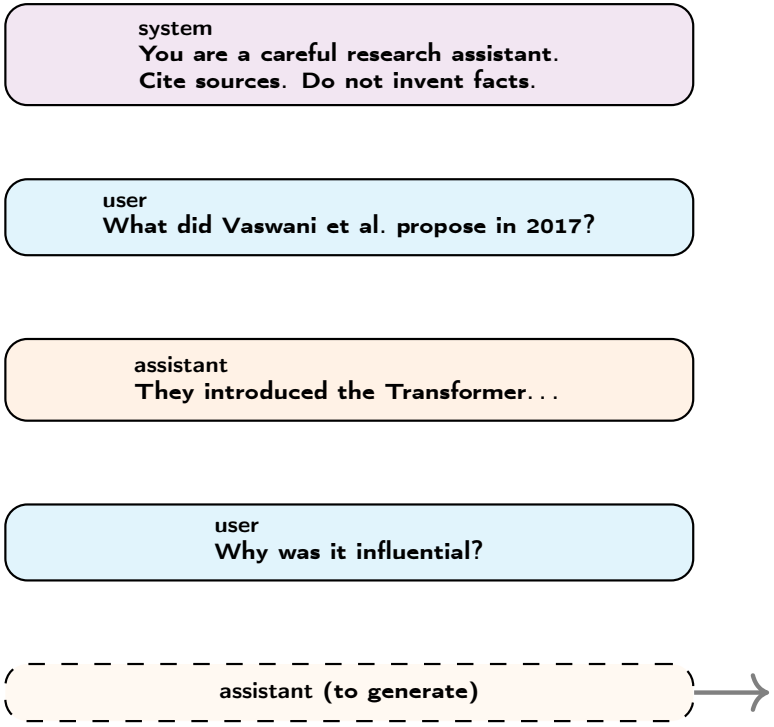
Rule of thumb: start with the simplest pattern that works. Add complexity only when measurably needed.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

Anatomy of a Chat Prompt

Modern instruction-tuned LLMs expect input as a sequence of **role-tagged messages**. Roles are baked into the chat template at fine-tuning time.



Role conventions:

`system`
: Persistent instructions, persona, output format, safety rules. Set once.

`user`
: The request to answer.

`assistant`
: Previous model responses (multi-turn history).

`tool`
: Result of a tool call (see later).

Under the hood the chat template flattens these into a single token stream with special tokens like `<|im_start|>`, `<|im_end|>`. *The model has no notion of "role" beyond what those tokens mean.*

Prompt Engineering: Practical Principles

Prompting is the cheapest way to steer behavior: no training, no data, instant feedback. A few rules consistently help.

Be specific and concrete State the task, the input, the desired output format, and any constraints explicitly. "Summarize" is ambiguous; "summarize in 3 bullets, $\leq$ 15 words each" is not.

Show, don't tell Add 1–5 worked examples (*few-shot*) when the task is hard to describe but easy to demonstrate (formatting, style, edge cases).

Decompose Break a hard task into named sub-steps. Either chain calls (workflow) or ask the model to follow the steps in one go.

Give the model an out Allow "I don't know" / "insufficient information" as a valid answer to reduce hallucination.

Constrain the output Request JSON, XML tags, or a specific schema when the output will be parsed by code. Many APIs support *structured output* / grammar-constrained decoding.

Put important things at the boundaries Models attend most to the start and end of the context (positional bias, "lost in the middle"). Place key instructions in the system prompt and the question last.

Iterate with evals Treat prompts like code: keep them under version control and measure changes against a small held-out set of inputs.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

In-Context Learning (Few-Shot Prompting)

[Brown et al., *"Language Models are Few-Shot Learners"*, 2020] showed that sufficiently large LMs can learn new tasks from a handful of examples placed *in the prompt*, with no weight updates. This is called **in-context learning** (ICL).

Three regimes:

Zero-shot Just a task description.

One-shot Description + 1 example.

Few-shot Description + k examples ($k \approx 2$–32).

When to use ICL:

- Task is hard to specify in words but easy to demonstrate.
- You need a specific output format.
- You have <100 labeled examples (otherwise consider fine-tuning).
- Task distribution is small enough to fit in the context window.

Practical tips:

- Use *diverse* examples covering edge cases.
- Order matters: recent examples have more influence (recency bias).
- Class balance matters for classification ICL.

Few-shot example: sentiment

```
Classify the sentiment as
positive, negative or neutral.

Review: "Loved every minute."
Sentiment: positive

Review: "It was fine, I guess."
Sentiment: neutral

Review: "Total waste of money."
Sentiment: negative

Review: "Best purchase this year!"
Sentiment:
```

The model continues with `positive`, **having induced both the format and the label set.**

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

Chain-of-Thought (CoT) Prompting

[Wei et al., *"Chain-of-Thought Prompting Elicits Reasoning in Large Language Models"*, 2022]: For multi-step problems, asking the model to "think step by step" before answering dramatically improves accuracy on arithmetic, commonsense, and symbolic reasoning benchmarks.

Standard prompting

```
Q: Roger has 5 balls. He buys
2 cans of 3. How many now?
A: 11
Q: A cafeteria had 23 apples,
used 20, bought 6 more. How many?
A: ???
```

Chain-of-thought prompting

```
Q: Roger has 5 balls. He buys
2 cans of 3. How many now?
A: He had 5. 2 cans * 3 = 6.
   5 + 6 = 11. Answer: 11.
Q: A cafeteria had 23 apples,
used 20, bought 6 more. How many?
A: ???
```

Why does it work?

- A single forward pass has fixed compute per token. Letting the model emit *intermediate tokens* effectively gives it more compute to "think".
- Intermediate steps act as a working memory written into the context.
- Each step conditions the next: the joint distribution over (rationale, answer) is easier to model than the answer alone.

Zero-shot CoT [Kojima et al., *"Large Language Models are Zero-Shot Reasoners"*, 2022]: simply appending "*Let's think step by step.*" to the prompt unlocks much of the gain without any examples. Modern reasoning-tuned models (o1, R1) emit long internal CoTs by default.

Beyond Linear CoT: Self-Consistency and Tree of Thoughts

Self-consistency [Xuezhi Wang et al., *"Self-Consistency Improves Chain of Thought Reasoning in Language Models"*, 2023]

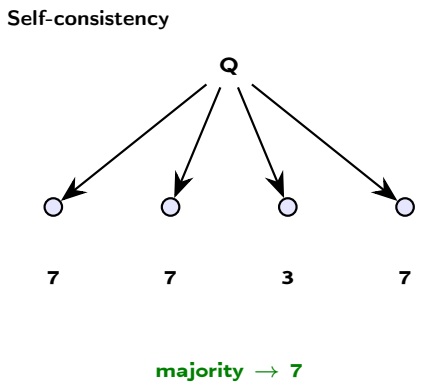
- Sample N different CoTs for the same question (temperature > 0).
- Extract the final answer from each.
- **Majority vote** over answers.

Trades $N\times$ compute for substantially higher accuracy on tasks with a unique verifiable answer. The right reasoning path tends to converge on the same answer from many starts.

Tree of Thoughts [Yao, Dian Yu, et al., *"Tree of Thoughts: Deliberate Problem Solving with Large Language Models"*, 2023]

- Treat reasoning as *search over a tree* of partial thoughts.
- At each node, the LLM *proposes* several next steps and *evaluates* how promising each is.
- Use BFS/DFS with backtracking and pruning.

Useful for puzzles (Game of 24, crosswords) where one wrong step is fatal and exploration matters.



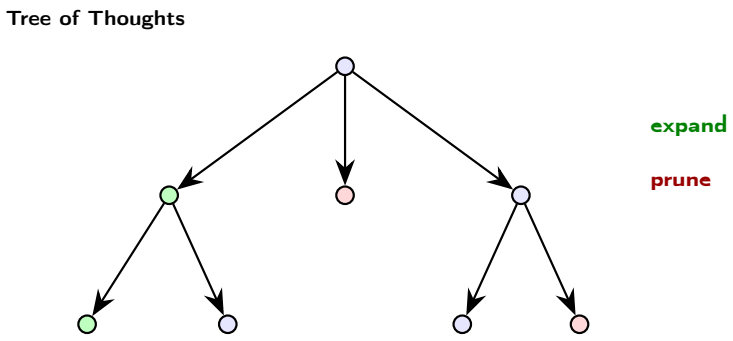


Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

Tool Use: Letting the Model Act

Why tools? A pretrained LLM cannot

- compute 17389 * 23491 reliably
- know today's weather, stock price, or git log
- read a file on the user's disk or query a database
- run unit tests or execute Python

Yet all of these are easy for tiny pieces of conventional code. **Tool use** bridges the LLM and that code.

The protocol (function calling, used by Claude, GPT, Gemini, ... [Anthropic, *Tool Use with Claude*, 2024]):

1. Developer registers a list of *tools*: name, natural-language description, JSON schema for arguments.
2. The schemas are inserted into the prompt (typically the system message).
3. When the model wants to call a tool, it emits a structured `tool_use` block instead of a normal answer.
4. The runtime parses the block, executes the tool, and appends the result as a `tool_result` message.
5. The model is invoked again with the augmented conversation and either calls more tools or produces a final answer.

[Schick et al., *"Toolformer: Language Models Can Teach Themselves to Use Tools"*, 2023]
Toolformer: train the model itself to insert API calls into its outputs by self-supervised data augmentation. Modern models are RL-fine-tuned on tool-use trajectories.

Tool Calling: Sequence Diagram

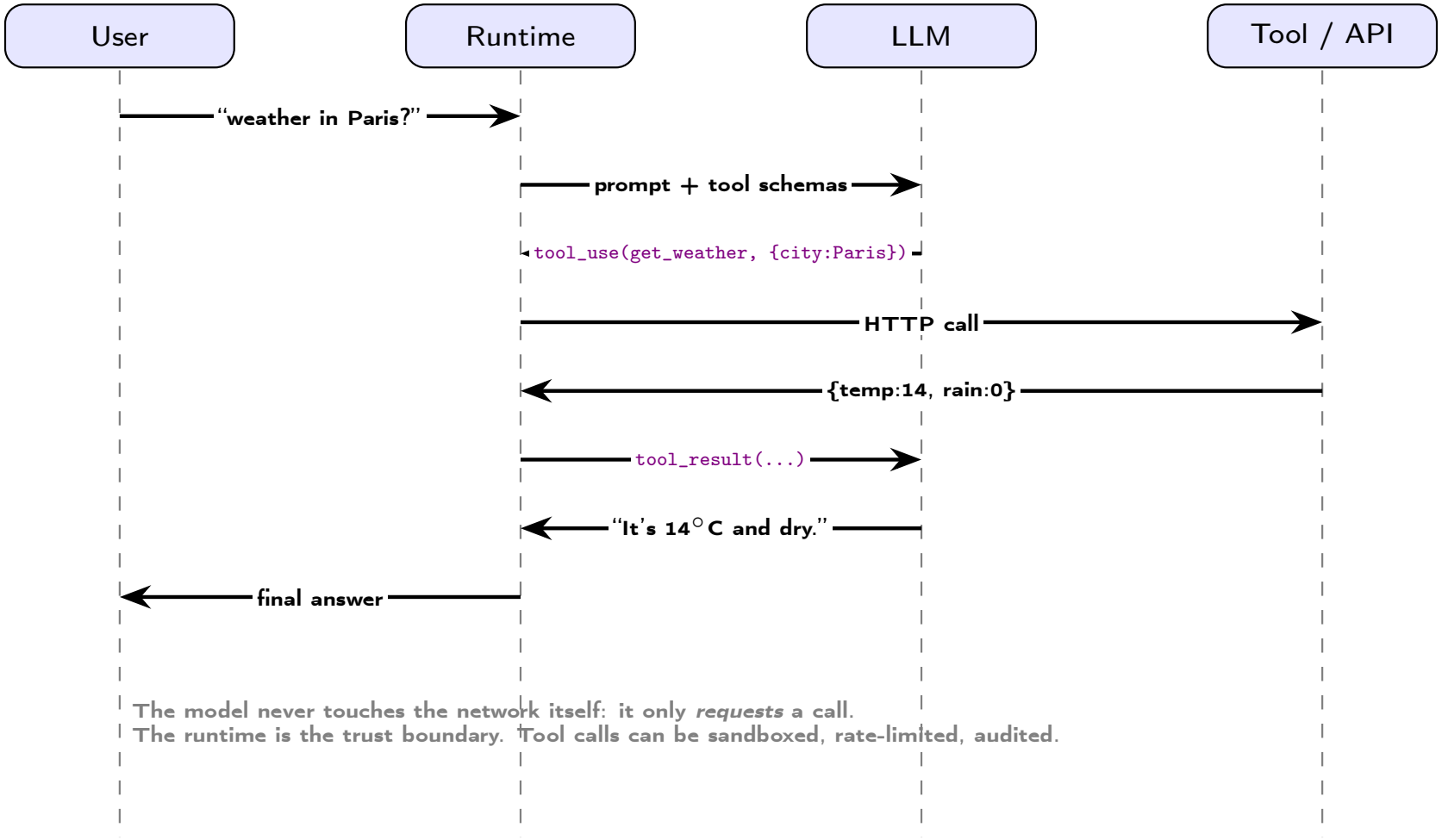


Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

Retrieval-Augmented Generation: Why and What

[Lewis et al., *"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"*, 2020]: Combine a parametric LM with a non-parametric *external memory* (a corpus of documents). At inference, retrieve relevant passages and feed them into the prompt as context.

What problems does it solve?

Knowledge cutoff Update the corpus, not the weights.

Private data Index company docs, codebases, mails, that the base model has never seen.

Hallucination Conditioning on retrieved text reduces fabrication when prompted to "answer only from the sources".

Citation You can show the user *which document* the answer came from.

Cost Smaller models with retrieval can match larger models without it on knowledge-intensive tasks.

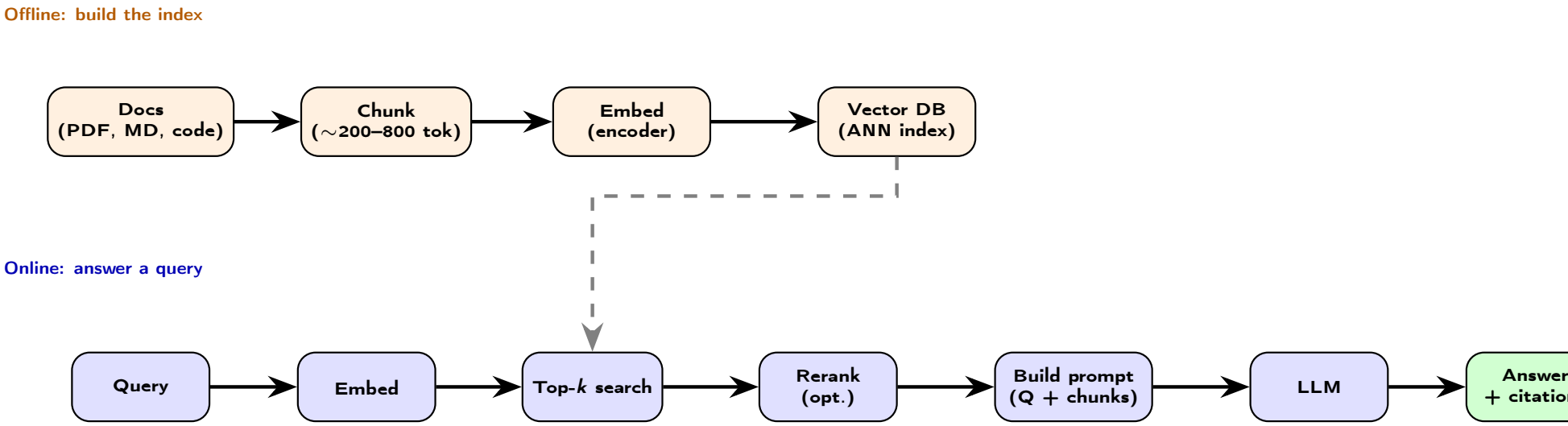
Two paradigms.

Closed-book Model relies only on its weights. (Standard chat.)

Open-book / RAG Prompt = `<system>` + `<retrieved chunks>` + `<question>`. Model is told to answer using the chunks.

See [Y. Gao et al., *"Retrieval-Augmented Generation for Large Language Models: A Survey"*, 2023] for an extensive survey.

The RAG Pipeline



Two stages, two distinct concerns.

Indexing (offline, batch) chunking strategy, embedding model choice, metadata to keep alongside vectors. Done once per corpus.

Querying (online, latency-sensitive) embedding the query, ANN search (FAISS, HNSW), optional reranking, prompt assembly, generation.

Practical RAG: Pitfalls and Tricks

Embeddings and retrieval

- **Dense retrieval** [Karpukhin et al., *"Dense Passage Retrieval for Open-Domain Question Answering"*, 2020]: encode query and passage with the same model, take cosine similarity. Captures semantic similarity.
- **Sparse retrieval** (BM25): keyword overlap. Cheap, strong baseline, complementary to dense.
- **Hybrid**: linear combination of BM25 and dense scores. Almost always better than either alone.
- **Rerankers**: a cross-encoder (e.g. ColBERT [Khattab and Zaharia, *"ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT"*, 2020]) scores (query, chunk) pairs jointly. Slow, but applied only to the top k candidates.

Chunking

- Too small: loses context. Too large: dilutes the relevant passage and wastes tokens.
- Respect document structure (headings, paragraphs, code blocks). Use overlap (e.g. 10%) to avoid losing information at boundaries.

Common failure modes

- **Retriever miss:** relevant chunk is not in the top k. The generator cannot recover. Fix the retriever first.
- **Distracting context:** top k contains plausible but wrong chunks. Use rerankers, lower k.
- **Stale index:** corpus changed but vectors did not.
- **Lost in the middle:** long context dilutes attention. Put the most relevant chunk first or last.

Table of Contents

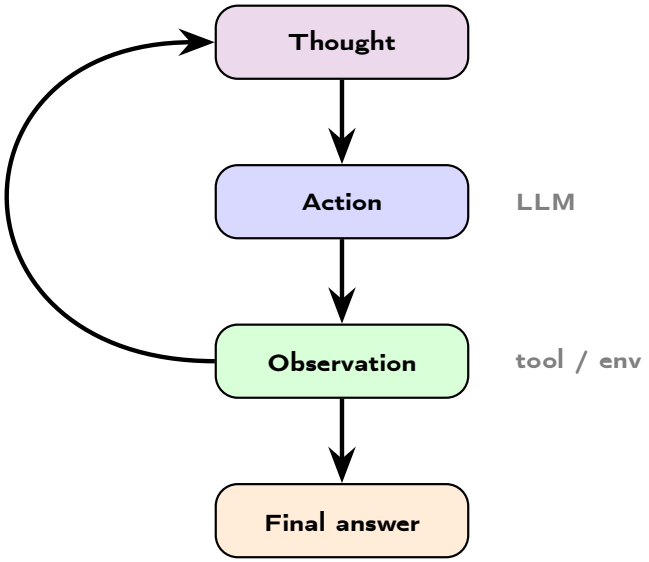
1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

ReAct: Reason + Act

[Yao, J. Zhao, et al., *"ReAct: Synergizing Reasoning and Acting in Language Models"*, 2023]: Interleave **Thought** (internal reasoning), **Action** (tool call), and **Observation** (tool result) inside a single rolling context. The LLM drives its own control flow.

ReAct trace (HotpotQA-style)

```
Q: Who directed the 1982 film
   that won Best Picture?

Thought: I need to find the 1982
Best Picture, then its director.
Action: search("1982 Best Picture")
Observation: "Gandhi"

Thought: Now find Gandhi's director.
Action: search("Gandhi 1982 director")
Observation: "Richard Attenborough"

Thought: I have the answer.
Action: finish("Richard Attenborough")
```



Reasoning helps decide *which* action; observations ground reasoning in real facts. Without reasoning, tool selection degrades; without acting, the model is stuck in its head.

The Generic Agent Loop

Algorithm 18: Generic LLM agent loop

Input: user goal g, tool set $\mathcal{T}$, max steps $T_{\max}$
Output: final answer a
Initialize context $C \leftarrow [\textsc{system prompt}, \textsc{tools}(\mathcal{T}), g]$
for $t = 1, \ldots, T_{\max}$ **do**
  $r \leftarrow \mathrm{LLM}(C)$ `// text + optional tool call`
  if *r contains a tool call $\tau(\cdot)$* **then**
    $o \leftarrow \tau(\textsc{args}(r))$ `// execute tool`
    $C \leftarrow C \,\|\, r \,\|\, \textsc{tool-result}(o)$
  else
    return final answer extracted from r
return "failed: step budget exhausted"

Key design choices.

Step budget $T_{\max}$ hard cap to bound cost and prevent infinite loops.

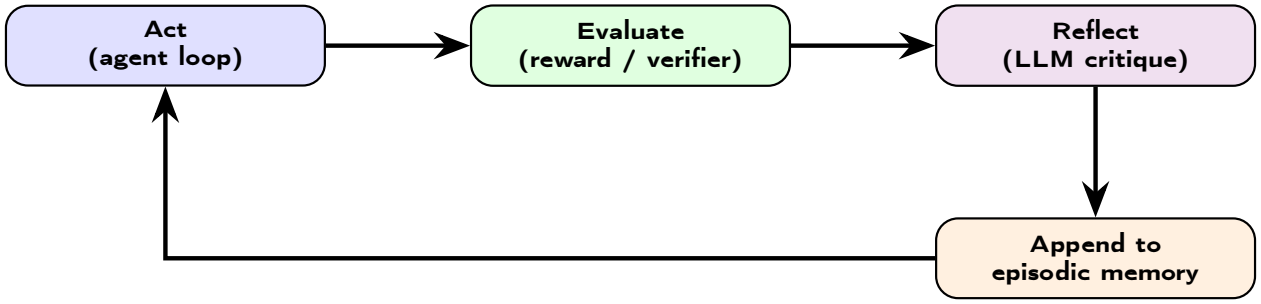
Stopping condition explicit `finish` tool, or model stops calling tools.

Error handling tool failures returned as observations; the model can react.

Context management long traces blow up the window; summarize, truncate, or offload to memory (next subsection).

Reflection: Learning Within an Episode

[Shinn et al., *"Reflexion: Language Agents with Verbal Reinforcement Learning"*, 2023]: When an agent fails (test fails, wrong answer, tool error), ask the LLM itself to *verbally critique* the trajectory and store the critique as text in the next attempt's context.



Why it works.

- No weight updates. The "policy improvement" is implemented as *text added to the prompt*: "last time I tried X and it failed because Y, this time try Z".
- Effective when there is a cheap verifier (unit tests, exact-match, compiler).
- Free signal for environments where RL would be expensive or unstable.

Variant: **self-critique** / **self-refine**, where reflection runs every step rather than only after failure.

Common Agentic Patterns

[Anthropic, *Building Effective Agents*, 2024] catalogues the patterns that recur in production systems.

Prompt chaining Decompose a task into a fixed sequence of LLM calls ($\text{step}_1 \to \text{step}_2 \to \ldots$). Predictable, easy to debug.

Routing A small classifier LLM picks one of several specialized handlers (e.g. "coding question" vs "billing question").

Parallelization Run several LLM calls in parallel and aggregate (vote, sum, take best). Useful for evaluation, multi-perspective answers.

Orchestrator/worker One LLM plans and dispatches sub-tasks to other LLM "workers"; the orchestrator merges results.

Evaluator/optimizer loop One LLM produces a candidate, another grades it, the first revises. Continue until the grader is satisfied.

Autonomous agent The LLM itself decides at each step what to do next (tool call, retrieve, ask user, finish). Most flexible, hardest to bound.

Guideline: prefer the simplest pattern that meets the requirement. Workflows are easier to test and cheaper to run; reach for fully autonomous agents only when the task path is genuinely unpredictable.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

Memory: Beyond the Context Window

An LLM is *stateless*: each call is just $p(\text{tokens} \mid \text{prompt})$. "Memory" in agentic systems means **external state** that the runtime shuttles in and out of the prompt.

Working memory The current conversation / agent trace inside the context window. Volatile, capped by max sequence length.

Episodic memory Logs of past interactions. "Last Tuesday the user asked about X and we agreed on Y."

Semantic memory Distilled facts and preferences. "The user is a Go developer working on the ADL repo."

Procedural memory Reusable how-to knowledge: prompts, playbooks, learned tool-use sequences.

Why not just use a longer context?

- Attention cost grows quadratically with context length.
- Long contexts dilute attention (lost in the middle).
- Cross-session continuity needs persistence anyway.
- Privacy: not all stored facts should be in every prompt.

Memory Architectures in Practice

Storage backends

Scratchpad / file
Plain text file the agent reads and writes via tools. Simple, debuggable.

Key-value store
Named slots: `user_role`, `project_X_status`. Index by name.

Vector store
Each memory is embedded; retrieve by semantic similarity to the current query (RAG over memories).

Knowledge graph
Structured facts as (subject, relation, object) triples.

Operations on memory

Write
At end of turn, summarize what was learned and store it.

Read
On a new query, retrieve a few relevant memories (semantic search, recency, importance).

Update
Overwrite a slot when new info contradicts old.

Forget
Drop stale or low-value entries to bound total size.

Generative Agents [Park et al., *"Generative Agents: Interactive Simulacra of Human Behavior"*, 2023]

Simulated characters in a sandbox town store every observation as a memory entry. Retrieval combines *recency*, *importance* (LLM-scored), and *relevance* (cosine similarity) into a single ranking. Periodic *reflection* steps consolidate raw observations into higher-level beliefs. A general recipe for long-horizon agent memory.

Table of Contents

1. Introduction
2. Language Generation
3. Attention & Transformers
4. Training a GPT-2 Model
5. Flash Attention
6. Sequence Models
7. State Space Models and Mamba
8. Mixture of Experts
9. Vision Models
10. Multimodal Models
12. Optimizers: From Adam to Muon
13. Reinforcement Learning for LLMs
14. Knowledge Distillation
15. Memory-Efficient Training Techniques
16. Parallelism for LLM Training and Inference
17. Using LLMs: Prompting, Tools, RAG and Agents
    - From Chat Completion to Agentic Systems
    - Prompting
    - In-Context Learning
    - Chain of Thought
    - Tool Use
    - Retrieval-Augmented Generation (RAG)
    - Agents and the Agent Loop
    - Memory
    - Putting It Together

Designing an Agentic System: Checklist

1. **Start with the task, not the architecture.**
   - Can a single well-crafted prompt do it? Stop here.
   - Does it need fresh / private knowledge? $\Rightarrow$ add RAG.
   - Does it need to compute, query, or act? $\Rightarrow$ add tools.
   - Is the path dynamic and unknowable? $\Rightarrow$ add an agent loop.
   - Does it need cross-session continuity? $\Rightarrow$ add memory.
2. **Make it observable.**
   - Log every prompt, completion, tool call, and retrieval.
   - Build evals before adding complexity. Treat prompts as code.
3. **Bound the blast radius.**
   - Sandbox tools that touch the file system, the network, or the user's accounts.
   - Cap step budgets, cost budgets, and tool-call rates.
   - Require confirmation for irreversible actions (delete, send, deploy).
4. **Iterate on the cheapest layer first.**
   - Prompt $<$ retrieval $<$ tool design $<$ scaffolding $<$ fine-tuning.
   - Most agent failures are prompt or retrieval failures, not model capability gaps.

Take-away: agentic AI is mostly *systems engineering around an LLM*. The cleverness is in the loop, the tools, and the data flowing through them, not in the model call itself.

Table of Contents

1 Introduction

2 Language Generation

3 Attention & Transformers

4 Training a GPT-2 Model

5 Flash Attention

6 Sequence Models

7 State Space Models and Mamba

8 Mixture of Experts

9 Vision Models

10 Multimodal Models

11 Hyperparameter Selection

12 Optimizers: From Adam to Muon

13 Reinforcement Learning for LLMs

14 Knowledge Distillation

15 Memory-Efficient Training Techniques

16 Parallelism for LLM Training and Inference

17 Using LLMs: Prompting, Tools, RAG and Agents

18 References

References I

Agarwal, Rishabh et al. (2024). "On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2306.13649.

Ainslie, Joshua et al. (2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. URL: https://arxiv.org/abs/2305.13245.

Akiba, Takuya et al. (2019). "Optuna: A Next-generation Hyperparameter Optimization Framework". In: *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. URL: https://arxiv.org/abs/1907.10902.

Alammar, Jay (2018). *The Illustrated Transformer*. URL: https://jalammar.github.io/illustrated-transformer/.

Anthropic (2024a). *Building Effective Agents*. https://www.anthropic.com/research/building-effective-agents.

— (2024b). *Introducing Computer Use, a New Claude 3.5 Sonnet, and Claude 3.5 Haiku*. https://www.anthropic.com/news/3-5-models-and-computer-use.

— (2024c). *Tool Use with Claude*. https://docs.anthropic.com/en/docs/build-with-claude/tool-use.

References II

Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio (2015). "Neural Machine Translation by Jointly Learning to Align and Translate". In: *International Conference on Learning Representations (ICLR)*.

Bengio, Yoshua et al. (2003). "A Neural Probabilistic Language Model". In: *Journal of Machine Learning Research* 3, pp. 1137–1155.

Bergstra, James, Rémi Bardenet, et al. (2011). "Algorithms for Hyper-Parameter Optimization". In: *Advances in Neural Information Processing Systems*. Vol. 24. URL: https://papers.nips.cc/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html.

Bergstra, James and Yoshua Bengio (2012). "Random Search for Hyper-Parameter Optimization". In: *Journal of Machine Learning Research* 13, pp. 281–305. URL: https://jmlr.org/papers/v13/bergstra12a.html.

Bernstein, Jeremy et al. (2025). "Power Lines: Scaling Laws for Weight Decay and Batch Size in LLM Pre-training". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2505.13738.

Bernstein, Jeremy and Laker Newhouse (2024). *Old Optimizer, New Norm: An Anthology*. URL: https://arxiv.org/abs/2409.20325.

Beyer, Lucas et al. (2024). "PaliGemma: A Versatile 3B VLM for Transfer". In: *arXiv preprint arXiv:2407.07726*. URL: https://arxiv.org/abs/2407.07726.

Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Springer.

References III

Bishop, Christopher M. and Hugh Bishop (2024). *Deep Learning: Foundations and Concepts*. Springer. URL: https://www.bishopbook.com.

bloc97 (2023). *NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and target length training*. Reddit post. URL: https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/.

Brandon, William, Aniruddha Mishra, et al. (2023). "Striped Attention: Faster Ring Attention for Causal Transformers". In: *arXiv preprint arXiv:2311.09431*. URL: https://arxiv.org/abs/2311.09431.

Brandon, William, Mayank Mishra, et al. (2024). "Reducing Transformer Key-Value Cache Size with Cross-Layer Attention". In: *arXiv preprint arXiv:2405.12981*. URL: https://arxiv.org/abs/2405.12981.

Brown, Tom et al. (2020). "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems* 33. URL: https://arxiv.org/abs/2005.14165.

Bucilă, Cristian, Rich Caruana, and Alexandru Niculescu-Mizil (2006). "Model Compression". In: *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

Campbell, Murray, A Joseph Hoane Jr, and Feng-hsiung Hsu (2002). "Deep Blue". In: *Artificial Intelligence* 134.1–2, pp. 57–83.

Carion, Nicolas et al. (2020). "End-to-End Object Detection with Transformers". In: *European Conference on Computer Vision (ECCV)*. URL: https://arxiv.org/abs/2005.12872.

References IV

Carlson, David et al. (2015). "Stochastic Spectral Descent for Restricted Boltzmann Machines". In: *International Conference on Artificial Intelligence and Statistics (AISTATS)*. URL: https://arxiv.org/abs/1502.05696.

Chen, Shouyuan et al. (2023). "Extending Context Window of Large Language Models via Positional Interpolation". In: *arXiv preprint arXiv:2306.15595*. URL: https://arxiv.org/abs/2306.15595.

Chen, Tianqi et al. (2016). "Training Deep Nets with Sublinear Memory Cost". In: *arXiv preprint arXiv:1604.06174*. URL: https://arxiv.org/abs/1604.06174.

Cheng, Bowen, Ishan Misra, et al. (2022). "Masked-attention Mask Transformer for Universal Image Segmentation". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/2112.01527.

Cheng, Bowen, Alexander G. Schwing, and Alexander Kirillov (2021). "Per-Pixel Classification is Not All You Need for Semantic Segmentation". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2107.06278.

Cho, Kyunghyun et al. (2014). "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734. URL: https://arxiv.org/abs/1406.1078.

Chomsky, Noam (1957). *Syntactic Structures*. Mouton.

References V

Chowdhery, Aakanksha, Sharan Narang, Jacob Devlin, et al. (2023). "PaLM: Scaling Language Modeling with Pathways". In: *Journal of Machine Learning Research* 24. URL: https://arxiv.org/abs/2204.02311.

Collobert, Ronan et al. (2011). "Natural Language Processing (Almost) from Scratch". In: *Journal of Machine Learning Research*. Vol. 12, pp. 2493–2537.

Dai, Damai et al. (2024). "DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models". In: *arXiv preprint arXiv:2401.06066*. URL: https://arxiv.org/abs/2401.06066.

Dao, Tri (2023). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *arXiv preprint arXiv:2307.08691*. URL: https://arxiv.org/abs/2307.08691.

Dao, Tri, Dan Fu, et al. (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2205.14135.

Dao, Tri and Albert Gu (2024). "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2405.21060.

DeepSeek-AI (2024a). "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model". In: *arXiv preprint arXiv:2405.04434*. URL: https://arxiv.org/abs/2405.04434.

References VI

— (2024b). "DeepSeek-V3 Technical Report". In: *arXiv preprint arXiv:2412.19437*. URL: https://arxiv.org/abs/2412.19437.

— (2025). "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning". In: *arXiv preprint arXiv:2501.12948*.

Defazio, Aaron et al. (2024). "The Road Less Scheduled". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2405.15682.

Dehghani, Mostafa et al. (2023). "Getting ViT in Shape: Scaling Laws for Compute-Optimal Model Design". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2305.13035.

Devlin, Jacob et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of NAACL-HLT*, pp. 4171–4186.

Dosovitskiy, Alexey et al. (2021). "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2010.11929.

Falkner, Stefan, Aaron Klein, and Frank Hutter (2018). "BOHB: Robust and Efficient Hyperparameter Optimization at Scale". In: *International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/1807.01774.

References VII

Fedus, William, Barret Zoph, and Noam Shazeer (2022). "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity". In: *Journal of Machine Learning Research* 23.120, pp. 1–39. URL: https://arxiv.org/abs/2101.03961.

Fleuret, François (2023). *The Little Book of Deep Learning*. URL: https://fleuret.org/francois/lbdl.html.

Furlanello, Tommaso et al. (2018). "Born-Again Neural Networks". In: *International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/1805.04770.

Gao, Yunfan et al. (2023). "Retrieval-Augmented Generation for Large Language Models: A Survey". In: *arXiv preprint arXiv:2312.10997*. URL: https://arxiv.org/abs/2312.10997.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. MIT Press. URL: https://www.deeplearningbook.org.

Goodfire, Kevin, Kevin Frans, and Thinking Machines (2025). *On-Policy Distillation*. Thinking Machines Lab Blog. URL: https://thinkingmachines.ai/blog/on-policy-distillation/.

Goyal, Priya et al. (2017). "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour". In: *arXiv preprint arXiv:1706.02677*. URL: https://arxiv.org/abs/1706.02677.

Grattafiori, Aaron, Abhimanyu Dubey, Abhinav Jauhri, et al. (2024). "The Llama 3 Herd of Models". In: *arXiv preprint arXiv:2407.21783*. URL: https://arxiv.org/abs/2407.21783.

References VIII

Gu, Albert and Tri Dao (2024). "Mamba: Linear-Time Sequence Modeling with Selective State Spaces". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2312.00752.

Gu, Albert, Karan Goel, and Christopher Ré (2022). "Efficiently Modeling Long Sequences with Structured State Spaces". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2111.00396.

Gupta, Vineet, Tomer Koren, and Yoram Singer (2018). "Shampoo: Preconditioned Stochastic Tensor Optimization". In: *International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/1802.09568.

He, Kaiming, Xinlei Chen, et al. (2022). "Masked Autoencoders Are Scalable Vision Learners". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/2111.06377.

He, Kaiming, Xiangyu Zhang, et al. (2016). "Deep Residual Learning for Image Recognition". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/1512.03385.

Hinton, Geoffrey, Li Deng, et al. (2012). "Deep neural networks for acoustic modeling in speech recognition". In: *IEEE Signal Processing Magazine* 29.6, pp. 82–97.

Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean (2015). "Distilling the Knowledge in a Neural Network". In: *NeurIPS Deep Learning Workshop*. URL: https://arxiv.org/abs/1503.02531.

References IX

Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780.

Hoffer, Elad, Itay Hubara, and Daniel Soudry (2017). "Train longer, generalize better: closing the generalization gap in large batch training". In: *Advances in Neural Information Processing Systems*. Vol. 30. URL: https://arxiv.org/abs/1705.08741.

Hoffmann, Jordan et al. (2022). "Training Compute-Optimal Large Language Models". In: *arXiv preprint arXiv:2203.15556*. URL: https://arxiv.org/abs/2203.15556.

Holtzman, Ari et al. (2020). "The Curious Case of Neural Text Degeneration". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/1904.09751.

Houlsby, Neil et al. (2019). "Parameter-Efficient Transfer Learning for NLP". In: *International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/1902.00751.

Hu, Edward J et al. (2022). "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2106.09685.

Huang, Yanping et al. (2019). "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/1811.06965.

References X

Jamil, Umar (2024). *Flash Attention Explained*. URL: https://www.youtube.com/watch?v=zy8ChVd_oTM.

Jelinek, Frederick (2005). "Some of My Best Friends are Linguists". In: *Language and Linguistics: Emerging Trends*. Widely attributed, c. 1988.

Jiang, Albert Q. et al. (2023). "Mistral 7B". In: *arXiv preprint arXiv:2310.06825*. URL: https://arxiv.org/abs/2310.06825.

Jiang, Ziheng et al. (2024). "MegaScale: Scaling Model Training to More Than 10,000 GPUs". In: *arXiv preprint arXiv:2402.15627*. URL: https://arxiv.org/abs/2402.15627.

Joachims, Thorsten (1998). "Text Categorization with Support Vector Machines: Learning with Many Relevant Features". In: *European Conference on Machine Learning (ECML)*, pp. 137–142.

Jordan, Keller (2024). *Muon: An optimizer for hidden layers in neural networks*. https://kellerjordan.github.io/posts/muon/.

Kaplan, Jared et al. (2020). "Scaling Laws for Neural Language Models". In: *arXiv preprint arXiv:2001.08361*. URL: https://arxiv.org/abs/2001.08361.

Karpukhin, Vladimir et al. (2020). "Dense Passage Retrieval for Open-Domain Question Answering". In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. URL: https://arxiv.org/abs/2004.04906.

References XI

Khattab, Omar and Matei Zaharia (2020). "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT". In: *ACM SIGIR Conference on Research and Development in Information Retrieval*. URL: https://arxiv.org/abs/2004.12832.

Kingma, Diederik P. and Jimmy Ba (2015). "Adam: A Method for Stochastic Optimization". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/1412.6980.

Kneser, Reinhard and Hermann Ney (1995). *Improved Backing-off for M-gram Language Modeling*.

Kojima, Takeshi et al. (2022). "Large Language Models are Zero-Shot Reasoners". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2205.11916.

Korthikanti, Vijay Anand et al. (2023). "Reducing Activation Recomputation in Large Transformer Models". In: *Proceedings of Machine Learning and Systems (MLSys)*. URL: https://arxiv.org/abs/2205.05198.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E Hinton (2012). "ImageNet classification with deep convolutional neural networks". In: *Advances in Neural Information Processing Systems*. Vol. 25.

Lamy-Poirier, Joel (2023). "Breadth-First Pipeline Parallelism". In: *Proceedings of Machine Learning and Systems (MLSys)*. URL: https://arxiv.org/abs/2211.05953.

References XII

Lester, Brian, Rami Al-Rfou, and Noah Constant (2021). "The Power of Scale for Parameter-Efficient Prompt Tuning". In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. URL: https://arxiv.org/abs/2104.08691.

Lewis, Patrick et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2005.11401.

Li, Houyi et al. (2025). "Predictable Scale: Part I, Step Law — Optimal Hyperparameter Scaling Law in Large Language Model Pretraining". In: *arXiv preprint arXiv:2503.04715*. URL: https://arxiv.org/abs/2503.04715.

Li, Lisha et al. (2017). "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization". In: *Journal of Machine Learning Research*. Vol. 18. 185, pp. 1–52. URL: https://arxiv.org/abs/1603.06560.

Li, Shen et al. (2020). "PyTorch Distributed: Experiences on Accelerating Data Parallel Training". In: *Proceedings of the VLDB Endowment* 13.12. URL: https://arxiv.org/abs/2006.15704.

Li, Xiang Lisa and Percy Liang (2021). "Prefix-Tuning: Optimizing Continuous Prompts for Generation". In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*. URL: https://arxiv.org/abs/2101.00190.

References XIII

Liu, Hao, Matei Zaharia, and Pieter Abbeel (2023). "Ring Attention with Blockwise Transformers for Near-Infinite Context". In: *arXiv preprint arXiv:2310.01889*. URL: https://arxiv.org/abs/2310.01889.

Liu, Ze et al. (2021). "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows". In: *International Conference on Computer Vision (ICCV)*. URL: https://arxiv.org/abs/2103.14030.

Liu, Zichen et al. (2025). "Understanding R1-Zero-Like Training: A Critical Perspective". In: *arXiv preprint arXiv:2503.20783*. URL: https://arxiv.org/abs/2503.20783.

Loshchilov, Ilya and Frank Hutter (2017). "SGDR: Stochastic Gradient Descent with Warm Restarts". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/1608.03983.

— (2019). "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/1711.05101.

Marcus, Mitchell P., Beatrice Santorini, and Mary Ann Marcinkiewicz (1993). "Building a Large Annotated Corpus of English: The Penn Treebank". In: *Computational Linguistics*. Vol. 19. 2, pp. 313–330.

McCandlish, Sam et al. (2018). "An Empirical Model of Large-Batch Training". In: *arXiv preprint arXiv:1812.06162*. URL: https://arxiv.org/abs/1812.06162.

References XIV

Meinhardt, Tim et al. (2022). "TrackFormer: Multi-Object Tracking with Transformers". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/2101.02702.

Micikevicius, Paulius et al. (2018). "Mixed Precision Training". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/1710.03740.

Mikolov, Tomas et al. (2013). "Distributed Representations of Words and Phrases and Their Compositionality". In: *Advances in Neural Information Processing Systems*.

Murphy, Kevin P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press. URL: https://probml.github.io/pml-book/book1.html.

— (2023). *Probabilistic Machine Learning: Advanced Topics*. MIT Press. URL: https://probml.github.io/pml-book/book2.html.

Narayanan, Deepak, Aaron Harlap, et al. (2019). "PipeDream: Generalized Pipeline Parallelism for DNN Training". In: *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*. URL: https://arxiv.org/abs/1806.03377.

Narayanan, Deepak, Mohammad Shoeybi, et al. (2021). "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM". In: *arXiv preprint arXiv:2104.04473*. URL: https://arxiv.org/abs/2104.04473.

OpenAI (2023). "GPT-4 Technical Report". In: *arXiv preprint arXiv:2303.08774*.

References XV

— (2024). *Learning to Reason with LLMs*. https://openai.com/index/learning-to-reason-with-llms/.

— (2025). *Introducing OpenAI o3 and o4-mini*. https://openai.com/index/introducing-o3-and-o4-mini/.

Oquab, Maxime, Timothée Darcet, Théo Moutakanni, et al. (2023). "DINOv2: Learning Robust Visual Features without Supervision". In: *arXiv preprint arXiv:2304.07193*. URL: https://arxiv.org/abs/2304.07193.

Ouyang, Long et al. (2022). "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2203.02155.

Park, Joon Sung et al. (2023). "Generative Agents: Interactive Simulacra of Human Behavior". In: *ACM Symposium on User Interface Software and Technology (UIST)*. URL: https://arxiv.org/abs/2304.03442.

Peng, Bowen et al. (2023). "YaRN: Efficient Context Window Extension of Large Language Models". In: *arXiv preprint arXiv:2309.00071*. URL: https://arxiv.org/abs/2309.00071.

Pennington, Jeffrey, Richard Socher, and Christopher D. Manning (2014). "GloVe: Global Vectors for Word Representation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1532–1543.

References XVI

Peters, Matthew E. et al. (2018). "Deep Contextualized Word Representations". In: *arXiv preprint arXiv:1802.05365*.

Prince, Simon J.D. (2023). *Understanding Deep Learning*. MIT Press. URL: https://udlbook.github.io/udlbook/.

Qi, Penghui et al. (2024). "Zero Bubble Pipeline Parallelism". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2401.10241.

Radford, Alec, Jong Wook Kim, et al. (2021). "Learning Transferable Visual Models From Natural Language Supervision". In: *International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/2103.00020.

Radford, Alec, Karthik Narasimhan, et al. (2018). *Improving Language Understanding by Generative Pre-Training*. OpenAI Blog.

Radford, Alec, Jeffrey Wu, et al. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Blog. URL: https://openai.com/research/language-unsupervised.

Rafailov, Rafael et al. (2024). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model". In: *Advances in Neural Information Processing Systems*. Vol. 36. URL: https://arxiv.org/abs/2305.18290.

Raffel, Colin et al. (2020). "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In: *Journal of Machine Learning Research* 21.140, pp. 1–67. URL: https://arxiv.org/abs/1910.10683.

References XVII

Rajbhandari, Samyam et al. (2020). "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models". In: *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. URL: https://arxiv.org/abs/1910.02054.

Sanh, Victor et al. (2019). "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter". In: *arXiv preprint arXiv:1910.01108*. URL: https://arxiv.org/abs/1910.01108.

Schick, Timo et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2302.04761.

Schneider, Oliver and Sebastian Wallkötter (2023). *Reinforcement Learning Course Materials*. University of Paderborn. URL: https://github.com/upb-lea/reinforcement_learning_course_materials.

Schulman, John et al. (2017). "Proximal Policy Optimization Algorithms". In: *arXiv preprint arXiv:1707.06347*. URL: https://arxiv.org/abs/1707.06347.

Sennrich, Rico, Barry Haddow, and Alexandra Birch (2016). "Neural Machine Translation of Rare Words with Subword Units". In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1715–1725. URL: https://arxiv.org/abs/1508.07909.

Shannon, Claude E. (1951). "Prediction and Entropy of Printed English". In: *Bell System Technical Journal* 30.1, pp. 50–64.

References XVIII

Shao, Zhihong et al. (2024). "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models". In: *arXiv preprint arXiv:2402.03300*. URL: https://arxiv.org/abs/2402.03300.

Shazeer, Noam (2019). "Fast Transformer Decoding: One Write-Head is All You Need". In: *arXiv preprint arXiv:1911.02150*. URL: https://arxiv.org/abs/1911.02150.

Shazeer, Noam et al. (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/1701.06538.

Shinn, Noah et al. (2023). "Reflexion: Language Agents with Verbal Reinforcement Learning". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2303.11366.

Shoeybi, Mohammad et al. (2020). "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism". In: *arXiv preprint arXiv:1909.08053*. URL: https://arxiv.org/abs/1909.08053.

Silver, David (2015). *Lectures on Reinforcement Learning*. URL: https://www.davidsilver.uk/teaching/.

Silver, David et al. (2017). "Mastering the game of Go without human knowledge". In: *Nature* 550.7676, pp. 354–359.

References XIX

Smith, Leslie N. (2017). "Cyclical Learning Rates for Training Neural Networks". In: *IEEE Winter Conference on Applications of Computer Vision (WACV)*. URL: https://arxiv.org/abs/1506.01186.

Snoek, Jasper, Hugo Larochelle, and Ryan P. Adams (2012). "Practical Bayesian Optimization of Machine Learning Algorithms". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 25. URL: https://arxiv.org/abs/1206.2944.

Su, Jianlin et al. (2024). "RoFormer: Enhanced Transformer with Rotary Position Embedding". In: *Neurocomputing* 568, p. 127063. URL: https://arxiv.org/abs/2104.09864.

Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le (2014). "Sequence to Sequence Learning with Neural Networks". In: *Advances in Neural Information Processing Systems*.

Sutton, Richard (2019). *The Bitter Lesson*. URL: http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

Sutton, Richard S and Andrew G Barto (2018). *Reinforcement Learning: An Introduction*. 2nd. MIT Press. URL: http://www.incompleteideas.net/book/the-book-2nd.html.

Tazi, Nouamane et al. (2025). *The Ultra-Scale Model Training Playbook*. Hugging Face. URL: https://huggingface.co/spaces/nanotron/ultrascale-playbook.

Tillet, Philippe, H.T. Kung, and David Cox (2019). *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations*. URL: https://www.eecs.harvard.edu/~htk/publication/2019-mapl-tillet-kung-cox.pdf.

References XX

Tolstikhin, Ilya O. et al. (2021). "MLP-Mixer: An all-MLP Architecture for Vision". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2105.01601.

Touvron, Hugo, Matthieu Cord, et al. (2021). "Training data-efficient image transformers & distillation through attention". In: *International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/2012.12877.

Touvron, Hugo, Thibaut Lavril, et al. (2023). "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971*. URL: https://arxiv.org/abs/2302.13971.

Touvron, Hugo, Louis Martin, et al. (2023). "Llama 2: Open Foundation and Fine-Tuned Chat Models". In: *arXiv preprint arXiv:2307.09288*. URL: https://arxiv.org/abs/2307.09288.

Tuddenham, Alex et al. (2022). "Orthogonalising gradients to speed up neural network optimisation". In: *arXiv preprint arXiv:2202.07052*. URL: https://arxiv.org/abs/2202.07052.

Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. URL: https://arxiv.org/abs/1706.03762.

Wang, Xuezhi et al. (2023). "Self-Consistency Improves Chain of Thought Reasoning in Language Models". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2203.11171.

References XXI

Wei, Jason et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2201.11903.

Wightman, Ross, Hugo Touvron, and Hervé Jégou (2021). "ResNet Strikes Back: An Improved Training Procedure in `timm`". In: *arXiv preprint arXiv:2110.00476*. URL: https://arxiv.org/abs/2110.00476.

Xie, Saining et al. (2017). "Aggregated Residual Transformations for Deep Neural Networks". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. URL: https://arxiv.org/abs/1611.05431.

Yang, Greg et al. (2022). "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer". In: *International Conference on Machine Learning (ICML)*. URL: https://arxiv.org/abs/2203.03466.

Yao, Shunyu, Dian Yu, et al. (2023). "Tree of Thoughts: Deliberate Problem Solving with Large Language Models". In: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: https://arxiv.org/abs/2305.10601.

Yao, Shunyu, Jeffrey Zhao, et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2210.03629.

References XXII

Yu, Qiying et al. (2025). "DAPO: An Open-Source LLM Reinforcement Learning System at Scale". In: *arXiv preprint arXiv:2503.14476*. URL: https://arxiv.org/abs/2503.14476.

Zhai, Xiaohua et al. (2023). "Sigmoid Loss for Language Image Pre-Training". In: *International Conference on Computer Vision (ICCV)*. URL: https://arxiv.org/abs/2303.15343.

Zhang, Linfeng et al. (2019). "Be Your Own Teacher: Improve the Performance of Convolutional Neural Networks via Self Distillation". In: *IEEE/CVF International Conference on Computer Vision (ICCV)*. URL: https://arxiv.org/abs/1905.08094.

Zhu, Xizhou et al. (2021). "Deformable DETR: Deformable Transformers for End-to-End Object Detection". In: *International Conference on Learning Representations (ICLR)*. URL: https://arxiv.org/abs/2010.04159.